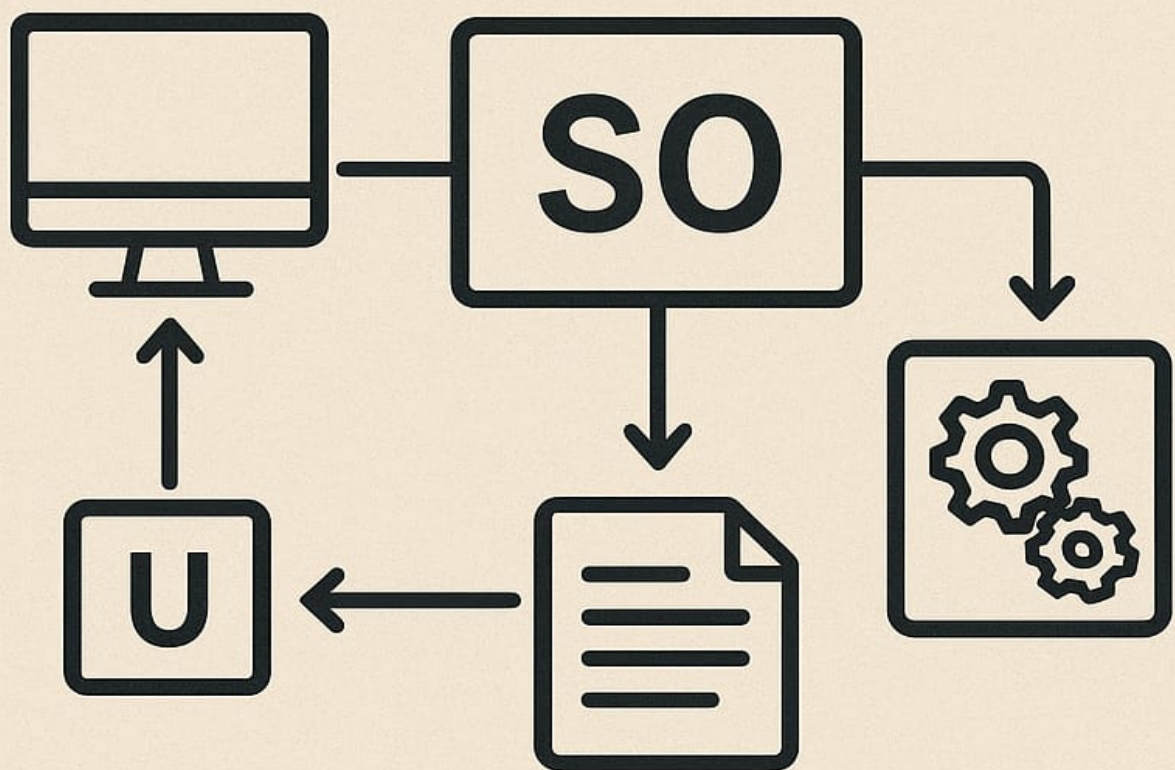


Introducción a los **Sistemas Operativos**

Fundamentos teóricos, ejercicios y código de ejemplo en C



Daniel Peralta

Introducción a los Sistemas Operativos

Este libro ofrece una introducción clara y práctica al fascinante mundo de los sistemas operativos. Pensado para estudiantes universitarios y autodidactas, combina fundamentos teóricos con ejercicios tipo parcial resueltos y código de ejemplo en C para que puedas aprender haciendo.

A lo largo de sus capítulos, exploraremos desde los conceptos básicos y la evolución de los sistemas operativos hasta la gestión de procesos, memoria, archivos, dispositivos y concurrencia, finalizando con temas avanzados como sistemas distribuidos y virtualización.

¿Qué encontrarás en este libro?



Fundamentos teóricos explicados de manera simple y directa.



Ejemplos y programas en C para entender cómo funcionan los conceptos por dentro.



Ejercicios prácticos tipo parcial UTN, con enunciados y soluciones detalladas.



Diagramas y recursos visuales para facilitar la comprensión de los temas clave.



Ideal como material de estudio, repaso o consulta durante la cursada y más allá.

Contenidos

- 1 Repaso Arquitectura de Computadores
- 2 Fundamentos de los Sistemas Operativos
- 3 Procesos
- 4 Planificación de Procesos
- 5 Hilos (Threads)
- 6 Sincronización
- 7 Interbloqueo (Deadlock)
- 8 Gestión de Memoria Real
- 9 Gestión de Memoria Virtual
- 10 Sistemas de Archivos (File System)
- 11 I/O - Gestión de Dispositivos



Daniel Peralta

Ayudante de cátedra y estudiante avanzado en ingeniería en sistemas de información en la Universidad Tecnológica Nacional, apasionado por la enseñanza y la divulgación de conceptos complejos de manera simple y práctica. Creador de contenido educativo en YouTube.



Conocé más recursos en:
dperalta.com.ar

“La mejor manera de aprender
sistemas operativos es
entenderlos por dentro.”

ISBN 978-631-01-5312-4



Introducción a los Sistemas Operativos

Fundamentos teóricos, ejercicios y código de ejemplo en C

Daniel Isaías Peralta

1ª edición — Mayo 2026

Longchamps, Argentina

Peralta, Daniel Isaías

Introducción a los sistemas operativos : fundamentos teóricos, ejercicios y código de ejemplo en C / Daniel Isaías Peralta. — 1ª ed. — Longchamps : Daniel Isaías Peralta, 2026.

Libro digital, PDF

Archivo digital: descarga y online

Creada PARCIALMENTE con IA / ChatGPT (OpenAI)

ISBN 978-631-01-5312-4

1. Sistemas Operativos. I. Título.

CDD 003

Primera edición: mayo de 2026

Versión del archivo: 1.0

Licencia: Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0).

Este material puede ser compartido, copiado, distribuido y adaptado libremente, siempre que se cite al autor y se mantenga la misma licencia en las obras derivadas.

<https://creativecommons.org/licenses/by-sa/4.0/>

Contacto:

hola@dperalta.com.ar — www.dperalta.com.ar

*A mi esposa Cecilia por ser ese equilibrio constante:
la que escucha cada idea, incluso las más desordenadas,
la que baja todo a tierra cuando hace falta,
pero nunca para frenar, sino para ayudarme a seguir mejor.
Por acompañarme sin imponer, por sostenerme sin limitar.*

*A mis hijos Tomás y Jazmín, por ser el motor y el motivo.
Por recordarme, todos los días,
para qué vale la pena hacer las cosas bien.*

Este libro surge en el marco de la materia **Sistemas Operativos** de la carrera de Ingeniería en Sistemas de Información

Universidad Tecnológica Nacional
Facultad Regional Buenos Aires (UTN-FRBA)

Este es un material de elaboración independiente creado por un estudiante de la carrera. No constituye material oficial de la cátedra ni de la Universidad Tecnológica Nacional — FRBA, y no ha sido revisado ni avalado por ninguna autoridad académica de la institución.

Índice de Contenidos

1	Prefacio	1
1.1	¿Qué vas a encontrar en este libro?	1
1.2	Código real, no pseudocódigo	1
1.3	Estructura progresiva	1
1.4	Preparación para evaluaciones	2
1.5	Metodología de estudio sugerida	2
1.6	Código fuente y contacto	2
2	Mensaje a los Estudiantes	3
3	Repaso Arquitectura de Computadores	5
3.1	Introducción y Contexto	5
3.1.1	¿Por qué necesitamos entender la arquitectura?	5
3.1.2	Los problemas que resuelve esta arquitectura	5
3.2	Conceptos Fundamentales	6
3.2.1	Arquitectura de von Neumann	6
3.3	Buses de Comunicación	7
3.4	Registros del Procesador	7
3.5	Program Status Word (PSW)	8
3.5.1	Ciclo de Instrucción	10
3.6	Sistema de Interrupciones	10
3.6.1	Interrupciones Enmascarables (Maskable Interrupts)	11
3.6.2	Interrupciones No Enmascarables (NMI - Non-Maskable Interrupts)	12
3.7	Jerarquía de Prioridades	12
3.8	IRQ (Interrupt Request Lines)	13
3.8.1	Controlador PIC (Programmable Interrupt Controller)	13
3.9	Interrupt Handlers (Manejadores de Interrupción)	13
3.9.1	Estructura de un Handler	14
3.9.2	Interrupciones Anidadas	14
3.10	Estados de Interrupciones	15
3.10.1	Disable/Enable a Nivel de Sistema	15
3.10.2	Disable/Enable por IRQ Específico	16
3.10.3	Modos de Operación del Procesador	16
3.10.4	Cambio de Procesos	17
3.10.5	Resumen y Hoja de Ruta	20

4	Fundamentos de los Sistemas Operativos	23
4.1	Objetivos de Aprendizaje	23
4.2	Introducción a los Sistemas Operativos	23
4.2.1	¿Qué es realmente un Sistema Operativo?	24
4.2.2	Los Cuatro Pilares Fundamentales	24
4.2.3	¿Por qué es tan Complejo?	25
4.2.4	El Enfoque Unix/Linux	26
4.2.5	Principales Tareas del Sistema Operativo	27
4.2.6	System Calls (Llamadas al Sistema)	28
4.2.7	Fast System Calls	29
4.2.8	Arquitecturas de Kernel	30
4.3	Análisis Técnico (Leelo sólo por curiosidad...)	32
4.3.1	Proceso de Arranque (Bootstrap)	32
4.3.2	Manejo de Interrupciones	33
4.3.3	Jerarquía de Memoria	33
4.4	Código en C	34
4.4.1	Ejemplo: Información del Sistema	34
4.4.2	Ejemplo: Manejo Básico de Señales	34
4.5	Casos de Estudio	35
4.5.1	Caso 1: Análisis de una Syscall	35
4.6	Síntesis	36
4.6.1	Puntos Clave del Capítulo	36
4.6.2	Conexiones con Próximos Capítulos	37
4.6.3	Errores Comunes en Parciales	37
5	Procesos	39
5.1	Objetivos de Aprendizaje	39
5.2	Introducción y Contexto	39
5.2.1	¿Por qué existen los procesos?	39
5.3	Conceptos Fundamentales	40
5.3.1	Programa vs Proceso: La Diferencia Esencial	40
5.3.2	Evolución de los Sistemas de Procesamiento	40
5.3.3	Grado de Multiprogramación	42
5.3.4	Estructuras de Control del Sistema Operativo	42
5.3.5	Imagen de un Proceso	45
5.3.6	Process Control Block (PCB)	46
5.3.7	Diagramas de Estado de Procesos	48
5.4	Análisis Técnico	50
5.4.1	Creación de Procesos en Unix/Linux	50
5.4.2	Carga de Programas: exec()	51

5.4.3	Terminación de Procesos	52
5.4.4	Procesos Zombies y Huérfanos	52
5.5	Código en C	54
5.5.1	Creación Básica de Procesos	54
5.5.2	Usando exec() para cargar programas	55
5.5.3	Manejo de múltiples hijos	56
5.5.4	Comunicación entre procesos: pipes	57
5.6	Casos de Estudio	59
5.6.1	Caso de Estudio 1: Análisis de fork() múltiple	59
5.6.2	Caso de Estudio 2: Problema de procesos zombie	60
5.7	Síntesis	61
5.7.1	Puntos Clave para Parcial	61
5.7.2	Errores Comunes y Tips	62
5.7.3	Conexión con Próximos Temas	63
5.7.4	Preguntas de Reflexión	63
6	Planificación de Procesos	65
6.1	Objetivos de Aprendizaje	65
6.2	Introducción y Contexto	65
6.2.1	¿Por qué necesitamos planificación?	65
6.2.2	El problema fundamental	65
6.2.3	Objetivos conflictivos	66
6.3	Conceptos Fundamentales	66
6.3.1	Niveles de Planificación	66
6.3.2	Tipos de Planificación	67
6.3.3	Métricas de Rendimiento	68
6.3.4	Ráfagas de CPU y I/O	69
6.3.5	Eventos que Provocan Planificación	70
6.4	Análisis Técnico	70
6.4.1	First-Come First-Served (FCFS)	70
6.4.2	Shortest Job First (SJF)	71
6.4.3	Shortest Remaining Time (SRT)	73
6.4.4	Round Robin (RR)	74
6.4.5	Virtual Round Robin (VRR)	75
6.4.6	Highest Response Ratio Next (HRRN)	76
6.4.7	Planificación por Prioridades	77
6.4.8	Multilevel Feedback Queue	78
6.5	Casos de Estudio	79
6.5.1	Planificación con Round Robin	79
6.5.2	Caso de Estudio: Planificación con SJF	81

6.5.3	Manejo de Prioridades en Eventos Simultáneos	82
6.6	Síntesis	83
6.6.1	Puntos Clave para Parcial	83
6.6.2	Ejemplo simulación Round Robin	84
6.6.3	Decisiones de Diseño	87
7	Hilos (Threads)	89
7.1	Objetivos de Aprendizaje	89
7.2	Introducción y Motivación	89
7.2.1	El problema de los procesos pesados	89
7.2.2	La solución: hilos como procesos livianos	90
7.3	Estructura y Definiciones Fundamentales	90
7.3.1	Anatomía de un hilo	90
7.3.2	Modelo de memoria con hilos	91
7.4	User-Level Threads vs Kernel-Level Threads	92
7.4.1	User-Level Threads (ULT)	92
7.4.2	Kernel-Level Threads (KLT)	94
7.4.3	Modelos de mapeo	95
7.5	Implementación en C con pthreads	97
7.5.1	Creación básica de hilos	98
7.5.2	Diferencias clave respecto a procesos	100
7.6	Gestión avanzada de hilos	101
7.6.1	Workers y pool de hilos	101
7.7	Planificación de hilos	104
7.7.1	Planificación de ULT	104
7.7.2	Planificación de KLT	105
7.8	Ejercicio práctico completo	106
7.8.1	Enunciado del problema	106
7.8.2	Análisis de la planificación con combinación de KLT y ULT	106
7.9	Análisis comparativo y casos de uso	108
7.9.1	Overhead de implementaciones	108
7.9.2	Casos de uso recomendados	109
7.10	Problemas comunes y debugging	110
7.10.1	Race conditions y datos compartidos	110
7.10.2	Herramientas de debugging	111
7.10.3	Patterns de sincronización básicos	112
7.11	Tendencias y tecnologías modernas	113
7.11.1	Green threads y corrutinas	114
7.11.2	Thread affinity y NUMA	114

7.12	Síntesis y conexiones	115
7.12.1	Puntos clave para evaluación	115
7.12.2	Errores conceptuales frecuentes	116
7.12.3	Preparación para próximos capítulos	116
7.12.4	Laboratorio propuesto	117
8	Sincronización	119
8.1	Objetivos de Aprendizaje	119
8.2	Introducción y Contexto	119
8.2.1	¿Por qué necesitamos sincronización?	119
8.3	El Problema Fundamental: Race Conditions	120
8.4	Sección Crítica y Condiciones de Bernstein	121
8.4.1	Sección Crítica	121
8.4.2	Condiciones de Bernstein	122
8.5	Soluciones a Nivel Software	122
8.5.1	Evolución Histórica de las Soluciones	122
8.5.2	Solución de Peterson (1981)	124
8.6	Soluciones a Nivel Hardware	125
8.6.1	Primitivos Atómicos	125
8.7	Soluciones del Sistema Operativo: Semáforos	127
8.7.1	Definición y Operaciones	127
8.7.2	Tipos de Semáforos	129
8.7.3	Usos Principales de Semáforos	130
8.8	Ejemplo Práctico: Control de Cochera	131
8.8.1	Planteamiento del Problema	132
8.8.2	Solución con Semáforos	132
8.9	Uso de Arrays de Semáforos	136
8.9.1	Problema: Múltiples Recursos del Mismo Tipo	136
8.9.2	Problema Clásico: Productor-Consumidor	139
8.9.3	Problema Lectores-Escritores	142
8.10	Síntesis	146
9	Interbloqueo (Deadlock)	149
9.1	Objetivos de Aprendizaje	149
9.2	Introducción y Contexto	149
9.2.1	El problema de la espera circular	149
9.2.2	Problemas relacionados: Starvation y Livelock	150
9.3	Condiciones para Deadlock	150
9.3.1	Las cuatro condiciones necesarias	150
9.3.2	Representación mediante grafos	151

9.4	Estrategias para manejar Deadlocks	152
9.4.1	El algoritmo del avestruz: “Do nothing”	153
9.5	Prevención de Deadlocks	154
9.5.1	Rompiendo la exclusión mutua	154
9.5.2	Rompiendo retención y espera	155
9.5.3	Rompiendo no expropiación	155
9.5.4	Rompiendo espera circular	155
9.6	Evasión de Deadlocks (Algoritmo del Banquero)	156
9.6.1	Estado seguro vs. estado inseguro	156
9.6.2	Estructuras de datos del algoritmo	157
9.6.3	Inicialización y verificación de seguridad	157
9.6.4	Solicitud de recursos	157
9.6.5	Ejemplo 1: Estado seguro	158
9.6.6	Ejemplo 2: Solicitud que lleva a estado seguro	158
9.7	Detección y Recuperación	159
9.7.1	Algoritmo de detección (similar al Safety Algorithm)	159
9.7.2	Recuperación de deadlocks	160
9.8	Deadlock en Propósito General vs. Sistemas Críticos	160
9.8.1	Sistemas de propósito general (PCs, servidores web)	160
9.8.2	Sistemas críticos (aeronáutica, medicina, industria)	161
9.9	Ejercicios Tipo Parcial	161
9.10	Síntesis	163
10	Gestión de Memoria Real	165
10.1	Objetivos de Aprendizaje	165
10.2	Introducción y Contexto	165
10.2.1	¿Por qué necesitamos gestionar la memoria?	165
10.2.2	El problema fundamental	166
10.2.3	Evolución histórica	166
10.3	Conceptos Fundamentales	166
10.3.1	Espacios de Direcciones	166
10.3.2	Binding de Direcciones	167
10.3.3	Componentes Hardware	169
10.3.4	Fragmentación	170
10.4	Técnicas de Asignación Contigua	171
10.4.1	Particiones Fijas	172
10.4.2	Particiones Dinámicas	172
10.4.3	Algoritmos de Asignación	173
10.5	Paginación Simple	176
10.5.1	Formato de Dirección Lógica	177

10.5.2 Traducción de Direcciones	178
10.5.3 Tabla de Páginas	179
10.5.4 Fragmentación Interna en Paginación	180
10.6 Segmentación	180
10.6.1 Concepto y Motivación	181
10.6.2 Formato de Dirección Lógica en Segmentación	182
10.6.3 Ventajas de Segmentación	183
10.6.4 Segmentación con Paginación	184
10.7 Técnicas Avanzadas	184
10.7.1 Buddy System	184
10.7.2 Paginación Multinivel	187
10.7.3 Tabla de Páginas Invertida	189
10.8 Compactación y Defragmentación	190
10.9 Protección y Compartición	191
10.9.1 Mecanismos de Protección	191
10.9.2 Compartición de Memoria	192
10.10 Código en C	193
10.10.1 Conceptos Básicos de Memoria y Punteros (Bonus)	193
10.10.2 Aritmética de Punteros	193
10.10.3 Asignación Dinámica con malloc	194
10.10.4 Mapeo de Memoria con mmap	195
10.10.5 Ejemplo Integrador: Simulación de Tabla de Páginas	196
10.11 Casos de Estudio	200
10.11.1 Ejercicio Simple: Traducción de Dirección Lógica	200
10.11.2 Ejercicio Complejo: Deducción y Traducción	201
10.12 Síntesis	204
10.12.1 Puntos Clave del Capítulo	204
10.12.2 Conexiones con Otros Temas	204
10.12.3 Errores Comunes en Parciales	205
10.12.4 Preguntas de Reflexión	205
10.12.5 Ejercicios Propuestos	206
10.12.6 Material para Profundizar	206
11 Gestión de Memoria Virtual	207
11.1 Objetivos de Aprendizaje	207
11.2 Introducción y Contexto	207
11.2.1 El Problema Fundamental	207
11.2.2 ¿Cómo es posible?	209
11.3 Recap: Conceptos del Capítulo 7	209
11.3.1 Paginación Básica (Recap)	209

11.3.2	TLB (Recap)	210
11.3.3	Lo Nuevo en Memoria Virtual	210
11.4	Conceptos Fundamentales	211
11.4.1	Espacio de Direcciones Virtual vs Físico	211
11.4.2	El Bit Presente (Present bit): El Héroe de Memoria Virtual	212
11.4.3	Page Fault: Evento Normal, NO Error	213
11.4.4	Swap Space (Backing Store)	214
11.5	Page Fault: Anatomía Completa	215
11.5.1	Flujo Paso a Paso	215
11.5.2	Resumen de Casos de Page Fault	216
11.5.3	Tiempos Aproximados	216
11.5.4	Casos Especiales	217
11.6	Principio de Localidad	218
11.6.1	¿Qué es el Principio de Localidad?	219
11.6.2	Localidad Temporal	219
11.6.3	Localidad Espacial	220
11.6.4	Localidad en Programas Reales	221
11.6.5	Working Set	221
11.6.6	¿Por Qué Funciona en Práctica?	222
11.7	Demand Paging vs Prepaging	223
11.7.1	Demand Paging (Paginación Bajo Demanda)	224
11.7.2	Prepaging (Paginación Anticipada)	225
11.8	Shared Pages y Técnicas Avanzadas	226
11.8.1	Copy-on-Write (COW)	226
11.8.2	Memory-Mapped Files	228
11.8.3	Shared Libraries	230
11.8.4	TLB Reach	231
11.9	Algoritmos de Reemplazo de Páginas	231
11.9.1	Objetivo del Algoritmo de Reemplazo	231
11.9.2	FIFO (First-In-First-Out)	232
11.9.3	Óptimo (OPT o MIN)	233
11.9.4	LRU (Least Recently Used)	233
11.9.5	Clock (Second Chance)	234
11.9.6	Clock-M (Clock Mejorado / Enhanced Second Chance)	236
11.9.7	Ejercicio Comparativo de Algoritmos	237
11.9.8	Comparación Final	239
11.10	Thrashing	240
11.10.1	¿Qué es Thrashing?	240
11.10.2	¿Por Qué Ocurre?	241

11.10.3 Detección de Thrashing	242
11.10.4 Prevención y Recuperación	242
11.10.5 Preguntas Típicas de Examen	243
11.11 Código en C: Simulador de Clock-M	244
11.12 Conclusiones del Capítulo	250
12 Sistema de Archivos (File System)	253
12.1 Objetivos de Aprendizaje	253
12.2 File System: Definición y Objetivos	253
12.2.1 ¿Por qué necesitamos File Systems?	253
12.2.2 Objetivos del File System	254
12.3 Archivos: Abstracción de Usuario	255
12.3.1 Concepto de Archivo	255
12.3.2 Atributos de un Archivo	255
12.3.3 File Control Block (FCB)	256
12.3.4 Operaciones sobre Archivos	256
12.3.5 Métodos de Acceso	257
12.4 Directorios: Organización Jerárquica	259
12.4.1 Concepto de Directorio	259
12.4.2 Estructuras de Directorios	259
12.4.3 Rutas: Absolutas y Relativas	261
12.4.4 Operaciones sobre Directorios	262
12.5 Estructuras Administrativas del SO	262
12.5.1 Arquitectura de Tres Niveles	263
12.5.2 Nivel 1: File Descriptor Table (por proceso)	263
12.5.3 Nivel 2: Open File Table (system-wide)	263
12.5.4 Nivel 3: Inode Table	264
12.5.5 Ejemplo de Interacción	265
12.5.6 File Locking: Sincronización de Acceso	265
12.6 Protección y Permisos	267
12.6.1 Modelo de Permisos Unix	267
12.6.2 Verificación de Permisos	268
12.6.3 Umask: Permisos por Defecto	269
12.6.4 Cambio de Permisos	269
12.7 Implementación Física: Disco	270
12.7.1 Hardware: Sectores vs Bloques	270
12.7.2 ¿Por qué usar bloques en vez de sectores directamente?	271
12.7.3 Fragmentación Interna	271
12.8 Métodos de Asignación de Espacio	272
12.8.1 Método 1: Asignación Contigua	272

12.8.2	Método 2: Asignación Enlazada	272
12.8.3	Método 3: Asignación Indexada	274
12.8.4	Comparación de Métodos	274
12.9	Manejo de Espacio Libre	275
12.9.1	Método 1: Bitmap de Bloques	275
12.9.2	Método 2: Lista Enlazada de Bloques Libres	276
12.9.3	Método 3: Agrupamiento	276
12.9.4	Método Usado en Práctica	276
12.10	Seguridad y Recuperación	276
12.10.1	Journaling: Registro de Transacciones	276
12.10.2	fsck / chkdsk: File System Consistency Check	277
12.11	Caso de Estudio 1: FAT (File Allocation Table)	278
12.11.1	Características Generales	279
12.11.2	Componentes de FAT	279
12.11.3	Concepto de Cluster	281
12.11.4	FAT12, FAT16, FAT32: Evolución	281
12.11.5	Ejemplo de Operación: Lectura de Archivo	282
12.12	Caso de Estudio 2: EXT2 / UFS (Unix File System)	283
12.12.1	Filosofía de Diseño	283
12.12.2	Estructura General: Block Groups	283
12.12.3	Componentes de un Block Group	284
12.12.4	El Inodo: Corazón de EXT2	285
12.12.5	Esquema de Punteros del Inodo	286
12.12.6	Cálculo de Tamaño Máximo Teórico	287
12.12.7	Ejemplo de Acceso a Archivo en EXT2	287
12.13	Comparación: FAT vs EXT2	288
12.14	Links: Hard Links y Soft Links	289
12.14.1	Hard Links	290
12.14.2	Soft Links (Symbolic Links)	291
12.14.3	Comparación Hard Link vs Soft Link	292
12.15	Ejercicios integradores	292
12.15.1	Ejercicio 1: lectura simple en EXT2	292
12.15.2	Ejercicio 2: lectura con múltiples niveles de indirección en EXT2	293
12.16	Otros sistemas de archivos	296
12.16.1	EXT3 y EXT4	297
12.16.2	NTFS	297
12.16.3	exFAT	297
12.16.4	APFS, XFS, Btrfs y ZFS	297
12.16.5	Cuadro comparativo	298

12.17 Síntesis y puntos clave	298
12.17.1 Conceptos fundamentales	298
12.17.2 Operaciones y estructuras del sistema operativo	299
12.17.3 Métodos de asignación de espacio	299
12.17.4 FAT y EXT2 comparados	300
12.17.5 Links	300
12.17.6 Consistencia y recuperación	301
12.18 Guía de resolución para ejercicios numéricos	301
12.18.1 Procedimiento paso a paso	301
12.18.2 Errores frecuentes	302
12.19 Conexiones con otros capítulos	302
13 I/O - Gestión de Dispositivos	305
13.1 Objetivos de Aprendizaje	305
13.2 Introducción: ¿Por qué es complejo el I/O?	305
13.2.1 Desafíos del I/O	306
13.2.2 Objetivos del Subsistema de I/O	306
13.3 Hardware de I/O	307
13.3.1 Tipos de Dispositivos	307
13.3.2 Controladores de Dispositivo	308
13.3.3 Interfaces de I/O	308
13.4 Técnicas de I/O	309
13.4.1 Técnica 1: I/O Programado (Polling)	309
13.4.2 Técnica 2: I/O por Interrupciones	310
13.4.3 Técnica 3: DMA (Direct Memory Access)	311
13.4.4 Comparación de Técnicas	313
13.5 Discos Magnéticos: Estructura y Rendimiento	314
13.5.1 Geometría del Disco	314
13.5.2 Tiempos de Acceso a Disco	316
13.5.3 Ejercicio: Cálculo de Tiempo de Acceso	318
13.6 Scheduling de I/O: Algoritmos de Disco	319
13.6.1 Algoritmo 1: FCFS (First-Come, First-Served)	320
13.6.2 Algoritmo 2: SSTF (Shortest Seek Time First)	320
13.6.3 Algoritmo 3: SCAN (Elevator Algorithm)	321
13.6.4 Algoritmo 4: C-SCAN (Circular SCAN)	322
13.6.5 Algoritmo 5: LOOK	323
13.6.6 Algoritmo 6: C-LOOK (Circular LOOK)	323
13.6.7 Algoritmo 7: FSCAN (Freeze SCAN)	324
13.6.8 Algoritmo 8: N-STEP-SCAN	324
13.6.9 Comparación de Algoritmos	325

13.7	Ejercicio Integrador: Scheduling de Disco	326
13.8	RAID (Redundant Array of Independent Disks)	328
13.8.1	RAID 0: Striping (sin redundancia)	328
13.8.2	RAID 1: Mirroring (espejo completo)	329
13.8.3	RAID 5: Striping con Paridad Distribuida	330
13.8.4	Comparación de RAID	333
13.9	Buffering y Caching	333
13.9.1	Buffering	334
13.9.2	Caching	336
13.9.3	Spooling	337
13.9.4	Ejemplo - Print spooler:	338
13.10	Caso de Estudio: I/O en Linux	338
13.10.1	Arquitectura de I/O en Linux	338
13.10.2	Archivos Especiales en /dev	339
13.10.3	Schedulers de I/O en Linux	340
13.10.4	ioctl: Operaciones Específicas de Dispositivo	340
13.11	10. Síntesis y Puntos Clave	341
13.11.1	Conceptos Fundamentales	341
13.11.2	Estructura y Rendimiento de Discos:	342
13.11.3	Scheduling de Disco	342
13.11.4	RAID	342
13.11.5	Software de I/O	343
13.11.6	Errores Comunes a Evitar	343
13.11.7	Tips para Ejercicios Numéricos	344
13.12	12. Conexiones con Otros Capítulos	344
13.13	13. Ejercicios Adicionales (Práctica)	346
13.13.1	Ejercicio 1: Técnicas de I/O	346
13.13.2	Ejercicio 2: Tiempos de Acceso	346
13.13.3	Ejercicio 3: RAID Capacity	346
13.13.4	Ejercicio 4: Scheduling Avanzado	347
14	Recursos Adicionales	349
14.1	Bibliografía Recomendada	349
14.2	Sobre este Libro	349
15	Glosario de Términos	351
15.1	A	351
15.2	B	351
15.3	C	352
15.4	D	352

15.5 F	352
15.6 I	353
15.7 K	353
15.8 L	353
15.9 M	354
15.10O	354
15.11P	354
15.12Q	355
15.13R	355
15.14S	355
15.15T	356
15.16U	356
15.17V	357
15.18W	357
15.19Z	357
15.20Referencias	357

1 Prefacio

Bienvenido a **Introducción a los Sistemas Operativos**, un material didáctico diseñado específicamente para estudiantes de Ingeniería que enfrentan una de las materias más desafiantes y fascinantes de su carrera.

1.1. ¿Qué vas a encontrar en este libro?

Este no es otro manual básico ni un resumen de diapositivas. Es un material con un enfoque práctico que busca conectar la teoría rigurosa con la implementación real. A lo largo de estos capítulos, los conceptos se presentan desde lo concreto hacia lo abstracto, porque siempre es más fácil entender una abstracción cuando primero viste cómo funciona en la práctica.

El libro combina rigor académico sin caer en complejidad innecesaria. Cada definición teórica viene acompañada de ejemplos reales, y las conexiones entre teoría y práctica están explícitamente señaladas. El enfoque central es no asumir que “ya lo sabés” ni que “es obvio”: estos temas son densos y merecen explicaciones claras.

1.2. Código real, no pseudocódigo

Vas a encontrar código C funcional y comentado en detalle. No son pseudocódigos ni ejemplos simplificados que “casi funcionan”. Son implementaciones reales que podés compilar, ejecutar y experimentar. Las llamadas al sistema (*syscalls*) de Unix/Linux están explicadas línea por línea.

1.3. Estructura progresiva

Cada capítulo se construye sobre los anteriores y prepara el terreno para los siguientes. Al final de cada tema hay una síntesis que te ayuda a conectar lo que acabás de aprender con lo que viene. La idea es que desarrolles una visión integral del sistema operativo como un todo coherente, no como una colección de temas inconexos.

1.4. Preparación para evaluaciones

El libro incluye ejercicios tipo parcial resueltos paso a paso, con casos de estudio basados en evaluaciones reales. Los errores más frecuentes están marcados explícitamente para que puedas anticiparlos.

1.5. Metodología de estudio sugerida

Cada capítulo está estructurado de forma deliberada para maximizar el aprendizaje. El flujo recomendado es el siguiente:

1. **Leé los objetivos** al inicio de cada capítulo para saber exactamente qué vas a aprender y por qué es importante.
2. **Entrá al contexto**: entender por qué existe un tema y qué problema resuelve es fundamental para que los detalles técnicos tengan sentido.
3. **Dominá los conceptos** con la base teórica sólida que se presenta en cada sección. No saltes esta parte pensando que “ya después lo entendés con el código”. La teoría te da el marco conceptual que hace que el código sea comprensible.
4. **Analizá la técnica**: cómo se implementa realmente lo que acabás de aprender en teoría. Aquí es donde los conceptos abstractos se conectan con decisiones concretas de implementación.
5. **Programá y experimentá** con el código de ejemplo. No te limites a leerlo: modificalo, rompeló, arreglalo. Esa experimentación activa es donde realmente se consolida el aprendizaje.
6. **Practicá con casos reales** resolviendo los ejercicios tipo parcial.
7. **Hacé la síntesis final** para integrar todo con el panorama completo del sistema operativo.

1.6. Código fuente y contacto

El código fuente de todos los ejemplos, las correcciones y las actualizaciones del libro están disponibles en el repositorio público:

```
1 github.com/dperalta86/Libro-Sistemas-Operativos
```

Para reportar erratas o sugerencias, abrí un *issue* en GitHub. También podés escribirme directamente a hola@dperalta.com.ar o visitar www.dperalta.com.ar.

2 Mensaje a los Estudiantes

Quiero compartirles algo antes de que empiecen: al momento de escribir estas páginas, todavía sigo siendo estudiante. Sigo aprendiendo, cometiendo errores y descubriendo nuevas formas de entender estos temas.

Este material recopila y organiza los conceptos fundamentales de Sistemas Operativos con un enfoque tanto teórico como práctico. Mi intención no es reemplazar las clases ni los apuntes, sino ofrecer una guía clara que pueda servir de apoyo al momento de estudiar, repasar o preparar exámenes.

Por eso, más que un “manual cerrado”, este libro es una invitación a investigar, conocer y aprender. Si te resulta útil para aclarar dudas, ordenar ideas o ganar confianza en la materia, entonces habrá cumplido su propósito.

¡Muchos éxitos en la cursada!

Daniel Isaías Peralta

3 Repaso Arquitectura de Computadores

3.1. Introducción y Contexto

3.1.1. ¿Por qué necesitamos entender la arquitectura?

Imaginá que querés entender cómo funciona un auto. Podrías aprender a manejarlo sin saber nada del motor, pero si quieres ser mecánico, necesitas entender pistones, válvulas, y transmisión. Con los sistemas operativos pasa lo mismo. Podés usar una computadora sin entender qué hay “debajo del capó”, pero para diseñar, optimizar o debuggear un SO, necesitas entender el hardware que administra. Esta comprensión no es opcional: es el fundamento sobre el cual se construyen todos los conceptos posteriores.

3.1.2. Los problemas que resuelve esta arquitectura

Todo sistema operativo moderno enfrenta tres desafíos fundamentales que la arquitectura de hardware debe resolver. Veamos cada uno:

El primer desafío es la comunicación entre software y hardware. El procesador solo entiende instrucciones binarias, mientras que los dispositivos tienen interfaces completamente diferentes. Se necesita un “traductor” capaz de convertir las intenciones de alto nivel de un programa en señales eléctricas específicas que cada componente pueda comprender.

El segundo desafío es la protección del sistema. Sin mecanismos de seguridad, un programa con errores podría crashear toda la máquina, o peor aún, múltiples programas competirían por los mismos recursos sin ningún tipo de arbitraje. La arquitectura debe proporcionar “niveles de privilegio” que limiten lo que cada programa puede hacer.

El tercer desafío es el manejo de eventos impredecibles. El usuario puede presionar una tecla en cualquier momento, los datos pueden llegar por la red de forma asincrónica, un proceso puede solicitar un archivo mientras otro está escribiendo en disco. El sistema necesita un mecanismo para “interrumpir” la ejecución normal y atender estos eventos de manera ordenada y eficiente.

3.2. Conceptos Fundamentales

3.2.1. Arquitectura de von Neumann

La arquitectura de von Neumann es el modelo fundamental sobre el cual se construyen prácticamente todas las computadoras modernas. Aunque han surgido variaciones y optimizaciones, los principios básicos permanecen intactos desde su concepción en la década de 1940.

La característica distintiva de la arquitectura de von Neumann es que *programa y datos comparten el mismo espacio de memoria*. Esto permite que los programas sean tratados como datos, habilitando conceptos como compiladores, intérpretes y sistemas operativos que pueden cargar y ejecutar otros programas dinámicamente.

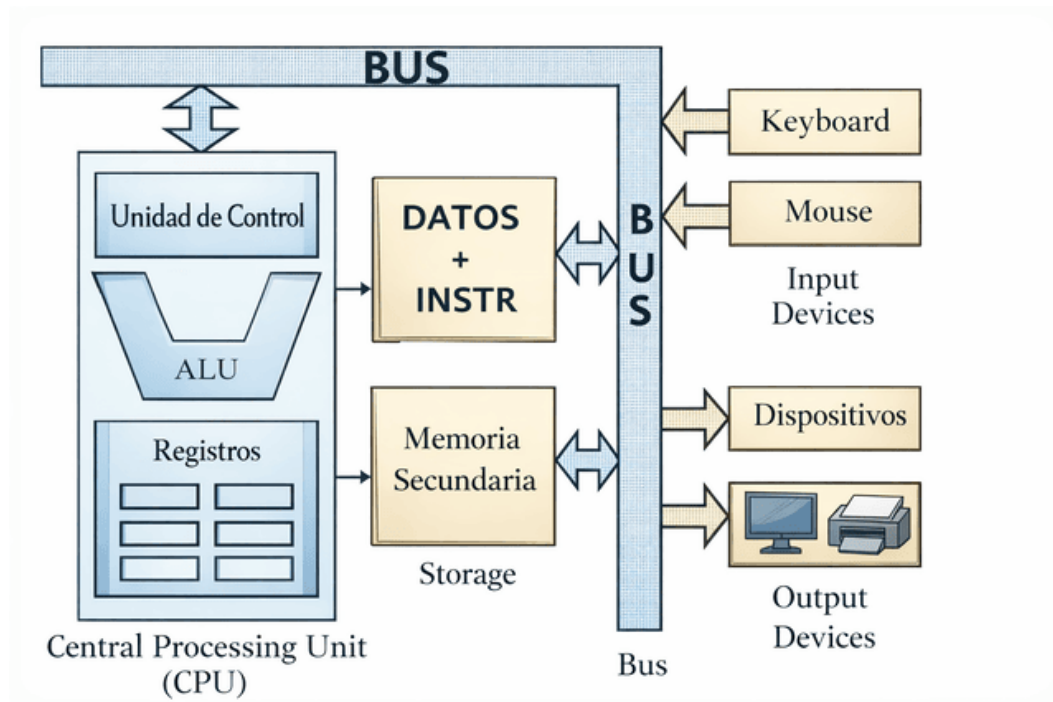


Diagrama de la arquitectura básica de una computadora, basado en el modelo de von Neumann, donde se muestran los principales componentes del sistema y su interconexión: CPU, memoria principal y dispositivos de entrada/salida.

Esta arquitectura se compone de cuatro subsistemas principales que trabajan en conjunto. La **CPU** (Unidad Central de Procesamiento) es el cerebro del sistema, conteniendo la Unidad de Control que decodifica y ejecuta instrucciones, la Unidad Aritmético-Lógica que realiza operaciones matemáticas y lógicas, y un conjunto de registros que proporcionan almacenamiento ultrarrápido dentro del procesador mismo.

La **Memoria Principal** (RAM) almacena tanto programas como datos de forma temporal. Su característica de acceso directo y aleatorio significa que cualquier ubicación puede ser accedida en tiempo constante, pero su naturaleza volátil implica que todo su contenido se pierde al apagar el sistema.

El **Almacenamiento Secundario** complementa a la RAM proporcionando persistencia. Los discos duros, SSD y otros medios de almacenamiento son significativamente más lentos que la RAM, pero mantienen la información incluso sin energía eléctrica y ofrecen capacidades mucho mayores. Los **Dispositivos de E/S** son las interfaces que permiten al sistema interactuar con el mundo exterior: teclado, mouse, pantalla, tarjetas de red, y una variedad de periféricos especializados.

3.3. Buses de Comunicación

Los componentes de la arquitectura de von Neumann no pueden funcionar de forma aislada. Los **buses** son las “autopistas” que permiten la comunicación entre CPU, memoria y dispositivos. Sin ellos, tendríamos componentes poderosos pero incapaces de colaborar.

El sistema de buses se divide en tres tipos especializados, cada uno con una función específica:

El **Bus de Datos** transporta la información real entre componentes. Su ancho, medido en bits, determina cuánta información puede transferirse simultáneamente. Un bus de 64 bits, por ejemplo, puede mover 8 bytes en cada ciclo de reloj, lo que impacta directamente en el rendimiento del sistema.

El **Bus de Direcciones** especifica la ubicación de memoria que se desea acceder. Su ancho determina el espacio máximo de direcciones que el sistema puede manejar. Con 32 bits podemos direccionar hasta 4GiB de memoria, mientras que con 64 bits ese límite se extiende hasta 16 exabytes, una cantidad astronómica para estándares actuales.

El **Bus de Control** coordina todas las operaciones mediante señales especializadas. Señales como READ, WRITE, IRQ y RESET sincronizan las transferencias entre componentes y aseguran que cada operación se complete correctamente antes de iniciar la siguiente.

3.4. Registros del Procesador

Los registros son la forma más rápida de memoria disponible en un sistema de cómputo. Están físicamente integrados en el chip del procesador, lo que los hace extremadamente veloces pero también extremadamente caros. Por esta razón, su número es limitado y cada uno tiene propósitos específicos.

La jerarquía de velocidad de memoria va desde los registros (1 ciclo de CPU) hasta el almacenamiento secundario (millones de ciclos). Esta diferencia de velocidad determina muchas de las decisiones de diseño de un sistema operativo.

Los registros se dividen en dos categorías principales según su nivel de privilegio: Los **Registros Visibles al Usuario** pueden ser utilizados por programas en modo usuario. Entre ellos encontramos los registros de propósito general como EAX, EBX, ECX y EDX en arquitecturas x86-32, o sus equivalentes de 64 bits RAX, RBX, RCX y RDX. Estos registros son la memoria de trabajo del procesador, almacenando valores temporales durante cálculos y operaciones.

Los registros de índice ESI (Source Index) y EDI (Destination Index) están optimizados para operaciones con strings y arrays, facilitando copias y comparaciones de bloques de memoria. El registro ESP/RSP actúa como puntero de stack, apuntando siempre al tope de la pila actual, mientras que EBP/RBP sirve como frame pointer, facilitando el acceso a parámetros y variables locales de funciones.

Los **Registros Privilegiados** solo pueden ser accedidos en modo kernel. Los registros de control CR0 a CR4 configuran características fundamentales del procesador. CR0 controla modos de operación básicos, CR2 almacena la dirección que causó el último page fault (crucial para memoria virtual), CR3 contiene el directorio de páginas actual para la MMU, y CR4 habilita extensiones modernas del procesador. Los registros de segmento en modo kernel (CS, DS, SS) son críticos para la protección de memoria, especificando qué regiones de memoria puede acceder cada proceso y con qué privilegios.

3.5. Program Status Word (PSW)

El **PSW**, también conocido como registro FLAGS, es un registro especial que contiene información sobre el estado actual del procesador. Cada bit en este registro tiene un significado específico y afecta cómo se ejecutan las instrucciones subsiguientes.

Los flags de condición reflejan el resultado de operaciones aritméticas y lógicas. El Zero Flag (ZF) se activa cuando una operación produce resultado cero, el Carry Flag (CF) indica acarreo en operaciones aritméticas, el Sign Flag (SF) señala si el resultado es negativo, y el Overflow Flag (OF) detecta desbordamientos en aritmética con signo.

Los flags de control modifican el comportamiento del procesador. El Interrupt Enable Flag (IF) determina si las interrupciones enmascarables están habilitadas, el Direction Flag (DF) controla la dirección de operaciones con strings, y el Trap Flag (TF) activa el modo single-step para debugging. Los flags de sistema controlan aspectos avanzados de la ejecución. El I/O Privilege Level (IOPL) especifica el nivel de privilegio necesario para operaciones de E/S, el Nested Task Flag (NT) indica si estamos en una tarea anidada, y el Resume Flag (RF) ayuda en el control de debugging.

Program Status Word (PSW) – Registro de 32 bits



Propósito: Almacena el estado actual del procesador y flags de control.

Tamaño: 32 bits (en arquitecturas x86).

Representación Binaria (bits 31-0)

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	0	0	0	0	0	0	0	0	0	0	0	ID	VP	VF	XC	RF

Condición (Status)

Control

Sistema

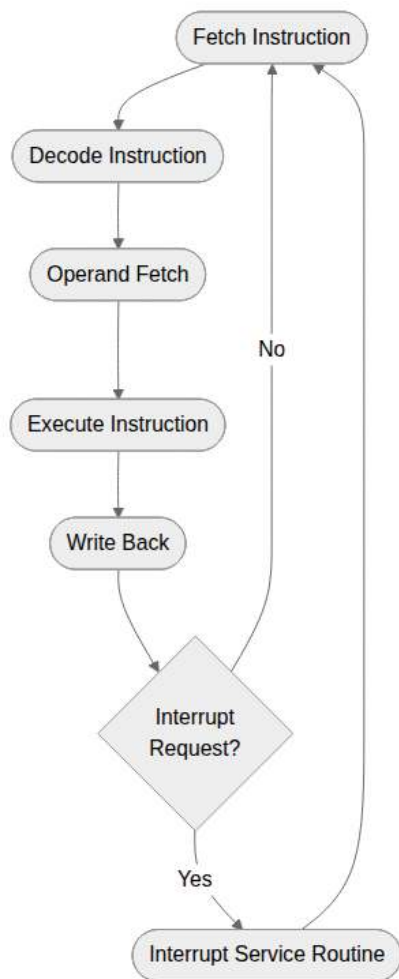
Reservado / Siempre 0 o 1

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	0	NT	IO	PL	OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

Descripción de Flags

Bit(s)	Flag	Nombre	Descripción	Tipo
0	CF	Carry Flag	Indica acarreo en operaciones aritméticas	Condición
2	PF	Parity Flag	Paridad del resultado (par/impar)	Condición
4	AF	Auxiliary Flag	Acarreo auxiliar (BCD)	Condición
6	ZF	Zero Flag	Se activa si el resultado es cero	Condición
7	SF	Sign Flag	Indica si el resultado es negativo	Condición
8	TF	Trap Flag	Modo single-step para debugging	Control
9	IF	Interrupt Flag	Habilita/deshabilita interrupciones	Control
10	DF	Direction Flag	Dirección para operaciones de string	Control
11	OF	Overflow Flag	Desbordamiento en aritmética con signo	Condición
12-13	IOPL	I/O Privilege Level	Nivel de privilegio para operaciones E/S	Sistema
14	NT	Nested Task Flag	Indica tarea anidada	Sistema
16	RF	Resume Flag	Control de debugging	Sistema

3.5.1. Ciclo de Instrucción



Fase 1: FETCH

PC apunta a próxima instrucción → se carga en IR → PC se incrementa automáticamente.

Fase 2: DECODE

Unidad de Control interpreta la instrucción → determina operación y operandos → prepara rutas de datos.

Fase 3: OPERAND FETCH

Si la instrucción requiere datos: dirección en bus → dato desde memoria → carga en registro temporal/ALU.

Fase 4: EXECUTE

ALU ejecuta la operación → actualización de registros y flags → resultados a memoria si aplica.

Fase 5: WRITE BACK \ Resultados transferidos a registro destino → PSW actualizado → CPU lista para nuevo ciclo. \[2mm]

Fase 0: CHECK INTERRUPT \ Se verifica interrupción → si hay ISR: guarda contexto, salta a rutina de servicio → al terminar, retorno al ciclo normal.

3.6. Sistema de Interrupciones

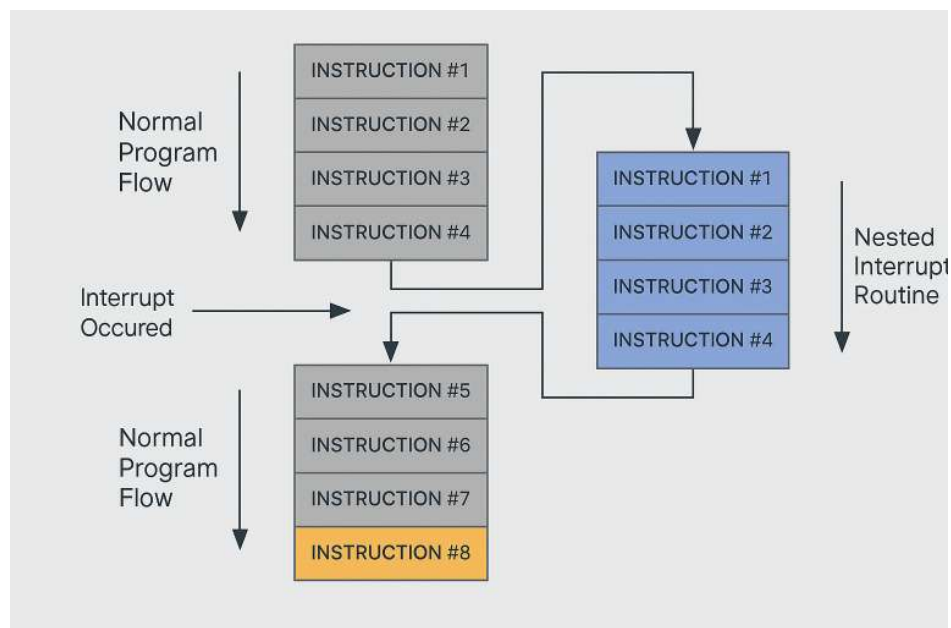
Las interrupciones son posiblemente el mecanismo de hardware más importante para un sistema operativo. Sin ellas, el SO no podría mantener control sobre el sistema ni implementar multitasking efectivo.

Definición: Una interrupción es un mecanismo hardware que permite detener temporalmente la ejecución normal del procesador para atender un evento urgente, sea este generado por hardware externo, por software, o por condiciones excepcionales durante la ejecución.

Existen tres categorías principales de interrupciones, cada una con características y propósitos distintos. Las interrupciones de hardware son generadas por dispositivos externos al procesador, como el teclado cuando se presiona una tecla, el timer del sistema que marca intervalos regulares, o la tarjeta de red al recibir paquetes. Estas interrupciones son asincrónicas: pueden ocurrir en cualquier momento, independientemente de lo que esté ejecutando el procesador.

Las interrupciones de software son generadas explícitamente por instrucciones del programa. Las system calls, por ejemplo, son interrupciones de software que solicitan servicios al sistema operativo. A diferencia de las interrupciones de hardware, estas son sincrónicas: ocurren en puntos predecibles del programa.

Las excepciones son eventos síncronos generados por condiciones de error durante la ejecución. Una división por cero, un acceso a memoria inválida, o una instrucción ilegal generan excepciones que deben ser manejadas inmediatamente.



3.6.1. Interrupciones Enmascarables (Maskable Interrupts)

Pueden ser temporalmente deshabilitadas por software mediante el control del bit IF (Interrupt Flag) en el registro de estado del procesador (PSW/EFLAGS).

Control:

- **CLI** (Clear Interrupt Flag): Deshabilita interrupciones
- **STI** (Set Interrupt Flag): Habilita interrupciones

Ejemplos:

- Timer del sistema (IRQ0 - genera multitasking preemptivo)
- Teclado (IRQ1) y mouse (IRQ12)
- Tarjeta de red (IRQ variable)
- Controladores de disco (IRQ14, IRQ15)
- Puertos serie (IRQ3, IRQ4)

```
1 // Pseudocódigo de sección crítica
2 cli();           // Deshabilitar interrupciones
3 // ... código crítico que no debe ser interrumpido ...
4 // Ejemplo: manipulación de estructuras del kernel
5 update_process_table();
6 modify_memory_mapping();
7 sti();           // Rehabilitar interrupciones
```

Deshabilitar interrupciones por períodos prolongados puede causar pérdida de eventos y afectar la responsividad del sistema. Las secciones críticas deben ser lo más breves posible.

3.6.2. Interrupciones No Enmascarables (NMI - Non-Maskable Interrupts)

Algunas situaciones son tan críticas que no pueden esperar. Las interrupciones no enmascarables tienen prioridad absoluta y no pueden ser deshabilitadas por software, ni siquiera por el kernel. Estas interrupciones se reservan para eventos que requieren atención inmediata del sistema.

Los eventos que generan NMI típicamente indican condiciones catastróficas: errores de paridad en memoria RAM que podrían corromper datos, fallas críticas de hardware como sobrecalentamiento extremo, watchdog timers que detectan que el sistema está completamente colgado, errores en el bus del sistema, o fallos de alimentación eléctrica inminentes.

Las NMI pueden interrumpir incluso al kernel en medio de secciones críticas. Esto significa que el handler de NMI debe ser extremadamente cuidadoso y no puede asumir que las estructuras del kernel están en estado consistente.

3.7. Jerarquía de Prioridades

```
1 NMI (Prioridad 0 - más alta)↓
2
3 Interrupciones de Hardware IRQ0-IRQ15 (Prioridad por IRQ)↓
4
```

```
5 Excepciones del Procesador (Divide by zero, Page Fault)↓
6
7 Interrupciones de Software (INT 0x80, syscalls)↓
8
9 Trampas y Breakpoints (más baja)
```

3.8. IRQ (Interrupt Request Lines)

3.8.1. Controlador PIC (Programmable Interrupt Controller)

PIC Primario (IRQ0-7):

```
1 IRQ 0 → Timer del sistema (8253/8254)
2 IRQ 1 → Teclado PS/2
3 IRQ 2 → Cascade a PIC secundario
4 IRQ 3 → Puerto serie COM2/COM4
5 IRQ 4 → Puerto serie COM1/COM3
6 IRQ 5 → Tarjeta de sonido/LPT2
7 IRQ 6 → Controlador de disquete
8 IRQ 7 → Puerto paralelo LPT1
```

PIC Secundario (IRQ8-15) - conectado via IRQ2:

```
1 IRQ 8 → Reloj de tiempo real (CMOS)
2 IRQ 9 → Redireccionado desde IRQ2
3 IRQ 10 → Libre (tarjetas de red)
4 IRQ 11 → Libre (USB, tarjetas PCI)
5 IRQ 12 → Mouse PS/2
6 IRQ 13 → Coprocesador matemático
7 IRQ 14 → Controlador IDE primario
8 IRQ 15 → Controlador IDE secundario
```

En sistemas modernos con APIC, el número de IRQs disponibles se extiende hasta 256, y múltiples procesadores pueden recibir interrupciones simultáneamente. Sin embargo, el modelo conceptual permanece similar.

3.9. Interrupt Handlers (Manejadores de Interrupción)

Cada tipo de interrupción debe tener asociado un handler: una rutina de código que sabe cómo responder a ese evento específico. Los handlers son las piezas de código más críticas del sistema operativo, y deben seguir reglas estrictas.

3.9.1. Estructura de un Handler

Un handler de interrupción debe ser eficiente y predecible. La estructura típica sigue este patrón:

```
1 // Prototipo genérico de handler
2 void interrupt_handler(int irq_number, struct pt_regs *regs) {
3     // 1. Guardar contexto (automático en entrada)
4     // 2. Identificar fuente de interrupción
5     // 3. Ejecutar lógica específica
6     // 4. Enviar EOI (End of Interrupt)
7     // 5. Restaurar contexto (automático en salida)
8 }
9
10 // Ejemplo: Handler del timer del sistema
11 void timer_interrupt_handler(int irq, struct pt_regs *regs) {
12     // Incrementar jiffies (contador global de tiempo)
13     jiffies++;
14
15     // Actualizar estadísticas del proceso actual
16     current->utime++;
17
18     // Verificar si el quantum del proceso expiró
19     if (--current->time_slice <= 0) {
20         current->need_resched = 1; // Marcar para replanificación
21     }
22
23     // Ejecutar timers pendientes
24     run_timer_list();
25
26     // EOI al controlador de interrupciones
27     send_eoi(IRQ_TIMER);
28 }
```

Este ejemplo muestra el handler del timer del sistema, una de las interrupciones más importantes. Cada “tick” del timer permite al sistema operativo mantener noción del tiempo transcurrido y decidir si es momento de cambiar al siguiente proceso.

3.9.2. Interrupciones Anidadas

Una característica avanzada de muchos sistemas es la capacidad de permitir que una interrupción de mayor prioridad interrumpa el procesamiento de una de menor prioridad. Esto se conoce como **anidamiento de interrupciones**.

```
1 // Ejemplo de manejo de interrupciones anidadas
2 void high_priority_handler(int irq, struct pt_regs *regs) {
3     // Esta interrupción puede ser interrumpida por NMI
4     disable_interrupts(); // Opcional: crear sección crítica
5
6     // Procesar evento crítico
7     handle_critical_event();
8
9     enable_interrupts(); // Permitir interrupciones anidadas
```

```
10
11     // Continuar con procesamiento menos crítico
12     handle_normal_processing();
13 }
```

El anidamiento ofrece ventajas significativas: mejor tiempo de respuesta para eventos críticos, priorización automática, y maximización del throughput del sistema. Sin embargo, introduce desafíos de complejidad en la gestión del stack, posibles deadlocks si no se maneja correctamente, y riesgo de overflow del stack si las interrupciones se anidan demasiado profundamente.

```
1 // Control de profundidad de anidamiento
2 #define MAX_NESTED_INTERRUPTS 8
3 static int nested_count = 0;
4
5 void generic_handler(int irq, struct pt_regs *regs) {
6     if (++nested_count > MAX_NESTED_INTERRUPTS) {
7         panic("Interrupt nesting overflow");
8     }
9
10    // ... lógica del handler ...
11
12    --nested_count;
13 }
```

3.10. Estados de Interrupciones

El sistema operativo necesita control fino sobre cuándo las interrupciones pueden ocurrir. Esto se logra mediante mecanismos que permiten habilitar y deshabilitar interrupciones a diferentes niveles de granularidad.

3.10.1. Disable/Enable a Nivel de Sistema

El control global de interrupciones se realiza mediante instrucciones que afectan el flag IF del procesador:

```
1 // Macros comunes en kernels Unix
2 #define local_irq_disable()    asm volatile("cli" ::: "memory")
3 #define local_irq_enable()    asm volatile("sti" ::: "memory")
4
5 // Con salvado y restauración de estado
6 unsigned long flags;
7 local_irq_save(flags);        // Guarda estado actual y deshabilita
8 // ... sección crítica ...
9 local_irq_restore(flags);     // Restaura estado previo
```

La variante con salvado y restauración es preferible porque no asume que las interrupciones estaban habilitadas antes de entrar a la sección crítica.

3.10.2. Disable/Enable por IRQ Específico

En ocasiones es necesario deshabilitar solo una fuente de interrupciones específica, sin afectar a las demás:

```
1 // Deshabilitar IRQ específico en PIC
2 void disable_irq(int irq) {
3     uint16_t port;
4     uint8_t value;
5
6     if (irq < 8) {
7         port = 0x21;    // PIC primario
8     } else {
9         port = 0xA1;    // PIC secundario
10        irq -= 8;
11    }
12
13    value = inb(port) | (1 << irq);
14    outb(port, value);
15 }
```

Este nivel de control permite, por ejemplo, deshabilitar el IRQ del teclado durante una actualización crítica de las estructuras de entrada, sin afectar el timer del sistema o la red.

El sistema de interrupciones es fundamental para implementar:

- **Multitasking preemptivo:** Timer interrupts permiten cambios de contexto forzados
- **Manejo eficiente de E/S:** Los dispositivos notifican al CPU cuando tienen datos
- **Gestión de memoria virtual:** Page faults son excepciones que permiten demand paging
- **Comunicación inter-procesos:** Señales se implementan sobre el sistema de interrupciones
- **Detección de errores:** Excepciones capturan condiciones anómalas inmediatamente

3.10.3. Modos de Operación del Procesador

La arquitectura moderna del procesador distingue entre dos modos de operación fundamentales que son la base de toda la seguridad y estabilidad del sistema.

En **Modo Kernel** (también llamado Supervisor o modo privilegiado), el procesador tiene acceso sin restricciones. Puede ejecutar cualquier instrucción, modificar registros críticos del sistema, acceder directamente a cualquier dispositivo hardware, y reconfigurar aspectos fundamentales de la operación del procesador. Este modo es extremadamente poderoso, pero también extremada-

mente peligroso: un error en modo kernel puede crashear todo el sistema. Por esta razón, solo el código del sistema operativo ejecuta en este modo.

En **Modo Usuario** (user mode), el procesador opera con un subconjunto restringido de instrucciones. No puede acceder directamente a hardware, no puede modificar configuraciones críticas del sistema, y no puede acceder a memoria que no le pertenece. Todas las aplicaciones de usuario ejecutan en este modo, lo que las aísla tanto del hardware como entre ellas.

Esta separación de modos es la base de la seguridad y estabilidad del sistema. Sin ella, cualquier programa podría crashear la máquina o acceder a datos privados de otros procesos. Es la razón por la que un navegador web con errores puede cerrarse sin afectar al resto del sistema.

3.10.4. Cambio de Procesos

Cuándo por algún motivo existe un cambio de proceso en ejecución (los motivos se verán en detalle mas adelante), se realizan varias tareas que son “transparentes” para el usuario final.

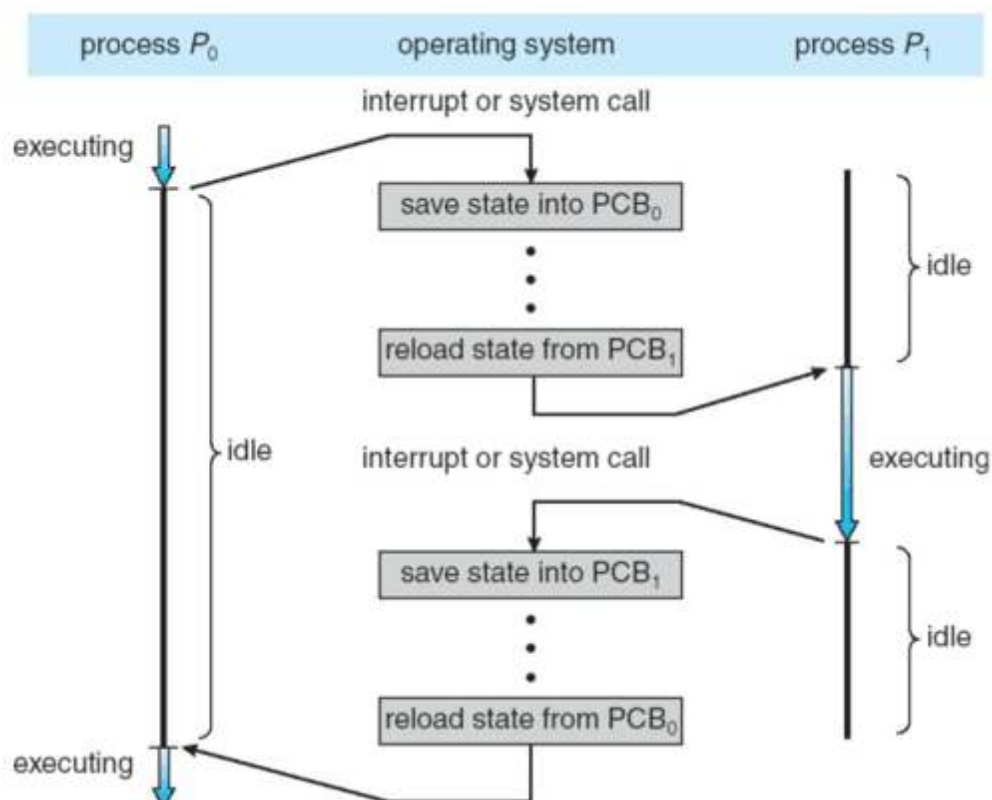


Diagrama del cambio de contexto entre procesos, donde el sistema operativo guarda el estado del proceso en ejecución en su PCB y restaura el estado de otro proceso para continuar su ejecución.

Cambio de procesos en sistemas Unix

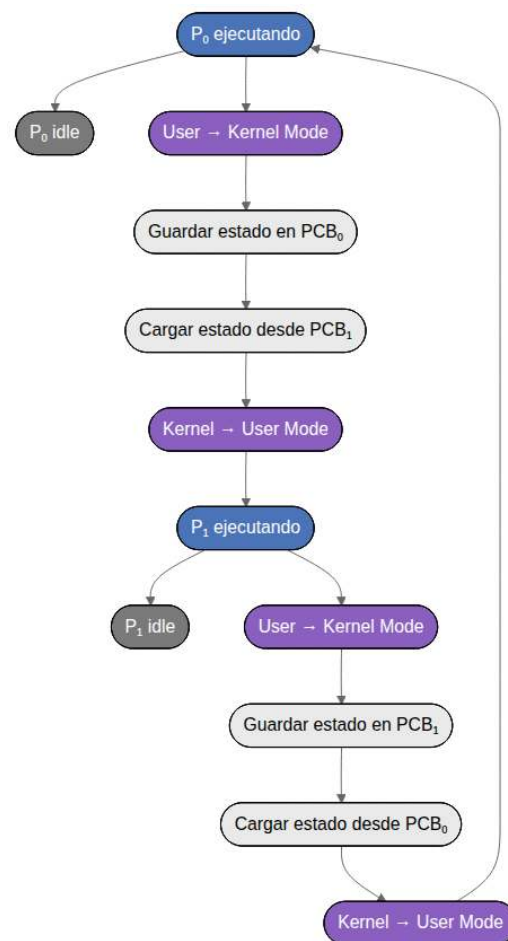
Cuando el sistema operativo interrumpe la ejecución de un proceso y decide continuar con otro, se produce un cambio de contexto. Este mecanismo implica la intervención tanto del hardware como del software (kernel).

Rol del Hardware:

El hardware se encarga de detectar la interrupción (timer, E/S o syscall), cambiar automáticamente a modo kernel y guardar información mínima como el contador de programa y los flags. Luego transfiere el control al vector de interrupción correspondiente.

Rol del Software (Kernel):

A partir de allí, el software del kernel toma el control: guarda el estado completo del proceso en su PCB, actualiza las estructuras del planificador y contabiliza uso de CPU y señales pendientes. El scheduler selecciona el nuevo proceso a ejecutar y el kernel restaura su estado desde el PCB.



Finalización del Proceso:

Antes de retornar al usuario, se actualiza el espacio de direcciones (MMU/TLB) y se realiza el cambio de modo kernel → usuario.

Consideraciones de Performance:

Este procedimiento asegura la correcta continuidad de los procesos, pero introduce un overhead inevitable: acceso a memoria para guardar/restaurar registros, invalidación de TLB y caches, y lógica extra del planificador. Por este motivo, sistemas I/O-bound suelen generar más cambios de contexto que los CPU-bound, y su rendimiento depende en gran medida de la eficiencia del scheduler y de las optimizaciones del hardware.

Fases del Context Switch en Sistemas Unix

Fase Inicial - Ejecución Normal:

0. P_0 ejecuta en modo usuario → Estado normal usando su espacio de direcciones y time slice vigente.

Detección del Evento [Hardware]:

- 1.** Se dispara evento → Timer interrupt, IRQ de E/S o trap por syscall/exception
- 2.** Cambio automático a modo kernel → CPU guarda PC y PSW/EFLAGS, cambia a stack de kernel
- 3.** Búsqueda en vector/IDT → Hardware obtiene dirección del handler apropiado

Guardar Estado [Software]:

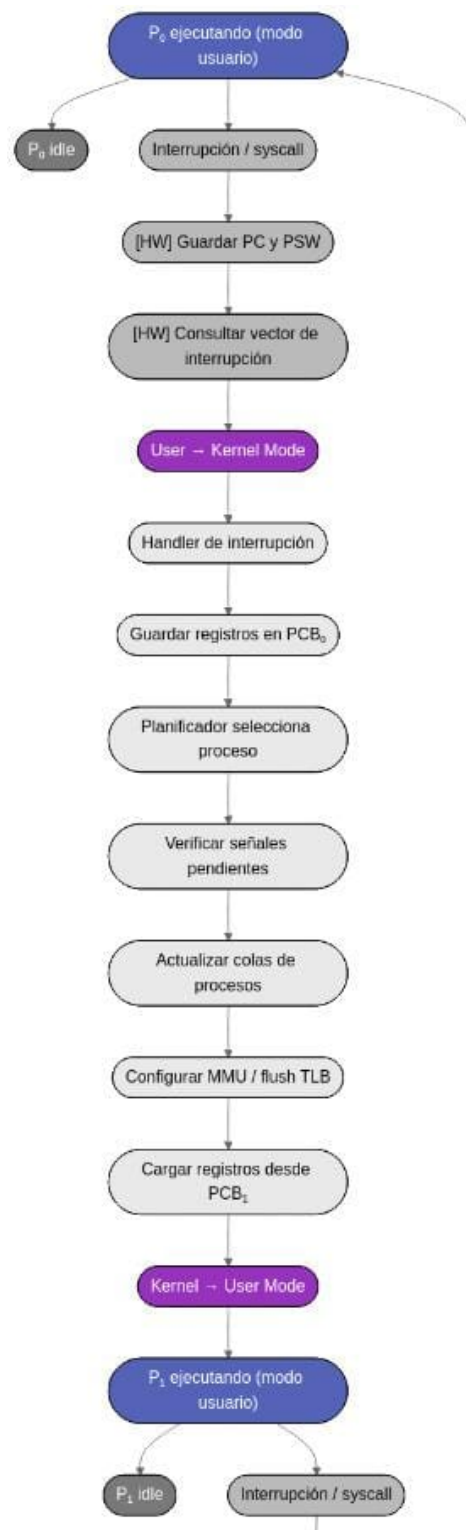
- 4.** Prólogo del handler → Kernel salva registros volátiles, arma frame de interrupción
- 5.** Decisión de reschedule → Marca need_resched según tipo de evento
- 6.** Guardar contexto de P_0 → Registros GPR, SP, FP, TLS; FPU/SIMD en forma lazy
- 7.** Contabilidad y señales → Actualiza uso de CPU, entrega señales pendientes

Selección y Carga [Software]:

- 8.** Actualizar colas y estado → Mueve P_0 de running → ready/blocked; ajusta prioridad
- 9.** Scheduler elige P_1 → Aplica política de planificación (CFS, prioridades, afinidad CPU)
- 10.** Preparar espacio de P_1 → Carga CR3/tablas de páginas; puede requerir TLB flush
- 11.** Recargar contexto de P_1 → Restaura registros, SP, FP, TLS, stack de usuario

Retorno a Ejecución [Hardware]:

- 12.** Epílogo y retorno → Prepara iret/sysret, re-habilita interrupciones
- 13.** Kernel → modo usuario → CPU restaura PSW/EFLAGS y PC, salta a P_1



14. P_1 ejecuta en modo usuario → Continúa hasta el próximo evento

Overhead: *El costo inevitable del cambio de contexto*

Cada cambio de contexto tiene un precio en rendimiento que no podemos evitar. Primero está el acceso a memoria para guardar y restaurar todos los registros del procesador, una operación que, aunque rápida, no es instantánea. Pero el impacto va más allá: cuando cambiamos de proceso, la TLB (Translation Lookaside Buffer) y las cachés del procesador quedan invalidadas porque contienen datos del proceso anterior, forzando al nuevo proceso a "cargar" estas estructuras nuevamente desde cero.

A esto se suma la lógica del planificador mismo: decidir qué proceso ejecutar siguiente, actualizar estadísticas, verificar prioridades. Todo esto consume ciclos de CPU que podrían estar haciendo trabajo útil. Particularmente crítico es entender que los sistemas I/O-bound (que pasan mucho tiempo esperando entrada/salida) generan significativamente más cambios de contexto que los sistemas CPU-bound (que hacen cálculos intensivos). Esto es porque cada operación de I/O típicamente bloquea el proceso, forzando un context switch. Un servidor web manejando miles de conexiones puede estar cambiando de contexto miles de veces por segundo.

3.10.5. Resumen y Hoja de Ruta

En este libro recorreremos el sistema operativo desde sus cimientos hasta sus aspectos más sofisticados, construyendo conocimiento de forma incremental y conectada.

Los primeros capítulos establecen los **fundamentos**. Comenzamos con arquitectura de computadores porque para entender cómo funciona un sistema operativo necesitás entender el hardware que gestiona. Después introducimos el concepto de proceso, que es la abstracción fundamental sobre la que se construye todo lo demás.

Estructura del Libro:

- Capítulos 1-2: Fundamentos - Arquitectura de computadores y concepto de proceso
- Capítulos 3-7: Concurrencia - Planificación, hilos, sincronización e interbloqueos
- Capítulos 8-9: Memoria - Gestión de memoria real y virtual
- Capítulo 10: Almacenamiento - Sistemas de archivos
- Capítulo 11: I/O - Gestión de Dispositivos

La sección de **concurrencia** es el corazón del libro. Acá estudiamos cómo el sistema operativo crea la ilusión de múltiples actividades simultáneas: cómo planifica qué proceso se ejecuta en cada momento, cómo los hilos permiten paralelismo dentro de un proceso, cómo se sincronizan accesos a

recursos compartidos, y cómo se previenen y resuelven interbloqueos. Esta sección es particularmente importante porque los problemas de concurrencia están entre los más sutiles y difíciles de la programación de sistemas.

La **gestión de memoria** empieza con lo básico: cómo se asigna y libera memoria, cómo se protegen los espacios de direcciones de diferentes procesos. Después avanzamos hacia memoria virtual, una de las abstracciones más elegantes de la computación moderna, que hace que cada proceso “vea” su propio espacio de direcciones continuo e independiente, sin importar cómo esté realmente organizada la memoria física.

El capítulo de **sistemas de archivos** muestra cómo toda la información que los procesos manipulan en memoria puede persistirse en almacenamiento permanente. Aquí conectamos muchos conceptos anteriores: procesos que acceden a archivos, memoria que se mapea desde archivos, y sincronización para accesos concurrentes al sistema de archivos.

Finalmente, la **gestión de I/O y dispositivos** cierra el círculo completando el cuarto pilar fundamental. Este capítulo integra todo lo anterior: procesos que esperan por operaciones de entrada/salida, planificación que debe considerar dispositivos lentos vs. rápidos, memoria que se usa como buffer para I/O, y la compleja interacción entre el sistema operativo y el hardware heterogéneo. Entender I/O es entender cómo el sistema operativo realmente se comunica con el mundo exterior.

Cada tema se construye deliberadamente sobre los anteriores. No podés entender memoria virtual sin entender procesos, no podés entender sincronización sin entender concurrencia, no podés entender sistemas de archivos sin entender gestión de memoria, y no podés entender I/O sin comprender cómo todos estos subsistemas interactúan. Por eso es importante seguir el orden, aunque después puedas volver a capítulos específicos para referencia o repaso.

Al final de este recorrido, vas a tener una comprensión integral de cómo funciona realmente el software más importante de tu computadora: ese conjunto de abstracciones, políticas y mecanismos que transforman metal y silicio en una máquina útil, segura y eficiente.

4 Fundamentos de los Sistemas Operativos

4.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

- Identificar los componentes principales de la arquitectura de von Neumann y su relación con el sistema operativo
- Explicar la diferencia entre modo kernel y modo usuario, y por qué es fundamental para el SO
- Describir el proceso de arranque (boot) y cómo se carga el sistema operativo
- Analizar los mecanismos de interrupciones y llamadas al sistema (syscalls)
- Relacionar la arquitectura del hardware con las funciones básicas del sistema operativo

4.2. Introducción a los Sistemas Operativos

Ahora que comprendemos la arquitectura de hardware sobre la cual operan, podemos finalmente definir con precisión qué es un sistema operativo.

Definición: Un Sistema Operativo es un conjunto de rutinas y procedimientos manuales y automáticos que permiten la operatoria en un sistema de computación. Es, fundamentalmente, un **programa de control** que media entre el hardware y los programas de aplicación.

El sistema operativo no es un programa más: es el programa que hace posible que existan todos los demás programas. Sus responsabilidades abarcan desde el momento en que se enciende la computadora hasta que se apaga, y cada milisegundo de ese tiempo está dedicado a alguna tarea crítica.

Sus principales tareas incluyen lanzar el Init Program Loader durante el arranque, gestionar todos los recursos del hardware de manera eficiente y justa, proporcionar servicios estandarizados a los programas de aplicación, actuar como intermediario entre usuarios y el hardware bruto, y mantener la seguridad e integridad de todo el sistema.

4.2.1. ¿Qué es realmente un Sistema Operativo?

Imagina que acabás de comprar una computadora nueva. La conectás, la prendés y... ¿qué esperás ver? No circuitos ni señales eléctricas, sino una interfaz que te permita ejecutar programas, crear archivos, conectarte a internet. Esa “magia” que transforma el hardware crudo en una máquina útil es el **sistema operativo**.

Pero esta descripción, aunque intuitiva, se queda corta. Un sistema operativo es fundamentalmente un *programa que administra recursos*, pero no es cualquier programa: es el programa más privilegiado de tu computadora, el único con acceso directo al hardware, y el responsable de que todos los demás programas funcionen de manera ordenada y eficiente.

Pensá en el sistema operativo como el gerente de un edificio de oficinas. No hace el trabajo de cada inquilino, pero coordina quién usa qué oficina, cuándo se pueden usar las salas de reuniones, cómo se distribuye la electricidad, y resuelve conflictos cuando dos inquilinos quieren usar el mismo recurso. Sin ese gerente, tendrías el caos: oficinas duplicadas, cortes de luz, reuniones interrumpiéndose mutuamente.

4.2.2. Los Cuatro Pilares Fundamentales

Todo sistema operativo moderno, desde el que corre en tu teléfono hasta el que controla supercomputadoras, se construye sobre cuatro pilares fundamentales. Entender estos pilares es entender la esencia de lo que hace un sistema operativo.

El primer pilar es la **gestión de procesos**. ¿Cómo puede tu computadora ejecutar el navegador, el reproductor de música y el editor de texto “al mismo tiempo” si solo tiene una CPU? La respuesta está en la capacidad del sistema operativo de crear procesos, programar su ejecución, coordinar su interacción y terminarlos cuando ya no son necesarios. Esta ilusión de simultaneidad es uno de los trucos más elegantes de la computación moderna.

Un proceso es un programa en ejecución. No es solo el código, sino también el estado completo de esa ejecución: variables, memoria asignada, archivos abiertos, posición en el código. El sistema operativo mantiene toda esta información y la restaura cada vez que le toca el turno al proceso.

El segundo pilar es la **gestión de memoria**. Con múltiples programas ejecutándose, ¿cómo se decide quién usa qué porción de memoria? ¿Qué pasa cuando se agota? ¿Cómo evitamos que un programa corrupto escriba en la memoria de otro? El sistema operativo debe distribuir este recurso crítico, protegerlo contra accesos indebidos, y optimizar su uso para que el sistema completo funcione eficientemente.

La gestión de memoria no es trivial: involucra crear abstracciones que hagan que cada programa

“piense” que tiene toda la memoria para sí mismo, mapear direcciones virtuales a físicas, decidir qué hacer cuando la memoria se agota, y mantener todo esto funcionando a velocidades que no degraden el rendimiento general del sistema.

El tercer pilar es la **gestión de almacenamiento**. Desde guardar un documento hasta instalar un programa, el sistema operativo debe organizar información en dispositivos permanentes como discos duros o SSDs. Pero no solo eso: debe crear abstracciones útiles como “archivos” y “directorios”, garantizar la integridad de los datos ante fallos de energía o errores de hardware, y proporcionar mecanismos de control de acceso para que no cualquiera pueda leer tus archivos privados.

Cuando guardás un archivo de texto de 1KiB, el sistema operativo decide en qué sectores del disco físico va a almacenarlo, actualiza las estructuras de metadatos que permiten encontrarlo después, potencialmente lo fragmenta si no hay espacio contiguo, mantiene una caché en memoria para accesos rápidos, y registra toda la operación para poder recuperarse si hay un fallo.

El cuarto pilar es la **gestión de E/S y comunicación**. Tu computadora tiene una increíble diversidad de dispositivos: teclado, mouse, pantalla, red, impresora, cámara, micrófonos. Cada uno funciona de manera diferente, con protocolos distintos y velocidades que varían por órdenes de magnitud. El sistema operativo debe manejar esta heterogeneidad y proporcionar interfaces uniformes que permitan a los programas comunicarse con estos dispositivos sin conocer los detalles de bajo nivel de cada uno.

Además, el sistema operativo facilita la comunicación entre procesos: cuando copiás texto de un programa y lo pegás en otro, cuando un servidor web recibe una petición de un navegador, o cuando dos aplicaciones intercambian datos, es el sistema operativo el que hace posible esta comunicación de manera segura y eficiente.

4.2.3. ¿Por qué es tan Complejo?

Si los pilares fundamentales suenan razonables y hasta obvios, te preguntarás: ¿por qué los sistemas operativos tienen fama de ser tan complejos? La respuesta está en los desafíos únicos que enfrentan, desafíos que raramente aparecen juntos en otros dominios de la computación.

Primero está el problema de la *conurrencia*. No solo hay múltiples actividades simultáneas, sino que estas actividades pueden interferir entre sí de formas sutiles y difíciles de predecir. Dos procesos que intentan actualizar el mismo archivo, un programa que lee mientras otro escribe, hilos que acceden a estructuras de datos compartidas: estos escenarios crean condiciones de carrera, interbloqueos y otros problemas que pueden ocurrir solo una vez en mil ejecuciones, haciendo que sean particularmente difíciles de detectar y corregir.

Los errores de concurrencia son notoriamente difíciles de reproducir y depurar. Un programa puede funcionar perfectamente en pruebas y fallar catastróficamente en producción debido a una condición de carrera que solo ocurre bajo ciertas combinaciones de timing específicas.

Luego están los *recursos limitados*. A diferencia de los algoritmos que estudiamos en cursos anteriores, donde a menudo podemos asumir memoria infinita o tiempo ilimitado, los sistemas operativos funcionan en un mundo de límites físicos duros. La CPU tiene un número finito de ciclos por segundo, la memoria tiene un tamaño máximo, el ancho de banda de red es limitado, y la batería se agota. Cada decisión del sistema operativo debe considerar estos límites y hacer compromisos (trade-offs) inteligentes.

La *heterogeneidad* es otro desafío fundamental. El sistema operativo debe funcionar con procesadores de diferentes arquitecturas, dispositivos de almacenamiento con características distintas, aplicaciones con necesidades completamente diferentes. Un servidor web tiene prioridades diferentes que un videojuego, que a su vez son diferentes de un sistema de procesamiento de transacciones financieras. El sistema operativo debe adaptarse a todos estos contextos sin favorecer injustamente a ninguno.

La *confiabilidad* es crítica porque un error en el sistema operativo puede afectar todo el sistema. Si tu navegador se cuelga, cerrás ese programa y listo. Pero si el sistema operativo falla, toda la computadora se detiene. Esta responsabilidad requiere niveles de robustez y manejo de errores que van más allá de lo que necesitan la mayoría de las aplicaciones.

La *seguridad* es un requisito transversal: el sistema operativo debe proteger datos y recursos de accesos no autorizados, prevenir que programas maliciosos comprometan el sistema, y garantizar que diferentes usuarios puedan coexistir sin interferirse mutuamente. Y todo esto debe hacerse sin impactar significativamente el rendimiento.

Finalmente está el desafío del *rendimiento*. Los usuarios esperan que sus sistemas sean rápidos, consuman poca energía, y respondan inmediatamente. Pero optimizar para velocidad puede aumentar el consumo energético, priorizar capacidad de respuesta puede reducir el throughput general, y así sucesivamente. El sistema operativo debe equilibrar constantemente estos objetivos contradictorios.

4.2.4. El Enfoque Unix/Linux

Este libro se centra en sistemas Unix/Linux, no por preferencia ideológica, sino por razones pedagógicas concretas que hacen que estos sistemas sean ideales para aprender cómo funcionan realmente los sistemas operativos.

La primera razón es la *transparencia*. A diferencia de sistemas propietarios donde el funcionamiento interno es opaco, en Unix/Linux el código fuente está disponible para estudiar. Esto significa

que cuando decimos “así funciona la planificación de procesos”, podés ir al código del kernel y ver exactamente cómo está implementado. Esta capacidad de “ver por dentro” es invaluable para el aprendizaje profundo.

Unix fue diseñado en los años 70 con una filosofía específica: “cada cosa hace una cosa bien”. Esta simplicidad conceptual, donde funciones complejas se construyen componiendo herramientas simples, hace que sea más fácil entender el sistema como un todo. Muchos sistemas modernos han abandonado esta filosofía en favor de características más complejas, pero Unix/Linux mantiene estos principios en su núcleo.

Los *estándares abiertos* son otra ventaja. POSIX (Portable Operating System Interface) define interfaces estándar que hacen que el código sea portable entre diferentes sistemas Unix-like. Esto significa que lo que aprendés sobre Linux es aplicable a BSD, macOS, y otros sistemas. Esta estandarización también facilita el aprendizaje porque podés confiar en que ciertos comportamientos son consistentes.

La *relevancia industrial* no puede ignorarse. Unix/Linux domina en servidores (más del 90 % de los servidores web corren Linux), sistemas embebidos (desde routers hasta SmartTVs), supercomputadoras (el 100 % del Top500 corre Linux), y cada vez más en desktop. Aprender Unix/Linux no es un ejercicio académico: es una habilidad directamente aplicable en la industria.

Finalmente, el *ambiente de desarrollo* en Unix/Linux es excepcional. Las herramientas para programación en C, el lenguaje en que están escritos los sistemas operativos, son nativas y poderosas. Los compiladores, debuggers, profilers, y otras herramientas que necesitás para experimentar con código de sistemas están integrados y bien documentados.

4.2.5. Principales Tareas del Sistema Operativo

El sistema operativo moderno es un software extraordinariamente complejo que debe manejar múltiples responsabilidades simultáneamente. Veamos cada una de sus áreas principales:

La **gestión de procesos** involucra crear nuevos procesos cuando un programa necesita ejecutarse, planificar cuál proceso debe usar la CPU en cada momento, y terminar procesos de manera ordenada cuando finalizan. Además, debe proporcionar mecanismos de comunicación entre procesos que los permitan colaborar, y manejar la sincronización para evitar condiciones de carrera y deadlocks.

La **gestión de memoria** requiere asignar bloques de memoria a procesos cuando la necesitan y liberarla cuando ya no la usan. Para maximizar el uso de la RAM limitada, implementa memoria virtual que permite usar disco como extensión de la RAM, y protege los espacios de memoria de cada proceso para que uno no pueda corromper accidentalmente (o maliciosamente) la memoria de otro.

La **gestión de almacenamiento** organiza los archivos en sistemas de archivos jerárquicos, controla el acceso a dispositivos de almacenamiento físico como discos y SSDs, e implementa políticas de respaldo y recuperación para proteger contra pérdida de datos.

La **gestión de E/S** debe controlar la enorme variedad de dispositivos que pueden conectarse al sistema, desde teclados y ratones hasta impresoras 3D y sensores especializados. Proporciona una interfaz uniforme que abstrae las diferencias entre dispositivos, y implementa técnicas como buffering, caching y spooling para maximizar la eficiencia.

Los aspectos de **seguridad y protección** incluyen autenticar usuarios para verificar identidades, controlar qué usuarios pueden acceder a qué recursos y con qué privilegios, y proteger procesos entre sí y proteger al sistema operativo mismo de procesos maliciosos.

Finalmente, **la interfaz de usuario** puede tomar la forma de una línea de comandos (CLI) para usuarios avanzados, interfaces gráficas (GUI) para usuarios generales, o APIs bien diseñadas para programadores que necesitan acceder a servicios del sistema.

4.2.6. System Calls (Llamadas al Sistema)

Aquí llegamos a uno de los conceptos más fundamentales en sistemas operativos: el mecanismo mediante el cual programas en modo usuario pueden solicitar servicios del kernel sin comprometer la seguridad del sistema.

Definición: Las System Calls son el mecanismo controlado mediante el cual los programas en modo usuario pueden solicitar servicios al sistema operativo, como acceso a hardware, creación de procesos, o manipulación de archivos.

Las system calls existen por razones fundamentales de protección, abstracción, eficiencia y portabilidad. Sin ellas, cada programa necesitaría incluir código específico para cada modelo de disco duro, cada tarjeta de red, cada tipo de pantalla. Además, cualquier programa podría acceder directamente al hardware de otros procesos, robando datos o causando crashes.

El *mecanismo tradicional* de system calls en arquitecturas x86 sigue estos pasos: el programa prepara los parámetros en registros o stack, ejecuta una instrucción INT (interrupt de software), la CPU cambia automáticamente a modo kernel, el kernel identifica qué syscall fue solicitada consultando una tabla, ejecuta la función correspondiente en modo privilegiado, y finalmente retorna el resultado y devuelve control al programa en modo usuario.

Mecanismo Tradicional:

- 1 1. Programa prepara parámetros
- 2 2. Ejecuta instrucción INT (interrupt)
- 3 3. CPU cambia a modo kernel
- 4 4. Kernel identifica syscall solicitada
- 5 5. Kernel ejecuta función correspondiente

```
6 6. Kernel retorna resultado
7 7. CPU vuelve a modo usuario
```

Ejemplo en Linux (x86): En Linux x86, una syscall tradicional se ve así: Cada línea tiene un propósito específico: EAX identifica qué syscall queremos, EBX-EDX contienen los argumentos, y la instrucción INT 0x80 es el portal al kernel.

```
1 mov eax, 4      ; syscall number para sys_write
2 mov ebx, 1      ; file descriptor (stdout)
3 mov ecx, msg    ; puntero al mensaje
4 mov edx, len    ; longitud del mensaje
5 int 0x80        ; interrupción de software
```

4.2.7. Fast System Calls

El mecanismo tradicional de syscalls mediante interrupciones de software, aunque funcional, tiene un problema de rendimiento significativo. Las interrupciones requieren guardar todo el contexto del procesador, consultar la tabla de vectores de interrupción, y manejar overhead considerable en el cambio de modo. Para un sistema que realiza millones de syscalls por segundo, esta ineficiencia se vuelve crítica.

Los procesadores modernos introducen instrucciones especializadas para hacer syscalls más rápidas. Intel desarrolló SYSENTER/SYSEXIT, mientras que AMD (posteriormente adoptado también por Intel en x86-64) creó SYSCALL/SYSRET.

```
1 ; SYSENTER/SYSEXIT (Intel)
2 mov eax, syscall_number
3 mov edx, return_address
4 mov ecx, user_stack
5 sysenter          ; Entrada rápida al kernel
6 ; ... kernel ejecuta syscall ...
7 sysexit           ; Retorno rápido a usuario
```

```
1 ; SYSCALL/SYSRET (AMD/Intel x86-64)
2 mov rax, syscall_number
3 mov rdi, arg1
4 mov rsi, arg2
5 syscall           ; Entrada directa al kernel
6 ; Resultado en RAX al retornar
```

Estas instrucciones especializadas son 50-70 % *más rápidas* que INT tradicional. Logran esto reduciendo el overhead de cambio de contexto, usando registros especializados en lugar de stack, y proporcionando transición directa sin necesidad de consultar vectores de interrupción.

En Linux moderno, la biblioteca glibc automáticamente elige el método más eficiente disponible en el procesador. El programador simplemente llama a funciones como `write()` o `read()`, y glibc se encarga de usar SYSCALL en x86-64, SYSENTER en x86-32, o SVC en ARM, según corresponda.

4.2.8. Arquitecturas de Kernel

La forma en que se organiza internamente el código del sistema operativo tiene profundas implicaciones en rendimiento, seguridad y mantenibilidad. Existen dos filosofías principales que representan extremos opuestos en el espectro de diseño.

Kernel Monolítico:

En un kernel monolítico, todo el sistema operativo ejecuta en modo kernel como un solo programa grande. Los drivers de dispositivos, el sistema de archivos, el stack de red, y todos los subsistemas comparten un único espacio de direcciones. La comunicación entre componentes se realiza mediante llamadas a función directas, como en cualquier programa normal.

Ventajas del enfoque monolítico:

El rendimiento es superior porque no hay overhead de comunicación entre componentes. Un driver puede llamar directamente a una función del filesystem sin cambios de contexto ni paso de mensajes. Esta eficiencia es la razón por la cual Linux, siendo monolítico, puede competir en rendimiento con sistemas diseñados específicamente para alta performance. La simplicidad de debugging también es notable: cuando algo falla, todo está en un solo lugar. Las herramientas de debugging pueden seguir el flujo de ejecución sin saltar entre diferentes espacios de direcciones.

Desventajas del enfoque monolítico:

La menor estabilidad es el precio que se paga por la eficiencia. Un bug en cualquier driver, por obscuro que sea, puede crashear todo el sistema porque todo comparte el mismo espacio de direcciones. Un error de puntero en el driver de una webcam puede corromper las estructuras del scheduler.

La seguridad también sufre: todo el código tiene privilegios máximos. No hay aislamiento entre un driver de red y el código que maneja contraseñas. El tamaño del kernel es considerable, y cada línea de código es un potencial punto de fallo.

Ejemplos: Linux, Unix tradicional, y parcialmente Windows (que combina elementos monolíticos con diseño en capas)

Microkernel:

La filosofía del microkernel es radicalmente diferente: mantener el kernel lo más pequeño posible, implementando solo las funciones absolutamente esenciales. Todo lo demás ejecuta como procesos separados en modo usuario.

El kernel mínimo se limita a scheduling de procesos básico, comunicación inter-procesos (IPC), y manejo elemental de memoria. Los drivers de dispositivos, sistemas de archivos, y servicios de red ejecutan como procesos normales que se comunican mediante paso de mensajes.

Ventajas del enfoque microkernel:

La estabilidad mejora dramáticamente: si un driver falla, es solo un proceso más que terminó. El sistema puede reiniciar el driver sin afectar al resto. La seguridad se beneficia de que cada servicio puede ejecutar con solo los privilegios mínimos necesarios.

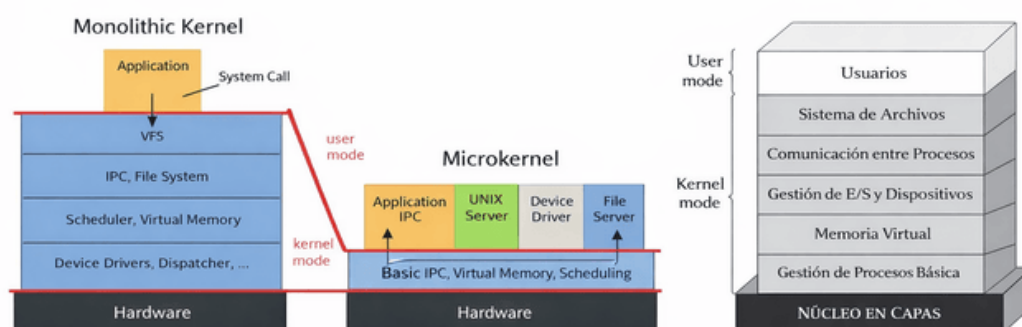
La modularidad facilita agregar, quitar, o actualizar componentes. Querer probar un nuevo filesystem no requiere recompilar el kernel, solo lanzar un nuevo proceso servidor. El debugging se simplifica porque los componentes están aislados.

Desventajas del enfoque microkernel:

El rendimiento sufre debido al overhead de IPC entre componentes. Una operación simple puede requerir múltiples cambios de contexto y múltiples mensajes. La complejidad de diseño e implementación es considerablemente mayor: diseñar un buen sistema de IPC eficiente es extremadamente difícil.

Ejemplos: Minix (el sistema usado para enseñar sistemas operativos), QNX (usado en sistemas embebidos críticos), seL4 (kernel formalmente verificado), y GNU Hurd (aún en desarrollo después de décadas).

Enfoques Híbridos: La realidad es que la mayoría de sistemas operativos modernos no son puramente monolíticos ni microkernels puros, sino que combinan elementos de ambos enfoques según convenga. Windows NT usa un microkernel modificado pero mueve algunos servicios críticos de performance al kernel. macOS combina el microkernel Mach con componentes monolíticos de BSD. Linux, aunque principalmente monolítico, usa módulos cargables que pueden agregarse y removerse dinámicamente.



Comparación de las principales arquitecturas de núcleo de un sistema operativo: kernel monolítico, microkernel y kernel multicapa, destacando la organización de sus componentes y su relación con el hardware.

Aspecto	Monolítico	Microkernel
Rendimiento	Excelente	Bueno
Estabilidad	Regular	Excelente
Seguridad	Regular	Excelente
Simplicidad	Alta	Baja
Tiempo de desarrollo	Menor	Mayor

4.3. Análisis Técnico (Leelo sólo por curiosidad...)

Esta sección profundiza en aspectos más técnicos que, si bien no son esenciales para un primer entendimiento, te darán una apreciación más profunda de la complejidad involucrada. Podés leerla por curiosidad o saltarla y volver más adelante.

4.3.1. Proceso de Arranque (Bootstrap)

El proceso de arranque es fascinante porque debe resolver un problema aparentemente imposible: ¿cómo puede una computadora apagada comenzar a ejecutar software si el software mismo necesita estar ejecutándose para cargar software? La respuesta involucra una cuidadosa secuencia de etapas, cada una más compleja que la anterior.

La secuencia comienza con el *Power-On Self Test* (POST), un programa pequeño almacenado en firmware (BIOS o UEFI) que ejecuta verificaciones básicas de hardware. Verifica que la RAM funcione, que haya dispositivos de boot disponibles, y que los componentes críticos respondan.

Una vez que POST completa exitosamente, se carga el *bootloader*, un programa pequeño que sabe cómo cargar el sistema operativo. Este programa típicamente reside en el Master Boot Record (MBR) en sistemas BIOS legacy, o en una partición EFI en sistemas modernos UEFI.

El bootloader lee el kernel del sistema operativo desde disco y lo carga en memoria. Luego transfiere control al punto de entrada del kernel, que comienza su propia fase de inicialización: detecta e inicializa todo el hardware disponible, configura estructuras de datos internas como tablas de procesos y sistemas de archivos, y finalmente arranca el primer proceso (tradicionalmente llamado `init`).

4.3.2. Manejo de Interrupciones

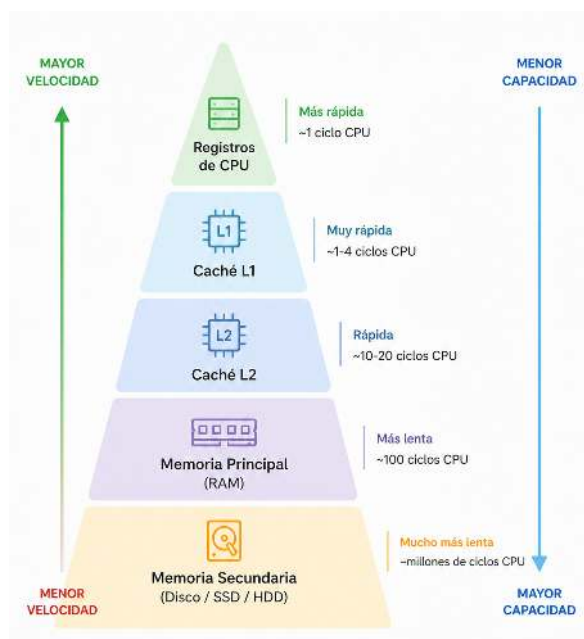
El manejo de interrupciones involucra una estructura de datos crítica llamada *vector de interrupciones* o *Interrupt Descriptor Table* (IDT) en x86. Esta tabla mapea cada número de interrupción a la dirección de memoria de su handler correspondiente.

Cuando ocurre una interrupción, el proceso completo se desarrolla así: el hardware detiene la ejecución actual y guarda el contexto mínimo (PC y PSW) en el stack. Consulta el vector de interrupciones para encontrar la dirección del handler apropiado. El procesador cambia a modo kernel si no estaba ya en ese modo. El handler ejecuta, realizando las acciones necesarias para atender la interrupción. Al completar, se restaura el contexto guardado y se retorna al punto donde fue interrumpido, como si nada hubiera pasado.

Esta capacidad de interrumpir transparentemente la ejecución, atender un evento, y continuar como si nada hubiera pasado es fundamental para la ilusión de que múltiples cosas están ocurriendo simultáneamente en un procesador que solo puede ejecutar una instrucción a la vez.

4.3.3. Jerarquía de Memoria

Los diferentes niveles de memoria en un sistema moderno forman una pirámide donde velocidad y capacidad están inversamente relacionados:



Las implicaciones para el sistema operativo son profundas. La gestión eficiente de cache es crítica para rendimiento, ya que la diferencia entre un cache hit y un cache miss puede ser de dos órdenes

de magnitud en latencia. La memoria virtual se vuelve necesaria cuando la RAM es insuficiente para todos los procesos activos. Los algoritmos de reemplazo deben optimizar para minimizar accesos al nivel más lento.

4.4. Código en C

Veamos cómo estos conceptos se traducen en código real. Los siguientes ejemplos te permitirán experimentar directamente con system calls y manejo de señales.

4.4.1. Ejemplo: Información del Sistema

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/utsname.h>
4
5  int main() {
6      struct utsname system_info;
7
8      // Syscall para obtener información del sistema
9      if (uname(&system_info) == -1) {
10         perror("uname failed");
11         return 1;
12     }
13
14     printf("Sistema Operativo: %s\n", system_info.sysname);
15     printf("Nombre del Host: %s\n", system_info.nodename);
16     printf("Release: %s\n", system_info.release);
17     printf("Versión: %s\n", system_info.version);
18     printf("Arquitectura: %s\n", system_info.machine);
19
20     // Otras syscalls útiles
21     printf("Process ID: %d\n", getpid());
22     printf("Parent Process ID: %d\n", getppid());
23     printf("User ID: %d\n", getuid());
24
25     return 0;
26 }
```

Este programa demuestra varias system calls en acción. La función `uname()` es una syscall que obtiene información del sistema operativo. Cada llamada provoca una transición de modo usuario a modo kernel, donde el kernel recopila la información solicitada y la retorna de manera segura. Las funciones `getpid()`, `getppid()` y `getuid()` son syscalls adicionales que obtienen identificadores del proceso actual, su proceso padre, y el usuario que lo ejecuta.

4.4.2. Ejemplo: Manejo Básico de Señales

```
1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 // Handler para la señal SIGINT (Ctrl+C)
6 void signal_handler(int sig) {
7     printf("\n;Recibida señal %d! Pero no me voy a cerrar...\n", sig);
8     printf("Presiona Ctrl+\ para terminar realmente.\n");
9 }
10
11 int main() {
12     // Registrar nuestro handler para SIGINT
13     signal(SIGINT, signal_handler);
14
15     printf("Programa ejecutándose... Presiona Ctrl+C para probar.\n");
16     printf("PID: %d\n", getpid());
17
18     // Loop infinito para demostrar el manejo de señales
19     while(1) {
20         printf("Trabajando...\n");
21         sleep(2); // Syscall que suspende ejecución por 2 segundos
22     }
23
24     return 0;
25 }
```

Este ejemplo ilustra el manejo de señales, un mecanismo de comunicación asíncrona fundamental. Cuando presionás Ctrl+C, el terminal envía la señal SIGINT al proceso. Normalmente esto terminaría el programa, pero como registramos nuestro propio handler, nuestro código ejecuta en su lugar.

Las señales son un caso fascinante de cómo el sistema operativo puede interrumpir un proceso para entregarle información. El handler se ejecuta en el contexto del proceso interrumpido, pero fue invocado por el kernel. Es un ejemplo perfecto de cómo hardware (la tecla Ctrl+C), kernel (detección y entrega de señal), y proceso de usuario (handler) colaboran.

4.5. Casos de Estudio

4.5.1. Caso 1: Análisis de una Syscall

Enunciado: Analice qué sucede cuando un programa ejecuta `printf("Hola mundo\n");` desde las perspectivas de hardware y sistema operativo.

Resolución Paso a Paso:

Comencemos analizando el código. La función `printf()` es parte de la biblioteca estándar de C (libc), no del kernel. Internamente, `printf()` formatea el string y luego llama a la syscall `write`

(), que sí es parte del kernel. Esta syscall debe escribir en stdout, representado por el file descriptor 1.

La secuencia de ejecución completa es:

```
1 Programa (modo usuario)→  
2 printf()→  
3 write(1, "Hola mundo\n", 11)→  
4 Interrupción de software (trap)→  
5 Modo kernel→  
6 sys_write() en el kernel→  
7 Driver de terminal→  
8 Hardware de pantalla→  
9 Retorno a modo usuario
```

Durante la transición de usuario a kernel, se guardan todos los registros del proceso en su kernel stack. Se cambia el puntero de stack para usar el stack del kernel. El modo de operación del procesador cambia a supervisor/kernel. En la transición inversa, se restauran los registros desde el kernel stack, se cambia de vuelta al stack de usuario, y el valor de retorno de la syscall se coloca en el registro apropiado (típicamente EAX/RAX).

Puntos clave para recordar:

- `printf()` NO es una syscall directa, es una función de biblioteca
- Hay cambio de modo de operación (usuario → kernel → usuario)
- El kernel valida permisos antes de permitir la escritura
- El file descriptor 1 representa stdout por convención POSIX
- Todo el proceso es transparente para el programador

4.6. Síntesis

4.6.1. Puntos Clave del Capítulo

Hemos recorrido un largo camino desde los componentes básicos del hardware hasta los conceptos fundamentales del sistema operativo. Tres ideas principales deben quedar cristalinas:

Primero, la arquitectura hardware no es un detalle de implementación irrelevante: determina las capacidades fundamentales del sistema operativo. La arquitectura de von Neumann establece el modelo básico de cómo funcionan las computadoras. Los modos de operación permiten la protección necesaria para la estabilidad del sistema. Las interrupciones son el mecanismo que habilita el multitasking y la respuesta a eventos externos.

Segundo, el sistema operativo es el intermediario esencial entre software y hardware. Las system calls proporcionan servicios controlados y seguros. El SO abstrae las complejidades del hardware, permitiendo que los programadores se concentren en la lógica de sus aplicaciones. Mantiene la seguridad y estabilidad que damos por sentada.

Tercero, el rendimiento de un sistema depende críticamente de entender la jerarquía de memoria. Los principios de localidad temporal y espacial son fundamentales para el diseño de algoritmos eficientes. La gestión de cache puede hacer la diferencia entre un sistema lento y uno rápido. Los accesos a disco son inevitablemente el cuello de botella principal en muchos sistemas.

4.6.2. Conexiones con Próximos Capítulos

Los conceptos que hemos explorado aquí forman la base sobre la cual construiremos todo lo que sigue. En el próximo capítulo sobre Procesos, veremos cómo los modos de operación y las syscalls permiten al SO crear y gestionar la abstracción fundamental de “programa en ejecución”.

Cuando estudiemos Planificación, las interrupciones de timer que apenas mencionamos aquí se volverán centrales: son el mecanismo que permite al SO implementar multitasking preemptivo, quitándole la CPU a un proceso que se está ejecutando para darle turno a otro.

En los capítulos sobre Gestión de Memoria, la jerarquía de memoria y la MMU (Memory Management Unit) que solo tocamos superficialmente se convertirán en protagonistas, permitiéndonos entender cómo un sistema puede hacer que múltiples procesos crean que tienen toda la memoria para ellos solos.

4.6.3. Errores Comunes en Parciales

Terminemos identificando algunos malentendidos frecuentes que aparecen en exámenes:

Error común: “printf() es una syscall” **Realidad:** printf() es una función de biblioteca (libc) que *usa* la syscall write() internamente. Esta distinción es importante: la biblioteca proporciona conveniencia (formateo de strings), mientras que la syscall proporciona funcionalidad fundamental (escribir bytes a un file descriptor).

Error común: “Las interrupciones solo vienen del hardware” **Realidad:** También existen interrupciones de software (syscalls, excepciones). De hecho, las syscalls son típicamente implementadas como interrupciones de software. Las excepciones, como división por cero o acceso a memoria inválida, también son formas de interrupción.

Error común: “El modo kernel es más rápido” **Realidad:** El modo kernel tiene más privilegios, no necesariamente más velocidad. De hecho, el cambio de modo usuario a kernel tiene overhead. Lo que hace que el kernel parezca rápido es que puede acceder directamente a hardware sin pasar por capas de abstracción.

5 Procesos

5.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Definir qué es un proceso y diferenciarlo de un programa
- Explicar las diferencias entre monoprogramación, multiprocesamiento, multiprogramación y multitarea
- Describir las estructuras de control del SO para gestionar procesos
- Analizar la imagen de un proceso y la estructura del PCB
- Interpretar los diagramas de estados (3, 5 y 7 estados) y sus transiciones
- Identificar el rol del dispatcher y los diferentes planificadores
- Implementar operaciones básicas con procesos usando syscalls de Unix/Linux
- Resolver ejercicios sobre creación, comunicación y terminación de procesos
- Manejar correctamente procesos zombie y huérfanos

5.2. Introducción y Contexto

5.2.1. ¿Por qué existen los procesos?

Imaginate una computadora de los años 1950: un solo programa ejecutándose, ocupando toda la memoria y el procesador hasta terminar. Si querías ejecutar otro programa, debías esperar. Si el programa se colgaba, reiniciabas toda la máquina. Este escenario, aunque funcional, era profundamente ineficiente. Los procesos nacieron como respuesta a una pregunta fundamental que revolucionó la computación: ¿cómo hacer que múltiples programas compartan eficientemente los recursos de una sola computadora? La respuesta transformó no solo la arquitectura de los sistemas operativos, sino la manera misma en que pensamos sobre la ejecución de software.

Un *proceso* es mucho más que un programa ejecutándose. Es una abstracción sofisticada que el sistema operativo construye para lograr cuatro objetivos críticos. Primero, aísla programas entre sí, de modo que un proceso no puede corromper la memoria de otro, garantizando estabilidad y seguridad. Segundo, permite compartir recursos como CPU, memoria y archivos de manera controlada y justa. Tercero, facilita la concurrencia, dando la ilusión de que múltiples tareas se ejecutan simultáneamente incluso en sistemas con una sola CPU. Finalmente, establece mecanismos seguros de comunicación entre programas, permitiendo que colaboren sin comprometer la integridad

del sistema.

La abstracción del proceso representa uno de los conceptos más fundamentales en sistemas operativos. Sin ella, la computación moderna tal como la conocemos sería imposible: no podríamos ejecutar múltiples aplicaciones simultáneamente, no tendríamos protección entre programas, y un error en cualquier software colapsaría todo el sistema.

5.3. Conceptos Fundamentales

5.3.1. Programa vs Proceso: La Diferencia Esencial

La distinción entre programa y proceso es sutil pero crucial. Un programa es una entidad completamente estática: un archivo ejecutable almacenado en disco que contiene código fuente compilado en instrucciones de máquina. Es una secuencia pasiva de instrucciones que no consume ningún recurso del sistema hasta el momento de su ejecución. Ejemplos típicos incluyen archivos como `/bin/ls` en Linux o `notepad.exe` en Windows.

Un proceso, en contraste, es una entidad dinámica y viva. Representa una instancia específica de un programa en ejecución, y abarca no solo el código sino también los datos, el contexto de ejecución completo, y todos los recursos asignados. Es una entidad activa que puede realizar acciones, tomar decisiones, y consume recursos del sistema como memoria, tiempo de CPU, y descriptores de archivos. Por ejemplo, cuando ejecutás el comando `ls` en tu terminal, el sistema operativo crea un proceso con un identificador único (PID 1234, digamos) que ejecuta ese programa.

Podés pensar en un programa como una receta de cocina escrita en un libro: contiene todas las instrucciones, pero no produce ningún resultado por sí misma. Un proceso es como un cocinero siguiendo esa receta en tiempo real: usa ingredientes (datos), ocupa un espacio en la cocina (memoria), y produce resultados concretos. Además, varios cocineros (procesos) pueden estar siguiendo la misma receta (programa) simultáneamente, cada uno en su propia estación de trabajo.

5.3.2. Evolución de los Sistemas de Procesamiento

Para entender realmente qué son los procesos y por qué existen, necesitamos recorrer la evolución histórica de cómo las computadoras manejan programas. Esta evolución no fue arbitraria, sino una respuesta a limitaciones concretas que cada generación de sistemas enfrentó.

La monoprogramación representa el modelo más primitivo. En estos sistemas, un solo programa se ejecutaba a la vez, monopolizando completamente todos los recursos. El problema fundamen-

tal era evidente: cuando el programa realizaba operaciones de entrada/salida (como leer del disco), la CPU quedaba completamente ociosa, esperando. Esta ineficiencia era catastrófica: la utilización de CPU raramente superaba el 5-10 %. Los sistemas batch simples de los años 1950 operaban bajo este modelo, donde los programas se procesaban uno tras otro en secuencia estricta.

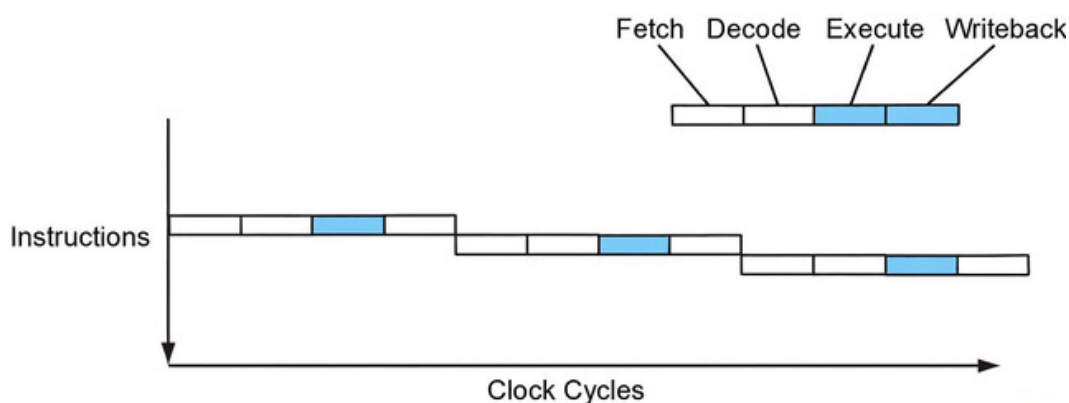


Diagrama de instrucciones secuenciales.

La multiprogramación atacó el problema de la ociosidad de manera elegante. En lugar de esperar que un programa termine, estos sistemas mantienen múltiples programas cargados en memoria simultáneamente. Una sola CPU alterna entre ellos de manera inteligente: cuando un proceso hace una operación de I/O y queda bloqueado, otro proceso usa la CPU. El objetivo central es maximizar la utilización de CPU, transformando el tiempo muerto en tiempo productivo.

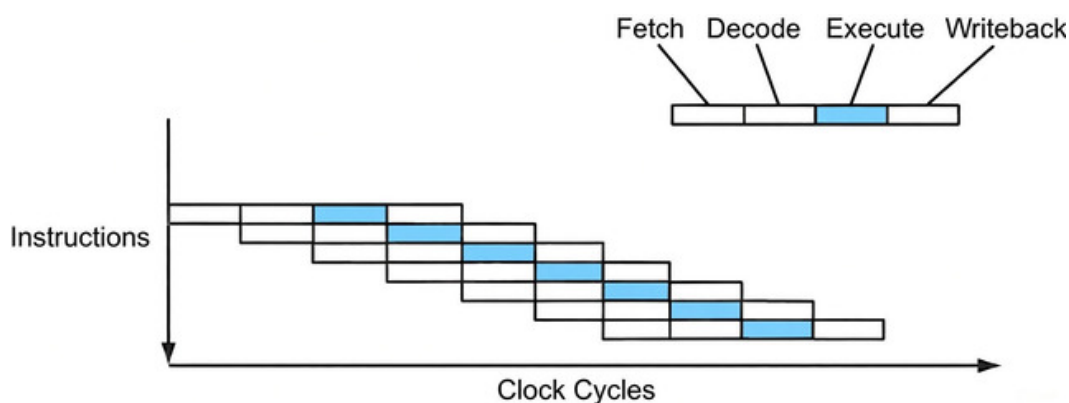


Diagrama de ejecución pipeline, donde las etapas de una instrucción (fetch, decode, execute y writeback) se superponen en distintos ciclos de reloj para mejorar el rendimiento.

El multiprocesamiento tomó un camino diferente: en lugar de mejorar cómo se usa una CPU, agregó múltiples CPUs físicas a la misma máquina. Esto permitió verdadero paralelismo a nivel hardware, donde cada CPU puede ejecutar un proceso completamente diferente al mismo tiempo. Los

sistemas SMP (Symmetric Multiprocessing) democratizaron este enfoque, permitiendo que cualquier CPU ejecute cualquier proceso sin restricciones especiales.

La multitarea extendió la multiprogramación con un concepto revolucionario: time-sharing. No solo los procesos comparten la CPU durante operaciones de I/O, sino que el sistema operativo puede interrumpir forzosamente un proceso en ejecución (preemptive scheduling) para darle turno a otro. Cada proceso recibe pequeños intervalos de tiempo llamados quantum o time slices, típicamente de 10-100 milisegundos. Esta rapidez en el cambio crea la ilusión de que todos los programas se ejecutan simultáneamente, logrando la interactividad que esperamos de los sistemas modernos.

Es común confundir multiprogramación con multitarea. La diferencia clave está en la preemption: en multiprogramación pura, un proceso solo cede la CPU voluntariamente (al hacer I/O). En multitarea, el sistema operativo puede quitarle la CPU a un proceso en cualquier momento, garantizando que ningún proceso monopolice el sistema.

5.3.3. Grado de Multiprogramación

El grado de multiprogramación es una métrica fundamental que define cuántos procesos residen simultáneamente en memoria principal. No se trata de cuántos procesos existen en total, ni cuántos están en ejecución en un momento dado, sino específicamente cuántos están cargados en RAM al mismo tiempo.

Este número está limitado por varios factores críticos. El más obvio es la memoria disponible: más procesos en memoria significa menos memoria disponible por proceso, lo que puede forzar el uso de memoria virtual y degradar el rendimiento. Los recursos del sistema también imponen límites, como la cantidad de file descriptors, sockets, o conexiones de red que el sistema puede mantener. Finalmente, existe un overhead del sistema operativo: cada proceso adicional requiere estructuras de datos (PCBs, tablas de páginas), y el costo del cambio de contexto aumenta con más procesos compitiendo por CPU.

En sistemas modernos, el grado de multiprogramación puede ser de cientos o incluso miles de procesos. Linux, por ejemplo, puede manejar fácilmente 10,000 procesos en hardware adecuado. Sin embargo, solo unos pocos estarán realmente activos en cualquier momento dado, el resto estará bloqueado esperando eventos.

5.3.4. Estructuras de Control del Sistema Operativo

Para gestionar eficientemente todos estos procesos, el sistema operativo mantiene un conjunto sofisticado de tablas interconectadas. Estas estructuras forman la columna vertebral del manejo

de recursos.

La **tabla de procesos** actúa como punto de entrada hacia todas las demás tablas. Contiene un PCB (Process Control Block) por cada proceso en el sistema, y cada PCB mantiene referencias cruzadas hacia las entradas correspondientes en las tablas de memoria, I/O y archivos. Esta estructura centralizada permite al sistema operativo localizar rápidamente toda la información relacionada con cualquier proceso.

Tabla de Procesos – Process Control Block (PCB)

**Propósito:** Mantiene la información de control de cada proceso del sistema.
Características: Estado del proceso, contexto de CPU, asignación de recursos y jerarquía de procesos.

PID (Identificador)	Proceso State (Estado)	Program Counter (PC)	CPU Registers (Contexto de CPU)	Memory Pointers (Rango de memoria)	Open Files (Archivos abiertos)	Priority (Prioridad)	Parent PID (Padre)	CPU Time (Uso de CPU)
1001	RUNNING	0x4A2B	EAX: 0xFF EBX: 0x12	0x1000 - 0x2FFF	fd3, fd5, fd7	0	1000	150 ms
1002	WAITING	0x3C4F	EAX: 0x00 EBX: 0xAB	0x3000 - 0x4FFF	fd4, fd8	1	1001	75 ms
1003	READY	0x5B8A	EAX: 0xCC EBX: 0x34	0x5000 - 0x6FFF	fd6, fd9, fd10	2	1000	200 ms

Legenda de Estados:
RUNNING En ejecución READY Listo para ejecutar WAITING En espera (bloqueado) TERMINATED Terminado

Descripción de Campos

- PID:** Identificador único del proceso.
- Process State:** Estado actual del proceso en el sistema.
- PC:** Program Counter (PC): Dirección de la siguiente instrucción a ejecutar.
- CPU Registers:** Contenido de los registros de la CPU asociados al proceso.
- Memory Pointers:** Rango de memoria asignado al proceso (código, datos y pila).
- Open Files:** Lista de archivos abiertos por el proceso.
- Priority:** Prioridad de planificación (menor valor = mayor prioridad).
- Parent PID:** Identificador del proceso padre (jerarquía).
- CPU Time:** Tiempo total de CPU utilizado por el proceso.

Estructura Interna de un PCB

PID

Estado del Proceso

Program Counter (PC)

Registros de CPU

EAX EBX ECX EDX ...

Punteros de Memoria

Bases (Inicio) Límite (Fin)

Archivos Abiertos

fd1 fd2 fd3 ...

Prioridad

PID del Padre

Tiempo de CPU

Información Adicional (credenciales, señales, etc.)

Contexto de Ejecución

Administración de Recursos

Planificación y Control

Cola de Listos (Ejemplo)

PID 1003 (READY) → PID 1005 (READY) → PID 1006 (READY) → ...

CPU

Ejecutando: PID 1001 (RUNNING)

Nota: La PCB contiene toda la información necesaria para suspender y reanudar la ejecución de un proceso, permitiendo la multitarea y el cambio de contexto.

La **tabla de memoria** rastrea cómo se asigna la memoria del sistema. Registra qué bloques de memoria están asignados a cada proceso, qué memoria está libre y disponible, los atributos de protección de cada región (lectura, escritura, ejecución), y toda la información necesaria para memoria virtual, como tablas de páginas y segmentos.

La **tabla de I/O** gestiona los dispositivos del sistema. Mantiene el estado actual de cada dispositivo (libre, ocupado, error), las colas de operaciones pendientes por dispositivo, los buffers asociados a cada operación en curso, y referencias a los controladores de dispositivos activos. Esta tabla es crucial para la multiprogramación: cuando un proceso se bloquea esperando I/O, el sistema necesita saber exactamente qué está esperando.

Tabla de Memoria – Memory Management Unit (MMU)

Propósito: Gestiona la asignación de memoria principal a cada proceso. Características: Control de límites, protección de memoria, gestión de páginas y segmentos.							
Process ID (PID)	Base Address (Registro base)	Limit Address (Límite superior)	Size (Tamaño)	Page Table (Dir. tabla de páginas)	Segment Table (Dir. tabla de segmentos)	State (Estado)	Protection (Protección)
1001	0x1000	0x2FFF	8192 bytes (8 KB)	0xA000	0x8000	ALLOCATED	RWX (111)
1002	0x3000	0x4FFF	8192 bytes (8 KB)	0xA100	0x8100	ALLOCATED	RW- (110)
1003	0x5000	0x6FFF	8192 bytes (8 KB)	0xA200	0x8200	ALLOCATED	R-X (101)
—	0x7000	0x8FFF	8192 bytes (8 KB)	—	—	FREE	--- (000)

Descripción de Campos

Process ID (PID):

Identificador único del proceso.

Base Address:

Dirección física inicial asignada al proceso.

Limit Address:

Dirección física límite (no inclusiva).

Size (Tamaño):

Cantidad de memoria asignada al proceso.

Page Table:

Dirección base de la tabla de páginas del proceso.

Segment Table:

Dirección base de la tabla de segmentos del proceso.

State (Estado):

Indica si el bloque de memoria está asignado o libre.

Protection (Protección):

Permisos de acceso a la memoria.

- R: Read (lectura)
- W: Write (escritura)
- X: eXecute (ejecución)

Vista de Asignación en Memoria

0x0000

0x1000

0x3000

0x5000

0x7000

0x9000

Proceso 1001 (8 KB)

Proceso 1002 (8 KB)

Proceso 1003 (8 KB)

Libre (8 KB)

0x1000 – 0x2FFF

0x3000 – 0x4FFF

0x5000 – 0x6FFF

0x7000 – 0x8FFF

Estados

ALLOCATED

Memoria asignada a un proceso.

FREE

Memoria disponible para asignación.

Protección

Permisos	R (Lectura)	W (Escritura)	X (Ejecución)
RWX (111)	✓	✓	✓
RW- (110)	✓	✓	✗
R-X (101)	✓	✗	✓
--- (000)	✗	✗	✗

Leyenda

Asignado (ALLOCATED)

Libre (FREE)

0xXXXX

Dirección hexadecimal

N KB

Tamaño en kilobytes

Nota:

Las direcciones mostradas son ejemplos. En un sistema real, los tamaños y direcciones dependen de la configuración de hardware y del SO.

Tabla de E/S – Device Control Table (DCT)

Propósito: Controla el acceso y estado de los dispositivos de entrada/salida. Características: Gestión de colas de espera, asignación de dispositivos y manejo de interrupciones.							
Device ID	Device Type	Status	Assigned Process	Memory Address (Registro base)	Queue (Cola de espera)	Driver (Rutina de servicio)	Interrupt Vector (Nº de interrupción)
DEV_001	DISK	BUSY	1001 (PID 1001)	0xC000	PID 1001, 1003	0xD000	14
DEV_002	KEYBOARD	IDLE	—	0xC100	empty	0xD100	1
DEV_003	PRINTER	BUSY	1002 (PID 1002)	0xC200	PID 1002	0xD200	7
DEV_004	NETWORK	IDLE	—	0xC300	empty	0xD300	11

BUSY

Dispositivo ocupado

IDLE

Dispositivo libre

0xXXXX

Dirección/Vector en hexadecimal

—

No asignado / Sin proceso

Descripción de Campos

Device ID:

Identificador único del dispositivo en el sistema.

Device Type:

Tipo o clase de dispositivo.

Status:

Indica si el dispositivo está ocupado (BUSY) o libre (IDLE).

Assigned Process:

Proceso que tiene actualmente asignado el dispositivo.

Memory Address (Registro base):

Dirección base de los registros de control del dispositivo.

Queue (Cola de espera):

Procesos en espera para usar el dispositivo.

Driver (Rutina de servicio):

Dirección de la rutina del controlador (driver) asociada.

Interrupt Vector (Nº de interrupción):

Número de interrupción asociado al dispositivo.

Flujo de E/S

Proceso (PID 1001)

SO (Gestor de E/S)

DCT (Tabla de E/S)

Dispositivo (DEV_001 - DISK)

Driver (0xD000)

Interrupción (Vector 14)

solicita E/S

asigna

despierta / continúa

notifica

genera

Notas:

La DCT permite al sistema operativo administrar eficientemente múltiples dispositivos, controlando su estado, colas de espera e interrupciones asociadas.

Convenciones

PID:

Identificador de Proceso

—

No asignado

0xXXXX:

Direcciones en hexadecimal

empty:

Cola vacía

La **tabla de archivos** coordina el acceso al sistema de archivos. Registra todos los archivos abier-

tos en el sistema, la ubicación de cada archivo en el almacenamiento, el estado de acceso actual (lectura, escritura, compartido), y los locks y permisos por archivo. Un mismo archivo puede estar abierto por múltiples procesos, y esta tabla asegura que las operaciones concurrentes se manejen correctamente.

1. Tabla de Archivos (File Table)

Propósito: Mantiene información de todos los archivos abiertos en el sistema.

Características: Estructura en memoria que relaciona los descriptores de archivos con sus FCB correspondientes.

FD (Descriptor)	Puntero al FCB	Modo de Acceso	Posición Actual	Contador de Referencias	Estado	Proceso Propietario
0	→ FCB #3	Lectura	2048	1	Abierto	PID 1001
1	→ FCB #4	Lectura/Escritura	0	1	Abierto	PID 1000
2	→ FCB #5	Lectura/Escritura	8192	1	Abierto	PID 1002
3	→ FCB #6	Escritura	512	1	Abierto	PID 1003
4	—	—	—	0	Libre	—

→ Puntero al FCB

Entrada en uso

Entrada libre

Descripción de Campos

FD: Descriptor de archivo (asignado por el sistema)

Puntero al FCB: Referencia al bloque de control del archivo

Modo de Acceso: Permisos con los que fue abierto

Posición Actual: Offset actual dentro del archivo

Contador de Referencias: Número de descriptores que hacen referencia al mismo FCB

Estado: Indica si la entrada está en uso o libre

Proceso Propietario: Proceso que abrió el archivo

2. File Control Block (FCB)

Propósito: Contiene toda la información administrativa de un archivo en el sistema.

Características: Estructura persistente en disco que describe y gestiona cada archivo.

File Name	system.log	Nombre del archivo
File Descriptor (ID)	3	Identificador único del archivo
Owner (UID)	1001	Usuario propietario
Permissions	0644 (rw-r--r--)	Permisos de acceso
File Pointer (Size)	2048	Tamaño actual del archivo (bytes)
Location (Bloques)	[Block_456, Block_457, Block_458]	Bloques asignados en disco
Open Count	2	Número de referencias (entradas en la tabla de archivos)
Created	2024-03-10 10:30:00	Fecha y hora de creación
Modified	2024-03-15 14:30:00	Fecha y hora de última modificación

FCB (Ejemplo)

Relación entre estructuras

Proceso (PID 1001) → open() → Tabla de Archivos (en memoria)

Tabla de Archivos (en memoria) → Referencia → FCB #3 (en disco)

FCB #3 (en disco) → Bloques de Disco

5.3.5. Imagen de un Proceso

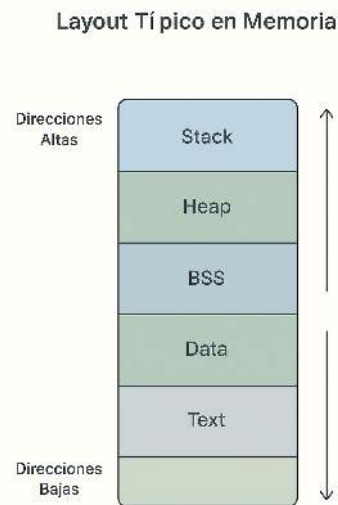
La imagen del proceso representa la huella completa que un proceso deja en memoria. No es simplemente el código ejecutable, sino una estructura compleja dividida en segmentos especializados, cada uno con un propósito específico.

El **text segment** contiene las instrucciones ejecutables del programa. Este segmento es read-only para prevenir modificaciones accidentales del código, y puede ser compartido entre múltiples procesos que ejecutan el mismo programa. Se carga directamente desde el archivo ejecutable al iniciar el proceso.

El **data segment** se divide en dos regiones. La sección de datos inicializados contiene variables globales y estáticas que tienen un valor inicial definido en el código fuente. La sección BSS (*Block Started by Symbol*) contiene variables globales y estáticas no inicializadas, que el sistema operativo inicializa automáticamente a cero. Este segmento es read-write y específico para cada proceso.

El **heap** es donde vive la memoria dinámica solicitada por el proceso mediante funciones como `malloc()` o el operador `new`. Crece hacia direcciones de memoria altas según el proceso solicita más memoria, y es completamente gestionado por el proceso (y las bibliotecas de manejo de memoria).

El **stack** contiene variables locales de funciones, parámetros pasados a funciones, direcciones de retorno, y frame pointers. Crece hacia direcciones de memoria bajas, en dirección opuesta al heap. Esta organización permite que heap y stack compartan el espacio disponible de manera flexible.



La separación entre heap y stack creciendo en direcciones opuestas no es arbitraria. Si crecieran en la misma dirección, sería necesario decidir de antemano cuánto espacio asignar a cada uno. Con este diseño, heap y stack pueden crecer dinámicamente hasta encontrarse, maximizando el uso eficiente de la memoria disponible.

5.3.6. Process Control Block (PCB)

El PCB es, sin exageración, la estructura de datos más importante para el manejo de procesos en cualquier sistema operativo. Es el “documento de identidad completo” de cada proceso, conteniendo absolutamente toda la información que el sistema operativo necesita para gestionar ese proceso.

El PCB existe porque el sistema operativo debe ser capaz de suspender un proceso en ejecución en cualquier momento y luego reanudarlo exactamente donde quedó, como si nunca hubiera sido interrumpido. Para lograr esta ilusión de ejecución continua, el PCB debe capturar el estado completo del proceso con precisión absoluta.

```
1 struct process_control_block {
2     // Identificación del proceso
3     pid_t pid;                // Process ID único
4     pid_t ppid;              // Parent Process ID
5     uid_t uid;               // User ID del propietario
6     gid_t gid;               // Group ID
7
8     // Estado del proceso
9     int state;                // NEW, READY, RUNNING, etc.
```

```

10     int priority;                // Prioridad de scheduling
11
12     // Contexto del procesador
13     struct cpu_context {
14         unsigned long regs[16]; // Registros de propósito general
15         unsigned long pc;       // Program Counter
16         unsigned long sp;       // Stack Pointer
17         unsigned long psw;      // Program Status Word
18     } context;
19
20     // Información de memoria
21     struct memory_map {
22         unsigned long code_start, code_end;
23         unsigned long data_start, data_end;
24         unsigned long heap_start, heap_end;
25         unsigned long stack_start, stack_end;
26         struct page_table *pgd; // Page Global Directory
27     } mm;
28
29     // Información de archivos
30     struct files_struct {
31         int max_fds; // Máximo file descriptors
32         struct file **fd_array; // Array de archivos abiertos
33     } files;
34
35     // Manejo de señales
36     struct signal_struct {
37         unsigned long pending; // Señales pendientes
38         struct sigaction actions[32]; // Handlers por señal
39     } signals;
40
41     // Información de scheduling
42     int time_slice; // Quantum restante
43     unsigned long cpu_time_used; // Tiempo de CPU acumulado
44
45     // Enlaces en listas del SO
46     struct list_head run_list; // Lista de procesos READY
47     struct list_head children; // Lista de procesos hijos
48     struct pcb *parent; // Puntero al proceso padre
49 };

```

Esta estructura, aunque simplificada, revela los componentes esenciales. La sección de identificación del proceso incluye el PID único del proceso, el PPID de su proceso padre, y los identificadores de usuario y grupo del propietario. La sección de estado del proceso mantiene el estado actual (NEW, READY, RUNNING, etc.) y la prioridad de scheduling.

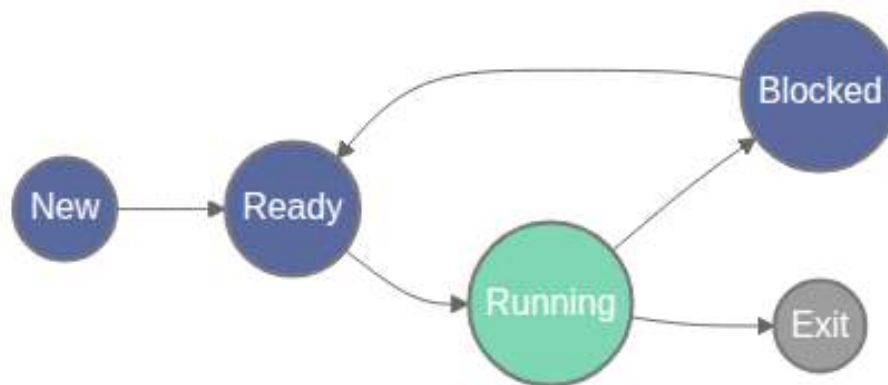
El contexto del procesador es quizás la parte más crítica: contiene todos los registros de propósito general, el program counter (PC) que indica la próxima instrucción a ejecutar, el stack pointer (SP), y el program status word (PSW) con flags y modos del procesador. Esta información es exactamente lo que se guarda y restaura durante un context switch.

La información de memoria describe el mapa de memoria del proceso: dónde comienzan y terminan los segmentos de código, datos, heap y stack, y punteros a las estructuras de memoria virtual como la tabla de páginas. La sección de archivos mantiene la tabla de file descriptors abiertos por

el proceso. El manejo de señales registra qué señales están pendientes de entrega y los handlers definidos para cada señal. Finalmente, la información de scheduling incluye el quantum restante del proceso, el tiempo de CPU acumulado, y enlaces a las diferentes listas del sistema operativo (ready queue, wait queues, lista de hijos, etc.).

Cuando presionás Ctrl+Z en tu terminal para suspender un programa, el sistema operativo guarda el estado completo del proceso en su PCB y lo mueve a una lista de procesos suspendidos. Cuando después ejecutás fg para continuarlo, el sistema restaura ese estado desde el PCB, y el programa continúa exactamente donde lo dejaste, completamente inconsciente de que estuvo suspendido.

5.3.7. Diagramas de Estado de Procesos



El modelo de cinco estados representa el comportamiento fundamental de los procesos. Un proceso comienza su vida en el estado NEW, recién creado pero aún no admitido al pool de procesos ejecutables.

Una vez que el sistema operativo lo admite (transición admit), pasa al estado READY, donde está listo para ejecutarse y simplemente espera que el scheduler le asigne tiempo de CPU.

Cuando el dispatcher selecciona el proceso (transición dispatch), entra al estado RUNNING, donde sus instrucciones se ejecutan activamente en la CPU. Desde este estado, pueden ocurrir tres cosas. Si el proceso realiza una operación bloqueante como una syscall de I/O (transición block), pasa al estado BLOCKED, donde espera que el evento se complete. Si su quantum de tiempo expira o llega un proceso de mayor prioridad (transición preempt), vuelve a READY para esperar su próximo turno. Finalmente, cuando el proceso completa su ejecución o es terminado (transición exit), alcanza el estado EXIT o terminal. Cuando un proceso en estado BLOCKED recibe la notificación de que su evento se completó (transición wakeup), no vuelve inmediatamente a RUNNING, sino que pasa a READY. Esto es crucial: el proceso debe esperar su turno como cualquier otro proceso, evitando que operaciones de I/O den prioridad injusta.

La razón por la cual BLOCKED va a READY y no directamente a RUNNING es fundamental para la justicia del sistema: un proceso que completa su I/O no debe "saltarse la fila" de procesos que ya estaban esperando pacientemente en la ready queue.

El Dispatcher es el componente del sistema operativo responsable de ejecutar el context switch. Su lógica es conceptualmente simple pero técnicamente compleja:

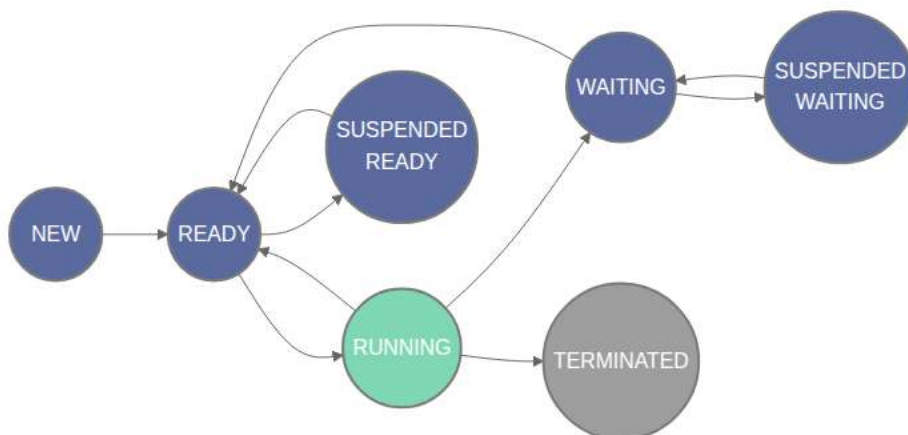
```

1 void dispatcher() {
2     while (sistema_activo) {
3         proceso_actual = scheduler(); // Seleccionar próximo proceso
4         context_switch(proceso_anterior, proceso_actual);
5         // Al retornar aquí, proceso_actual ha ejecutado
6     }
7 }

```

Este loop nunca termina mientras el sistema esté activo. Llama al scheduler para seleccionar el próximo proceso, ejecuta el cambio de contexto completo, y cuando ese proceso eventualmente cede la CPU (por preemption, bloqueo, o terminación), el dispatcher retoma el control y el ciclo continúa.

Diagrama de 7 Estados (Con Swapping):



El modelo de siete estados extiende el modelo básico para incorporar *swapping*, la capacidad del sistema operativo de mover procesos entre memoria principal y disco para gestionar escasez de memoria. Esto agrega dos estados nuevos: **READY/SUSPENDED**, donde un proceso está listo para ejecutar pero ha sido swapeado a disco, y **BLOCKED/SUSPENDED**, donde el proceso está esperando un evento y también ha sido movido a disco.

Las razones para hacer swapping son pragmáticas. Cuando la memoria es insuficiente para mantener todos los procesos activos, el sistema debe liberar espacio moviendo algunos procesos a disco. Los procesos que llevan mucho tiempo inactivos son candidatos naturales. El sistema operativo

también puede decidir hacer swapping para balancear la carga del sistema, priorizando procesos más importantes o con mayor actividad.

El swapping introduce latencia significativa. Mover un proceso de disco a memoria puede tomar milisegundos, una eternidad en términos de CPU. Por eso, los sistemas operativos modernos intentan minimizar el swapping mediante algoritmos sofisticados de manejo de memoria y predicción de comportamiento de procesos.

Tres Niveles de Planificación:

Los sistemas operativos modernos emplean tres niveles de planificación, cada uno operando en escalas de tiempo diferentes y con objetivos distintos.

El **Long-Term Scheduler** (también llamado *job scheduler* o *admission scheduler*) controla el grado de multiprogramación decidiendo qué procesos admitir desde el estado NEW a READY. Ejecuta con poca frecuencia, típicamente cada varios segundos, y su objetivo es mantener un balance saludable entre procesos CPU-bound (que usan intensivamente el procesador) y procesos I/O-bound (que pasan mucho tiempo esperando operaciones de entrada/salida).

El **Medium-Term Scheduler** (o *swapper*) decide qué procesos mover entre memoria y disco. Opera en una escala de tiempo intermedia, ejecutando cada varios segundos, y gestiona qué procesos deben permanecer en memoria activa. Típicamente suspende procesos que no están usando la CPU activamente o que llevan mucho tiempo bloqueados.

El **Short-Term Scheduler** (o *CPU scheduler* o *dispatcher*) es el más frecuente y crítico. Ejecuta cada 10-100 milisegundos y decide qué proceso de la ready queue debe recibir la CPU en cada momento. Implementa los algoritmos de scheduling como FIFO, Shortest Job First, Round Robin, scheduling por prioridades, etc.

La frecuencia de ejecución del short-term scheduler explica por qué debe ser extremadamente eficiente. Si el scheduler tarda 1 ms en tomar una decisión y otorga quanta de 10 ms, entonces el 10 % del tiempo de CPU se desperdicia en overhead de scheduling. Esta es la razón por la cual los algoritmos de scheduling deben ser no solo justos y eficientes, sino también computacionalmente baratos.

5.4. Análisis Técnico

5.4.1. Creación de Procesos en Unix/Linux

En Unix y sistemas derivados como Linux, los procesos se crean mediante una syscall peculiar y elegante llamada `fork()`. Su comportamiento es único en la historia de las interfaces de progra-

mación: crea una copia casi exacta del proceso que la invoca.

Las características de `fork()` son fascinantes. Cuando un proceso llama a `fork()`, el sistema operativo crea un nuevo proceso hijo que es una copia exacta del padre. Ambos procesos continúan ejecutándose desde el punto inmediatamente después del llamado a `fork()`, pero el valor de retorno es diferente en cada uno, permitiendo que ambos identifiquen su rol. En el proceso padre, `fork()` retorna el PID del hijo recién creado. En el proceso hijo, `fork()` retorna 0. Si ocurre un error, `fork()` retorna -1.

Una optimización crucial en implementaciones modernas es *Copy-on-Write* (CoW). En lugar de copiar inmediatamente todas las páginas de memoria del padre al hijo, ambos procesos inicialmente comparten las mismas páginas físicas marcadas como read-only. Solo cuando alguno intenta modificar una página, el sistema operativo hace una copia real de esa página. Esto hace que `fork()` sea extremadamente eficiente, especialmente cuando el hijo inmediatamente ejecuta un `exec()`.

El algoritmo simplificado de `fork()` revela su funcionamiento interno:

```
1 1. Asignar nuevo PID al proceso hijo
2 2. Copiar PCB del padre al hijo
3 3. Copiar espacio de direcciones (CoW)
4 4. Agregar hijo a la tabla de procesos
5 5. Retornar:
6   - En el padre: PID del hijo (> 0)
7   - En el hijo: 0
8   - Error: -1
```

El diseño de `fork()` refleja la filosofía Unix de "hacer una cosa y hacerla bien". Separa la creación del proceso de la carga de un nuevo programa, dando flexibilidad máxima. Entre el `fork()` y un eventual `exec()`, el proceso hijo puede configurar su entorno, redireccionar archivos, cerrar descriptores, cambiar su directorio de trabajo, etc.

5.4.2. Carga de Programas: `exec()`

Mientras `fork()` crea procesos, la familia de funciones `exec()` transforma procesos existentes cargando un nuevo programa. Esta separación de responsabilidades es un diseño brillante que habilita capacidades poderosas.

Los pasos de `exec()` son sistemáticos y destructivos para el proceso que lo invoca. Primero, verifica que el archivo ejecutable existe y que el proceso tiene permisos para ejecutarlo. Luego, lee los headers del archivo (formato ELF en Linux, Mach-O en macOS, PE en Windows) para entender su estructura. A continuación, libera toda la memoria anterior del proceso, destruyendo completamente el código, datos, heap y stack previos. Carga los nuevos segmentos del programa (text,

data, bss) en memoria e inicializa un nuevo stack con los argumentos proporcionados. Finalmente, transfiere el control al punto de entrada del nuevo programa.

Un detalle crítico: `exec()` NO crea un nuevo proceso, transforma el proceso actual. El PID permanece igual, el PPID permanece igual, y muchos atributos del proceso se conservan (como file descriptors que no tienen el flag `FD_CLOEXEC`). El único cambio drástico es que el código en ejecución es completamente diferente.

Cuando ejecutás un comando en tu shell como `ls -l`, el shell primero hace `fork()` para crear un proceso hijo, y luego ese hijo hace `exec("/bin/ls", "ls", "-l", NULL)` para transformarse en el comando `ls`. El shell (proceso padre) espera con `wait()` a que el hijo termine. Esta es la mecánica fundamental de cómo funciona cualquier shell Unix.

5.4.3. Terminación de Procesos

Los procesos pueden terminar de dos maneras fundamentalmente diferentes: normalmente o anormalmente. Cada una tiene implicaciones distintas para el sistema.

La **terminación normal** ocurre cuando un proceso completa su tarea exitosamente. Puede llamar explícitamente a `exit(status)` con un código de salida que indica el resultado de su ejecución, o puede simplemente ejecutar un `return` en la función `main()`, que el compilador traduce automáticamente a una llamada a `exit()` con el valor retornado.

La **terminación anormal** sucede cuando algo sale mal. Las señales son el mecanismo principal: `SIGKILL` termina el proceso inmediatamente sin posibilidad de cleanup, `SIGSEGV` indica un acceso inválido a memoria, `SIGABRT` es enviada cuando el programa llama a `abort()`, etc. Estas terminaciones a menudo indican errores de programación o condiciones excepcionales.

Después de la terminación, el proceso entra en estados especiales que merecen atención particular. Un **proceso zombie** es aquel que ha terminado completamente su ejecución pero cuyo PCB permanece en el sistema porque el proceso padre aún no ha leído su código de salida mediante `wait()`. Un **proceso huérfano** es aquel cuyo padre murió antes que él, dejándolo sin supervisor.

5.4.4. Procesos Zombies y Huérfanos

Estos dos estados especiales representan casos extremos en el ciclo de vida de los procesos, y entenderlos es crucial para programación de sistemas robusta.

Un **proceso zombie** se define formalmente como un proceso que ha completado su ejecución pero cuyo PCB permanece en la tabla de procesos. La causa es simple pero tiene consecuencias: el proceso padre no ha llamado a `wait()` para leer el exit status del hijo. En el output de comandos como `ps`, estos procesos aparecen como “defunct” o con estado “Z”. El problema con los zombies

es que consumen entradas en la tabla de procesos, un recurso finito. Si un programa crea muchos hijos sin hacer `wait()`, eventualmente puede agotar este recurso.

Ejemplo de Zombie:

```
1 if (fork() == 0) {
2     // Hijo termina rápidamente
3     exit(42);
4 }
5 // Padre NO hace wait() y continúa ejecutando
6 sleep(60); // Hijo queda zombie por 1 minuto
```

La solución a los zombies tiene dos enfoques principales. El padre debe llamar a `wait()` o `waitpid()` oportunamente para leer el estado de terminación de sus hijos. Alternativamente, puede configurar un handler para la señal `SIGCHLD` que haga el cleanup automáticamente. Es importante notar que ejecutar kill sobre un proceso zombie no tiene efecto: ya está muerto, solo esperando que su padre lea su estado final.

Un proceso zombie no consume memoria ni CPU, solo una entrada en la tabla de procesos. Sin embargo, en un sistema con miles de procesos, esta puede ser una fuga de recursos significativa. Los zombies son síntoma de programación descuidada, no un bug del sistema operativo.

Un **proceso huérfano** tiene una historia diferente. Ocurre cuando el padre de un proceso muere antes que el hijo, dejándolo sin supervisor. El sistema operativo no permite esta orfandad permanente: automáticamente reasigna el proceso huérfano como hijo del proceso `init` (PID 1, o `systemd` en sistemas modernos). Este proceso especial adopta a todos los huérfanos del sistema. El comportamiento del huérfano no cambia; continúa ejecutándose normalmente. Cuando eventualmente termina, `init` hace `wait()` automáticamente, evitando que se convierta en zombie.

Ejemplo de Huérfano:

```
1 if (fork() == 0) {
2     // Hijo duerme por mucho tiempo
3     sleep(300);
4     exit(0);
5 }
6 // Padre termina inmediatamente
7 exit(0);
8 // Hijo queda huérfano, adoptado por init
```

Los procesos huérfanos son en realidad inofensivos e incluso útiles. Los daemons (servicios que corren en segundo plano) a menudo se vuelven huérfanos deliberadamente: el proceso inicial hace `fork()`, configura el hijo como daemon, y termina, dejando al daemon como huérfano que `init` adopta.

5.5. Código en C

5.5.1. Creación Básica de Procesos

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7      pid_t pid;
8      int status;
9
10     printf("Antes del fork - PID: %d\n", getpid());
11
12     // Crear proceso hijo
13     pid = fork();
14
15     if (pid == -1) {
16         // Error en fork()
17         perror("fork failed");
18         exit(1);
19     }
20     else if (pid == 0) {
21         // Código del HIJO
22         printf("Soy el hijo - PID: %d, PPID: %d\n",
23             getpid(), getppid());
24
25         // Simular trabajo
26         sleep(2);
27         printf("Hijo terminando...\n");
28         exit(42); // Código de salida
29     }
30     else {
31         // Código del PADRE
32         printf("Soy el padre - PID: %d, hijo PID: %d\n",
33             getpid(), pid);
34
35         // Esperar al hijo
36         wait(&status);
37
38         // Analizar cómo terminó el hijo
39         if (WIFEXITED(status)) {
40             printf("Hijo terminó normalmente con código: %d\n",
41                 WEXITSTATUS(status));
42         }
43     }
44
45     printf("Proceso %d terminando\n", getpid());
46     return 0;
47 }
```

Este ejemplo demuestra todos los elementos fundamentales del uso de `fork()`. La línea con `getpid()` muestra el PID del proceso antes del `fork`, estableciendo un punto de referencia. El llamado a `fork()` es el momento crucial donde un proceso se convierte en dos.

El manejo de errores en las líneas 15-18 es esencial: `fork()` puede fallar si el sistema ha alcanzado límites de procesos o si no hay recursos disponibles. Las líneas 19-26 contienen código que SOLO ejecuta el proceso hijo, identificado porque `fork()` retornó 0. El hijo puede llamar a `getppid()` para obtener el PID de su padre, confirmando la relación padre-hijo.

El código del padre en las líneas 27-38 es igualmente importante. La llamada a `wait(&status)` es bloqueante: el padre se suspende hasta que el hijo termine. Las macros `WIFEXITED()` y `WEXITSTATUS()` extraen información del status: si el hijo terminó normalmente y cuál fue su código de salida.

La última línea `printf("Proceso %d terminando", getppid())` se ejecutará dos veces: una por el padre y otra por el hijo. Sin embargo, verás que la línea del hijo aparece antes de que el padre imprima el análisis del status, porque el padre estaba bloqueado en `wait()`.

5.5.2. Usando `exec()` para cargar programas

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7      pid_t pid;
8
9      pid = fork();
10
11     if (pid == -1) {
12         perror("fork");
13         exit(1);
14     }
15     else if (pid == 0) {
16         // HIJO: ejecutar comando "ls -l"
17         printf("Hijo antes de exec - PID: %d\n", getppid());
18
19         // execl: último parámetro debe ser NULL
20         execl("/bin/ls", "ls", "-l", NULL);
21
22         // Si llegamos aquí, exec falló
23         perror("exec failed");
24         exit(1);
25     }
26     else {
27         // PADRE: esperar al hijo
28         int status;
29         wait(&status);
30
31         printf("Comando terminó con status: %d\n",
32               WEXITSTATUS(status));
33     }
34
35     return 0;
```

```
36 }
```

Este código ilustra el patrón clásico fork-exec que todos los shells utilizan. El proceso padre hace `fork()` para crear un hijo que se sacrificará transformándose en otro programa. En la línea 18, antes del `exec()`, imprimimos el PID del hijo, demostrando que existe como proceso independiente.

La llamada a `exec()` en la línea 20 es el momento de transformación. El primer argumento es el path completo al ejecutable, el segundo es el `argv[0]` que el programa verá (por convención, el nombre del programa), seguido de los argumentos reales, y terminando obligatoriamente con `NULL`.

Las líneas 22-24 son código que idealmente nunca se ejecuta. Si `exec()` tiene éxito, el proceso se transforma completamente y este código deja de existir. Solo si `exec()` falla (archivo no encontrado, sin permisos, formato inválido) se ejecutarán estas líneas para reportar el error.

El padre espera pacientemente con `wait()`, y cuando el comando termina, examina su código de salida. Es notable que el padre puede esperar al hijo con `wait()` incluso después de que hizo `exec()`: el PID del hijo no cambió, solo su contenido.

5.5.3. Manejo de múltiples hijos

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  #define NUM_HIJOS 3
7
8  int main() {
9      pid_t hijos[NUM_HIJOS];
10     int i, status;
11
12     // Crear múltiples hijos
13     for (i = 0; i < NUM_HIJOS; i++) {
14         hijos[i] = fork();
15
16         if (hijos[i] == -1) {
17             perror("fork");
18             exit(1);
19         }
20         else if (hijos[i] == 0) {
21             // Código del hijo
22             printf("Hijo %d - PID: %d iniciando\n", i, getpid());
23
24             // Simular trabajo variable
25             sleep(i + 1);
26
27             printf("Hijo %d terminando\n", i);
28             exit(i); // Cada hijo retorna su número
29         }
```

```
30     // El padre continúa el loop
31 }
32
33 // Padre espera a TODOS los hijos
34 printf("Padre esperando a %d hijos...\n", NUM_HIJOS);
35
36 for (i = 0; i < NUM_HIJOS; i++) {
37     pid_t pid_terminado = wait(&status);
38
39     printf("Hijo PID %d terminó con código %d\n",
40           pid_terminado, WEXITSTATUS(status));
41 }
42
43 printf("Todos los hijos terminaron\n");
44 return 0;
45 }
```

Este ejemplo demuestra cómo manejar varios hijos simultáneamente, un escenario común en servidores y aplicaciones paralelas. El loop en las líneas 13-30 crea tres hijos secuencialmente, pero hay un detalle sutil: después de cada `fork()`, AMBOS procesos (padre e hijo) continúan ejecutando el loop. Sin embargo, el hijo encuentra `hijos[i] == 0` e inmediatamente ejecuta su código y termina, evitando crear sus propios hijos.

Cada hijo simula trabajo diferente (línea 25) durmiendo por `i + 1` segundos, lo que significa que terminarán en orden diferente al que fueron creados. Esto es crucial para el siguiente punto.

El segundo loop (líneas 36-41) es donde el padre recolecta a sus hijos. Nota que usa `wait()` sin especificar qué hijo esperar, lo que significa que `wait()` retorna para el primer hijo que termine, sin importar su orden de creación. La variable `pid_terminado` contiene el PID del hijo que terminó, permitiendo al padre identificarlo.

Si necesitás esperar a un hijo específico, usá `waitpid(pid, &status, 0)` en lugar de `wait(&status)`. Esto bloquea hasta que el hijo específico con ese PID termine, ignorando otros hijos que pudieran terminar antes.

5.5.4. Comunicación entre procesos: pipes

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main() {
8      int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
9      pid_t pid;
10     char buffer[100];
11
12     // Crear pipe antes del fork
```

```
13     if (pipe(pipefd) == -1) {
14         perror("pipe");
15         exit(1);
16     }
17
18     pid = fork();
19
20     if (pid == -1) {
21         perror("fork");
22         exit(1);
23     }
24     else if (pid == 0) {
25         // HIJO: escritor del pipe
26         close(pipefd[0]); // Cerrar extremo de lectura
27
28         char *mensaje = "Hola desde el hijo!";
29         write(pipefd[1], mensaje, strlen(mensaje) + 1);
30
31         close(pipefd[1]); // Cerrar extremo de escritura
32         exit(0);
33     }
34     else {
35         // PADRE: lector del pipe
36         close(pipefd[1]); // Cerrar extremo de escritura
37
38         // Leer mensaje del hijo
39         ssize_t bytes_leidos = read(pipefd[0], buffer, sizeof(buffer));
40
41         if (bytes_leidos > 0) {
42             printf("Padre recibió: %s\n", buffer);
43         }
44
45         close(pipefd[0]); // Cerrar extremo de lectura
46         wait(NULL);       // Esperar al hijo
47     }
48
49     return 0;
50 }
```

Los pipes son el mecanismo más simple de IPC (Inter-Process Communication) en Unix, perfecto para comunicación unidireccional entre procesos relacionados. Este ejemplo muestra el flujo completo. La llamada a `pipe(pipefd)` en la línea 13 crea el pipe ANTES del fork, lo cual es esencial. Esto asegura que tanto padre como hijo tengan acceso a los mismos file descriptors del pipe. El array `pipefd` recibe dos descriptors: `pipefd[0]` es el extremo de lectura, `pipefd[1]` es el extremo de escritura.

Después del fork, cada proceso cierra el extremo que no va a usar (líneas 25 y 35). Esto no es opcional: es crítico para el correcto funcionamiento. Si el hijo no cierra `pipefd[0]`, el pipe nunca indicará EOF cuando el escritor termine. Si el padre no cierra `pipefd[1]`, podría bloquearse indefinidamente esperando datos que nunca llegarán.

El hijo escribe su mensaje (línea 28) y cierra su extremo de escritura (línea 30). El padre lee del pipe (línea 38) y el sistema garantiza que recibirá exactamente los bytes que el hijo escribió, en orden.

La llamada a `read()` es bloqueante: si no hay datos disponibles, el padre espera.

Los pipes tienen un buffer limitado (típicamente 4KB-64KB). Si un proceso escribe más datos de los que el buffer puede contener sin que nadie los lea, `write()` se bloqueará. Esto puede causar deadlocks si no se maneja correctamente la comunicación bidireccional.

5.6. Casos de Estudio

5.6.1. Caso de Estudio 1: Análisis de `fork()` múltiple

Este tipo de ejercicio aparece frecuentemente en exámenes porque revela comprensión profunda de cómo opera `fork()`. El desafío es rastrear mentalmente la multiplicación de procesos.

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int x = 5;
6
7      fork();
8      fork();
9      x++;
10
11     printf("PID: %d, x = %d\n", getpid(), x);
12     return 0;
13 }
```

La resolución requiere seguir la ejecución paso a paso, prestando atención a que cada proceso continúa ejecutando el código secuencialmente. Comenzamos con un solo proceso donde `x = 5`. El primer `fork()` crea un hijo, resultando en dos procesos ejecutando la misma instrucción siguiente. Cuando ambos ejecutan el segundo `fork()`, cada uno crea su propio hijo, llevando el total a cuatro procesos.

El incremento `x++` ocurre en cada uno de los cuatro procesos independientemente. Recordá que después de un `fork()`, padre e hijo tienen copias separadas de todas las variables. Modificar `x` en un proceso no afecta a los otros. Cada proceso incrementa su propia copia de `x` de 5 a 6.

El `printf()` final se ejecuta cuatro veces, una por cada proceso, mostrando cuatro PIDs diferentes pero el mismo valor de `x = 6`. Los PIDs serán asignados por el sistema operativo y variarán en cada ejecución.

La fórmula general 2^n procesos con `n` forks secuenciales funciona porque cada `fork` dobla el número de procesos existentes. Con 0 forks hay 1 proceso. Con 1 fork hay 2 procesos. Con 2 forks hay 4 procesos. Con 3 forks hay 8 procesos, y así sucesivamente.

Un error común es pensar que el segundo `fork()` solo lo ejecuta el padre original. En realidad, después del primer `fork()`, HAY DOS PROCESOS ejecutando el código, y ambos llegan al segundo `fork()` y lo ejecutan.

5.6.2. Caso de Estudio 2: Problema de procesos zombie

Este caso ilustra un anti-patrón común: crear procesos y olvidarse de ellos. Es un bug de recursos que puede degradar seriamente un sistema con el tiempo.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     int i;
7
8     for (i = 0; i < 5; i++) {
9         if (fork() == 0) {
10             // Hijo termina rápidamente
11             printf("Hijo %d terminando\n", i);
12             exit(0);
13         }
14     }
15
16     // Padre NO hace wait()
17     printf("Padre durmiendo 30 segundos...\n");
18     sleep(30); // Mientras tanto, ejecutar: ps aux | grep Z
19
20     return 0;
21 }
```

El problema es evidente pero sutil. El loop crea cinco hijos rápidamente. Cada hijo imprime un mensaje y termina inmediatamente con `exit(0)`. Mientras tanto, el padre no hace ningún `wait()`, sino que simplemente duerme por 30 segundos. Durante esos 30 segundos, los cinco hijos están muertos pero sus PCBs permanecen en la tabla de procesos como zombies.

Podés verificar esto ejecutando el programa y, mientras el padre duerme, usar `ps aux | grep Z` en otra terminal. Verás cinco procesos con estado “Z” o “defunct”. Estos procesos no consumen memoria ni CPU, pero ocupan slots en la tabla de procesos.

La Solución 1 es directa: después del loop de creación, agregar un loop que haga `wait()` cinco veces. Esto bloqueará al padre hasta que los cinco hijos terminen, pero asegura que sus recursos se liberen correctamente.

```
1 // Después del bucle
2 for (i = 0; i < 5; i++) {
3     wait(NULL);
4 }
```

La Solución 2 usa un enfoque más elegante para procesos que no necesitan conocer el exit status de sus hijos. Al establecer el handler de `SIGCHLD` a `SIG_IGN`, le decimos al kernel que automáticamente limpie los hijos terminados sin crear zombies. Esto se llama “auto-reaping”.

```
1 #include <signal.h>
2
3 int main() {
4     signal(SIGCHLD, SIG_IGN); // Auto-reaping de hijos
5     // ... resto del código
6 }
```

Ignorar `SIGCHLD` no es equivalente a no hacer nada. El comportamiento default de `SIGCHLD` es *ser ignorado* (no terminar el proceso), pero eso NO previene zombies. Solo establecer explícitamente `signal(SIGCHLD, SIG_IGN)` activa el auto-reaping. Esta distinción confunde a muchos programadores.

5.7. Síntesis

5.7.1. Puntos Clave para Parcial

Esta sección destila los conceptos que más frecuentemente aparecen en exámenes y que forman la base conceptual del capítulo.

La diferencia entre *proceso* y *programa* es fundamental: un programa es código estático en disco, mientras que un proceso es esa código ejecutándose con contexto completo (registros, memoria, archivos abiertos, etc.). El PCB es la estructura de datos que materializa esta diferencia, conteniendo absolutamente todo lo que el sistema operativo necesita saber sobre un proceso.

Los *estados de un proceso* no son arbitrarios sino que reflejan las realidades del hardware y la concurrencia: `NEW` cuando está siendo creado, `READY` cuando está listo pero esperando CPU, `RUNNING` cuando está ejecutándose, `BLOCKED` cuando espera un evento externo, y `TERMINATED` cuando ha finalizado. Los modelos extendidos agregan estados `SUSPENDED` para manejar swapping.

Las *syscalls* fundamentales tienen comportamientos que debés memorizar. `fork()` retorna el PID del hijo al padre, 0 al hijo, y -1 en caso de error. `exec()` reemplaza completamente la imagen del proceso pero mantiene el mismo PID. `wait()` bloquea al padre hasta que un hijo termine, evitando procesos zombie. `getpid()` y `getppid()` permiten a un proceso conocer su identidad y ascendencia.

Las diferencias entre modelos de procesamiento se prestan a preguntas de opción múltiple. La monoprogramación ejecuta un programa por vez, desperdiciando CPU durante I/O. La multiprogramación mantiene múltiples programas en memoria, alternando entre ellos para maximizar uso

de CPU. La multitarea extiende esto con preemption, permitiendo interrumpir procesos arbitrariamente. El multiprocesamiento agrega múltiples CPUs físicos para verdadero paralelismo.

Las *estructuras del SO* son interconectadas: la tabla de procesos es el punto de entrada que referencia a las tablas de memoria, I/O, y archivos. La imagen del proceso en memoria se divide en segmentos especializados: text (código read-only), data (variables globales), BSS (datos no inicializados), heap (memoria dinámica creciendo hacia arriba), y stack (variables locales creciendo hacia abajo). Los diagramas de estado formalizan las transiciones posibles. El modelo de 5 estados incluye: admit (NEW→READY), dispatch (READY→RUNNING), timeout/preempt (RUNNING→READY), block (RUNNING→BLOCKED), wakeup (BLOCKED→READY), y exit (RUNNING→TERMINATED). El modelo de 7 estados agrega estados suspended y transiciones de swapping.

5.7.2. Errores Comunes y Tips

Identificar errores típicos ayuda a evitarlos en código propio y reconocerlos en exámenes. No verificar el retorno de `fork()` es el error más básico pero sorprendentemente común. Sin verificación, el código del padre y del hijo se mezcla indistinguiblemente.

No cerrar extremos no usados de pipes causa bloqueos sutiles. Si el proceso que lee no cierra el extremo de escritura, `read()` nunca retornará EOF. Si el escritor no cierra el extremo de lectura, desperdicia recursos y puede causar condiciones de carrera.

No hacer `wait()` de los hijos crea zombies que acumulan recursos. En un sistema de producción, esto puede eventualmente agotar la tabla de procesos, causando que `fork()` falle para todos los usuarios. Asumir orden de ejecución entre padre e hijo es un error conceptual profundo. Después de un `fork()`, el orden de ejecución es completamente no determinístico. Nunca escribas código que dependa de que el padre o el hijo ejecute primero.

Confundir grado de multiprogramación con multiprocesamiento revela falta de comprensión terminológica. Grado de multiprogramación = número de procesos en memoria. Multiprocesamiento = número de CPUs físicos. Son conceptos ortogonales: *podés tener multiprogramación en una sola CPU, o multiprocesamiento con bajo grado de multiprogramación.*

No distinguir entre estados blocked y suspended es otro error conceptual. Un proceso BLOCKED está en memoria, esperando un evento. Un proceso BLOCKED/SUSPENDED está en disco, esperando un evento. La diferencia afecta dramáticamente el tiempo de respuesta cuando el evento ocurre.

Un ejercicio mental útil: imaginá un proceso que hace `read()` de un socket de red. Si está en BLOCKED, cuando lleguen los datos puede comenzar a ejecutarse en milisegundos. Si está en BLOCKED/SUSPENDED, primero debe ser swapeado de vuelta a memoria, lo que puede tomar decenas o cientos de milisegundos adicionales.

5.7.3. Conexión con Próximos Temas

Los procesos no son un tema aislado sino la fundación sobre la cual se construyen todos los conceptos subsiguientes del curso.

El capítulo de **Planificación** explora en profundidad las transiciones `READY` → `RUNNING` → `READY` que apenas tocamos aquí. Los tres niveles de planificadores (long-term, medium-term, short-term) deciden el flujo de procesos a través del sistema, y los algoritmos de scheduling (FIFO, SJF, Round Robin, prioridades) determinan qué proceso de la ready queue ejecutar y por cuánto tiempo.

Los **Hilos** representan múltiples flujos de ejecución dentro de un solo proceso. Comparten el mismo espacio de direcciones (segmentos text, data, heap) pero tienen stacks separados, permitiendo paralelismo real en sistemas multiprocesador sin el overhead de procesos completos.

La **Sincronización** se vuelve necesaria cuando procesos o hilos comparten recursos. Las race conditions surgen cuando múltiples flujos de ejecución acceden datos compartidos concurrentemente sin coordinación. Semáforos, mutexes, y monitores son las herramientas para resolver estos problemas.

El **Interbloqueo** es un problema exclusivo de sistemas concurrentes: procesos pueden bloquearse mutuamente esperando recursos que otros poseen. Las cuatro condiciones necesarias (exclusión mutua, hold-and-wait, no preemption, espera circular) y los algoritmos de prevención/detección se estudian en detalle.

La **Gestión de Memoria** profundiza en cómo cada proceso obtiene su espacio de direcciones virtual aparentemente privado. El PCB mantiene información crucial de memory management (page tables, segmentos), y el swapping de estados `SUSPENDED` mueve procesos entre memoria y disco según políticas complejas.

El **Sistema de Archivos** se conecta porque los procesos acceden archivos mediante file descriptors mantenidos en el PCB. La herencia de descriptors en `fork()` y su cierre/preservación en `exec()` habilitan patrones poderosos como redirección de I/O en shells.

5.7.4. Preguntas de Reflexión

Estas preguntas van más allá de la memorización, requiriendo síntesis de múltiples conceptos.

¿Por qué `fork()` + `exec()` en lugar de una sola syscall “`create_process()`”? Esta separación proporciona flexibilidad extraordinaria. Entre el `fork()` y el `exec()`, el proceso hijo puede configurar su entorno: cambiar su working directory, modificar variables de entorno, redirigir `stdin/stdout/stderr`, cerrar file descriptors innecesarios, cambiar su user/group ID, etc. Esta flexibilidad habilita shells sofisticados, pipes, redirección, y job control. Además, refleja la filosofía Unix de herramientas pequeñas con funciones únicas que se combinan poderosamente.

¿Qué pasaría si no existieran los niveles de privilegio (modo kernel/usuario)? El sistema sería fundamentalmente inseguro e inestable. Cualquier programa podría acceder directamente a hardware, corromper memoria de otros procesos, leer archivos de otros usuarios, o incluso modificar el código del sistema operativo. No habría aislamiento, protección, ni garantías de seguridad. Un solo bug en cualquier programa podría colapsar todo el sistema.

¿Por qué el SO mantiene procesos zombie en lugar de eliminarlos inmediatamente? El padre necesita poder leer el exit status del hijo para conocer el resultado de su ejecución. Además, el sistema operativo mantiene información de accounting (tiempo de CPU usado, memoria máxima, etc.) que el padre puede necesitar. Eliminar inmediatamente el proceso destruiría esta información. Finalmente, mantener la consistencia del modelo padre-hijo: el padre debe poder hacer `wait()` en cualquier momento después de que el hijo termine, sin condiciones de carrera.

¿Cuál es la ventaja real de la multiprogramación si solo hay una CPU? Aunque parezca que dividir el tiempo de CPU entre múltiples programas simplemente hace que todos vayan más lentos, la realidad es diferente. Los programas típicamente pasan la mayoría de su tiempo esperando I/O (teclado, red, disco). Con multiprogramación, mientras un proceso espera I/O, otro puede usar la CPU productivamente. Esto puede aumentar la utilización de CPU del 5-10 % (monoprogramación) al 70-90 % (multiprogramación), y mejora dramáticamente el tiempo de respuesta percibido por usuarios en sistemas interactivos.

Pregunta de reflexión final: Si entendiste este capítulo, deberías poder explicar por qué cuando terminás un programa con Ctrl+C en la terminal, no se "rompe" el resto del sistema. ¿Cuál es el mecanismo de aislamiento?

Respuesta: Cada programa ejecuta en un proceso separado con su propio espacio de direcciones protegido por la MMU (Memory Management Unit). Ctrl+C envía la señal SIGINT solo al proceso en foreground del terminal, no afecta otros procesos ni al kernel. El sistema operativo mantiene aislamiento estricto mediante protección de memoria por hardware (cada proceso solo puede acceder a su propio espacio de direcciones) y separación de PCBs (el estado de cada proceso está completamente independiente). Incluso si el proceso mata termina de manera anormal (segmentation fault, abort), solo ese proceso muere, el resto del sistema continúa normalmente.

Próximo capítulo: Planificación de Procesos - ¿Cómo decide el SO cuál proceso ejecutar y por cuánto tiempo? Exploraremos algoritmos de scheduling, métricas de rendimiento como tiempo de respuesta y throughput, y casos reales de scheduling en sistemas operativos modernos.

6 Planificación de Procesos

6.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Explicar por qué es necesaria la planificación de procesos en sistemas multiprogramados
- Distinguir entre planificación a corto, mediano y largo plazo
- Analizar algoritmos de planificación: FCFS, SJF, SRT, Round Robin, VRR, HRRN, prioridades y feedback
- Determinar las características de cada algoritmo: preemptivo/no preemptivo, overhead, starvation, aging
- Calcular métricas de rendimiento: tiempo de espera y tiempo de respuesta
- Resolver ejercicios con diagramas de Gantt considerando interrupciones y prioridades
- Evaluar qué algoritmo es más apropiado según el tipo de sistema
- Aplicar criterios de desempate cuando múltiples procesos compiten simultáneamente

6.2. Introducción y Contexto

6.2.1. ¿Por qué necesitamos planificación?

Imaginemos una cafetería con un solo barista y 10 clientes esperando. ¿En qué orden debe atender a los clientes? Esta pregunta, aparentemente simple, esconde un problema de optimización complejo. Podría atender por orden de llegada, lo cual parece justo. O quizás debería priorizar los pedidos más rápidos para maximizar la cantidad de clientes atendidos por hora. Tal vez tenga sentido un sistema de turnos rotativos, o dar prioridad a clientes VIP.

Cada estrategia tiene ventajas y desventajas, y lo que funciona bien en un contexto puede ser desastroso en otro. Este mismo dilema se presenta en los sistemas operativos cuando deben decidir qué proceso ejecutar en el CPU.

6.2.2. El problema fundamental

En un sistema con múltiples procesos pero un solo CPU (o pocos núcleos), el sistema operativo enfrenta tres decisiones críticas que debe tomar miles de veces por segundo: qué proceso ejecu-

tar, cuándo cambiar de proceso, y por cuánto tiempo cada proceso debe mantener el control del CPU.

La planificación de procesos es el mecanismo mediante el cual el sistema operativo decide qué proceso de la cola de listos (READY) debe ejecutarse a continuación en el CPU.

Estas decisiones impactan directamente en el rendimiento del sistema, la experiencia del usuario y la utilización eficiente de los recursos disponibles. Un algoritmo de planificación mal diseñado puede hacer que aplicaciones interactivas se sientan lentas y poco responsivas, mientras que un algoritmo bien ajustado puede hacer que un sistema parezca mucho más rápido de lo que realmente es.

6.2.3. Objetivos conflictivos

Los algoritmos de planificación buscan optimizar múltiples métricas que frecuentemente entran en conflicto entre sí. Maximizar el throughput (cantidad de procesos completados por unidad de tiempo) a menudo significa darle prioridad a procesos cortos, lo que puede hacer que procesos largos esperen indefinidamente. Ser absolutamente justo con todos los procesos puede resultar en un sistema menos eficiente globalmente.

No existe el algoritmo de planificación "perfecto". Cada algoritmo representa un compromiso entre objetivos en competencia, y la elección apropiada depende del contexto de uso del sistema.

La clave está en entender estos compromisos y elegir el algoritmo más apropiado para cada situación. Un sistema batch que procesa trabajos científicos tiene prioridades muy diferentes a un smartphone que debe responder instantáneamente a la interacción del usuario.

6.3. Conceptos Fundamentales

6.3.1. Niveles de Planificación

El sistema operativo no tiene un solo planificador, sino una jerarquía de tres niveles que operan en diferentes escalas de tiempo. Cada nivel se encarga de decisiones distintas, y juntos forman un sistema cohesivo de gestión de procesos.

6.3.1.1. Planificación a Largo Plazo (Job Scheduler)

El *planificador a largo plazo* decide qué programas nuevos ingresan al sistema, controlando la transición de NEW a READY. Esta decisión se toma con poca frecuencia, típicamente cuando un usuario ejecuta un nuevo programa o cuando se inicia un trabajo batch.

Su función principal es controlar el grado de multiprogramación del sistema, es decir, cuántos procesos pueden coexistir simultáneamente en memoria. Si admite demasiados procesos, el sistema puede saturarse y degradar su rendimiento. Si admite muy pocos, el CPU puede quedarse ocioso esperando trabajo.

Este nivel de planificación es común en sistemas batch tradicionales, pero muchos sistemas operativos modernos como Linux o Windows prácticamente no tienen un planificador a largo plazo explícito: admiten todos los procesos que el usuario solicita, dejando que otros mecanismos gestionen la carga.

6.3.1.2. Planificación a Mediano Plazo (Memory Scheduler)

Operando en una escala temporal de segundos, el planificador a mediano plazo decide qué procesos mantener en memoria principal y cuáles mover temporalmente al disco mediante swapping. Cuando la memoria RAM se llena, este planificador puede suspender procesos inactivos, liberando espacio para procesos más activos.

Este mecanismo es crucial para mantener el balance entre rendimiento y utilización de memoria. Un proceso suspendido no consume RAM, pero reactivarlo requiere traerlo de vuelta desde el disco, una operación costosa en tiempo.

6.3.1.3. Planificación a Corto Plazo (CPU Scheduler)

El planificador a corto plazo (dispatcher) es el corazón del sistema de planificación y el enfoque principal de este capítulo. Opera a velocidades de milisegundos, decidiendo constantemente qué proceso de la cola READY debe ejecutarse en el CPU.

En un sistema típico, el planificador a corto plazo puede tomar decisiones cada 10-100 milisegundos, mientras que el planificador a largo plazo puede tomar decisiones cada varios segundos o incluso minutos.

6.3.2. Tipos de Planificación

La distinción más fundamental entre algoritmos de planificación es si son preemptivos o no preemptivos, una característica que define completamente su comportamiento y casos de uso.

6.3.2.1. Planificación No Preemptiva (No Expropiativa)

En un esquema no preemptivo, una vez que un proceso obtiene el CPU, lo mantiene hasta que voluntariamente lo libera, ya sea porque termina su ejecución o porque se bloquea esperando una operación de I/O. El sistema operativo no puede forzar al proceso a ceder el CPU.

Esta simplicidad tiene ventajas: menor overhead computacional porque hay menos cambios de contexto, y mayor previsibilidad en el tiempo de ejecución. Sin embargo, presenta un problema crítico: un solo proceso mal comportado puede monopolizar el CPU indefinidamente, dejando a todos los demás procesos sin oportunidad de ejecutarse.

6.3.2.2. Planificación Preemptiva (Expropiativa)

Los algoritmos preemptivos otorgan al sistema operativo la capacidad de interrumpir un proceso en ejecución y asignar el CPU a otro proceso. Esta interrupción puede ocurrir por varios motivos: el quantum de tiempo asignado se agotó, llegó un proceso de mayor prioridad, o simplemente el sistema decidió que es momento de darle una oportunidad a otro proceso.

La preemptividad es esencial para sistemas interactivos modernos. Sin ella, un programa que entra en un ciclo infinito podría congelar todo el sistema. Con preemptividad, el sistema operativo mantiene siempre el control final sobre el CPU.

El costo de esta flexibilidad es el overhead de los frecuentes cambios de contexto. Cada vez que el sistema operativo quita el CPU a un proceso para dárselo a otro, debe guardar el estado completo del primer proceso y restaurar el estado del segundo, una operación que consume tiempo.

6.3.3. Métricas de Rendimiento

Para evaluar y comparar algoritmos de planificación necesitamos métricas objetivas. Estas se dividen en dos categorías según su perspectiva.

Las métricas orientadas al sistema incluyen la utilización del CPU (porcentaje de tiempo que el CPU está ejecutando procesos productivos en lugar de estar ocioso) y el throughput (cantidad de procesos que el sistema completa por unidad de tiempo). Estas métricas interesan principalmente a los administradores de sistemas y diseñadores de algoritmos.

Las métricas orientadas al usuario capturan la experiencia percibida. El **tiempo de retorno** (turnaround time) mide cuánto tiempo total transcurre desde que un proceso llega al sistema hasta que termina completamente:

$$t_{\text{retorno}} = t_{\text{terminacion}} - t_{\text{llegada}}$$

El **tiempo de espera** (waiting time) cuantifica cuánto tiempo un proceso pasó esperando en la cola READY, sin ejecutarse:

$$t_{espera} = t_{retorno} - t_{ejecucion}$$

El **tiempo de respuesta** (response time) es particularmente importante en sistemas interactivos, midiendo cuánto tiempo transcurre desde que un proceso llega hasta que comienza su primera ejecución:

$$t_{respuesta} = t_{primera_ejecucion} - t_{llegada}$$

Un usuario percibe la velocidad de un sistema principalmente a través del tiempo de respuesta, no del tiempo de retorno. Un proceso que comienza a ejecutarse inmediatamente pero tarda mucho en terminar puede sentirse más responsivo que uno que termina rápido pero tarda en comenzar.

6.3.4. Ráfagas de CPU y I/O

Los procesos no utilizan el CPU de manera continua. En lugar de eso, alternan entre períodos de uso intensivo del CPU (CPU burst) y períodos esperando operaciones de entrada/salida (I/O burst). Esta distinción es fundamental para diseñar algoritmos de planificación efectivos.

Un proceso CPU-bound se caracteriza por ráfagas de CPU largas y pocas operaciones de I/O. Ejemplos típicos incluyen simulaciones científicas, renderizado de video, o minería de criptomonedas. Estos procesos pueden ejecutarse por segundos o incluso minutos sin bloquearse.

Por otro lado, un proceso I/O-bound tiene ráfagas de CPU muy cortas, bloqueándose frecuentemente para operaciones de disco, red, o interacción con el usuario. Los editores de texto, navegadores web, y la mayoría de aplicaciones interactivas caen en esta categoría.

Un editor de texto pasa la mayor parte del tiempo esperando que el usuario presione una tecla. Cuando esto ocurre, procesa la entrada en milisegundos y vuelve a bloquearse. Su ráfaga de CPU típica puede ser de solo 1-10 milisegundos.

Esta clasificación importa porque un buen algoritmo de planificación debería favorecer procesos I/O-bound para mantener la responsividad del sistema, mientras que los procesos CPU-bound pueden tolerar más latencia sin que el usuario lo note.

6.3.5. Eventos que Provocan Planificación

El planificador no toma decisiones en momentos arbitrarios, sino que se invoca en respuesta a eventos específicos del sistema. Estos eventos tienen diferentes prioridades, lo que determina su orden de procesamiento cuando ocurren simultáneamente.

La **interrupción de reloj** tiene la prioridad más alta. Ocurre cuando el quantum de tiempo asignado a un proceso se agota en un sistema preemptivo. Esta interrupción garantiza que el sistema operativo retome el control periódicamente, evitando que un proceso monopolice el CPU.

Las **interrupciones de finalización de I/O** tienen prioridad media. Cuando un proceso bloqueado esperando I/O recibe sus datos, genera una interrupción que puede tener mayor prioridad que el proceso actualmente en ejecución, especialmente si el proceso que estaba esperando es más prioritario.

Finalmente, las **system calls** representan puntos donde el proceso actual voluntariamente cede el control, generalmente porque necesita bloquearse para I/O. Estas tienen la menor prioridad porque no requieren intervención urgente del sistema operativo.

Cuando múltiples procesos llegan simultáneamente a la cola READY, el criterio de desempate estándar es aplicar primero prioridades (si existen) y luego FCFS (primero en llegar, primero en ser servido) entre procesos de igual prioridad.

6.4. Análisis Técnico

Ahora que establecimos los fundamentos conceptuales, podemos explorar en detalle los algoritmos de planificación más importantes. Cada uno representa diferentes filosofías y compromisos, diseñados para contextos de uso específicos.

6.4.1. First-Come First-Served (FCFS)

El algoritmo más simple imaginable: atender los procesos en el orden exacto en que llegan. FCFS mantiene una cola FIFO (First In, First Out) y ejecuta cada proceso hasta que termina o se bloquea voluntariamente, sin ninguna posibilidad de expropiar el CPU.

Su implementación es trivial: una simple cola enlazada donde nuevos procesos se agregan al final y el planificador siempre toma el proceso del frente. No requiere información sobre duración de procesos ni cálculos complejos de prioridades.

La característica definitoria de FCFS es su justicia absoluta en términos de orden de llegada. No puede haber starvation (inanición) porque todos los procesos eventualmente llegarán al frente de

la cola. El algoritmo es completamente predecible: si conocés el orden de llegada y los tiempos de ejecución, podés calcular exactamente cuándo ejecutará cada proceso.

FCFS es óptimo en un sentido limitado: minimiza la varianza del tiempo de espera. Todos los procesos esperan aproximadamente lo mismo (relativo a su posición en la cola), sin favores ni discriminación.

Sin embargo, FCFS sufre del **efecto convoy**: un proceso largo puede retener todos los procesos cortos que llegan después, resultando en tiempos de espera promedio pobres. Imaginá una fila de supermercado donde alguien con un carrito lleno está adelante de cinco personas con un solo artículo cada una. El tiempo de espera promedio es terrible, aunque técnicamente sea “justo”.

Además, FCFS desperdicia oportunidades de paralelismo. Mientras el CPU ejecuta un proceso CPU-bound largo, múltiples procesos I/O-bound podrían estar realizando sus operaciones de I/O simultáneamente, pero en cambio esperan en la cola sin hacer nada productivo.

6.4.2. Shortest Job First (SJF)

SJF adopta un enfoque radicalmente diferente: en lugar de orden de llegada, ejecuta primero el proceso con menor tiempo de CPU estimado. Este simple cambio tiene consecuencias profundas para el rendimiento del sistema.

Cuando un proceso termina o se bloquea, el planificador examina todos los procesos en la cola READY, identifica el que requiere menos tiempo de CPU, y lo ejecuta hasta su terminación. No hay preemptividad: una vez que un proceso comienza, continúa hasta bloquearse o terminar.

SJF es matemáticamente óptimo para minimizar el tiempo de retorno promedio. Ningún otro algoritmo no preemptivo puede lograr un mejor tiempo de retorno promedio que SJF. Esta es una garantía teórica demostrable.

La intuición detrás de esta optimalidad es simple: ejecutar primero trabajos cortos significa que muchos procesos terminan rápidamente, reduciendo el tiempo promedio de espera. Pensá en la fila del supermercado: si las personas con pocos artículos pasan primero, el tiempo de espera promedio disminuye dramáticamente.

Pero SJF tiene dos problemas fundamentales que limitan su aplicabilidad práctica. Primero, requiere conocer el tiempo de ejecución de cada proceso de antemano, algo imposible en general. El sistema operativo puede estimar tiempos basándose en ráfagas anteriores usando un promedio móvil exponencial:

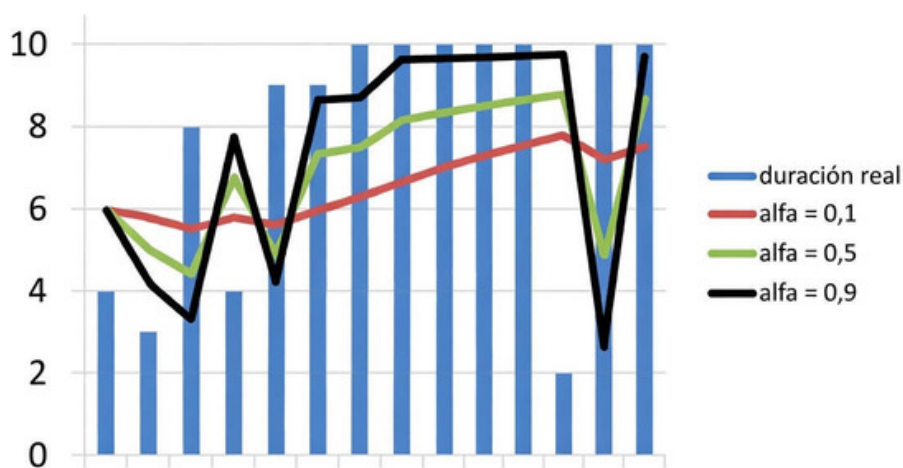
$$\tau_{(n+1)} = \alpha \times t_{(n)} + (1 - \alpha) \times \tau_{(n)}$$

donde $\tau_{(n+1)}$ es el tiempo estimado para la próxima ráfaga, $t_{(n)}$ fue el tiempo real de la ráfaga anterior, y α controla cuánto peso le damos a la historia reciente versus el pasado lejano.

6.4.2.1. El rol crítico de α en la predicción

El parámetro α determina fundamentalmente el comportamiento del predictor. Su valor, siempre entre 0 y 1, establece un balance delicado entre dos extremos:

Cuando α está cerca de 1 (por ejemplo, $\alpha = 0.9$), le damos mucho peso a la observación más reciente. El sistema tiene “memoria corta” y reacciona rápidamente a cambios en el comportamiento del proceso. Si un proceso que normalmente usa 10ms de CPU de repente necesita 100ms, la estimación se ajusta rápidamente. Esto es útil para procesos con comportamiento variable, pero hace que el predictor sea volátil y susceptible a ráfagas anómalas individuales. Por el contrario, cuando α está cerca de 0 (por ejemplo, $\alpha = 0.1$), privilegiamos la historia acumulada. La estimación cambia lentamente, suavizando fluctuaciones temporales. Esto es ideal para procesos con comportamiento estable y predecible, pero significa que el sistema tarda mucho en adaptarse cuando un proceso genuinamente cambia su patrón de uso de CPU. El valor $\alpha = 0.5$ representa un compromiso equilibrado, dando igual peso a la medición reciente y al historial acumulado. Es un punto de partida razonable cuando no conocemos el comportamiento del proceso.



Comparación entre la duración real de los procesos y la estimación de CPU utilizada por SJF para distintos valores de α , mostrando cómo el parámetro influye en la adaptación del algoritmo al comportamiento reciente.

En la práctica, sistemas operativos modernos suelen usar valores de α entre 0.5 y 0.8, favoreciendo levemente la reactividad sobre la estabilidad. Algunos sistemas incluso ajustan α dinámicamente basándose en la variabilidad observada del proceso.

El segundo problema que enfrenta **SJF** es más serio: starvation severa. Un proceso largo puede nunca ejecutarse si constantemente llegan procesos más cortos. En un sistema ocupado, un trabajo de varias horas podría quedar perpetuamente postergado, esperando un momento de calma que nunca llega.

SJF sin mecanismo de aging puede resultar en starvation indefinida para procesos largos. En sistemas de producción, siempre debe implementarse algún mecanismo que incremente gradualmente la prioridad de procesos que esperan mucho tiempo.

6.4.3. Shortest Remaining Time (SRT)

SRT es la versión preemptiva de SJF, llevando su filosofía al extremo. En lugar de considerar solo el tiempo total de ejecución, SRT examina el tiempo restante de cada proceso y siempre ejecuta el que tiene menos tiempo pendiente.

La diferencia crucial es que SRT puede expropiar un proceso en ejecución. Si llega un nuevo proceso cuyo tiempo de CPU es menor que el tiempo restante del proceso actual, SRT inmediatamente realiza un context switch para ejecutar el proceso nuevo.

```
1 // Pseudocódigo simplificado de SRT
2 al_llegar_nuevo_proceso(proceso_nuevo):
3     if proceso_nuevo.tiempo_restante < proceso_actual.tiempo_restante:
4         context_switch(proceso_nuevo)
5     else:
6         agregar_aCola(proceso_nuevo)
```

Esta agresividad en la preempción mejora aún más el tiempo de retorno promedio comparado con SJF. Procesos cortos que llegan tarde no necesitan esperar a que termine el proceso largo actualmente en ejecución; pueden apropiarse del CPU inmediatamente.

Sin embargo, el costo es considerable. El overhead de context switches puede volverse prohibitivo en sistemas con muchos procesos cortos llegando constantemente. Cada cambio de contexto implica guardar registros, actualizar tablas de páginas, invalidar cachés del CPU, y restaurar el estado del nuevo proceso.

Peor aún, la starvation se vuelve aún más severa que en SJF. Un proceso largo puede iniciarse, ejecutarse unos milisegundos, ser expropiado por un proceso corto, volver a iniciar brevemente, ser expropiado nuevamente, y así sucesivamente sin nunca llegar a completarse.

En un servidor web procesando miles de requests cortos por segundo, un proceso de backup que requiere 10 minutos de CPU podría ser constantemente expropiado y tomar horas o días en completarse, aunque técnicamente solo necesita 10 minutos de CPU.

El aging es absolutamente crítico en SRT. Sin él, el algoritmo es prácticamente inutilizable en sistemas de producción. Una estrategia común es incrementar artificialmente la prioridad de procesos

(o decrementar su “tiempo restante” percibido) en proporción al tiempo que han esperado.

6.4.4. Round Robin (RR)

Round Robin representa un cambio filosófico completo. En lugar de intentar optimizar el tiempo de retorno, RR prioriza la justicia (*fairness*) y el tiempo de respuesta. La idea es elegante en su simplicidad: cada proceso recibe un turno fijo de tiempo (*quantum*), y cuando termina su turno, va al final de la cola para esperar su próxima oportunidad.

El algoritmo mantiene una cola circular de procesos READY. El planificador asigna el CPU al proceso al frente de la cola por un máximo de Q unidades de tiempo. Si el proceso termina antes de agotar su quantum, el sistema pasa al siguiente proceso. Si el quantum expira, el proceso actual se mueve al final de la cola y el siguiente proceso obtiene el CPU.

Algoritmo:

```
1 1. Asignar quantum  $Q$  a cada proceso
2 2. Ejecutar proceso por máximo  $Q$  unidades de tiempo
3 3. Si no termina, mover al final de cola READY
4 4. Si se bloquea antes de  $Q$ , no pierde el quantum restante
5 5. Seleccionar siguiente proceso de la cola
```

La magia de Round Robin está en que garantiza progreso para todos los procesos. No importa cuántos procesos haya en el sistema; cada uno recibirá periódicamente su turno de ejecución. Esto elimina completamente la posibilidad de starvation.

El tiempo de respuesta en Round Robin está acotado: un proceso con n procesos en la cola nunca esperará más de $n \times Q$ unidades de tiempo antes de su próxima ejecución. Esta garantía es invaluable para sistemas interactivos.

La selección del quantum es un arte sutil que requiere balance cuidadoso. Un quantum demasiado pequeño (por ejemplo, 1 milisegundo) resulta en overhead excesivo: el sistema pasa más tiempo cambiando entre procesos que realmente ejecutándolos. Un quantum demasiado grande (por ejemplo, 1 segundo) degrada Round Robin prácticamente a FCFS: la mayoría de procesos terminarán antes de agotar su quantum.

La regla práctica tradicional sugiere quantums de 10-100 milisegundos, significativamente mayores que el tiempo de context switch (típicamente 0.1-1 milisegundo en hardware moderno). Esto mantiene el overhead de cambio de contexto bajo 1-10 % del tiempo total de CPU.

Round Robin no es óptimo para tiempo de retorno. De hecho, puede tener uno de los peores tiempos de retorno promedio entre todos los algoritmos. Un proceso que requiere 100 milisegundos de CPU y es el único en el sistema terminará en exactamente 100ms con cualquier algoritmo. Pero con Round Robin compartiendo el CPU con otros 9 procesos ($Q=10\text{ms}$), ese mismo proceso tardará

aproximadamente 1 segundo en completarse: 10ms por turno, 10 turnos total, con 90ms de espera entre cada turno.

Sin embargo, para sistemas interactivos modernos, este compromiso vale la pena. La experiencia del usuario mejora dramáticamente cuando todas las aplicaciones progresan constantemente, incluso si ninguna termina particularmente rápido.

"Atomicidad mata quantum..."

— Ing. Néstor Esquivel

Esta frase, cuando la dijo el profe, sonó casi zen y sin sentido, pero resume un principio crítico que aparece en ejercicios de planificación: **una operación atómica no puede ser interrumpida, ni siquiera cuando expira el quantum.**

Una operación atómica es indivisible: o se ejecuta completa o no se ejecuta. Para garantizar esta propiedad, el sistema debe deshabilitar interrupciones durante su ejecución. Esto significa que el timer interrupt que normalmente causa un context switch al expirar el quantum simplemente no puede ocurrir mientras la operación atómica está en progreso.

En ejercicios de planificación con Round-Robin:

Si un proceso está ejecutando una operación atómica de 15ms y su quantum es de 10ms, el proceso **NO** será expropiado a los 10ms. Continuará hasta completar los 15ms de la operación atómica, y recién entonces el scheduler podrá actuar. El quantum efectivo en ese caso fue de 15ms, no 10ms.

Este comportamiento puede parecer una violación del algoritmo RR, pero es necesario para mantener la corrección del sistema. La atomicidad es una garantía de seguridad que tiene prioridad sobre las políticas de planificación.

6.4.5. Virtual Round Robin (VRR)

Round Robin trata todos los procesos por igual, pero no todos los procesos son iguales. VRR es un refinamiento que reconoce una asimetría fundamental: los procesos I/O-bound generalmente merecen un trato ligeramente preferencial porque responden a interacciones del usuario y liberan el CPU rápidamente.

La innovación de VRR es simple pero efectiva: mantiene dos colas en lugar de una. La **cola READY** funciona como en Round Robin clásico, conteniendo procesos nuevos y procesos que agotaron su quantum completo. La **cola AUXILIARY** contiene procesos que se bloquearon voluntariamente para I/O antes de agotar su quantum.

El planificador siempre prefiere la cola AUXILIARY sobre la cola READY. Cuando un proceso de la

cola AUXILIARY ejecuta, recibe solo su quantum *restante* (no un quantum completo nuevo). Esto recompensa el comportamiento cooperativo: un proceso que se bloqueó rápidamente puede volver al CPU antes que procesos que intentaron usar todo su tiempo.

```

1 Estado inicial: Cola READY = [P1, P2, P3], Q=10ms
2
3 t=0-8: P1 ejecuta 8ms, se bloquea para I/O → va a AUXILIARY con
  quantum_restante=2ms
4 t=8-18: P2 ejecuta 10ms completos → va al final de READY
5 t=18-20: I/O de P1 termina
6 t=20-22: P1 (desde AUXILIARY) ejecuta sus 2ms restantes, termina
7 t=22-32: P3 ejecuta su turno

```

Esta modificación beneficia desproporcionadamente a procesos interactivos y editores de texto que se bloquean frecuentemente esperando input del usuario. Estos procesos obtienen respuesta más rápida sin penalizar significativamente a procesos CPU-bound.

VRR implementa una forma sutil de aging: el quantum restante efectivamente actúa como un boost de prioridad temporal. Procesos que se comportan "bien" bloqueándose voluntariamente reciben trato preferencial la próxima vez que necesiten el CPU.

El costo es complejidad de implementación adicional: el sistema operativo debe rastrear quantum restante para cada proceso y gestionar dos colas separadas. En la práctica, el overhead es modesto y el beneficio en responsividad justifica la complejidad extra.

6.4.6. Highest Response Ratio Next (HRRN)

HRRN intenta capturar lo mejor de SJF y FCFS en un solo algoritmo, balanceando la preferencia por trabajos cortos con un mecanismo de aging automático que previene starvation.

La métrica central es el *response ratio* (ratio de respuesta), calculado para cada proceso como:

$$ResponseRatio = (t_{espera} + t_{servicio}) / t_{servicio}$$

El planificador siempre selecciona el proceso con el mayor response ratio. Miremos qué significa esta fórmula. Un proceso que acaba de llegar tiene tiempo de espera cero, entonces su ratio es simplemente 1.0. A medida que espera, el numerador crece linealmente, aumentando su ratio y por lo tanto su prioridad.

Procesos cortos (Tiempo_servicio pequeño) naturalmente tienen ratios más altos, dándoles preferencia similar a SJF. Pero procesos largos que han esperado mucho tiempo eventualmente tendrán ratios muy altos también, garantizando que eventualmente ejecuten.

Proceso A: $t_{servicio} = 5ms, t_{espera} = 10ms \rightarrow ratio = (10 + 5)/5 = 3.0$

Proceso B: $t_{servicio} = 20ms, t_{espera} = 40ms \rightarrow ratio = (40 + 20)/20 = 3.0$

Ambos tienen el mismo ratio, aunque B es más largo. El tiempo de espera de B compensa su mayor longitud.

Esta fórmula implementa aging de manera elegante y automática. No requiere parámetros arbitrarios ni ajustes manuales: el balance entre favorecimiento de trabajos cortos y prevención de starvation emerge naturalmente de la matemática.

HRRN mantiene el enfoque no preemptivo, ejecutando cada proceso seleccionado hasta su terminación o bloqueo. Esto limita su efectividad para sistemas interactivos altamente responsivos, pero lo hace apropiado para sistemas batch que buscan optimizar throughput mientras mantienen fairness razonable.

El overhead computacional de HRRN es moderado. En cada decisión de planificación, el sistema debe calcular el response ratio para todos los procesos READY. Con cientos de procesos, esto puede volverse costoso. Además, HRRN hereda de SJF la necesidad de estimar tiempos de servicio, lo cual no siempre es preciso.

6.4.7. Planificación por Prioridades

Los algoritmos de prioridad reconocen explícitamente que no todos los procesos son igualmente importantes. El sistema operativo, el administrador, o el propio diseño del sistema pueden asignar diferentes niveles de prioridad, y el planificador simplemente ejecuta el proceso de mayor prioridad disponible.

Las **prioridades estáticas** se asignan cuando un proceso inicia y no cambian durante su vida. Esto es simple y eficiente, apropiado cuando las relaciones de importancia entre procesos son claras y estables. Por ejemplo, procesos del sistema operativo podrían tener prioridad 0 (máxima), aplicaciones de usuario prioridad 50, y tareas de mantenimiento prioridad 100.

Las **prioridades dinámicas** ajustan continuamente basándose en el comportamiento del proceso. Un proceso I/O-bound podría recibir prioridad más alta porque libera el CPU rápidamente. Un proceso que ha esperado mucho tiempo podría ver su prioridad incrementarse gradualmente.

La planificación por prioridades puede implementarse de manera preemptiva o no preemptiva. En el esquema no preemptivo, un proceso ejecuta hasta terminar o bloquearse incluso si llegan procesos de mayor prioridad. En el esquema preemptivo, la llegada de un proceso de mayor prioridad inmediatamente expulsa al proceso actual.

Planificación por prioridades sin aging sufre de starvation severa. Procesos de baja prioridad pueden literalmente nunca ejecutarse en un sistema moderadamente ocupado. Aging no es opcional, es esencial.

El aging en sistemas de prioridad típicamente incrementa la prioridad de un proceso en función de su tiempo de espera:

$$prioridad_{efectiva} = prioridad_{base} + (tiempo_{espera} / factor_{aging})$$

El `factor_aging` controla qué tan rápido crece la prioridad. Un factor pequeño significa que procesos de baja prioridad rápidamente alcanzan prioridad competitiva. Un factor grande significa que deben esperar mucho tiempo antes de volverse relevantes.

Los sistemas de prioridades brillan en contextos de tiempo real donde ciertos procesos tienen deadlines estrictos. Un controlador de motor en un auto autónomo debe ejecutarse cada 10ms sin excepción, mientras que la actualización de la pantalla de entretenimiento puede esperar. Las prioridades proporcionan las garantías necesarias para estos escenarios.

6.4.8. Multilevel Feedback Queue

Multilevel Feedback Queue representa el pináculo de complejidad en planificación de procesos, combinando ideas de múltiples algoritmos en un framework adaptativo sofisticado. El sistema mantiene múltiples colas de prioridad, cada una con su propio algoritmo de planificación, y los procesos pueden moverse dinámicamente entre colas basándose en su comportamiento observado.

Una configuración típica podría tener tres colas:

Cola 0 (prioridad más alta): Round Robin con quantum de 8ms. Todos los procesos nuevos inician aquí. Esta cola captura procesos interactivos y trabajos muy cortos.

Cola 1 (prioridad media): Round Robin con quantum de 16ms. Procesos que agotaron su quantum en Cola 0 descienden aquí. El quantum más largo es apropiado para procesos que claramente necesitan más tiempo de CPU.

Cola 2 (prioridad más baja): FCFS. Procesos que agotaron su quantum en Cola 1 descienden aquí. Estos son claramente trabajos CPU-bound largos; FCFS minimiza el overhead para ellos.

La regla de promoción es crucial: si un proceso se bloquea para I/O *antes* de agotar su quantum, mantiene su cola actual o incluso puede ser promovido a una cola de mayor prioridad. Esto recompensa el comportamiento I/O-bound, que generalmente corresponde a procesos interactivos.

```
1 // Comportamiento adaptativo del sistema
2 Proceso P inicia → Cola 0
```

```

3 P ejecuta 8ms completos → desciende a Cola 1
4 P ejecuta 16ms completos → desciende a Cola 2
5 P se bloquea después de 5ms → mantiene Cola 2
6 P frecuentemente se bloquea → puede ser promovido a Cola 1
7
8 // El sistema aprende qué tipo de proceso es P

```

Multilevel Feedback Queue implementa una forma de machine learning primitivo: observa el comportamiento pasado de cada proceso para predecir su comportamiento futuro y ajustar su tratamiento dinámicamente.

Este enfoque adaptativo es poderoso pero tiene costos. La complejidad de implementación es considerable: el sistema operativo debe rastrear múltiples colas, implementar múltiples algoritmos simultáneamente, aplicar reglas de promoción y demotion, y prevenir starvation mediante aging cuidadoso entre colas. Tuning el sistema también es un desafío. ¿Cuántas colas? ¿Qué quantum para cada una? ¿Cuándo promover procesos? ¿Con qué frecuencia aplicar aging? Cada parámetro afecta el comportamiento global del sistema de maneras a veces contraintuitivas.

A pesar de esta complejidad, muchos sistemas operativos modernos utilizan variantes de Multilevel Feedback Queue porque, cuando está bien tuneado, proporciona excelente rendimiento para cargas de trabajo heterogéneas. Linux, por ejemplo, utiliza el “Completely Fair Scheduler” (CFS) que incorpora ideas similares.

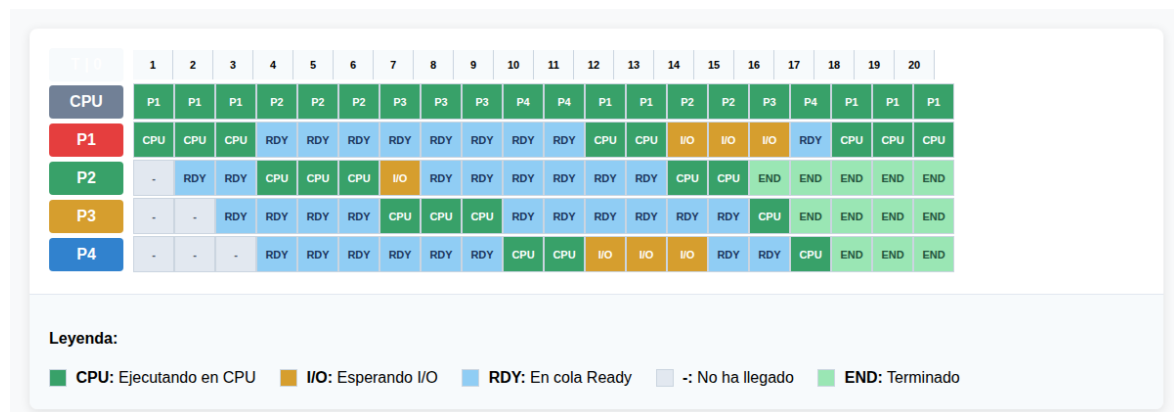
6.5. Casos de Estudio

6.5.1. Planificación con Round Robin

Consideremos cuatro procesos con comportamiento heterogéneo, incluyendo múltiples ráfagas de CPU intercaladas con operaciones de I/O. Este escenario refleja sistemas reales donde los procesos raramente ejecutan hasta completarse sin interrupciones.

	Proceso	Llegada	CPU Burst	I/O Burst	CPU Burst 2
1	P1	0	5	2	3
2	P2	1	3	1	2
3	P3	2	4	–	–
4	P4	3	2	3	1

Aplicaremos Round Robin con un quantum de $Q=3$. La clave está en trackear cuidadosamente el estado de cada proceso: cuánto tiempo de CPU ha consumido en su ráfaga actual, si está bloqueado esperando I/O, y cuándo puede volver a la cola READY.



Desarrollemos el timeline paso a paso. En $t=0$, P1 es el único proceso en el sistema y comienza a ejecutarse. Usará 3 de sus 5 milisegundos de CPU antes de que su quantum expire. Mientras tanto, P2 y P3 llegan al sistema en $t=1$ y $t=2$ respectivamente, ingresando a la cola READY.

En $t=3$, P1 ha agotado su quantum habiendo ejecutado solo 3 de sus 5ms necesarios. Va al final de la cola READY con 2ms de CPU pendientes. P2, que ha estado esperando desde $t=1$, ahora obtiene el CPU. Simultáneamente, P4 llega y se agrega a la cola.

P2 completa su primera ráfaga de CPU (3ms) en $t=6$, justo agotando su quantum. Se bloquea inmediatamente para realizar I/O durante 1ms. P3 ahora obtiene su turno, ejecutando durante 3ms de su ráfaga de 4ms total.

En $t=7$, P2 completa su I/O y vuelve a la cola READY, pero debe esperar su turno detrás de P1 y P4. P3 continúa ejecutando hasta $t=9$, cuando agota su quantum con 1ms de CPU aún pendiente.

Este patrón continúa: procesos ejecutan por su quantum o hasta bloquearse, nuevos procesos completan I/O y vuelven a la cola, y el planificador rota sistemáticamente entre todos los procesos READY.

Calculemos las métricas finales. Los tiempos de terminación son: P1 termina en $t=18$, P2 en $t=14$, P3 en $t=15$, y P4 en $t=19$.

```

1  Tiempos de terminación:
2  P1: 18, P2: 14, P3: 15, P4: 19
3
4  Tiempos de retorno:
5  P1: 18-0 = 18
6  P2: 14-1 = 13
7  P3: 15-2 = 13
8  P4: 19-3 = 16
9  Promedio: (18+13+13+16)/4 = 15
10
11 Tiempos de espera:
12 P1: 18-(5+3) = 10
13 P2: 13-(3+2) = 8
14 P3: 13-4 = 9
15 P4: 16-(2+1) = 13
16 Promedio: (10+8+9+13)/4 = 10
17
18 Tiempo de respuesta (primera ejecución):

```

```

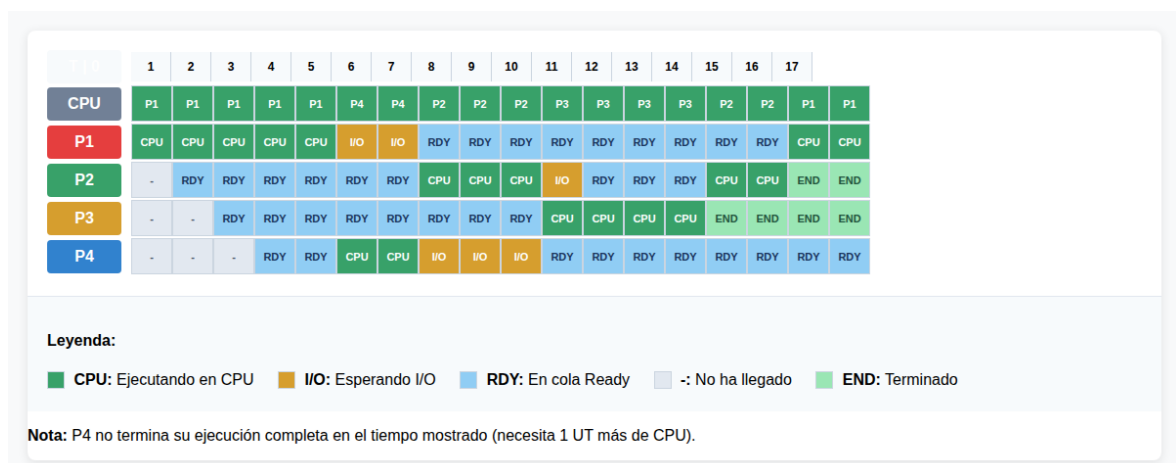
19 P1: 0-0 = 0
20 P2: 3-1 = 2
21 P3: 6-2 = 4
22 P4: 9-3 = 6
23 Promedio: (0+2+4+6)/4 = 3

```

Observá cómo el tiempo de respuesta promedio es excelente (solo 3ms), confirmando que Round Robin proporciona buena responsividad. Sin embargo, el tiempo de retorno promedio (15ms) es relativamente alto porque los procesos se turnan el CPU en lugar de completarse rápidamente.

6.5.2. Caso de Estudio: Planificación con SJF

Ahora analicemos el mismo conjunto de procesos usando SJF no preemptivo. Este algoritmo tomará decisiones completamente diferentes, priorizando la minimización del tiempo de retorno promedio sobre la justicia y la responsividad.



En $t=0$, P1 es el único proceso disponible y ejecuta sus 5ms completos, terminando su primera ráfaga en $t=5$ y bloqueándose para I/O. Durante este tiempo, P2, P3 y P4 han llegado y están esperando. Aquí está la primera decisión interesante de SJF. En $t=5$, tenemos tres procesos en la cola READY con las siguientes ráfagas de CPU pendientes: P2 necesita 3ms, P3 necesita 4ms, y P4 necesita 2ms. SJF selecciona P4 porque tiene la ráfaga más corta.

Pero hay un problema: P1 aún está usando el recurso de I/O. Dependiendo de la implementación del sistema, esto puede o no bloquear a P4. Asumiendo que el I/O es independiente, P4 ejecuta sus 2ms y se bloquea para su propio I/O en $t=7$.

En $t=7$, P1 ha completado su I/O y tiene una segunda ráfaga de CPU de 3ms pendiente. La cola READY ahora contiene: P1 (3ms), P2 (3ms), P3 (4ms). Con dos procesos empatados en tiempo más corto, SJF aplica FCFS como criterio de desempate: P2 llegó antes que P1 volvió de I/O, entonces P2 ejecuta. P2 completa su primera ráfaga en $t=10$, va a I/O brevemente, y vuelve en $t=11$ con su segunda ráfaga de 2ms. Mientras tanto, P3 finalmente obtiene su oportunidad en $t=12$, ejecutando

sus 4ms completos y terminando en $t=16$.

En este punto las colas se reordenan constantemente basándose en las ráfagas restantes. P4 completa su I/O y tiene una ráfaga final de 1ms (la más corta), P2 necesita 2ms, y P1 necesita 3ms. El orden de ejecución final es P4, luego P2, finalmente P1.

```

1  Tiempos de terminación:
2  P1: 20, P2: 13, P3: 16, P4: 16
3
4  Tiempos de retorno:
5  P1: 20-0 = 20
6  P2: 13-1 = 12
7  P3: 16-2 = 14
8  P4: 16-3 = 13
9  Promedio: (20+12+14+13)/4 = 14.75
10
11 Tiempos de espera:
12 P1: 20-(5+3) = 12
13 P2: 12-(3+2) = 7
14 P3: 14-4 = 10
15 P4: 13-(2+1) = 10
16 Promedio: (12+7+10+10)/4 = 9.75
17
18 Tiempo de respuesta:
19 P1: 0-0 = 0
20 P2: 7-1 = 6
21 P3: 12-2 = 10
22 P4: 5-3 = 2 (ejecuta cuando P1 va a I/O)
23 Promedio: (0+6+10+2)/4 = 4.5

```

Comparado con Round Robin, SJF logra mejor tiempo de retorno promedio (14.75 vs 15) y mejor tiempo de espera promedio (9.75 vs 10), confirmando su optimalidad teórica. Sin embargo, el tiempo de respuesta promedio es significativamente peor (4.5 vs 3), y notá cómo P3 tiene un tiempo de respuesta terrible de 10ms porque llegó cuando había procesos más cortos esperando.

6.5.3. Manejo de Prioridades en Eventos Simultáneos

Un aspecto sutil pero crítico de la planificación es cómo manejar múltiples eventos que ocurren en el mismo instante. Este escenario aparece frecuentemente en sistemas reales debido a la granularidad finita del reloj del sistema.

Imaginá la situación en $t=10$ donde tres eventos convergen simultáneamente:

- Interrupción de reloj: P1 agota su quantum
- Finalización de I/O: P2 completa su operación de disco y vuelve a READY
- System call: P3 se bloquea voluntariamente esperando red

El orden en que el sistema operativo procesa estos eventos determina qué proceso ejecuta a continuación. El orden de prioridad estándar es:

Interrupción de reloj (prioridad más alta)
 Finalización de I/O (prioridad media)
 System call / transición voluntaria (prioridad más baja)

Este ordenamiento no es arbitrario. Las interrupciones de reloj deben procesarse inmediatamente para mantener la integridad del sistema de tiempo. Las finalizaciones de I/O tienen prioridad sobre system calls porque representan eventos externos que el sistema debe reconocer rápidamente. Una vez procesados todos los eventos, el planificador evalúa la nueva configuración. Si P2 (que volvió de I/O) tiene mayor prioridad que P1 (que fue expropiado), entonces P2 ejecutará. Si tienen igual prioridad, generalmente continúa P1 porque ya estaba en posesión del CPU, minimizando context switches innecesarios.

El criterio de desempate cuando múltiples procesos tienen igual prioridad y están READY simultáneamente suele ser FCFS basado en el instante en que cada proceso originalmente se volvió READY, no en el instante del desempate actual.

6.6. Síntesis

6.6.1. Puntos Clave para Parcial

Resumen de algoritmos:

Algoritmo	Preemptivo	Overhead	Starvation	Aging	Mejor para
FCFS	No	Mínimo	No	No necesario	Batch simple
SJF	No	Bajo	Sí	Necesario	Batch conocido
SRT	Sí	Alto	Sí (severo)	Crítico	Trabajos cortos
RR	Sí	Medio	No	No necesario	Interactivo
VRR	Sí	Medio-Alto	No	Implícito	I/O intensivo
HRRN	No	Medio	No	Incorporado	Balanceado
Prioridades	Ambos	Bajo-Medio	Sí	Esencial	Tiempo real
Multilevel	Sí	Alto	Posible	Necesario	Propósito general

Fórmulas esenciales:

```
1 t_retorno = t_terminacion - t_llegada
```



```

2  t_espera = t_retorno - t_CPU-total
3  t_respuesta = t_primera-ejecucion - t_llegada
4
5  HRRN: Response Ratio = (t_espera + t_servicio) / t_servicio
6  Aging: Nueva Prioridad = prioridad-base + (t_espera / factor-aging)

```

Cuando resuelvas ejercicios de planificación, seguí esta metodología sistemática:

Primero, dibujá un timeline mostrando todas las llegadas y eventos importantes. Marcá claramente cuándo cada proceso llega, cuándo comienzan y terminan las ráfagas de CPU, y cuándo ocurren los I/O.

Segundo, identificá todas las interrupciones y sus prioridades relativas. Esto es especialmente importante cuando múltiples eventos ocurren simultáneamente.

Tercero, aplicá el algoritmo de planificación respetando estrictamente sus reglas de preemptividad. No asumas comportamientos: si el algoritmo es no preemptivo, un proceso ejecuta hasta bloquearse o terminar sin excepciones.

Cuarto, manejá las operaciones de I/O correctamente. Un proceso en I/O no está en la cola READY y no puede ser seleccionado para ejecución. Cuando completa su I/O, debe explícitamente retornar a READY antes de ser considerado.

Finalmente, calculá todas las métricas solicitadas para cada proceso individualmente, luego promedialas. Verificá que los números tengan sentido: el tiempo de espera nunca puede ser negativo, y el tiempo de retorno debe ser al menos igual al tiempo de CPU total.

6.6.2. Ejemplo simulación Round Robin

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      int pid;
6      int arrival_time;
7      int burst_time;
8      int remaining_time;    // Tiempo restante de ejecución
9      int completion_time;
10     int turnaround_time;
11     int waiting_time;
12     int response_time;
13     int first_response;    // Para calcular tiempo de respuesta
14 } Process;
15
16 void round_robin_schedule(Process processes[], int n, int quantum) {
17     int current_time = 0;
18     int completed = 0;
19     int queue[100]; // Cola circular simple
20     int front = 0, rear = 0;
21     int in_queue[10] = {0}; // Marca si proceso está en cola
22
23     printf("Round Robin (Quantum = %d):\n", quantum);
24     printf("Tiempo\tProceso ejecutándose\n");
25

```

```

26 // Inicializar campos
27 for (int i = 0; i < n; i++) {
28     processes[i].remaining_time = processes[i].burst_time;
29     processes[i].first_response = -1;
30 }
31
32 // Agregar procesos que llegan en tiempo 0
33 for (int i = 0; i < n; i++) {
34     if (processes[i].arrival_time == 0) {
35         queue[rear++] = i;
36         in_queue[i] = 1;
37     }
38 }
39
40 while (completed < n) {
41     if (front == rear) {
42         // Cola vacía, avanzar tiempo hasta próxima llegada
43         current_time++;
44         // Verificar nuevas llegadas
45         for (int i = 0; i < n; i++) {
46             if (processes[i].arrival_time == current_time && !in_queue[i]) {
47                 queue[rear++] = i;
48                 in_queue[i] = 1;
49             }
50         }
51         continue;
52     }
53
54     // Obtener proceso del frente de la cola
55     int current_process = queue[front++];
56     in_queue[current_process] = 0;
57
58     // Marcar primera respuesta
59     if (processes[current_process].first_response == -1) {
60         processes[current_process].first_response = current_time;
61         processes[current_process].response_time = current_time -
62             processes[
63                 current_process].
64                 arrival_time;
65
66         // Ejecutar por quantum o hasta terminar
67         int execution_time = (processes[current_process].remaining_time <
68             quantum) ?
69             processes[current_process].remaining_time :
70             quantum;
71
72         printf("%d-%d\tP%d\n", current_time, current_time + execution_time,
73             processes[current_process].pid);
74
75         processes[current_process].remaining_time -= execution_time;
76         current_time += execution_time;
77
78         // Verificar nuevas llegadas durante la ejecución
79         for (int i = 0; i < n; i++) {
80             if (processes[i].arrival_time <= current_time &&

```

```

78         processes[i].arrival_time > (current_time - execution_time)
79         &&
80         !in_queue[i] && processes[i].remaining_time > 0) {
81             queue[rear++] = i;
82             in_queue[i] = 1;
83         }
84     }
85     if (processes[current_process].remaining_time == 0) {
86         // Proceso terminado
87         completed++;
88         processes[current_process].completion_time = current_time;
89         processes[current_process].turnaround_time =
90             processes[current_process].completion_time -
91             processes[current_process].arrival_time;
92         processes[current_process].waiting_time =
93             processes[current_process].turnaround_time -
94             processes[current_process].burst_time;
95     } else {
96         // Proceso no terminado, vuelve a la cola
97         queue[rear++] = current_process;
98         in_queue[current_process] = 1;
99     }
100 }
101
102 // Imprimir resultados
103 printf("\nResultados:\n");
104 printf("PID\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse\n"
105 );
106 for (int i = 0; i < n; i++) {
107     printf("%d\t%d\t%d\t%d\t\t%d\t\t%d\t\t%d\n",
108         processes[i].pid, processes[i].arrival_time,
109         processes[i].burst_time, processes[i].completion_time,
110         processes[i].turnaround_time, processes[i].waiting_time,
111         processes[i].response_time);
112 }
113
114 int main() {
115     Process processes[] = {
116         {1, 0, 20, 0, 0, 0, 0, 0, -1},
117         {2, 0, 3, 0, 0, 0, 0, 0, -1},
118         {3, 0, 3, 0, 0, 0, 0, 0, -1}
119     };
120
121     int n = sizeof(processes) / sizeof(processes[0]);
122     int quantum = 4;
123
124     round_robin_schedule(processes, n, quantum);
125
126     return 0;
127 }

```

Este código muestra todos los detalles sutiles de una implementación real: gestión de la cola circular, tracking del tiempo restante de cada proceso, cálculo preciso de todas las métricas, y manejo correcto de procesos que terminan antes de agotar su quantum.

Cuando estudies para el parcial, no solo memorices algoritmos; enténdé por qué toman las decisiones que toman. ¿Por qué SJF minimiza el tiempo de retorno? Porque ejecutar trabajos cortos primero minimiza el tiempo total que todos los procesos pasan en el sistema. ¿Por qué Round Robin garantiza responsividad? Porque ningún proceso espera más de $n \times Q$ unidades de tiempo, sin importar cuán largo sea.

Tips para parcial:

1. Diagramas de Gantt son esenciales - Mostrar CPU e I/O separadamente
2. Marcar eventos importantes - Interrupciones, llegadas, cambios de estado
3. Verificar cálculos - Tiempo total debe ser consistente
4. Considerar overhead - Context switches tienen costo
5. Justificar decisiones - Explicar por qué se eligió cada proceso

6.6.3. Decisiones de Diseño

La elección del algoritmo apropiado depende fundamentalmente del contexto de uso del sistema. No existe una respuesta universal porque diferentes entornos priorizan diferentes objetivos.

Para **sistemas batch** que procesan trabajos largos sin intervención humana, algoritmos como SJF, HRRN o FCFS son apropiados. El objetivo principal es maximizar throughput (trabajos completados por hora) y utilizar eficientemente los recursos. La responsividad individual de cada proceso es menos importante porque no hay usuarios esperando resultados inmediatos. La starvation es menos crítica porque eventualmente el sistema procesará todos los trabajos durante la noche o el fin de semana.

Los **sistemas interactivos** como laptops, smartphones, o estaciones de trabajo tienen prioridades completamente diferentes. El tiempo de respuesta es crítico porque los usuarios perciben demoras de más de 100-200ms como lentitud frustrante. Algoritmos como Round Robin, VRR, o Multilevel Feedback Queue son ideales porque garantizan que todas las aplicaciones progresen constantemente. Un usuario puede tener 20 aplicaciones abiertas; todas deben sentirse responsivas aunque ninguna esté usando el CPU intensivamente.

Sistemas de tiempo real enfrentan restricciones más estrictas aún: ciertos procesos deben completarse antes de deadlines absolutos. Perder un deadline puede resultar en fallas catastróficas (considera un controlador de airbag que debe activarse en 10ms). Estos sistemas requieren planificación por prioridades con garantías matemáticamente verificables. La justicia es irrelevante; lo único que importa es cumplir deadlines.

Los sistemas de propósito general modernos como Linux o Windows enfrentan el desafío más complejo: deben manejar simultáneamente procesos batch de larga duración, aplicaciones interactivas que requieren respuesta instantánea, y componentes de tiempo real del sistema

operativo. La solución es típicamente alguna variante de Multilevel Feedback Queue que se adapta dinámicamente a patrones de carga heterogéneos.

Finalmente, recordá que la planificación de CPU es solo una pieza del rompecabezas de rendimiento del sistema. La gestión de memoria, el sistema de I/O, y el diseño de las aplicaciones mismas frecuentemente tienen mayor impacto en la experiencia del usuario que el algoritmo de planificación específico. Un algoritmo de planificación brillante no puede compensar aplicaciones mal diseñadas que bloquean la interfaz de usuario o realizan I/O ineficientemente.

Próximo capítulo: Hilos - Explorando la concurrencia dentro de los procesos y desafíos de planificación multinivel.

7 Hilos (Threads)

7.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Explicar qué son los hilos y por qué son necesarios en sistemas modernos
- Distinguir entre User-Level Threads (ULT) y Kernel-Level Threads (KLT)
- Analizar los modelos de mapeo entre hilos de usuario y kernel
- Implementar programas multihilo usando pthreads en C
- Resolver ejercicios de planificación combinando procesos e hilos
- Identificar problemas de concurrencia y sus soluciones básicas
- Evaluar cuándo usar hilos vs procesos según el contexto

7.2. Introducción y Motivación

7.2.1. El problema de los procesos pesados

Los procesos tradicionales resuelven la multiprogramación, pero presentan limitaciones importantes en aplicaciones modernas. La creación de un proceso mediante `fork()` implica copiar todo el espacio de direcciones, lo cual resulta costoso en términos de tiempo y memoria. El cambio de contexto entre procesos requiere actualizar el mapa de memoria completo, invalidar caches y la TLB (Translation Lookaside Buffer), operaciones que consumen cientos o miles de ciclos de CPU. La comunicación entre procesos necesita mecanismos especiales como IPC, pipes o memoria compartida, añadiendo complejidad al diseño del sistema. Además, el aislamiento de recursos, aunque beneficioso para la seguridad, dificulta compartir datos eficientemente entre tareas relacionadas.

Consideremos un navegador web moderno. Durante la carga de una página se ejecutan simultáneamente múltiples tareas: descarga de HTML, procesamiento y renderizado, descarga de imágenes, ejecución de JavaScript y manejo de eventos del usuario. Si estas tareas fueran procesos separados, el overhead de comunicación y context switching sería prohibitivo. Los hilos permiten que todas estas actividades compartan memoria y se comuniquen de forma directa, reduciendo la latencia y mejorando la experiencia del usuario.

7.2.2. La solución: hilos como procesos livianos

Hilo (Thread): Unidad básica de utilización del CPU dentro de un proceso, caracterizada por tener su propio flujo de ejecución independiente pero compartiendo el espacio de direcciones con otros hilos del mismo proceso.

Los hilos representan una abstracción fundamental que separa dos conceptos previamente unidos en los procesos: el espacio de direcciones y el flujo de ejecución. Mientras que los procesos tradicionales combinan ambos aspectos de forma inseparable, los hilos permiten múltiples flujos de ejecución dentro del mismo espacio de direcciones.

Esta separación tiene profundas implicaciones. Los hilos dentro de un proceso comparten el código ejecutable, las variables globales, el heap dinámico, los archivos abiertos y otros recursos del sistema. Sin embargo, cada hilo mantiene elementos privados esenciales para su ejecución independiente: su propio identificador (TID), el conjunto completo de registros del procesador incluyendo el program counter y stack pointer, un stack privado para variables locales y llamadas a funciones, y su propio estado de planificación.

La compartición del espacio de direcciones significa que las variables globales y la memoria dinámica son accesibles directamente por todos los hilos, sin necesidad de mecanismos complejos de IPC. Esta característica simplifica enormemente la comunicación entre hilos, aunque introduce nuevos desafíos relacionados con la sincronización que exploraremos en profundidad en capítulos posteriores.

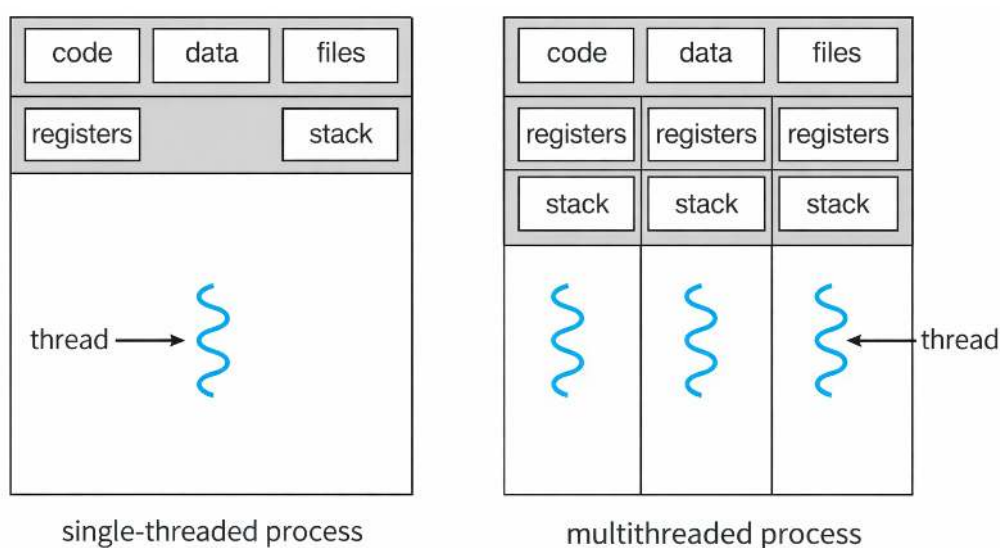
7.3. Estructura y Definiciones Fundamentales

7.3.1. Anatomía de un hilo

Cada hilo mantiene una estructura de control que el sistema operativo utiliza para gestionar su ejecución. Esta estructura, típicamente llamada Thread Control Block (TCB), contiene toda la información necesaria para suspender y reanudar el hilo:

```
1 typedef struct thread_control_block {
2     int tid;                // Thread ID único dentro del proceso
3     int state;              // READY, RUNNING, BLOCKED, TERMINATED
4     void* stack_pointer;    // Puntero al tope del stack privado
5     void* program_counter;  // Próxima instrucción a ejecutar
6     // NOTA: 'registers' es conceptual;
7     unsigned long registers[16]; // Simplificación didáctica
8     void* stack_base;       // Base del stack
9     size_t stack_size;      // Tamaño del stack
10    int priority;            // Prioridad de planificación
11    struct thread_control_block* next;
12 } tcb_t;
```

El TID (Thread ID) identifica únicamente al hilo dentro de su proceso. El estado refleja su situación actual en el sistema: ready cuando está listo para ejecutar, running cuando tiene asignado un CPU, blocked cuando espera por algún recurso, o terminated cuando ha finalizado. Los punteros `stack_pointer` y `program_counter` capturan exactamente dónde se encuentra el hilo en su ejecución. El campo `registers` preserva el estado completo de los registros del procesador, permitiendo que el hilo retome su ejecución exactamente donde la dejó después de un cambio de contexto.

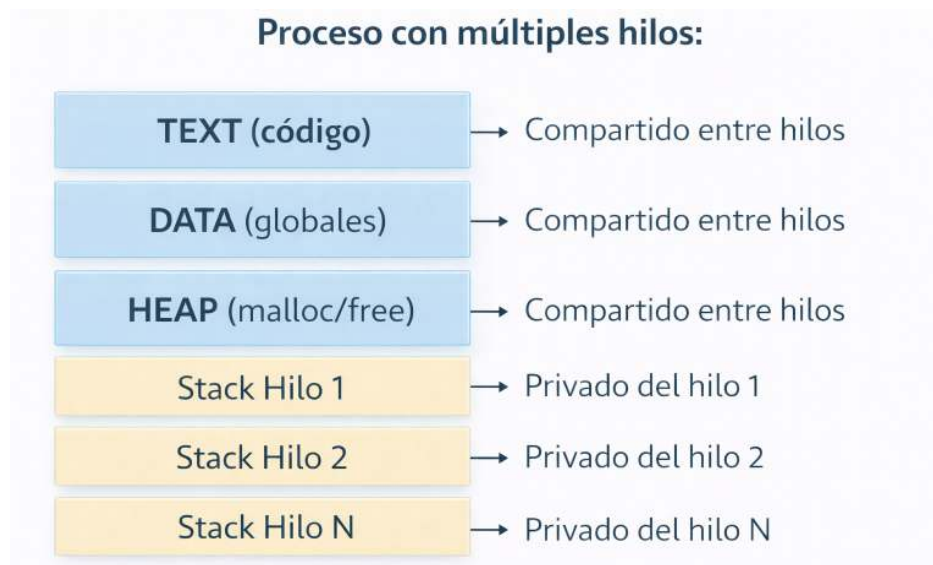


La información del stack es crítica. Cada hilo necesita su propio stack para almacenar variables locales, parámetros de funciones y direcciones de retorno. Un stack insuficiente causará stack overflow, mientras que stacks muy grandes desperdiciarán memoria. La mayoría de implementaciones usan stacks de 2-8MB por hilo, aunque sistemas embebidos pueden usar apenas 1-2KB.

7.3.2. Modelo de memoria con hilos

La organización del espacio de direcciones de un proceso multihilo revela claramente qué recursos son compartidos y cuáles son privados:

En la parte baja de la memoria encontramos el segmento de texto (código ejecutable), idéntico para todos los hilos. Inmediatamente después está el segmento de datos inicializados y el BSS (datos no inicializados), también compartidos. El heap dinámico, gestionado por `malloc()` y `free()`, crece hacia direcciones más altas y es accesible por todos los hilos del proceso.



La parte alta de la memoria se reserva para los stacks individuales de cada hilo. Estos crecen hacia direcciones más bajas y están separados por regiones de guarda (guard pages) que detectan desbordamientos. Esta separación física de los stacks garantiza que las variables locales de un hilo no interfieran con las de otro.

Las variables globales y el heap son compartidos entre hilos, lo que requiere sincronización explícita para evitar race conditions. En contraste, las variables locales declaradas dentro de funciones son automáticamente privadas de cada hilo, ya que residen en su stack individual. Esta distinción es fundamental para razonar sobre la corrección de programas multihilo.

7.4. User-Level Threads vs Kernel-Level Threads

La implementación de hilos puede realizarse en dos niveles fundamentalmente diferentes del sistema: en espacio de usuario o en espacio de kernel. Esta decisión arquitectural tiene profundas implicaciones en el rendimiento, el paralelismo y la complejidad del sistema.

7.4.1. User-Level Threads (ULT)

User-Level Threads (ULT): Hilos implementados completamente en espacio de usuario mediante bibliotecas especializadas, donde el kernel del sistema operativo no tiene conocimiento de los hilos individuales.

En el modelo ULT, desde la perspectiva del kernel existe únicamente un proceso con un solo hilo de ejecución. Una biblioteca especializada en espacio de usuario (como GNU Portable Threads o las bibliotecas de fibras) implementa toda la gestión de hilos: creación, destrucción, planificación y cambios de contexto.

El cambio de contexto entre ULT es extremadamente eficiente porque no requiere transición al modo kernel. La biblioteca simplemente guarda los registros del hilo actual en su TCB y restaura los del siguiente hilo a ejecutar. Esto puede realizarse en apenas 20-50 instrucciones, comparado con las miles de instrucciones necesarias para un cambio de contexto completo que involucre al kernel.

```
1 // Pseudocódigo simplificado de context switch ULT
2 void ult_context_switch(ult_t *from, ult_t *to) {
3     // 1. Guardar estado de registros en TCB
4     if (setjmp(from->context) == 0) {
5         // 2. Cambiar hilo actual en scheduler
6         current_ult = to;
7
8         // 3. Restaurar estado del nuevo hilo
9         longjmp(to->context, 1);
10    }
11    // Ejecución continúa aquí cuando este hilo vuelva a ejecutar
12 }
```

Esta implementación ilustra la simplicidad fundamental de los ULT. Las funciones `setjmp()` y `longjmp()` de la biblioteca estándar de C capturan y restauran el estado de ejecución, permitiendo “saltar” entre diferentes puntos del programa. El scheduler de usuario simplemente decide qué hilo debe ejecutar a continuación y realiza el salto correspondiente.

NOTA IMPORTANTE:

El uso de `setjmp()/longjmp()` aquí es únicamente una simplificación conceptual para mostrar cómo se guarda/restaura el contexto de ejecución.

En una implementación real de ULT se necesita también:

- Asignar stacks independientes (con `malloc` o `mmap`)
- Cambiar el stack pointer (`setjmp` NO lo hace automáticamente)
- Manejar señales o `sigaltstack` para cambios de contexto preemptivos

Bibliotecas reales como `Protothreads`, `libpth` o `Goroutines` usan técnicas más avanzadas.

La planificación de ULT típicamente sigue un modelo cooperativo. Cada hilo debe voluntariamente ceder el control, ya sea llamando explícitamente a una función de `yield` o al realizar operaciones de I/O. Esto elimina la necesidad de interrupciones de timer, pero introduce el riesgo de que un hilo monopolice el CPU si no coopera adecuadamente.

Las ventajas de los ULT incluyen cambios de contexto extremadamente rápidos (del orden de microsegundos), ausencia de overhead de system calls, la posibilidad de implementar políticas de planificación especializadas para cada aplicación, y soporte en sistemas operativos que no implementan hilos nativamente a nivel kernel. Sin embargo, presentan limitaciones significativas. No pueden aprovechar el paralelismo real en sistemas multicore, ya que el kernel ve un solo hilo de ejecución y lo asigna a un único procesador. Si un hilo realiza una operación de I/O bloqueante, todo el proceso se bloquea, ya que el kernel suspende el único "hilo" que conoce. La planificación cooperativa puede llevar a problemas de starvation si algunos hilos no ceden el control apropiadamente.

7.4.2. Kernel-Level Threads (KLT)

Kernel-Level Threads (KLT): Hilos implementados y gestionados directamente por el kernel del sistema operativo, donde cada hilo es una entidad de planificación independiente conocida por el SO.

En contraste con los ULT, los hilos a nivel kernel son entidades de primera clase en el sistema operativo. Cada KLT tiene su propia entrada en las tablas de planificación del kernel y es gestionado exactamente como un proceso ligero. El kernel mantiene un TCB completo para cada hilo y realiza todos los cambios de contexto.

```

1 // Context switch KLT (ejecutado por kernel)
2 void klt_context_switch(tcb_t *from, tcb_t *to) {
3     // 1. Guardar estado completo en kernel
4     save_processor_state(&from->cpu_state);
5     save_fpu_state(&from->fpu_state);
6
7     // 2. Actualizar planificador
8     update_scheduler_queues(from, to);
9
10    // 3. Cambiar contexto de memoria si es necesario
11    if (from->process_id != to->process_id) {
12        switch_address_space(to->process_id);
13        flush_tlb();
14    }
15
16    // 4. Restaurar estado del nuevo hilo
17    restore_processor_state(&to->cpu_state);
18    restore_fpu_state(&to->fpu_state);
19
20    // 5. Continuar ejecución
21    jump_to_thread(to);
22 }
```

Este pseudocódigo revela la complejidad adicional del cambio de contexto a nivel kernel. Además de guardar y restaurar los registros generales, el kernel debe preservar el estado de la unidad de

punto flotante, actualizar las colas de planificación, y potencialmente cambiar el espacio de direcciones completo si los hilos pertenecen a procesos diferentes. El flush de la TLB después de cambiar el espacio de direcciones es particularmente costoso, pudiendo tomar cientos de ciclos. La planificación de KLT es preventiva. El kernel puede suspender un hilo en cualquier momento mediante interrupciones de timer, garantizando que ningún hilo monopolice el CPU. Esta planificación justa viene al costo de mayor overhead, pero proporciona mejor respuesta interactiva y aprovecha automáticamente sistemas multicore.

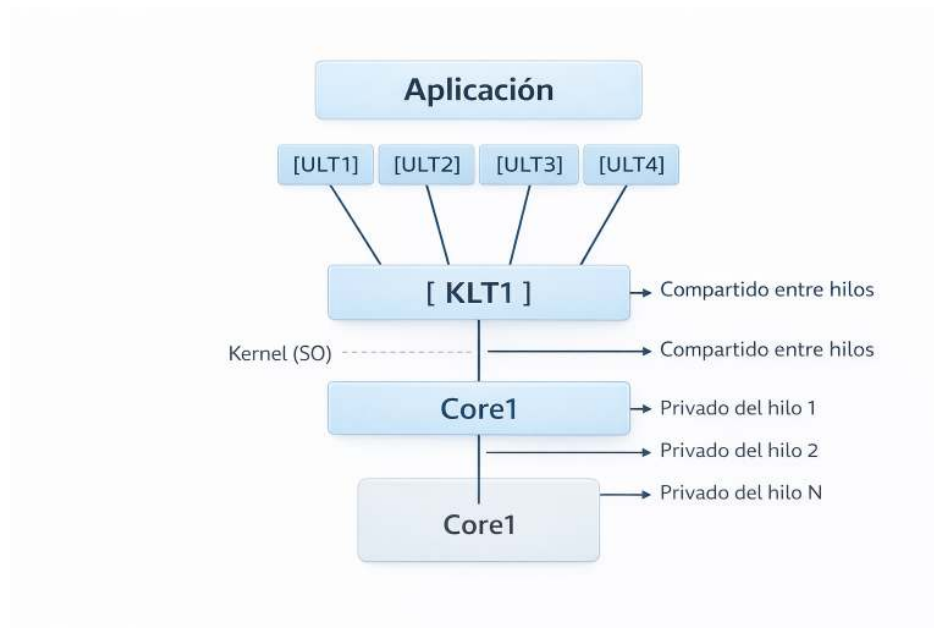
Los KLT ofrecen paralelismo real en sistemas multiprocesador o multicore. El kernel puede asignar diferentes hilos del mismo proceso a diferentes CPUs, permitiendo ejecución verdaderamente simultánea. Cuando un hilo se bloquea en I/O, solo ese hilo se suspende; otros hilos del mismo proceso continúan ejecutando. La planificación preventiva del kernel garantiza fairness entre hilos y permite prioridades sofisticadas. Las desventajas incluyen cambios de contexto significativamente más lentos (típicamente 500-2000 instrucciones), overhead de system calls para todas las operaciones de gestión de hilos, limitaciones en el número máximo de hilos determinadas por recursos del kernel, y mayor consumo de memoria en espacio kernel para mantener las estructuras de control.

7.4.3. Modelos de mapeo

La relación entre hilos de usuario y kernel puede estructurarse de varias formas, cada una con sus propias características de rendimiento y complejidad.

7.4.3.1. Modelo Many-to-One (N:1)

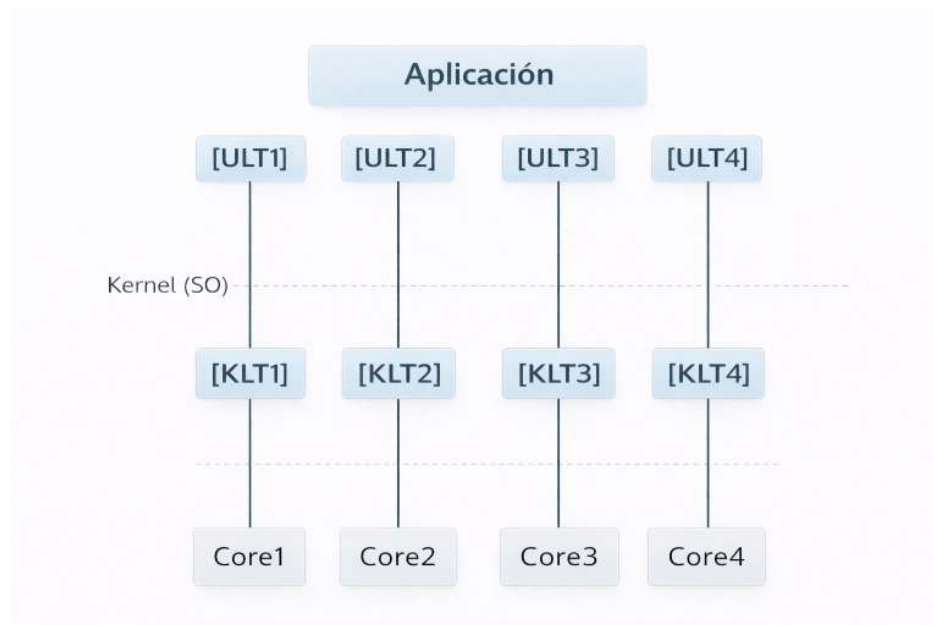
Múltiples hilos de usuario mapean a un solo hilo kernel. Este es el modelo clásico de ULT puro:



Este modelo maximiza la eficiencia del context switch y minimiza el uso de recursos del kernel, pero sacrifica completamente el paralelismo. Toda la aplicación se ejecuta como un único hilo desde la perspectiva del sistema operativo.

7.4.3.2. Modelo One-to-One (1:1)

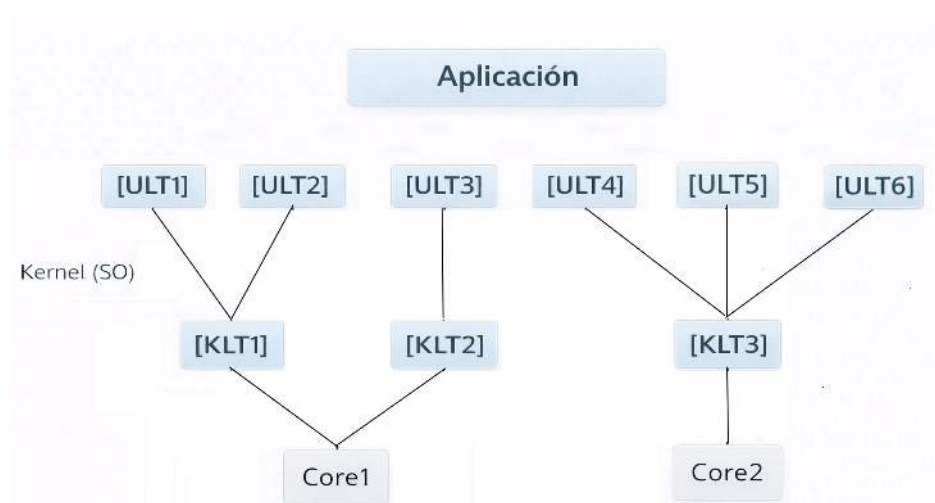
Cada hilo de usuario mapea a un hilo kernel dedicado. Este es el modelo más común en sistemas operativos modernos:



Este modelo maximiza el paralelismo y permite que el kernel gestione directamente todos los hilos, pero incrementa el overhead de cada operación. Linux, Windows y la mayoría de sistemas UNIX modernos utilizan este modelo para sus implementaciones de hilos POSIX (pthreads).

7.4.3.3. Modelo Many-to-Many (M:N)

M hilos de usuario mapean a N hilos kernel, donde $M > N$. Este modelo híbrido intenta combinar las ventajas de ambos enfoques:



El modelo M:N requiere un scheduler de dos niveles: uno en espacio de usuario que gestiona los hilos de usuario, y otro en el kernel que gestiona los hilos kernel. La biblioteca de hilos debe coordinar con el kernel para optimizar el mapeo dinámico, asignando hilos de usuario activos a los hilos kernel disponibles. Este modelo fue implementado en Solaris (como threads M:N) y en algunas versiones tempranas de Go, aunque la complejidad adicional ha llevado a muchos sistemas a preferir el modelo 1:1 más simple.

7.5. Implementación en C con pthreads

La biblioteca POSIX Threads (pthreads) proporciona una API estándar para programación multihilo en sistemas UNIX y Linux. Esta interfaz ha sido adoptada ampliamente y representa el estándar de facto para programación concurrente en C.

7.5.1. Creación básica de hilos

Vamos a explorar un ejemplo completo que ilustra los conceptos fundamentales de la programación con pthreads:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <unistd.h>
5
6  // Estructura para pasar múltiples argumentos al hilo
7  typedef struct {
8      int thread_id;
9      int iterations;
10     char* message;
11 } thread_args_t;
12
13 // Función que ejecutará cada hilo
14 void* thread_function(void* arg) {
15     thread_args_t* args = (thread_args_t*)arg;
16
17     printf("Hilo %d iniciado - TID: %lu, PID: %d\n",
18         args->thread_id, pthread_self(), getpid());
19
20     // Trabajo del hilo
21     for (int i = 0; i < args->iterations; i++) {
22         printf("Hilo %d: %s - iteración %d\n",
23             args->thread_id, args->message, i + 1);
24         sleep(1);
25     }
26
27     // Preparar valor de retorno
28     int* result = malloc(sizeof(int));
29     *result = args->iterations * args->thread_id;
30
31     printf("Hilo %d terminando con resultado %d\n", args->thread_id, *result);
32     return result;
33 }
34
35 int main() {
36     const int NUM_THREADS = 3;
37     pthread_t threads[NUM_THREADS];
38     thread_args_t thread_args[NUM_THREADS];
39
40     printf("Proceso principal - PID: %d\n", getpid());
41
42     // Configurar argumentos y crear hilos
43     for (int i = 0; i < NUM_THREADS; i++) {
44         thread_args[i].thread_id = i + 1;
45         thread_args[i].iterations = 3 + i; // 3, 4, 5 iteraciones
46         thread_args[i].message = "Trabajando";
47
48         int result = pthread_create(&threads[i], NULL,
49             thread_function, &thread_args[i]);
50
51         if (result != 0) {
```

```
52         fprintf(stderr, "Error creando hilo %d\n", i + 1);
53         exit(1);
54     }
55 }
56
57 // Esperar terminación y recolectar resultados
58 printf("\nEsperando que terminen todos los hilos...\n");
59
60 for (int i = 0; i < NUM_THREADS; i++) {
61     void* thread_result;
62     pthread_join(threads[i], &thread_result);
63
64     int* result_value = (int*)thread_result;
65     printf("Hilo %d retornó: %d\n", i + 1, *result_value);
66
67     free(result_value); // Importante: liberar memoria
68 }
69
70 printf("Proceso principal terminando\n");
71 return 0;
72 }
```

Compilación:

```
1 gcc -o hilos_basico hilos_basico.c -lpthread
```

Este programa demuestra varios aspectos cruciales de la programación multihilo. Primero, notemos que todos los hilos comparten el mismo PID (Process ID), confirmando que son parte del mismo proceso. Sin embargo, cada hilo tiene su propio TID (Thread ID) único, obtenido mediante `pthread_self()`.

La función `thread_function()` acepta un puntero genérico `void*` como argumento, permitiendo pasar cualquier tipo de dato. En este caso, pasamos una estructura `thread_args_t` que contiene múltiples parámetros. Esta técnica es común cuando necesitamos proporcionar más de un valor al hilo.

El valor de retorno del hilo también es un `void*`, permitiendo retornar cualquier tipo de dato. Aquí asignamos dinámicamente un entero con `malloc()`, lo asignamos y lo retornamos. El hilo principal recupera este valor mediante `pthread_join()` y debe recordar liberar la memoria asignada.

Es crítico entender el ciclo de vida de los datos pasados a los hilos. Las estructuras `thread_args[i]` deben permanecer válidas hasta que cada hilo las haya leído completamente. Por eso las declaramos como un array en la función principal en lugar de variables locales de un bucle. Si pasáramos la dirección de una variable local que cambia en cada iteración, todos los hilos podrían ver el mismo valor (el último) debido a una race condition.

7.5.2. Diferencias clave respecto a procesos

La programación con hilos difiere fundamentalmente de la programación con procesos en varios aspectos. La creación de un hilo mediante `pthread_create()` es órdenes de magnitud más rápida que `fork()` porque solo necesita asignar un nuevo stack y copiar los registros iniciales, sin duplicar el espacio de direcciones completo. El cambio de contexto entre hilos del mismo proceso requiere únicamente actualizar registros y cambiar al nuevo stack, mientras que entre procesos diferentes implica además cambiar el mapa completo de memoria y purgar la TLB.

La comunicación entre hilos es directa a través de variables globales y memoria compartida, sin necesidad de mecanismos especiales de IPC. Sin embargo, esta facilidad viene acompañada de la responsabilidad de sincronizar correctamente los accesos concurrentes para evitar race conditions. El aislamiento entre procesos proporciona seguridad: un error en un proceso no puede corromper directamente la memoria de otro. Con hilos, un error en cualquier hilo (como escribir fuera de los límites de un array) puede corromper el estado compartido y afectar a todos los hilos del proceso. Esta falta de aislamiento hace el debugging más complejo pero permite mayor eficiencia.

Consideremos una aplicación de procesamiento de imágenes. Si implementamos cada filtro (blur, sharpen, edge detection) como un proceso separado, necesitaremos copiar la imagen entre procesos o usar memoria compartida explícita. Con hilos, todos los filtros pueden acceder directamente a los mismos buffers de imagen, coordinando sus accesos mediante mutex cuando sea necesario. Esto reduce dramáticamente el overhead de datos y simplifica la arquitectura del código.

Aspecto	Procesos	Hilos
Creación	<code>fork()</code> copia memoria completa	<code>pthread_create()</code> solo crea stack
Memoria	Espacios separados	Espacio compartido
Comunicación	IPC, pipes, señales	Variables globales, heap
Context Switch	Cambio completo de MM	Solo registros y stack
Overhead	Alto (1-10ms)	Bajo (10-100µs)
Aislamiento	Total	Ninguno

7.6. Gestión avanzada de hilos

7.6.1. Workers y pool de hilos

Un patrón arquitectural fundamental en aplicaciones de alto rendimiento es el pool de hilos trabajadores. En lugar de crear y destruir hilos dinámicamente para cada tarea, mantenemos un conjunto fijo de hilos que esperan trabajo en una cola compartida.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <unistd.h>
5
6  #define POOL_SIZE 4
7  #define QUEUE_SIZE 100
8
9  // Variables globales para sincronización de finalización
10 pthread_mutex_t done_mutex;
11 pthread_cond_t done_cond;
12 int tasks_done = 0;
13 int total_tasks = 0;
14
15 // Estructura para tareas en cola
16 typedef struct task {
17     void (*function)(void*);
18     void* argument;
19     struct task* next;
20 } task_t;
21
22 // Pool de hilos
23 typedef struct {
24     pthread_t workers[POOL_SIZE];
25     task_t* task_queue;
26     task_t* queue_tail;
27     pthread_mutex_t queue_mutex;
28     pthread_cond_t queue_condition;
29     int shutdown;
30 } thread_pool_t;
31
32 thread_pool_t pool;
33
34 // Función del hilo trabajador
35 void* worker_thread(void* arg) {
36     printf("Worker iniciado - TID: %p\n", (void*)pthread_self());
37
38     while (1) {
39         pthread_mutex_lock(&pool.queue_mutex);
40
41         while (pool.task_queue == NULL && !pool.shutdown) {
42             pthread_cond_wait(&pool.queue_condition, &pool.queue_mutex);
43         }
44
45         if (pool.shutdown) {
46             pthread_mutex_unlock(&pool.queue_mutex);
47             break;
```

```

48     }
49
50     task_t* task = pool.task_queue;
51     pool.task_queue = task->next;
52     if (pool.task_queue == NULL) {
53         pool.queue_tail = NULL;
54     }
55
56     pthread_mutex_unlock(&pool.queue_mutex);
57
58     printf("Worker %p ejecutando tarea\n", (void*)pthread_self());
59     task->function(task->argument);
60     free(task);
61 }
62
63 printf("Worker %p terminando\n", (void*)pthread_self());
64 return NULL;
65 }
66
67 // Agregar tarea al pool
68 void pool_add_task(void (*function)(void*), void* argument) {
69     task_t* new_task = malloc(sizeof(task_t));
70     new_task->function = function;
71     new_task->argument = argument;
72     new_task->next = NULL;
73
74     pthread_mutex_lock(&pool.queue_mutex);
75
76     if (pool.queue_tail == NULL) {
77         pool.task_queue = pool.queue_tail = new_task;
78     } else {
79         pool.queue_tail->next = new_task;
80         pool.queue_tail = new_task;
81     }
82
83     pthread_cond_signal(&pool.queue_condition);
84     pthread_mutex_unlock(&pool.queue_mutex);
85 }
86
87 // Inicializar pool
88 void pool_init() {
89     pool.task_queue = NULL;
90     pool.queue_tail = NULL;
91     pool.shutdown = 0;
92
93     pthread_mutex_init(&pool.queue_mutex, NULL);
94     pthread_cond_init(&pool.queue_condition, NULL);
95
96     // Crear workers
97     for (int i = 0; i < POOL_SIZE; i++) {
98         pthread_create(&pool.workers[i], NULL, worker_thread, NULL);
99     }
100
101     printf("Pool de %d workers inicializado\n", POOL_SIZE);
102 }
103
104 // Tarea de ejemplo
105 void example_task(void* arg) {

```

```

106     int* task_id_ptr = (int*)arg;
107     int task_id = *task_id_ptr;
108     free(arg); // Liberar el argumento malloceado
109
110     printf(" Ejecutando tarea %d...\n", task_id);
111     sleep(2);
112     printf(" Tarea %d completada\n", task_id);
113
114     // Incrementar contador de tareas completadas
115     pthread_mutex_lock(&done_mutex);
116     tasks_done++;
117     if (tasks_done == total_tasks) {
118         pthread_cond_signal(&done_cond);
119     }
120     pthread_mutex_unlock(&done_mutex);
121 }
122
123
124 int main() {
125     pool_init();
126
127     // Variables para espera determinística
128     total_tasks = 10;
129     tasks_done = 0;
130     pthread_mutex_init(&done_mutex, NULL);
131     pthread_cond_init(&done_cond, NULL);
132
133     // Agregar tareas al pool (con memoria dinámica)
134     for (int i = 0; i < total_tasks; i++) {
135         int* arg = malloc(sizeof(int));
136         *arg = i + 1;
137         pool_add_task(example_task, arg);
138         printf("Tarea %d agregada al pool\n", i + 1);
139     }
140
141     // Esperar DETERMINÍSTICAMENTE a que terminen todas las tareas
142     pthread_mutex_lock(&done_mutex);
143     while (tasks_done < total_tasks) {
144         pthread_cond_wait(&done_cond, &done_mutex);
145     }
146     pthread_mutex_unlock(&done_mutex);
147
148     // Shutdown del pool
149     pthread_mutex_lock(&pool.queue_mutex);
150     pool.shutdown = 1;
151     pthread_cond_broadcast(&pool.queue_condition);
152     pthread_mutex_unlock(&pool.queue_mutex);
153
154     for (int i = 0; i < POOL_SIZE; i++) {
155         pthread_join(pool.workers[i], NULL);
156     }
157
158     // Limpiar recursos
159     pthread_mutex_destroy(&pool.queue_mutex);
160     pthread_cond_destroy(&pool.queue_condition);
161     pthread_mutex_destroy(&done_mutex);
162     pthread_cond_destroy(&done_cond);
163

```

```
164     printf("Pool terminado correctamente\n");
165     return 0;
166 }
```

El pool de hilos resuelve varios problemas importantes. Primero, elimina el overhead de creación y destrucción repetida de hilos. Aunque crear un hilo es más barato que crear un proceso, aún requiere asignar memoria para el stack (típicamente 2-8MB), inicializar estructuras kernel, y potencialmente interactuar con el scheduler. En aplicaciones que manejan miles de tareas cortas por segundo, este overhead se vuelve significativo.

Segundo, el pool controla el nivel de concurrencia. Sin un pool, una aplicación podría crear tantos hilos como tareas pendientes haya, potencialmente sobrecargando el sistema. Con un pool de tamaño fijo, limitamos el máximo número de tareas ejecutándose simultáneamente, previniendo sobrecargas y mejorando la previsibilidad del rendimiento.

La implementación usa un mutex para proteger la cola de tareas y una variable de condición para notificar a los workers cuando hay trabajo disponible. Los hilos esperan en `pthread_cond_wait()`, que atómicamente libera el mutex y suspende el hilo hasta recibir una señal. Cuando llega una nueva tarea, `pthread_cond_signal()` despierta un worker que la procesa.

Los pools de hilos son ubicuos en infraestructura moderna. Servidores web como Apache (Worker MPM) y Nginx usan pools para manejar conexiones. Bases de datos como PostgreSQL y MySQL mantienen pools de hilos para procesar queries. Frameworks como Java's Thread-PoolExecutor y Python's `concurrent.futures` proporcionan implementaciones robustas y configurables de este patrón.

7.7. Planificación de hilos

La planificación de hilos presenta desafíos únicos dependiendo de si están implementados a nivel usuario o kernel. Entender estas diferencias es crucial para diseñar sistemas concurrentes eficientes.

7.7.1. Planificación de ULT

Con hilos a nivel usuario, la biblioteca de hilos implementa su propio scheduler sin intervención del kernel. Este scheduler típicamente funciona de forma cooperativa:

```
1 // Scheduler Round Robin para ULT
2 void ult_scheduler() {
3     static int current = 0;
```

```

4     static int quantum_remaining = QUANTUM;
5
6     quantum_remaining--;
7     if (quantum_remaining <= 0 || threads[current].state != ULT_RUNNING) {
8         // Buscar próximo hilo READY
9         int next = current;
10        do {
11            next = (next + 1) % MAX_THREADS;
12        } while (threads[next].state != ULT_READY && next != current);
13
14        if (threads[next].state == ULT_READY) {
15            // Context switch
16            threads[current].state = ULT_READY;
17            threads[next].state = ULT_RUNNING;
18            ult_context_switch(&threads[current], &threads[next]);
19            current = next;
20            quantum_remaining = QUANTUM;
21        }
22    }
23 }

```

Este scheduler Round Robin mantiene un quantum para el hilo actual y rota entre hilos ready cuando el quantum expira o el hilo actual se bloquea. La función `ult_context_switch()` realiza el cambio de contexto en espacio de usuario, sin ningún system call.

El desafío principal de la planificación ULT es el problema del bloqueo. Si un hilo llama a una operación de I/O bloqueante como `read()`, el kernel bloquea todo el proceso (porque solo ve un hilo), suspendiendo incluso hilos ULT que podrían continuar ejecutando. Las bibliotecas ULT sofisticadas resuelven esto usando I/O no bloqueante y multiplexando manualmente las operaciones, añadiendo complejidad considerable.

7.7.2. Planificación de KLT

Con hilos a nivel kernel, el sistema operativo planifica cada hilo como una entidad independiente. Esto tiene una implicación importante pero a menudo pasada por alto:

En planificación fair entre hilos individuales, un proceso con muchos hilos puede monopolizar el CPU. Consideremos dos procesos: Proceso A con 10 hilos y Proceso B con 1 hilo. Si el scheduler distribuye tiempo equitativamente entre los 11 hilos totales, el Proceso A recibe $10/11 \approx 91$

Algunos sistemas operativos modernos implementan planificación consciente de procesos para mitigar este problema. El Completely Fair Scheduler (CFS) de Linux, por ejemplo, puede configurarse para distribuir tiempo de CPU entre grupos de hilos (cgroups) en lugar de hilos individuales, proporcionando fairness a nivel de proceso.

7.8. Ejercicio práctico completo

Vamos a trabajar un ejercicio complejo que integra todos los conceptos vistos: planificación Round Robin, hilos KLT, hilos ULT con scheduler interno, y operaciones de I/O.

7.8.1. Enunciado del problema

Un sistema con planificador Round Robin (quantum $Q=3ms$) debe ejecutar tres procesos con diferentes configuraciones de hilos:

Proceso P1 tiene un único hilo KLT: - P1-T1: Llega en $t=0ms$, requiere 6ms de CPU total, realiza I/O después de 4ms de uso de CPU durante 3ms

Proceso P2 tiene dos hilos KLT independientes: - P2-T1: Llega en $t=1ms$, requiere 3ms de CPU - P2-T2: Llega en $t=3ms$, requiere 6ms de CPU, realiza I/O después de 5ms de uso de CPU durante 2ms

Proceso P3 tiene arquitectura híbrida: un KLT que contiene dos ULT con scheduler interno FIFO: - P3-T1 (KLT): Llega en $t=2ms$, requiere 10ms de CPU total para ejecutar sus ULT - P3-T2 (ULT1): Llega en $t=4ms$, requiere 6ms de CPU - P3-T3 (ULT2): Llega en $t=2ms$, requiere 4ms de CPU

El aspecto más interesante de este ejercicio es P3. Los hilos ULT (P3-T2 y P3-T3) solo pueden ejecutar cuando su KLT contenedor (P3-T1) tiene asignado el CPU. El scheduler interno FIFO significa que P3-T2 debe completarse completamente antes de que P3-T3 pueda comenzar.

7.8.2. Análisis de la planificación con combinación de KLT y ULT

Analicemos la ejecución del sistema considerando que el planificador del kernel utiliza **Round Robin con quantum $Q = 3ms$** , y que los procesos presentan distintas arquitecturas de hilos.

En $t = 0ms$, solo el hilo **P1-T1 (KLT)** está presente en el sistema, por lo que comienza a ejecutarse inmediatamente. Durante este primer intervalo, se producen nuevas llegadas: **P2-T1** en $t = 1ms$, **P3-T1 (KLT)** en $t = 2ms$, y **P2-T2** en $t = 3ms$. Todos estos hilos son agregados a la cola *READY* mientras P1-T1 consume su quantum.

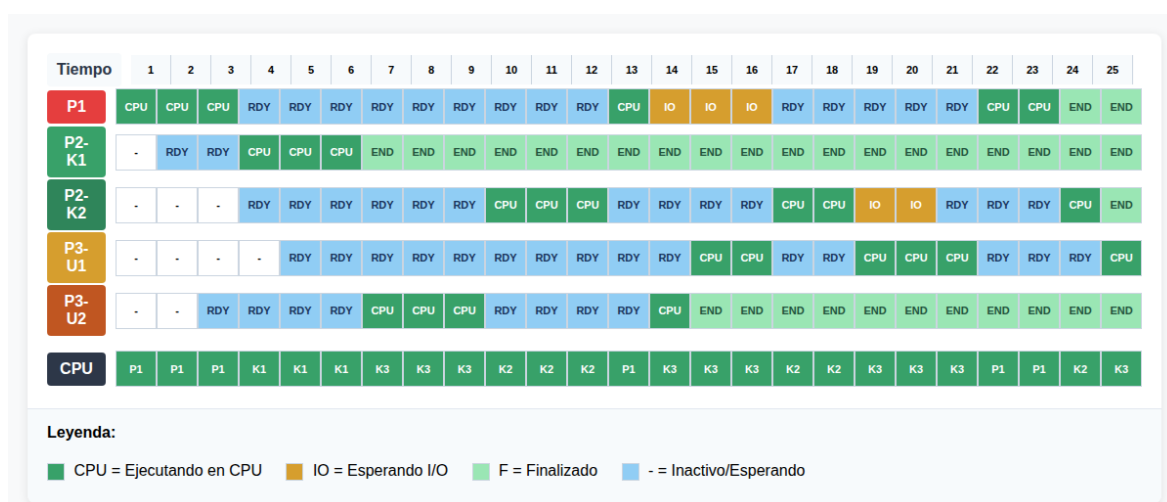
A partir de este punto, la planificación continúa siguiendo el esquema Round Robin clásico, alternando entre los distintos **KLT visibles para el sistema operativo**. En el caso de **P2**, cada uno de sus hilos (P2-T1 y P2-T2) es tratado como una entidad independiente por el kernel, pudiendo ser planificados y bloqueados por I/O de manera separada.

La situación más interesante ocurre cuando **P3-T1 obtiene el CPU**. Desde la perspectiva del sistema operativo, P3-T1 es un único hilo KLT. Sin embargo, internamente contiene dos **ULT**, gestionados por un scheduler en espacio de usuario con política FIFO. Cuando P3-T1 comienza a ejecutarse, su scheduler interno selecciona al ULT que llegó primero (**P3-ULT2**), que consume tiempo de CPU **dentro del quantum asignado al KLT**.

Durante este intervalo, el kernel no tiene visibilidad ni control sobre qué ULT está ejecutando. Para el sistema operativo, simplemente P3-T1 está usando CPU. Cuando el quantum del KLT se agota, el kernel desaloja a P3-T1, suspendiendo implícitamente la ejecución del ULT activo, aunque este no haya finalizado su trabajo.

Este comportamiento se repite cada vez que P3-T1 es planificado: el kernel decide *cuándo* ejecuta el KLT, mientras que el scheduler en espacio de usuario decide *qué ULT* se ejecuta dentro de ese intervalo. Recién cuando el primer ULT completa totalmente su ejecución, el scheduler interno permite que el siguiente ULT comience a ejecutarse.

Los ULT dependen completamente de su KLT contenedor. Desde el punto de vista del kernel, solo existen los KLT; la planificación interna de ULT es invisible para el sistema operativo. Esto permite una planificación más liviana y flexible, pero también implica que un bloqueo o desalojo del KLT afecta a todos los ULT asociados.



Observemos particularmente las secciones donde P3-T1 está activo. El diagrama puede mostrar internamente qué ULT está ejecutando, pero desde la perspectiva del scheduler del SO, solo P3-T1 está usando el CPU.

7.8.2.1. Consejo para encarar ejercicios con KLT y ULT

En ejercicios que combinan hilos a nivel kernel (KLT) y hilos a nivel usuario (ULT), resulta muy útil separar el análisis en **dos etapas**, evitando mezclar niveles de planificación desde el inicio.

Una estrategia recomendable es la siguiente:

1. **Planificar primero solo los KLT**, ignorando momentáneamente la existencia de ULT.
 - El kernel únicamente planifica KLT.

- En esta etapa se construye el diagrama de Gantt considerando llegadas, bloqueos por I/O y quantum del scheduler del sistema operativo.

2. Una vez obtenido el Gantt de KLT, agregar las filas correspondientes a los ULT.

- Para cada KLT que contiene ULT, se analiza cómo su scheduler interno distribuye el tiempo de CPU recibido.
- Los ULT consumen tiempo *dentro del quantum del KLT*, siguiendo su política interna (FIFO, en este caso).

Este enfoque puede pensarse como una **planificación anidada**:

el kernel decide *cuándo* ejecuta cada KLT, y cada KLT decide *qué ULT* ejecuta durante ese intervalo.

Adoptar este método ayuda a: - evitar errores conceptuales, - mantener claro qué decisiones toma el sistema operativo y cuáles no, - y entender mejor las limitaciones y ventajas de los ULT frente a los KLT.

7.9. Análisis comparativo y casos de uso

Habiendo explorado los fundamentos técnicos, consideremos ahora cuándo usar cada tipo de hilo en la práctica. Esta decisión arquitectural puede determinar el éxito o fracaso de un sistema concurrente.

7.9.1. Overhead de implementaciones

Métricas de rendimiento típicas:

Operación	ULT	KLT	Proceso
Creación	1-10 μ s	50-200 μ s	1-10 ms
Context Switch	0.1-1 μ s	5-50 μ s	100-1000 μ s
Memoria por entidad	2-8 KiB	8-16 KiB	4-8 MiB
Sincronización	Variables	Mutex/Sem	IPC

Estas métricas revelan que los ULT son 50-100x más rápidos que los KLT para context switch, y los KLT son 10-20x más rápidos que los procesos. Sin embargo, recordemos que velocidad no es el único criterio: el paralelismo real en multicore frecuentemente vale más que la velocidad de context switch.

7.9.2. Casos de uso recomendados

7.9.2.1. Usar ULT cuando necesitemos control fino y muchos context switches

Los ULT brillan en aplicaciones con tareas cortas y cooperativas que necesitan cambiar de contexto frecuentemente. Sistemas de eventos, servidores de juegos en tiempo real, y simulaciones con muchas entidades son candidatos ideales:

```
1 // ULT para manejar múltiples conexiones de jugadores
2 void* game_connection_handler(void* player_data) {
3     while (player_connected) {
4         process_player_input(); // CPU burst corto
5         update_game_state();   // CPU burst corto
6         ult_yield();           // Cooperativo
7         send_updates_to_player(); // I/O burst
8         ult_yield();           // Dar oportunidad a otros
9     }
10 }
```

Este patrón funciona bien con ULT porque cada conexión de jugador representa una tarea relativamente independiente que puede ceder el control voluntariamente en puntos bien definidos. El scheduler de usuario puede implementar políticas especializadas, como priorizar jugadores con mejor latencia o dar tiempo extra a sesiones importantes.

7.9.2.2. Usar KLT cuando necesitemos paralelismo real

Las aplicaciones que realizan cómputo intensivo y pueden dividirse en subtareas independientes deben usar KLT para aprovechar múltiples cores:

```
1 // KLT para aprovechar múltiples cores
2 void* image_processor_thread(void* args) {
3     image_region_t* region = (image_region_t*)args;
4
5     // Cada hilo procesa una región diferente de la imagen
6     for (int y = region->start_y; y < region->end_y; y++) {
7         for (int x = region->start_x; x < region->end_x; x++) {
8             apply_filter(image, x, y); // CPU intensivo
9         }
10    }
11    return NULL;
12 }
```

Aquí, cada hilo procesa una región diferente de la imagen de forma completamente independiente. Con KLT, el kernel puede asignar cada hilo a un core diferente, logrando speedup casi lineal (si hay suficientes cores disponibles). Con ULT, todos los hilos compartirían un solo core, negando cualquier beneficio de paralelismo.

7.9.2.3. Usar pool de hilos para controlar recursos

Los pools son especialmente valiosos en servidores que enfrentan carga variable e impredecible:

Un servidor web puede recibir 10 requests por segundo durante horas, luego súbitamente enfrentar 10,000 requests por segundo cuando un artículo se vuelve viral. Sin un pool, el servidor intentaría crear 10,000 hilos simultáneos, agotando memoria y provocando thrashing. Con un pool de, digamos, 200 hilos, el servidor mantiene latencia predecible procesando requests en cola, degradando gracefully bajo carga extrema en lugar de colapsar.

7.10. Problemas comunes y debugging

La programación multihilo introduce nuevas clases de errores que no existen en código secuencial. Entender estos problemas y sus soluciones es fundamental para escribir código correcto.

7.10.1. Race conditions y datos compartidos

Una race condition ocurre cuando el resultado de un programa depende del timing específico de múltiples hilos. Consideremos un ejemplo clásico:

```
1 // Variable global compartida entre hilos
2 int account_balance = 1000;
3
4 void* withdraw_money(void* amount_ptr) {
5     int amount = *(int*)amount_ptr;
6
7     // PROBLEMA: Esta secuencia NO es atómica
8     int current_balance = account_balance;    // LOAD
9     if (current_balance >= amount) {          // CHECK
10         sleep(1); // Simular procesamiento
11         account_balance = current_balance - amount; // STORE
12         printf("Retiro exitoso: $%d\n", amount);
13     } else {
14         printf("Fondos insuficientes\n");
15     }
16
17     return NULL;
18 }
```

Este código parece correcto a primera vista, pero contiene un bug sutil y peligroso. El problema está en la secuencia LOAD-CHECK-STORE que no es atómica:

```
1 Estado inicial: account_balance = 1000
2
3 Timeline paralelo:
```

```

4 T=0: Hilo 1 lee account_balance → current = 1000
5 T=1: Hilo 2 lee account_balance → current = 1000
6 T=2: Hilo 1 verifica: 1000 >= 800 ✓
7 T=3: Hilo 2 verifica: 1000 >= 600 ✓
8 T=4: Hilo 1 duerme (simulando procesamiento)
9 T=5: Hilo 2 duerme (simulando procesamiento)
10 T=6: Hilo 1 escribe: balance = 1000 - 800 = 200
11 T=7: Hilo 2 escribe: balance = 1000 - 600 = 400
12
13 Resultado final: balance = 400 (¡el último en escribir gana!)
14 Correcto sería: rechazar el segundo retiro (fondos insuficientes)

```

Este tipo de bug es particularmente insidioso porque puede no manifestarse durante testing. Si los hilos ejecutan con timings ligeramente diferentes, el código funciona correctamente. El bug solo aparece con ciertos timings específicos, haciéndolo difícil de reproducir y diagnosticar. Esto es un Heisenbug: un error que cambia su comportamiento cuando intentas observarlo.

La solución requiere hacer la secuencia completa CHECK-STORE atómica usando un mutex:

```

1 pthread_mutex_t balance_mutex = PTHREAD_MUTEX_INITIALIZER;
2
3 void* withdraw_money_safe(void* amount_ptr) {
4     int amount = *(int*)amount_ptr;
5
6     pthread_mutex_lock(&balance_mutex); // Sección crítica
7
8     int current_balance = account_balance;
9     if (current_balance >= amount) {
10         account_balance = current_balance - amount;
11         printf("Retiro exitoso: $%d, balance: $%d\n", amount, account_balance);
12     } else {
13         printf("Fondos insuficientes para $%d\n", amount);
14     }
15
16     pthread_mutex_unlock(&balance_mutex);
17
18     return NULL;
19 }

```

El mutex garantiza que solo un hilo puede ejecutar la sección crítica a la vez. Cuando un hilo llama `pthread_mutex_lock()`, adquiere el lock si está disponible, o se bloquea esperando si otro hilo lo posee. Solo cuando el hilo llama `pthread_mutex_unlock()` otro hilo puede entrar.

7.10.2. Herramientas de debugging

Los errores de concurrencia son notoriamente difíciles de detectar con testing tradicional. Afortunadamente, existen herramientas especializadas:

ThreadSanitizer es un detector de race conditions incluido en GCC y Clang. Instrumenta el código para rastrear todos los accesos a memoria y detectar patrones de acceso concurrente peligrosos:

```
1 # Compilar con ThreadSanitizer
2 gcc -g -fsanitize=thread -o programa programa.c -lpthread
3
4 # Ejecutar
5 ./programa
6
7 # Salida típica:
8 # WARNING: ThreadSanitizer: data race
9 # Write of size 4 at 0x7fff8b2c4020 by thread T2:
10 #   #0 withdraw_money programa.c:15
11 # Previous write of size 4 at 0x7fff8b2c4020 by thread T1:
12 #   #0 withdraw_money programa.c:15
```

ThreadSanitizer reporta exactamente qué líneas de código accedieron al mismo dato de forma no sincronizada, facilitando enormemente la localización del bug.

GDB también soporta debugging multihilo. Podemos listar todos los hilos activos, cambiar entre ellos, y examinar el stack de cada uno:

```
1 gdb ./programa
2
3 (gdb) run
4 (gdb) info threads          # Listar todos los hilos
5 (gdb) thread 2             # Cambiar al hilo 2
6 (gdb) bt                   # Backtrace del hilo actual
7 (gdb) thread apply all bt  # Backtrace de todos los hilos
```

Esta última funcionalidad es invaluable para diagnosticar deadlocks, donde múltiples hilos están bloqueados esperándose mutuamente.

7.10.3. Patterns de sincronización básicos

Algunos problemas de concurrencia aparecen tan frecuentemente que tienen soluciones establecidas. El patrón Producer-Consumer es fundamental:

```
1 #include <pthread.h>
2 #include <semaphore.h>
3
4 #define BUFFER_SIZE 10
5
6 int buffer[BUFFER_SIZE];
7 int in = 0, out = 0;
8
9 sem_t empty, full;
10 pthread_mutex_t buffer_mutex;
11
12 // Versión simplificada y didáctica
13 void* producer(void* arg) {
14     int item = 1;
15
```

```
16     while (1) {
17         sem_wait(&empty);
18         pthread_mutex_lock(&buffer_mutex);
19
20         buffer[in] = item++;
21         printf("Producido: %d en posición %d\n", buffer[in], in);
22         in = (in + 1) % BUFFER_SIZE;
23
24         pthread_mutex_unlock(&buffer_mutex);
25         sem_post(&full);
26
27         sleep(1); // Simular tiempo fijo (seguro y predecible)
28     }
29 }
30
31 void* consumer(void* arg) {
32     int consumer_id = *(int*)arg;
33
34     while (1) {
35         sem_wait(&full);
36         pthread_mutex_lock(&buffer_mutex);
37
38         int item = buffer[out];
39         printf("Consumidor %d tomó: %d de posición %d\n",
40             consumer_id, item, out);
41         out = (out + 1) % BUFFER_SIZE;
42
43         pthread_mutex_unlock(&buffer_mutex);
44         sem_post(&empty);
45
46         sleep(1); // Simular tiempo fijo
47     }
48 }
```

Este patrón usa dos semáforos para coordinar productores y consumidores. El semáforo `empty` cuenta los espacios libres en el buffer, mientras `full` cuenta los items disponibles. Los productores esperan en `empty` antes de producir (asegurando hay espacio), luego señalan `full` después de producir (indicando hay un nuevo item). Los consumidores hacen lo opuesto.

El `mutex buffer_mutex` protege el acceso al buffer compartido, garantizando que solo un hilo modifique las variables `in` y `out` a la vez. Esta combinación de semáforos (para coordinación) y mutex (para exclusión mutua) es un patrón ubicuo en programación concurrente.

7.11. Tendencias y tecnologías modernas

El campo de la concurrencia continúa evolucionando. Lenguajes y sistemas modernos exploran nuevos modelos de programación que simplifican o evitan completamente algunos problemas tradicionales de hilos.

7.11.1. Green threads y corrutinas

Go introdujo las goroutines, una implementación sofisticada del modelo M:N. Miles de goroutines (esencialmente ULT) se multiplexan sobre un número pequeño de threads OS usando un scheduler preemptive con work-stealing. El scheduler de Go puede migrar goroutines entre threads, balanceando carga automáticamente. Cada goroutine comienza con un stack de solo 2KB que crece dinámicamente si es necesario, permitiendo millones de goroutines concurrentes.

Rust toma un enfoque diferente. En lugar de hilos, Rust promueve futures y `async/await` para concurrencia. Una función `async` retorna un future que representa computación pendiente. El runtime (como Tokio) ejecuta múltiples futures concurrentemente en un pool de threads, pero sin el overhead de context switch entre hilos. Crucialmente, el sistema de ownership de Rust previene race conditions en tiempo de compilación: el compilador rechaza código que podría causar data races.

Estos modelos representan un cambio filosófico. Los hilos tradicionales son preemptive: el scheduler puede suspenderlos en cualquier momento. Las goroutines y corrutinas son cooperativas en la superficie pero preemptive internamente: ceden el control en puntos bien definidos (llamadas a funciones `async`, operaciones de I/O), pero el runtime puede preemptarlas en esos puntos. Esto combina la eficiencia de la cooperación con las garantías de progreso de la preemption.

7.11.2. Thread affinity y NUMA

En sistemas con múltiples procesadores o muchos cores, mantener un hilo ejecutando en el mismo core puede mejorar dramáticamente el rendimiento. Esto se debe a la jerarquía de caches: cada core tiene caches L1 y L2 privadas que contienen los datos que el hilo usó recientemente. Si el scheduler migra el hilo a otro core, esas caches se pierden y deben reconstruirse.

```
1 // NOTA: Esta función es específica de Linux (pthread_setaffinity_np).
2 // En sistemas BSD/macOS no está disponible.
3 #ifndef __linux__
4 #include <sched.h>
5
6 void set_thread_affinity(pthread_t thread, int cpu_core) {
7     cpu_set_t cpuset;
8     CPU_ZERO(&cpuset);
9     CPU_SET(cpu_core, &cpuset);
10
11     int result = pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);
12     if (result != 0) {
13         perror("pthread_setaffinity_np");
14     }
15 }
16 #else
17 void set_thread_affinity(pthread_t thread, int cpu_core) {
```

```
18 // No-op en sistemas no Linux
19 (void)thread;
20 (void)cpu_core;
21 printf("Advertencia: thread affinity solo disponible en Linux\n");
22 }
23 #endif
24
25 // Ejemplo de uso para optimizar cache locality
26 void create_affine_threads() {
27     pthread_t threads[4];
28
29     for (int i = 0; i < 4; i++) {
30         pthread_create(&threads[i], NULL, worker_function, &i);
31         set_thread_affinity(threads[i], i); // Cada hilo en core específico
32     }
33 }
```

La función `pthread_setaffinity_np()` restringe un hilo a ejecutar solo en cores específicos. Esto es especialmente importante en arquitecturas NUMA (Non-Uniform Memory Access), donde cada procesador tiene memoria local rápida y puede acceder memoria de otros procesadores más lentamente. Asignando hilos a cores específicos y asegurando que usen memoria local, podemos reducir la latencia de acceso a memoria significativamente.

Las bases de datos modernas usan thread affinity agresivamente. Por ejemplo, PostgreSQL puede configurarse para dedicar cores específicos a diferentes tipos de trabajos: algunos cores manejan queries transaccionales cortos, otros manejan analíticas largas, y otros gestionan I/O y checkpointing. Esta segregación previene que cargas de trabajo diferentes compitan por recursos de cache.

7.12. Síntesis y conexiones

Hemos recorrido un camino desde los conceptos fundamentales de hilos hasta implementaciones prácticas y desafíos reales. Es momento de consolidar las ideas clave y preparar el terreno para el siguiente capítulo.

7.12.1. Puntos clave para evaluación

La distinción entre ULT y KLT no es meramente académica: representa diferentes trade-offs fundamentales en el diseño de sistemas. Los ULT ofrecen eficiencia extrema a costa de paralelismo, mientras los KLT sacrifican algo de eficiencia para ganar paralelismo real y mejor integración con el sistema operativo. El modelo M:N intenta lo mejor de ambos mundos pero añade complejidad significativa. La problemática de planificación mixta, ilustrada en nuestro ejercicio detallado, revela cómo los hilos ULT están fundamentalmente limitados por el quantum asignado a su KLT contene-

dor. Este constraint debe considerarse cuidadosamente al diseñar sistemas que mezclan ambos tipos de hilos. Las APIs de pthreads proporcionan los building blocks para programación concurrente, pero usar estos bloques correctamente requiere entender profundamente el modelo de memoria compartida y los peligros de las race conditions. El ciclo completo `create` -> `join/detach` y la gestión cuidadosa de la sincronización no son opcionales sino absolutamente esenciales para código correcto.

7.12.2. Errores conceptuales frecuentes

Algunas misconcepciones comunes pueden llevar a diseños fundamentalmente incorrectos:

Más hilos siempre significa mejor rendimiento: Falso. Cada hilo adicional añade overhead de context switch y competencia por recursos compartidos. Más allá de cierto punto (típicamente relacionado con el número de cores disponibles), agregar hilos degrada el rendimiento debido a thrashing y contention.

Las variables locales son automáticamente thread-safe: Solo parcialmente cierto. Las variables locales en el stack son privadas de cada hilo, pero si un hilo pasa un puntero a su variable local a otro hilo (o almacena el puntero en una estructura compartida), esa variable se vuelve compartida y requiere sincronización.

Los ULT son más simples que los KLT: Paradójicamente falso. Aunque el context switch ULT es más simple, implementar un scheduler completo de ULT con todas sus estructuras de datos y políticas es significativamente más complejo que usar la API de pthreads que delega la complejidad al kernel.

7.12.3. Preparación para próximos capítulos

Los hilos introducen la necesidad crítica de sincronización. Hemos visto glimpses de esta problemática con nuestro ejemplo de race condition en retiros bancarios, pero apenas rasguñamos la superficie. El siguiente capítulo explorará sistemáticamente los mecanismos que permiten a hilos coordinarse de forma segura y eficiente.

Veremos mutexes para exclusión mutua, permitiendo que solo un hilo acceda a una sección crítica a la vez. Exploraremos semáforos, una primitiva más general que puede resolver problemas de sincronización complejos. Las condition variables nos permitirán implementar espera eficiente sin busy-waiting. Y los read-write locks optimizarán el caso común donde múltiples hilos pueden leer simultáneamente pero las escrituras requieren acceso exclusivo.

```
1 // Problema que veremos en el próximo capítulo
2 pthread_mutex_t resource_mutex;
3 pthread_cond_t resource_available;
4 int shared_resource_count = 0;
5
6 // ¿Cómo coordinamos múltiples hilos que compiten por recursos limitados?
7 // ¿Cómo evitamos deadlock cuando un hilo necesita múltiples recursos?
8 // ¿Cómo implementamos fairness entre hilos productores y consumidores?
```

También enfrentaremos problemas clásicos de concurrencia como el productor-consumidor con buffer limitado, lectores-escritores con diferentes políticas de prioridad, y el famoso problema de los filósofos comensales que ilustra deadlock y su prevención. Estos problemas no son meramente ejercicios académicos: representan patrones que aparecen constantemente en sistemas reales.

7.12.4. Laboratorio propuesto

Para consolidar tu comprensión, considera estas prácticas:

Práctica 1: Benchmark ULT vs KLT - Implementa una simulación simple de ULT usando `setjmp/longjmp` y compárala con pthreads. Mide context switches por segundo variando el trabajo realizado en cada switch. Grafica cómo el overhead relativo cambia con diferentes duraciones de quantum.

Práctica 2: Pool de hilos configurable - Extiende nuestro ejemplo de pool para que pueda manejar diferentes tipos de tareas con prioridades. Implementa métricas como latencia promedio de procesamiento, utilización de hilos, y tamaño de cola. Experimenta con diferentes tamaños de pool bajo cargas variables.

Práctica 3: Simulador de planificación mixta - Crea un simulador que reproduzca ejercicios como nuestro Gantt chart. Permite configurar procesos con diferentes números de KLT y ULT, schedulers internos (FIFO, Round Robin), y operaciones de I/O. Visualiza la ejecución y calcula métricas automáticamente.

La comprensión profunda de los hilos es fundamental para el diseño de sistemas concurrentes eficientes. En aplicaciones modernas, la elección correcta entre ULT, KLT o modelos híbridos, junto con una arquitectura de pool bien diseñada, puede determinar la diferencia entre un sistema que escala gracefully y uno que colapsa bajo carga. Armado con este conocimiento, estás preparado para explorar los mecanismos de sincronización que hacen posible la coordinación segura entre hilos.

8 Sincronización

8.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Identificar problemas de concurrencia: race conditions, deadlock, starvation
- Explicar qué son las operaciones atómicas y por qué son necesarias
- Comprender las condiciones de Bernstein para la ejecución concurrente
- Analizar soluciones de sincronización a nivel software y hardware
- Implementar soluciones usando mutex, semáforos y variables de condición
- Resolver problemas clásicos: Productor-Consumidor, Lectores-Escritores
- Aplicar sincronización en escenarios reales usando analogías cotidianas
- Analizar y prevenir deadlocks usando técnicas formales
- Programar soluciones thread-safe en C usando pthreads
- Evaluar el overhead de diferentes primitivas de sincronización

8.2. Introducción y Contexto

8.2.1. ¿Por qué necesitamos sincronización?

Los sistemas concurrentes son como una orquesta: múltiples instrumentos tocando simultáneamente pueden crear una sinfonía hermosa o un desastre. La diferencia está en la coordinación. En los sistemas operativos modernos, donde múltiples procesos e hilos compiten por recursos compartidos, la sincronización es el director de orquesta que mantiene todo en armonía.

Imaginemos un supermercado en hora pico. Tenemos 20 cajas funcionando independientemente, un empleado que debe ordenar las filas (pero solo puede estar en un lugar a la vez), un sistema de promociones que solo permite una aplicación simultánea, y clientes que llegan aleatoriamente eligiendo cajas según su conveniencia. Este escenario, aparentemente simple, está plagado de problemas potenciales de concurrencia.

Sin una coordinación adecuada, pueden ocurrir los siguientes problemas:

Race condition: Dos cajeros intentan usar el sistema de promociones simultáneamente, corrompiendo la base de datos
Starvation: Una caja siempre tiene fila larga porque el empleado

nunca la atiende Deadlock: El empleado espera que se libere una caja para ordenarla, pero el cajero espera que el empleado termine de ordenar para continuar Inconsistencia: El contador total de ventas se pierde cuando dos cajas lo actualizan al mismo tiempo

Esta analogía del supermercado nos permite mapear conceptos abstractos de sincronización a situaciones cotidianas. Las cajas registradoras representan un array de recursos limitados, el sistema de promociones es un recurso de exclusión mutua, el empleado ordenador funciona como un recurso único móvil, y el contador de ventas ejemplifica una variable compartida crítica. Los cajeros y clientes actúan como threads que acceden concurrentemente a estos recursos, mientras que la cola de clientes por caja implementa el clásico patrón productor-consumidor.

```

1 RECURSOS DEL SUPERMERCADO (Variables compartidas):
2 - Cajas registradoras (20) ← Array de recursos limitados
3 - Sistema de promociones (1) ← Recurso exclusivo mutuo
4 - Empleado ordenador (1) ← Recurso único móvil
5 - Contador total de ventas ← Variable compartida crítica
6 - Cola de clientes por caja ← Buffer productor-consumidor
7
8 PROCESOS/HILOS:
9 - Cajeros (threads) ← Acceden a recursos concurrentemente
10 - Clientes (threads) ← Productores de trabajo
11 - Sistema de facturación ← Consumidor de transacciones
12 - Empleado (thread especial) ← Administrador de recursos

```

8.3. El Problema Fundamental: Race Conditions

Una race condition ocurre cuando el resultado de una operación depende del orden específico de ejecución de múltiples operaciones concurrentes sobre datos compartidos. El término “race” (carrera) es apropiado: los procesos compiten entre sí, y el ganador determina el resultado final, haciendo que el comportamiento del sistema sea impredecible.

Consideremos un ejemplo concreto: dos cajeros actualizando el contador de ventas totales. A nivel de código de alto nivel, parece simple:

```

1 // Dos cajeros actualizando ventas totales
2 int ventas_totales = 0;
3
4 // Cajero 1 // Cajero 2
5 ventas_totales += 100; ventas_totales += 200;

```

Sin embargo, lo que parece una operación atómica en C en realidad se traduce a múltiples instrucciones de lenguaje de máquina. Cuando observamos el assembly generado, vemos la verdadera complejidad:

```

1 ; Cajero 1 ; Cajero 2
2 LOAD R1, [ventas_totales] LOAD R2, [ventas_totales]
3 ADD R1, 100 ADD R2, 200

```

```
4  STORE [ventas_totales], R1    STORE [ventas_totales], R2
```

El problema surge porque la operación *ventas_totales* + = *valor* no es atómica. En realidad, consiste en tres operaciones distintas:

- Cargar el valor actual de memoria a un registro
- Incrementar el valor en el registro
- Almacenar el nuevo valor de vuelta en memoria

Si dos threads ejecutan estas operaciones intercaladamente, el resultado final puede ser incorrecto, perdiendo una o ambas actualizaciones.

Los resultados posibles varían desde el correcto (*ventas_totales* = 300) hasta completamente incorrectos donde se pierde la venta de uno u otro cajero (*ventas_totales* = 100o200). Esta impredecibilidad es lo que hace a las race conditions tan peligrosas y difíciles de depurar: el bug puede no manifestarse en pruebas, apareciendo solo bajo condiciones específicas de carga en producción.

8.4. Sección Crítica y Condiciones de Bernstein

8.4.1. Sección Crítica

Cada programa concurrente tiene porciones de código donde accede a recursos compartidos. Estas porciones se denominan secciones críticas, y deben ejecutarse de manera atómica, es decir, sin interrupciones que permitan que otro proceso acceda simultáneamente al mismo recurso.

La estructura general de un programa con sección crítica sigue un patrón bien definido:

```
1  do {
2      // Protocolo de entrada
3      entrada_seccion_critica();
4
5      // SECCIÓN CRÍTICA
6      // Acceso a recursos compartidos
7
8      // Protocolo de salida
9      salida_seccion_critica();
10
11     // Sección no crítica
12     hacer_trabajo_local();
13
14 } while (true);
```

Para que una solución al problema de la sección crítica sea correcta, debe satisfacer cuatro requisitos fundamentales. Primero, debe garantizar exclusión mutua: solo un proceso puede estar en su sección crítica a la vez. Segundo, debe asegurar progreso: si ningún proceso está en su sección crítica, la decisión de quién entrará no puede posponerse indefinidamente. Tercero, debe propor-

cionar espera acotada: existe un límite en el número de veces que otros procesos pueden entrar a su sección crítica antes de que un proceso en espera pueda hacerlo. Finalmente, la solución no debe asumir nada sobre las velocidades relativas de los procesos.

8.4.2. Condiciones de Bernstein

Antes de aplicar mecanismos de sincronización, es útil determinar si dos procesos realmente necesitan sincronizarse. Las Condiciones de Bernstein proporcionan un criterio matemático para esta decisión.

Para que dos procesos P_1 y P_2 puedan ejecutarse concurrentemente de manera segura, deben cumplirse tres condiciones simultáneamente: Sean R_1 y R_2 los conjuntos de variables que leen los procesos respectivamente, y W_1 y W_2 los conjuntos de variables que escriben. Entonces:

$$R_1 \cap W_2 = \emptyset (P_1 \text{ no lee lo que } P_2 \text{ escribe})$$

$$W_1 \cap R_2 = \emptyset (P_1 \text{ no escribe lo que } P_2 \text{ lee})$$

$$W_1 \cap W_2 = \emptyset (P_1 \text{ y } P_2 \text{ no escriben las mismas variables})$$

Consideremos un ejemplo donde estas condiciones se violan:

```
1 // Proceso 1:  $_1R = \{x\}$ ,  $_1W = \{y\}$ 
2  $y = x + 10$ ;
3
4 // Proceso 2:  $_2R = \{y\}$ ,  $_2W = \{x\}$ 
5  $x = y * 2$ ;
```

En este caso, hay violaciones claras: $W_1 \cap R_2 = y$ porque el Proceso 1 escribe y que el Proceso 2 lee, y $R_1 \cap W_2 = x$ porque el Proceso 1 lee x que el Proceso 2 escribe. Por tanto, estos procesos no pueden ejecutarse concurrentemente sin sincronización, ya que el resultado dependerá del orden de ejecución.

8.5. Soluciones a Nivel Software

8.5.1. Evolución Histórica de las Soluciones

La historia de la sincronización en sistemas operativos es una historia de intentos, fallas y mejoras incrementales. Cada solución fallida nos enseñó algo sobre la complejidad del problema y nos acercó a soluciones correctas.

8.5.1.1. Primeras Aproximaciones: Variables de Control

Los primeros intentos de resolver el problema de la sección crítica usaron variables compartidas simples para coordinar el acceso. Aunque intuitivos, revelaron sutilezas inesperadas. El primer intento, el turno simple, usaba una variable compartida que indicaba qué proceso podía entrar:

```
1 int turno = 1;
2
3 // Proceso 1
4 while (turno != 1);
5 // Sección crítica
6 turno = 2;
7
8 // Proceso 2
9 while (turno != 2);
10 // Sección crítica
11 turno = 1;
```

Esta solución garantiza exclusión mutua: solo el proceso cuyo turno corresponde puede entrar. Sin embargo, viola gravemente la condición de progreso. Si un proceso termina de usar su sección crítica y no quiere volver a entrar inmediatamente, el otro proceso queda bloqueado indefinidamente esperando su turno, incluso si nadie más está usando el recurso. Es como tener dos personas compartiendo un baño, donde cada una solo puede usarlo en turnos estrictos, incluso si la otra persona está durmiendo.

El segundo intento mejoró la situación usando *flags* independientes:

```
1 bool flag[2] = {false, false};
2
3 // Proceso i
4 flag[i] = true;
5 while (flag[j]); // j = 1-i
6 // Sección crítica
7 flag[i] = false;
```

Aquí, cada proceso indica su intención de entrar levantando su flag, luego verifica si el otro proceso también está interesado. El problema es una race condition clásica: ambos procesos pueden leer el flag del otro como **false** antes de que cualquiera lo establezca en **true**, resultando en que ambos entren simultáneamente a la sección crítica. La exclusión mutua se viola.

El tercer intento intentó ser más cortés, haciendo que los procesos cedan ante conflictos:

```
1 bool flag[2] = {false, false};
2
3 // Proceso i
4 flag[i] = true;
5 while (flag[j]) {
6     flag[i] = false;
7     // Esperar tiempo aleatorio
8     flag[i] = true;
9 }
10 // Sección crítica
11 flag[i] = false;
```


Esta solución introduce un nuevo problema: livelock. Ambos procesos pueden entrar en un ciclo donde continuamente bajan y levantan sus flags, cada uno cediendo cortésmente al otro, pero ninguno progresando nunca. Es como dos personas en un pasillo estrecho, cada una haciéndose a un lado para que pase la otra, resultando en que ambas se mueven en la misma dirección indefinidamente.

8.5.2. Solución de Peterson (1981)

Gary Peterson finalmente resolvió el problema en 1981 con una solución elegante que combina ideas de los intentos anteriores. La solución de Peterson es la *primera solución correcta* puramente de software para dos procesos:

```
1  bool flag[2] = {false, false};
2  int turn = 0;
3
4  // Proceso i (donde j = 1-i)
5  void peterson_enter(int i) {
6      flag[i] = true;      // Mostrar interés
7      turn = j;            // Ceder el turno al otro
8      while (flag[j] && turn == j); // Esperar si el otro está interesado y
        tiene turno
9  }
10
11 void peterson_exit(int i) {
12     flag[i] = false;      // No tengo más interés
13 }
14
15 // Uso completo
16 void proceso_i() {
17     while (true) {
18         peterson_enter(i);
19
20         // SECCIÓN CRÍTICA
21         seccion_critica();
22
23         peterson_exit(i);
24
25         // Sección no crítica
26         seccion_no_critica();
27     }
28 }
```

La brillantez de Peterson está en cómo combina el flag de intención con la cesión de turno. Cuando un proceso quiere entrar, primero levanta su flag mostrando interés, luego cede el turno al otro proceso. Solo espera si el otro proceso también está interesado y tiene el turno. Esta combinación garantiza las cuatro propiedades necesarias.

La corrección de Peterson se puede demostrar formalmente: Exclusión Mutua: Si ambos procesos están en el while, uno debe tener $\text{turn} = i$ y el otro $\text{turn} = j$. Como turn es una variable única compartida, esto es imposible. Por tanto, al menos uno saldrá del while.

Progreso: Si un proceso no está interesado ($\text{flag}[j] = \text{false}$), el otro puede entrar inmediatamente sin importar el valor de turn .

Espera Acotada: El proceso que ejecutó $\text{turn} = j$ más recientemente cederá el turno, garantizando que el otro proceso pueda entrar después de a lo sumo una espera.

Sin embargo, la solución de Peterson tiene limitaciones importantes. Funciona solo para dos procesos, requiere busy waiting que desperdicia ciclos de CPU, y asume orden secuencial de memoria. En procesadores modernos con reordenamiento de instrucciones y cachés múltiples, puede fallar sin barreras de memoria explícitas.

8.6. Soluciones a Nivel Hardware

Las limitaciones de las soluciones puramente de software llevaron al desarrollo de primitivas atómicas implementadas directamente en hardware. Estas instrucciones ejecutan múltiples operaciones como una unidad indivisible, proporcionando los bloques fundamentales para construir mecanismos de sincronización más eficientes.

8.6.1. Primitivos Atómicos

Una operación atómica es aquella que se ejecuta completamente sin posibilidad de interrupción. Desde la perspectiva de otros procesos, la operación ocurre instantáneamente.

8.6.1.1. Test-and-Set (Hardware)

La instrucción Test-and-Set lee un valor, lo cambia a true, y retorna el valor original, todo en una operación atómica:

```
1 // Implementada en hardware - ATÓMICA
2 bool test_and_set(bool* target) {
3     bool old_value = *target;
4     *target = true;
5     return old_value;
6 }
7
8 // Uso para mutex
9 bool lock = false; // false = libre, true = ocupado
10
11 void acquire_lock() {
```

```
12     while (test_and_set(&lock)) {
13         // Busy waiting (spin lock)
14         // Continuar intentando hasta obtener el lock
15     }
16 }
17
18 void release_lock() {
19     lock = false;
20 }
```

Este primitivo permite implementar spin locks de manera correcta. Un proceso intenta adquirir el lock repetidamente con test-and-set hasta que tiene éxito. Cuando obtiene el lock (test-and-set retorna false), puede entrar a su sección crítica. Al salir, simplemente establece el lock en false.

Ventajas y desventajas de Test-and-Set La simplicidad de test-and-set lo hace fácil de implementar en hardware y funciona para cualquier número de procesos, garantizando exclusión mutua. Sin embargo, sufre de busy waiting: un proceso bloqueado continúa consumiendo ciclos de CPU chequeando el lock repetidamente. Además, no garantiza espera acotada, lo que puede causar starvation donde un proceso espera indefinidamente mientras otros continuamente obtienen el lock.

8.6.1.2. Compare-and-Swap (CAS)

Compare-and-Swap es más flexible que test-and-set, permitiendo actualizar un valor solo si tiene un valor esperado específico:

```
1 // Más flexible que test-and-set
2 bool compare_and_swap(int* ptr, int expected, int new_value) {
3     if (*ptr == expected) {
4         *ptr = new_value;
5         return true;
6     }
7     return false;
8 }
9
10 // Implementar contador atómico
11 void atomic_increment(int* counter) {
12     int old_value, new_value;
13     do {
14         old_value = *counter;
15         new_value = old_value + 1;
16     } while (!compare_and_swap(counter, old_value, new_value));
17 }
```

CAS es la base de muchas estructuras de datos lock-free modernas. En el ejemplo, incrementamos un contador atómicamente: leemos el valor actual, calculamos el nuevo valor, y usamos CAS para actualizarlo solo si no cambió entre la lectura y la actualización. Si falló (otro thread lo cambió), reintentamos.

8.6.1.3. Fetch-and-Add

Fetch-and-Add retorna el valor anterior de una variable y le suma un valor dado, todo atómicamente:

```
1 // Retorna valor anterior y suma atómicamente
2 int fetch_and_add(int* ptr, int value) {
3     int old_value = *ptr;
4     *ptr += value;
5     return old_value;
6 }
7
8 // Implementar ticket lock (espera acotada)
9 typedef struct {
10     int ticket;
11     int turn;
12 } ticket_lock_t;
13
14 void ticket_acquire(ticket_lock_t* lock) {
15     int my_ticket = fetch_and_add(&lock->ticket, 1);
16     while (lock->turn != my_ticket); // Esperar mi turno
17 }
18
19 void ticket_release(ticket_lock_t* lock) {
20     lock->turn++; // Dar turno al siguiente
21 }
```

Este primitivo permite implementar ticket locks, que garantizan espera acotada similar a un sistema de turnos en un banco. Cada proceso que llega obtiene un número de ticket con fetch-and-add, luego espera hasta que su número sea el turno actual. Esto garantiza fairness: los procesos entran en el orden que llegaron.

8.7. Soluciones del Sistema Operativo: Semáforos

Los primitivos de hardware resolvieron el problema de la atomicidad, pero dejaron el problema del busy waiting. Los sistemas operativos necesitaban un mecanismo de más alto nivel que bloqueara procesos eficientemente en lugar de desperdiciar CPU. *Edsger Dijkstra* inventó los semáforos en 1965, revolucionando la sincronización en sistemas operativos.

8.7.1. Definición y Operaciones

Un semáforo es esencialmente un contador entero no negativo con dos operaciones atómicas. A diferencia de los spin locks, un semáforo puede bloquear un proceso, poniéndolo a dormir hasta que el recurso esté disponible.

```
1 typedef struct {
2     int value;
```

```

3     queue_t waiting_queue;
4 } semaphore_t;
5
6 // P() o wait() - Decrementar y posiblemente bloquear
7 void sem_wait(semaphore_t* sem) {
8     sem->value--;
9     if (sem->value < 0) {
10         // Bloquear proceso y agregarlo a la cola
11         add_to_queue(&sem->waiting_queue, current_process);
12         block_current_process();
13     }
14 }
15
16 // V() o signal() - Incrementar y despertar
17 void sem_post(semaphore_t* sem) {
18     sem->value++;
19     if (sem->value <= 0) {
20         // Hay procesos esperando, despertar uno
21         process_t* p = remove_from_queue(&sem->waiting_queue);
22         wakeup_process(p);
23     }
24 }

```

La operación `sem_wait()` (también llamada P, por el holandés “proberen” = probar) decrementa el contador. Si el resultado es negativo, el proceso se bloquea y se agrega a una cola de espera. La operación `sem_post()` (también llamada V, por “verhogen” = incrementar) incrementa el contador. Si hay procesos esperando (valor ≤ 0), despierta uno de ellos.

La semántica del valor del semáforo es crucial para entender su funcionamiento:

- Valor positivo: número de recursos disponibles
- Valor cero: no hay recursos disponibles, pero tampoco procesos esperando
- Valor negativo: su valor absoluto indica el número de procesos esperando

Esta interpretación explica por qué `post` despierta un proceso cuando el valor es ≤ 0 : un valor no positivo implica que hay procesos bloqueados esperando el recurso.

Nota: Esta interpretación (valor negativo = procesos esperando) asume la implementación donde `sem_wait()` decrementa primero y luego bloquea si el valor es negativo. Esta es la implementación clásica mostrada en el pseudocódigo. En la práctica, las implementaciones reales de POSIX semáforos (`sem_wait()`/`sem_post()`) no exponen el valor negativo; mantienen un contador no negativo y una cola separada de procesos bloqueados.

8.7.2. Tipos de Semáforos

Los semáforos vienen en dos variedades principales, cada una optimizada para casos de uso específicos.

8.7.2.1. Semáforo Binario (Mutex)

Un semáforo binario, también llamado mutex (mutual exclusion), solo puede tener valores 0 o 1. Se usa principalmente para proteger secciones críticas:

```
1 semaphore_t mutex;
2 sem_init(&mutex, 1); // Inicializar en 1 (disponible)
3
4 void critical_section() {
5     sem_wait(&mutex); // P(mutex) - Obtener exclusión mutua
6
7     // SECCIÓN CRÍTICA
8     // Solo un proceso puede estar aquí
9
10    sem_post(&mutex); // V(mutex) - Liberar exclusión mutua
11 }
```

Cuando el mutex vale 1, el recurso está disponible. El primer proceso que ejecuta `sem_wait()` decrementa el mutex a 0 y entra. Cualquier otro proceso que intente entrar se bloquea hasta que el primer proceso ejecute `sem_post()`, incrementando el mutex de vuelta a 1.

8.7.2.2. Semáforo Contador

Un semáforo contador puede tener cualquier valor no negativo, representando múltiples instancias de un recurso:

```
1 #define POOL_SIZE 5
2 semaphore_t connection_pool;
3 sem_init(&connection_pool, POOL_SIZE);
4
5 void use_connection() {
6     sem_wait(&connection_pool); // Obtener conexión
7
8     // Usar conexión de base de datos
9     execute_query();
10
11    sem_post(&connection_pool); // Liberar conexión
12 }
```

Este patrón es ideal para administrar pools de recursos limitados. Un servidor web con 5 conexiones de base de datos usa un semáforo inicializado en 5. Cada cliente que obtiene una conexión ejecuta `sem_wait()`, decrementando el contador. Cuando las 5 conexiones están en uso, nuevos clientes se bloquean hasta que alguien libere una conexión con `sem_post()`.

8.7.3. Usos Principales de Semáforos

Los semáforos son herramientas versátiles que pueden resolver múltiples problemas de sincronización. Veamos sus patrones de uso más comunes.

Para *exclusión mutua*, un semáforo binario inicializado en 1 protege la sección crítica:

```
1 semaphore_t mutex = 1;
2
3 void proceso() {
4     sem_wait(&mutex);    // Entrar a sección crítica
5     // Sección crítica
6     sem_post(&mutex);    // Salir de sección crítica
7 }
```

Para *limitar acceso a N instancias* de un recurso, un semáforo contador inicializado en N controla cuántos procesos pueden usar el recurso simultáneamente:

```
1 semaphore_t recursos = N;
2
3 void usar_recurso() {
4     sem_wait(&recursos); // Obtener uno de N recursos
5     // Usar recurso
6     sem_post(&recursos); // Liberar recurso
7 }
```

Para *ordenar ejecución entre procesos*, un semáforo inicializado en 0 actúa como señal de sincronización:

```
1 semaphore_t sincronizacion = 0;
2
3 void proceso_A() {
4     // Hacer trabajo A
5     trabajo_A();
6     sem_post(&sincronizacion); // Señalar que A terminó
7 }
8
9 void proceso_B() {
10    sem_wait(&sincronizacion); // Esperar que A termine
11    // Hacer trabajo B (que depende de A)
12    trabajo_B();
13 }
```

Aquí, el proceso B no puede comenzar hasta que A termine. El semáforo en 0 garantiza que B se bloqueará en wait hasta que A señale completitud con post.

El patrón más complejo y útil es el problema *productor-consumidor*, que requiere tres semáforos trabajando en conjunto:

```
1 #define BUFFER_SIZE 10
2
3 semaphore_t empty = BUFFER_SIZE; // Espacios vacíos
4 semaphore_t full = 0;            // Elementos llenos
5 semaphore_t mutex = 1;           // Exclusión mutua
6
```

```
7 void productor() {
8     while (true) {
9         // Producir elemento
10        item = produce_item();
11
12        sem_wait(&empty);    // Esperar espacio vacío
13        sem_wait(&mutex);    // Obtener acceso al buffer
14
15        add_to_buffer(item); // Agregar al buffer
16
17        sem_post(&mutex);    // Liberar acceso al buffer
18        sem_post(&full);     // Señalar elemento disponible
19    }
20 }
21
22 void consumidor() {
23     while (true) {
24        sem_wait(&full);      // Esperar elemento disponible
25        sem_wait(&mutex);    // Obtener acceso al buffer
26
27        item = remove_from_buffer(); // Quitar del buffer
28
29        sem_post(&mutex);    // Liberar acceso al buffer
30        sem_post(&empty);    // Señalar espacio vacío
31
32        consume_item(item); // Consumir elemento
33    }
34 }
```

En este patrón, `empty` cuenta espacios vacíos disponibles (inicia en `BUFFER_SIZE`), `full` cuenta elementos disponibles para consumir (inicia en 0), y `mutex` protege el acceso al buffer compartido (inicia en 1). El productor primero espera un espacio vacío (`wait empty`), luego obtiene exclusión mutua (`wait mutex`), agrega su elemento, libera el mutex (`post mutex`), y finalmente señala un nuevo elemento disponible (`post full`). El consumidor hace lo inverso: espera un elemento (`wait full`), obtiene el mutex (`wait mutex`), extrae el elemento, libera el mutex (`post mutex`), y señala un espacio vacío (`post empty`). El orden de las operaciones es crítico. Si el productor obtuviera el mutex antes de verificar `empty`, podría quedarse bloqueado sosteniendo el mutex, impidiendo que el consumidor libere espacio, causando deadlock.

8.8. Ejemplo Práctico: Control de Cochera

Ahora que entendemos los semáforos, apliquémoslos a un problema realista. Vamos a diseñar el sistema de control para una cochera automatizada, un escenario que involucra múltiples recursos compartidos y diferentes patrones de sincronización.

8.8.1. Planteamiento del Problema

Una cochera tiene **20 espacios** para autos, **1 entrada** (con barrera), **2 salidas** (con barreras) y un **Sistema de control** que debe llevar cuenta de espacios ocupados.

Requerimientos: 1. No permitir entrada si cochera está llena 2. Controlar acceso exclusivo a entrada y salidas 3. Mantener contador preciso de autos 4. Evitar deadlock entre entrada y salidas

8.8.2. Solución con Semáforos

```
1  #include <stdio.h>
2  #include <semaphore.h>
3  #include <pthread.h>
4
5  #define CAPACIDAD_COCHERA 20
6  #define NUM_AUTOS 100
7
8  typedef struct {
9      // Semáforo contador para espacios disponibles
10     sem_t espacios_disponibles;
11
12     // Mutex para acceso exclusivo a entrada
13     sem_t mutex_entrada;
14
15     // Mutex para acceso exclusivo a cada salida
16     sem_t mutex_salida1;
17     sem_t mutex_salida2;
18
19     // Mutex para el contador global
20     sem_t mutex_contador;
21
22     // Estado de la cochera
23     int autos_dentro;
24     int total_entradas;
25     int total_salidas;
26 } cochera_t;
27
28 cochera_t cochera;
29
30 void inicializar_cochera() {
31     // Semáforo contador para espacios disponibles (compartido entre hilos)
32     sem_init(&cochera.espacios_disponibles, 0, CAPACIDAD_COCHERA);
33
34     // NOTA: Para exclusión mutua entre hilos, es más convencional usar
35     // pthread_mutex_t.
36     // Se usan semáforos binarios aquí por consistencia, pero en la práctica
37     // se prefiere mutex.
38     sem_init(&cochera.mutex_entrada, 0, 1);
39     sem_init(&cochera.mutex_salida1, 0, 1);
40     sem_init(&cochera.mutex_salida2, 0, 1);
41     sem_init(&cochera.mutex_contador, 0, 1);
42     // Inicializar estado
```

```

43     cochera.autos_dentro = 0;
44     cochera.total_entradas = 0;
45     cochera.total_salidas = 0;
46
47     printf("Cochera inicializada: %d espacios disponibles\n",
48           CAPACIDAD_COCHERA);
49 }
50 void* auto_entrando(void* arg) {
51     int auto_id = *(int*)arg;
52     free(arg); // Liberar el argumento dinámico
53
54     printf("Auto %d llegó a la cochera\n", auto_id);
55
56     // 1. Verificar si hay espacio disponible
57     printf("Auto %d esperando espacio...\n", auto_id);
58     sem_wait(&cochera.espacios_disponibles); // Bloquea si cochera llena
59
60     // 2. Hay espacio garantizado, obtener acceso exclusivo a entrada
61     printf("Auto %d esperando acceso a entrada...\n", auto_id);
62     sem_wait(&cochera.mutex_entrada);
63
64     // 3. SECCIÓN CRÍTICA: Procesar entrada
65     printf("Auto %d entrando a cochera\n", auto_id);
66
67     // Simular tiempo de entrada (abrir barrera, validar ticket, etc.)
68     sleep(1);
69
70     // Actualizar contador global de manera thread-safe
71     sem_wait(&cochera.mutex_contador);
72     cochera.autos_dentro++;
73     cochera.total_entradas++;
74     printf("Auto %d dentro. Total en cochera: %d/%d\n",
75           auto_id, cochera.autos_dentro, CAPACIDAD_COCHERA);
76     sem_post(&cochera.mutex_contador);
77
78     // 4. Liberar acceso a entrada
79     sem_post(&cochera.mutex_entrada);
80
81     printf("Auto %d estacionado exitosamente\n", auto_id);
82
83     // Simular tiempo estacionado
84     sleep(2 + (rand() % 5)); // 2-6 segundos estacionado
85
86     // Ahora el auto quiere salir
87     return NULL;
88 }
89
90 void* auto_saliendo(void* arg) {
91     int auto_id = *(int*)arg;
92     free(arg); // Liberar el argumento dinámico
93
94     printf("Auto %d quiere salir\n", auto_id);
95
96     // Elegir salida aleatoriamente (load balancing simple)
97     int salida = (rand() % 2) + 1;
98     sem_t* mutex_salida = (salida == 1) ? &cochera.mutex_salida1 : &cochera.
99                               mutex_salida2;

```

```

99
100 // 1. Obtener acceso exclusivo a la salida elegida
101 printf("Auto %d esperando acceso a salida %d...\n", auto_id, salida);
102 sem_wait(mutex_salida);
103
104 // 2. SECCIÓN CRÍTICA: Procesar salida
105 printf("Auto %d saliendo por salida %d\n", auto_id, salida);
106
107 // Simular tiempo de salida (validar pago, abrir barrera, etc.)
108 sleep(1);
109
110 // Actualizar contador global
111 sem_wait(&cochera.mutex_contador);
112 cochera.autos_dentro--;
113 cochera.total_salidas++;
114 printf("Auto %d salió. Total en cochera: %d/%d\n",
115        auto_id, cochera.autos_dentro, CAPACIDAD_COCHERA);
116 sem_post(&cochera.mutex_contador);
117
118 // 3. Liberar acceso a salida
119 sem_post(mutex_salida);
120
121 // 4. IMPORTANTE: Señalar que hay un espacio más disponible
122 sem_post(&cochera.espacios_disponibles);
123
124 printf("Auto %d salió exitosamente por salida %d\n", auto_id, salida);
125
126 return NULL;
127 }
128
129 void mostrar_estadisticas() {
130     sem_wait(&cochera.mutex_contador);
131
132     printf("\n ESTADÍSTICAS DE LA COCHERA:\n");
133     printf("    - Autos dentro: %d/%d\n", cochera.autos_dentro,
134           CAPACIDAD_COCHERA);
134     printf("    - Total entradas: %d\n", cochera.total_entradas);
135     printf("    - Total salidas: %d\n", cochera.total_salidas);
136     printf("    - Espacios libres: %d\n", CAPACIDAD_COCHERA - cochera.
137           autos_dentro);
138
139     sem_post(&cochera.mutex_contador);
140 }
141
142 int main() {
143     pthread_t hilos_entrada[NUM_AUTOS];
144     pthread_t hilos_salida[NUM_AUTOS];
145
146     // Inicializar sistema
147     inicializar_cochera();
148     srand(time(NULL));
149
150     printf("Iniciando simulación: %d autos intentarán usar la cochera\n\n",
151           NUM_AUTOS);
152
153     // Crear hilos de entrada (con argumentos dinámicos)
154     for (int i = 0; i < NUM_AUTOS; i++) {
155         int* arg = malloc(sizeof(int));

```

```

154     *arg = i + 1;
155     int rc = pthread_create(&hilos_entrada[i], NULL, auto_entrando, arg);
156     if (rc != 0) {
157         fprintf(stderr, "Error creando hilo de entrada: %s\n", strerror(
158             rc));
159         exit(1);
160     }
161
162     // Espaciar llegadas aleatoriamente
163     usleep(50000 + (rand() % 100000)); // 50-150ms entre llegadas
164 }
165
166 // Crear hilos de salida con delay
167 sleep(3); // NOTA: En producción se usaría una barrera, esto es didá
168 ctico
169
170 for (int i = 0; i < NUM_AUTOS; i++) {
171     int* arg = malloc(sizeof(int));
172     *arg = i + 1;
173     int rc = pthread_create(&hilos_salida[i], NULL, auto_saliendo, arg);
174     if (rc != 0) {
175         fprintf(stderr, "Error creando hilo de salida: %s\n", strerror(rc
176             ));
177         exit(1);
178     }
179     usleep(100000 + (rand() % 200000)); // 100-300ms entre salidas
180 }
181
182 // Mostrar estadísticas periódicamente
183 for (int i = 0; i < 10; i++) {
184     sleep(2);
185     mostrar_estadisticas();
186 }
187
188 // Esperar que todos los hilos de entrada terminen
189 for (int i = 0; i < NUM_AUTOS; i++) {
190     pthread_join(hilos_entrada[i], NULL);
191 }
192
193 // Esperar que todos los hilos de salida terminen
194 for (int i = 0; i < NUM_AUTOS; i++) {
195     pthread_join(hilos_salida[i], NULL);
196 }
197
198 printf("\nSimulación terminada\n");
199 mostrar_estadisticas();
200
201 // Cleanup
202 sem_destroy(&cochera.espacios_disponibles);
203 sem_destroy(&cochera.mutex_entrada);
204 sem_destroy(&cochera.mutex_salida1);
205 sem_destroy(&cochera.mutex_salida2);
206 sem_destroy(&cochera.mutex_contador);
207
208 return 0;
209 }

```

La solución usa cinco semáforos coordinados. `espacios_disponibles` es un semáforo

contador inicializado en 20 que representa los espacios libres. Cuando un auto quiere entrar, primero ejecuta wait en este semáforo, bloqueándose si la cochera está llena. Los tres mutex (`mutex_entrada`, `mutex_salida1`, `mutex_salida2`) garantizan acceso exclusivo a cada barrera. Finalmente, `mutex_contador` protege las variables de estadísticas compartidas.

La función `auto_entrando()` sigue un protocolo cuidadoso. Primero espera un espacio disponible (wait `espacios_disponibles`), lo que garantiza que solo entre si hay lugar. Luego obtiene acceso exclusivo a la entrada (wait `mutex_entrada`), simula el proceso de entrada, actualiza el contador de manera thread-safe, y libera la entrada (post `mutex_entrada`) para el siguiente auto. Notar que el auto retiene su espacio reservado (no hace post en `espacios_disponibles`) porque está ocupando ese espacio.

La función `auto_saliendo()` implementa load balancing simple eligiendo aleatoriamente entre las dos salidas. Obtiene acceso exclusivo a la salida elegida (wait `mutex_salida`), procesa la salida, actualiza el contador, libera la salida (post `mutex_salida`), y crucialmente, señala que hay un espacio más disponible (post `espacios_disponibles`). Este último post es fundamental: permite que autos esperando en la entrada puedan proceder.

El orden de las operaciones en `auto_entrando()` previene deadlock. Si esperaríamos el `mutex_entrada` antes de verificar `espacios_disponibles`, un auto podría obtener acceso a la entrada pero luego bloquearse esperando espacio, manteniendo el mutex y bloqueando todos los demás autos indefinidamente.

8.9. Uso de Arrays de Semáforos

A medida que los sistemas se vuelven más complejos, a menudo necesitamos múltiples recursos del mismo tipo pero con características ligeramente diferentes. Los arrays de semáforos permiten modelar estos escenarios de manera elegante.

8.9.1. Problema: Múltiples Recursos del Mismo Tipo

Consideremos un servidor web con un pool de 10 threads worker, donde cada request necesita exactamente un worker. Sin embargo, algunos requests requieren workers especializados: algunos workers son expertos en consultas de base de datos, otros en procesamiento de imágenes, otros en llamadas API. Necesitamos un mecanismo para asignar el worker correcto a cada tipo de request.

```
1 #define NUM_WORKERS 10
2 #define NUM_REQUESTS 50
3
4 typedef enum {
```

```

5     WORKER_GENERAL = 0,
6     WORKER_DATABASE = 1,
7     WORKER_IMAGE = 2,
8     WORKER_API = 3
9 } worker_type_t;
10
11 typedef struct {
12     sem_t workers[4];           // Array de semáforos por tipo
13     pthread_mutex_t worker_mutex[NUM_WORKERS]; // Mutex individual por
        worker
14     worker_type_t worker_types[NUM_WORKERS]; // Tipo de cada worker
15     bool worker_busy[NUM_WORKERS];           // Estado de cada worker
16 } server_pool_t;
17
18 server_pool_t server;
19
20 void inicializar_server() {
21     // Distribuir workers por tipo
22     int workers_por_tipo[] = {4, 3, 2, 1}; // General, DB, Image, API
23
24     for (int tipo = 0; tipo < 4; tipo++) {
25         sem_init(&server.workers[tipo], 0, workers_por_tipo[tipo]);
26     }
27
28     // Inicializar workers individuales
29     int worker_idx = 0;
30     for (int tipo = 0; tipo < 4; tipo++) {
31         for (int i = 0; i < workers_por_tipo[tipo]; i++) {
32             pthread_mutex_init(&server.worker_mutex[worker_idx], NULL);
33             server.worker_types[worker_idx] = tipo;
34             server.worker_busy[worker_idx] = false;
35             worker_idx++;
36         }
37     }
38 }
39
40 int obtener_worker(worker_type_t tipo_requerido) {
41     // 1. Esperar worker del tipo requerido
42     sem_wait(&server.workers[tipo_requerido]);
43
44     // 2. Encontrar worker específico de ese tipo
45     for (int i = 0; i < NUM_WORKERS; i++) {
46         if (server.worker_types[i] == tipo_requerido) {
47             if (pthread_mutex_trylock(&server.worker_mutex[i]) == 0) {
48                 if (!server.worker_busy[i]) {
49                     server.worker_busy[i] = true;
50                     return i; // Worker encontrado y reservado
51                 }
52                 pthread_mutex_unlock(&server.worker_mutex[i]);
53             }
54         }
55     }
56
57     // No debería llegar aquí si los semáforos están bien sincronizados
58     return -1;
59 }
60
61 void liberar_worker(int worker_id) {

```

```

62     pthread_mutex_lock(&server.worker_mutex[worker_id]);
63
64     server.worker_busy[worker_id] = false;
65     worker_type_t tipo = server.worker_types[worker_id];
66
67     pthread_mutex_unlock(&server.worker_mutex[worker_id]);
68
69     // Señalar que hay un worker más de este tipo disponible
70     sem_post(&server.workers[tipo]);
71 }
72
73 void* procesar_request(void* arg) {
74     int request_id = *(int*)arg;
75     worker_type_t tipo_necesario = rand() % 4; // Request aleatorio
76
77     printf("Request %d necesita worker tipo %d\n", request_id, tipo_necesario
78         );
79
80     // Obtener worker
81     int worker_id = obtener_worker(tipo_necesario);
82     if (worker_id == -1) {
83         printf("Request %d: Error obteniendo worker\n", request_id);
84         return NULL;
85     }
86
87     printf("Request %d asignado a worker %d (tipo %d)\n",
88         request_id, worker_id, tipo_necesario);
89
90     // Simular procesamiento
91     sleep(1 + (rand() % 3));
92
93     // Liberar worker
94     liberar_worker(worker_id);
95
96     printf("Request %d completado por worker %d\n", request_id, worker_id);
97
98     return NULL;
99 }

```

En esta implementación, usamos un array de semáforos `workers[4]` donde cada elemento representa un tipo diferente de worker. Los semáforos se inicializan según la distribución: 4 workers generales, 3 especializados en bases de datos, 2 en procesamiento de imágenes, y 1 en APIs.

Cuando llega un request, primero ejecuta wait en el semáforo correspondiente al tipo de worker que necesita. Esto garantiza que solo procederá si hay un worker de ese tipo disponible. Luego busca el worker específico dentro de ese tipo, usa mutex individuales por worker para evitar race conditions en la asignación, y marca el worker como ocupado.

Este patrón permite balanceo de carga automático: requests de diferentes tipos no compiten entre sí por workers, pero requests del mismo tipo se encolan apropiadamente. Si todos los workers de base de datos están ocupados, nuevos requests de DB se bloquean sin afectar requests de procesamiento de imágenes que puedan usar sus workers especializados.

8.9.2. Problema Clásico: Productor-Consumidor

El problema productor-consumidor es uno de los problemas de sincronización más estudiados en sistemas operativos. Aparece en casi cualquier sistema que procese datos asincrónicamente: pipelines de procesamiento, sistemas de mensajería, buffers de red, colas de impresión, y más.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  #define BUFFER_SIZE 5
8  #define NUM_ITEMS 20
9
10 typedef struct {
11     int buffer[BUFFER_SIZE];    // Buffer circular
12     int in;                     // Índice de inserción
13     int out;                     // Índice de extracción
14
15     sem_t empty;                // Espacios vacíos disponibles
16     sem_t full;                 // Elementos disponibles para consumir
17     pthread_mutex_t mutex;      // Exclusión mutua para buffer
18
19     int items_produced;         // Estadísticas
20     int items_consumed;
21 } buffer_t;
22
23 buffer_t shared_buffer;
24
25 void init_buffer() {
26     shared_buffer.in = 0;
27     shared_buffer.out = 0;
28     shared_buffer.items_produced = 0;
29     shared_buffer.items_consumed = 0;
30
31     // empty inicia con BUFFER_SIZE (todos los espacios están vacíos)
32     sem_init(&shared_buffer.empty, 0, BUFFER_SIZE);
33
34     // full inicia con 0 (no hay elementos para consumir)
35     sem_init(&shared_buffer.full, 0, 0);
36
37     pthread_mutex_init(&shared_buffer.mutex, NULL);
38
39     printf("Buffer inicializado (tamaño: %d)\n", BUFFER_SIZE);
40 }
41
42 void* productor(void* arg) {
43     int prod_id = *(int*)arg;
44
45     for (int i = 0; i < NUM_ITEMS; i++) {
46         // Producir elemento
47         int item = (prod_id * 100) + i;
48
49         printf("Productor %d creó item %d\n", prod_id, item);
50
51         // PASO 1: Esperar espacio vacío
```



```

52     printf("Productor %d esperando espacio...\n", prod_id);
53     sem_wait(&shared_buffer.empty);
54
55     // PASO 2: Obtener exclusión mutua sobre buffer
56     pthread_mutex_lock(&shared_buffer.mutex);
57
58     // PASO 3: SECCIÓN CRÍTICA - Insertar en buffer
59     shared_buffer.buffer[shared_buffer.in] = item;
60     printf("Item %d insertado en posición %d\n",
61           item, shared_buffer.in);
62
63     shared_buffer.in = (shared_buffer.in + 1) % BUFFER_SIZE;
64     shared_buffer.items_produced++;
65
66     // PASO 4: Liberar exclusión mutua
67     pthread_mutex_unlock(&shared_buffer.mutex);
68
69     // PASO 5: Señalar elemento disponible
70     sem_post(&shared_buffer.full);
71
72     // Simular tiempo de producción
73     usleep(200000 + (rand() % 300000)); // 200-500ms
74 }
75
76 printf("Productor %d terminó\n", prod_id);
77 return NULL;
78 }
79
80 void* consumidor(void* arg) {
81     int cons_id = *(int*)arg;
82
83     for (int i = 0; i < NUM_ITEMS; i++) {
84         // PASO 1: Esperar elemento disponible
85         printf("Consumidor %d esperando elemento...\n", cons_id);
86         sem_wait(&shared_buffer.full);
87
88         // PASO 2: Obtener exclusión mutua sobre buffer
89         pthread_mutex_lock(&shared_buffer.mutex);
90
91         // PASO 3: SECCIÓN CRÍTICA - Extraer del buffer
92         int item = shared_buffer.buffer[shared_buffer.out];
93         printf("Item %d extraído de posición %d por consumidor %d\n",
94               item, shared_buffer.out, cons_id);
95
96         shared_buffer.out = (shared_buffer.out + 1) % BUFFER_SIZE;
97         shared_buffer.items_consumed++;
98
99         // PASO 4: Liberar exclusión mutua
100        pthread_mutex_unlock(&shared_buffer.mutex);
101
102        // PASO 5: Señalar espacio vacío
103        sem_post(&shared_buffer.empty);
104
105        // Consumir elemento
106        printf("Consumidor %d procesando item %d\n", cons_id, item);
107        usleep(300000 + (rand() % 400000)); // 300-700ms
108    }
109 }

```

```
110     printf("Consumidor %d terminó\n", cons_id);
111     return NULL;
112 }
113
114 void mostrar_estado_buffer() {
115     pthread_mutex_lock(&shared_buffer.mutex);
116
117     printf("\nESTADO DEL BUFFER:\n");
118     printf("    Producidos: %d | Consumidos: %d\n",
119         shared_buffer.items_produced, shared_buffer.items_consumed);
120
121     // Mostrar contenido actual
122     printf("    Buffer: [");
123     for (int i = 0; i < BUFFER_SIZE; i++) {
124         if (i >= shared_buffer.out && i < shared_buffer.in) {
125             printf("%d", shared_buffer.buffer[i]);
126         } else {
127             printf("_");
128         }
129         if (i < BUFFER_SIZE - 1) printf(", ");
130     }
131     printf("]\n");
132     printf("    IN=%d, OUT=%d\n", shared_buffer.in, shared_buffer.out);
133
134     pthread_mutex_unlock(&shared_buffer.mutex);
135 }
136
137 int main() {
138     pthread_t prod_thread, cons_thread;
139     int prod_id = 1, cons_id = 1;
140
141     init_buffer();
142     srand(time(NULL));
143
144     printf("Iniciando Productor-Consumidor\n\n");
145
146     // Crear threads
147     pthread_create(&prod_thread, NULL, productor, &prod_id);
148     pthread_create(&cons_thread, NULL, consumidor, &cons_id);
149
150     // Monitor periódico
151     for (int i = 0; i < 10; i++) {
152         sleep(2);
153         mostrar_estado_buffer();
154     }
155
156     // Esperar terminación
157     pthread_join(prod_thread, NULL);
158     pthread_join(cons_thread, NULL);
159
160     printf("\nSimulación terminada\n");
161     mostrar_estado_buffer();
162
163     // Cleanup
164     sem_destroy(&shared_buffer.empty);
165     sem_destroy(&shared_buffer.full);
166     pthread_mutex_destroy(&shared_buffer.mutex);
167 }
```

```

168     return 0;
169 }

```

Notar como en esta solución tres semáforos trabajan juntos para garantizar corrección. El semáforo `empty` (inicializado en `BUFFER_SIZE`) cuenta espacios vacíos disponibles, `full` (inicializado en 0) cuenta elementos listos para consumir, y `mutex` (inicializado en 1) protege el acceso concurrente al buffer.

El productor sigue un protocolo de cuatro pasos. Primero, espera un espacio vacío (`wait empty`), bloqueándose si el buffer está lleno. Segundo, obtiene exclusión mutua (`lock mutex`) para acceder al buffer de manera segura. Tercero, inserta el elemento en el buffer circular, actualiza el índice `in` con aritmética módulo para wrap-around, e incrementa las estadísticas. Cuarto, libera el mutex (`unlock mutex`) y señala un elemento disponible (`post full`) para despertar consumidores en espera.

El consumidor ejecuta el protocolo inverso: espera un elemento (`wait full`), obtiene el mutex, extrae el elemento actualizando el índice `out`, libera el mutex, y señala un espacio vacío (`post empty`). Este último paso es crucial: permite que productores bloqueados puedan continuar agregando elementos.

El orden de las operaciones de semáforos es crítico para evitar deadlock. El productor debe verificar `empty` antes de obtener el mutex. Si obtuviera el mutex primero, podría bloquearse esperando espacio mientras sostiene el mutex, impidiendo que el consumidor libere espacio. Similarmente, el consumidor debe verificar `full` antes del mutex. La regla general: siempre esperar semáforos de recursos (`empty`, `full`) antes de obtener mutex de exclusión mutua.

8.9.3. Problema Lectores-Escritores

El problema lectores-escriores modela situaciones donde múltiples threads quieren leer un recurso compartido, pero las escrituras requieren acceso exclusivo. Bases de datos, cachés, y estructuras de datos compartidas enfrentan este desafío.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  typedef struct {
8      // Datos compartidos
9      int shared_data;
10
11      // Control de acceso
12      pthread_mutex_t mutex;          // Proteger reader_count
13      sem_t write_lock;              // Escritores exclusivos
14
15      // Estadísticas

```

```
16     int reader_count;           // Lectores activos
17     int total_reads;           // Total operaciones de lectura
18     int total_writes;          // Total operaciones de escritura
19
20 } shared_resource_t;
21
22 shared_resource_t resource;
23
24 void init_resource() {
25     resource.shared_data = 0;
26     resource.reader_count = 0;
27     resource.total_reads = 0;
28     resource.total_writes = 0;
29
30     pthread_mutex_init(&resource.mutex, NULL);
31     sem_init(&resource.write_lock, 0, 1); // Un escritor a la vez
32
33     printf("Recurso compartido inicializado\n");
34 }
35
36 void* lector(void* arg) {
37     int reader_id = *(int*)arg;
38
39     for (int i = 0; i < 5; i++) {
40         printf("Lector %d quiere leer\n", reader_id);
41
42         // PROTOCOLO DE ENTRADA - LECTORES
43         pthread_mutex_lock(&resource.mutex);
44
45         resource.reader_count++;
46
47         // El primer lector bloquea escritores
48         if (resource.reader_count == 1) {
49             printf("Primer lector %d bloqueando escritores\n", reader_id);
50             sem_wait(&resource.write_lock);
51         }
52
53         pthread_mutex_unlock(&resource.mutex);
54
55         // SECCIÓN CRÍTICA - LECTURA
56         printf("Lector %d leyendo: valor = %d\n",
57             reader_id, resource.shared_data);
58         resource.total_reads++;
59
60         // Simular tiempo de lectura
61         usleep(100000 + (rand() % 200000)); // 100-300ms
62
63         // PROTOCOLO DE SALIDA - LECTORES
64         pthread_mutex_lock(&resource.mutex);
65
66         resource.reader_count--;
67
68         // El último lector desbloquea escritores
69         if (resource.reader_count == 0) {
70             printf("Último lector %d desbloqueando escritores\n", reader_id);
71             sem_post(&resource.write_lock);
72         }
73     }
```

```

74     pthread_mutex_unlock(&resource.mutex);
75
76     printf("Lector %d terminó lectura %d\n", reader_id, i + 1);
77
78     // Tiempo entre lecturas
79     usleep(500000 + (rand() % 1000000)); // 0.5-1.5s
80 }
81
82 printf("Lector %d terminó todas las lecturas\n", reader_id);
83 return NULL;
84 }
85
86 void* escritor(void* arg) {
87     int writer_id = *(int*)arg;
88
89     for (int i = 0; i < 3; i++) {
90         printf("Escritor %d quiere escribir\n", writer_id);
91
92         // PROTOCOLO DE ENTRADA - ESCRITORES
93         // Esperar acceso exclusivo (bloquea otros escritores y lectores)
94         sem_wait(&resource.write_lock);
95
96         // SECCIÓN CRÍTICA - ESCRITURA
97         int new_value = (writer_id * 1000) + i;
98         printf("Escritor %d escribiendo: %d → %d\n",
99             writer_id, resource.shared_data, new_value);
100
101         resource.shared_data = new_value;
102         resource.total_writes++;
103
104         // Simular tiempo de escritura
105         usleep(300000 + (rand() % 400000)); // 300-700ms
106
107         // PROTOCOLO DE SALIDA - ESCRITORES
108         sem_post(&resource.write_lock);
109
110         printf("Escritor %d terminó escritura %d\n", writer_id, i + 1);
111
112         // Tiempo entre escrituras
113         usleep(800000 + (rand() % 1200000)); // 0.8-2.0s
114     }
115
116     printf("Escritor %d terminó todas las escrituras\n", writer_id);
117     return NULL;
118 }
119
120 void mostrar_estadisticas() {
121     pthread_mutex_lock(&resource.mutex);
122
123     printf("\nESTADÍSTICAS:\n");
124     printf("    Valor actual: %d\n", resource.shared_data);
125     printf("    Lectores activos: %d\n", resource.reader_count);
126     printf("    Total lecturas: %d\n", resource.total_reads);
127     printf("    Total escrituras: %d\n", resource.total_writes);
128
129     pthread_mutex_unlock(&resource.mutex);
130 }
131

```

```

132 int main() {
133     #define NUM_READERS 3
134     #define NUM_WRITERS 2
135
136     pthread_t readers[NUM_READERS];
137     pthread_t writers[NUM_WRITERS];
138     int reader_ids[NUM_READERS];
139     int writer_ids[NUM_WRITERS];
140
141     init_resource();
142     srand(time(NULL));
143
144     printf("Iniciando Lectores-Escritores\n\n");
145
146     // Crear lectores
147     for (int i = 0; i < NUM_READERS; i++) {
148         reader_ids[i] = i + 1;
149         pthread_create(&readers[i], NULL, lector, &reader_ids[i]);
150     }
151
152     // Crear escritores con pequeño delay
153     sleep(1);
154     for (int i = 0; i < NUM_WRITERS; i++) {
155         writer_ids[i] = i + 1;
156         pthread_create(&writers[i], NULL, escritor, &writer_ids[i]);
157     }
158
159     // Monitor periódico
160     for (int i = 0; i < 8; i++) {
161         sleep(2);
162         mostrar_estadisticas();
163     }
164
165     // Esperar terminación
166     for (int i = 0; i < NUM_READERS; i++) {
167         pthread_join(readers[i], NULL);
168     }
169     for (int i = 0; i < NUM_WRITERS; i++) {
170         pthread_join(writers[i], NULL);
171     }
172
173     printf("\nSimulación terminada\n");
174     mostrar_estadisticas();
175
176     // Cleanup
177     pthread_mutex_destroy(&resource.mutex);
178     sem_destroy(&resource.write_lock);
179
180     return 0;
181 }

```

La solución permite múltiples lectores simultáneos (ya que leer no modifica datos), pero garantiza que escritores tengan acceso exclusivo. El `write_lock` semáforo controla el acceso exclusivo: un escritor esperando en este semáforo bloquea tanto a otros escritores como eventualmente a nuevos lectores.

El protocolo de lectores es sofisticado. Cuando un lector quiere entrar, obtiene el mutex que pro-

tege `reader_count`, incrementa el contador, y si es el primer lector (`count == 1`), ejecuta `wait` en `write_lock` para bloquear escritores. Libera el mutex rápidamente para permitir que otros lectores entren concurrentemente. Múltiples lectores pueden estar leyendo simultáneamente porque solo el primero esperó el `write_lock`.

Al salir, el lector nuevamente obtiene el mutex, decrementa el contador, y si es el último lector (`count == 0`), ejecuta `post` en `write_lock` para permitir que escritores procedan. Este diseño garantiza que escritores esperen hasta que todos los lectores terminen.

Los escritores tienen un protocolo más simple: simplemente esperan acceso exclusivo (`wait write_lock`), realizan su escritura, y liberan (`post write_lock`). El semáforo garantiza que solo un escritor puede estar activo, y que escritores esperen hasta que todos los lectores salgan.

Esta solución favorece a los lectores: mientras lleguen lectores nuevos, los escritores esperarán indefinidamente (starvation de escritores). Existen variantes que dan prioridad a escritores o implementan fairness, cada una con diferentes trade-offs. La elección depende del patrón de acceso esperado: sistemas con lecturas frecuentes y escrituras raras favorecen esta implementación, mientras sistemas con muchas escrituras necesitan variantes con prioridad de escritor.

8.10. Síntesis

Hemos recorrido un largo camino desde las race conditions hasta soluciones sofisticadas con semáforos. La sincronización es fundamental en sistemas modernos: sin ella, el multithreading y la concurrencia simplemente no funcionarían de manera confiable.

Los puntos clave que debemos recordar son que las race conditions son la causa raíz de la mayoría de los bugs en programas concurrentes, difíciles de reproducir y depurar porque dependen del timing. La sección crítica debe protegerse con primitivas de sincronización apropiadas, eligiendo la herramienta correcta para cada problema. Los semáforos emergieron como la herramienta más versátil, permitiendo exclusión mutua, limitación de recursos, y sincronización de eventos con un único mecanismo. Finalmente, el overhead de sincronización debe balancearse cuidadosamente con las necesidades de concurrencia: demasiada sincronización serializa el programa innecesariamente, muy poca causa race conditions.

La sincronización conecta con prácticamente todos los temas del sistema operativo. El scheduling interactúa con sincronización cuando procesos bloqueados en semáforos van a colas de espera, cambiando su estado de running a blocked. El deadlock puede prevenirse con diseño cuidadoso del orden de adquisición de recursos, tema que exploraremos en profundidad en el próximo capítulo. Memory management requiere sincronización para mantener coherencia de caché cuando múltiples cores acceden a memoria compartida. Los sistemas de archivos usan sincronización

para controlar acceso concurrente a archivos y directorios. Finalmente, las redes requieren sincronización tanto a nivel local (acceso a sockets) como en sistemas distribuidos donde procesos en diferentes máquinas deben coordinarse.

Con este fundamento sólido en sincronización, estamos preparados para enfrentar el próximo gran desafío: detectar, prevenir y recuperarnos de deadlocks, donde la sincronización mal aplicada puede llevar al sistema a un estado de bloqueo permanente.

9 Interbloqueo (Deadlock)

9.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Definir formalmente deadlock e identificar sus cuatro condiciones necesarias
- Diferenciar entre deadlock, inanición (starvation) y livelock
- Aplicar estrategias de prevención rompiendo condiciones necesarias
- Implementar el Algoritmo del Banquero para evasión de deadlocks en sistemas con recursos fijos
- Explicar por qué Windows, Linux y macOS adoptan la estrategia “Do nothing” (Avestruz)
- Contrastar el manejo de deadlocks en sistemas de propósito general vs. sistemas críticos

9.2. Introducción y Contexto

9.2.1. El problema de la espera circular

Imaginemos un cruce de dos calles en una ciudad sin semáforos. En el mismo instante, llegan cuatro autos, uno por cada esquina. Cada auto necesita cruzar, pero el espacio en la intersección sólo permite que uno pase a la vez. Auto A espera a que B se mueva, B espera a C, C espera a D, y D espera a A. Resultado: *nadie se mueve jamás*. Esto es un deadlock en el mundo real.

En los sistemas operativos, ocurre exactamente lo mismo, pero con **procesos** y **recursos**. Un proceso necesita recursos para ejecutarse (impresoras, archivos, memoria, acceso a bases de datos). Si dos o más procesos se bloquean mutuamente esperando recursos que el otro tiene, y ninguno puede ceder porque necesita el otro para avanzar, tenemos un **interbloqueo** o **deadlock**.

Definición formal: Un deadlock es una situación en la que un conjunto de procesos está permanentemente bloqueado porque cada proceso del conjunto espera un recurso que sólo puede ser liberado por otro proceso del mismo conjunto (que también está bloqueado).

Deadlock NO es lo mismo que un programa lento o un sistema sobrecargado. En un deadlock, el sistema no progresa *nunca más* a menos que intervenga un agente externo (el administrador o el SO). El tiempo de espera no es “mucho”, es **infinito**.

9.2.2. Problemas relacionados: Starvation y Livelock

Antes de seguir, debemos distinguir deadlock de dos fenómenos que lo rodean pero son fundamentalmente distintos.

Inanición (Starvation) ocurre cuando un proceso nunca obtiene el recurso que necesita, no porque esté bloqueado en un ciclo, sino porque el sistema siempre prefiere a otros procesos. El proceso está listo para ejecutarse, pero el planificador (o el asignador de recursos) jamás lo selecciona.

En una impresora compartida, si el sistema siempre prioriza documentos cortos, un trabajo de 500 páginas podría sufrir *starvation*: nunca se imprime porque siempre llegan trabajos de 1 página antes que él. El proceso existe, está listo, pero nunca progresa.

Livelock es más sutil y frecuente en sistemas distribuidos. Los procesos no están bloqueados (como en deadlock), y tampoco están detenidos (como en starvation). Están activos, ejecutando código, *pero no progresan hacia su objetivo* porque responden a eventos externos que los hacen retroceder constantemente.

Dos personas se encuentran cara a cara en un pasillo angosto. Ambas se corren a la izquierda para dejarse pasar, pero justo cuando una se mueve, la otra también se mueve al mismo lado. Entonces ambas se corren a la derecha, y vuelven a chocar. Siguen moviéndose, pero nunca logran cruzarse. Eso es livelock.

En sistemas operativos, un ejemplo típico de livelock ocurre en protocolos de red: dos nodos intentan enviar un paquete al mismo tiempo, detectan colisión, esperan tiempos aleatorios, y vuelven a transmitir simultáneamente una y otra vez. La red está activa, el CPU no está bloqueado, pero el throughput es cero para esa comunicación.

9.3. Condiciones para Deadlock

9.3.1. Las cuatro condiciones necesarias

Para que ocurra un deadlock en un sistema, **deben cumplirse simultáneamente** cuatro condiciones. Si falta una sola, el deadlock es imposible.

1. **Exclusión mutua (Mutual Exclusion)**: Al menos un recurso debe ser no compartible. Es decir, sólo un proceso puede usar el recurso a la vez. Si otro proceso solicita el mismo recurso, debe esperar hasta que el primero lo libere.

2. **Retención y espera (Hold and Wait)**: Un proceso debe estar reteniendo al menos un recurso mientras espera por otros recursos que están siendo retenidos por otros procesos.
3. **No expropiación (No Preemption)**: Los recursos no pueden ser arrebatados forzosamente a un proceso. El proceso debe liberarlos voluntariamente. (Esto es cierto para la mayoría de los recursos: archivos, impresoras, semáforos. No aplica al CPU o memoria, que sí pueden ser expropiados).
4. **Espera circular (Circular Wait)**: Debe existir una cadena circular de procesos donde cada proceso espera un recurso que el siguiente proceso de la cadena posee.

Estas condiciones son necesarias: sin ellas, no hay deadlock. Pero no son **suficientes**: que se cumplan las cuatro no *garantiza* que haya deadlock, porque quizás los procesos liberen recursos antes de que se forme el ciclo. La condición **suficiente** para deadlock es **espera circular en un sistema que ya cumple las otras tres**.

9.3.2. Representación mediante grafos

Para analizar deadlocks visualmente, usamos el **Grafo de Asignación de Recursos**. Es un grafo dirigido con dos tipos de nodos:

- **Procesos**: dibujados como círculos $P_1, P_2, P_3, \dots$
- **Recursos**: dibujados como cuadrados $R_1, R_2, R_3, \dots$ (cada recurso puede tener k instancias si es múltiple)

Las aristas (flechas) pueden ser:

- **Solicitud**: $P_i \rightarrow R_j$ (el proceso P_i está esperando una instancia de R_j)
- **Asignación**: $R_j \rightarrow P_i$ (una instancia de R_j está asignada a P_i)



Grafo de Asignación de recursos. Deadlock.

Teorema del ciclo: Si el grafo no contiene ciclos, no hay deadlock. Si contiene un ciclo, **puede** haber deadlock: - Si cada recurso tiene una sola instancia: ciclo $\Rightarrow$ deadlock seguro. - Si algún recurso tiene múltiples instancias: ciclo no garantiza deadlock (podría haber instancias libres que rompan la espera).

Dos procesos compiten por dos impresoras (mismo tipo de recurso, múltiples instancias). P1 tiene una impresora y pide la otra; P2 tiene la otra y pide una. Gráficamente hay ciclo, pero si hay dos impresoras más libres, el sistema puede asignarlas y romper el ciclo. En cambio, si cada impresora es un recurso distinto (R1 y R2), entonces ciclo implica deadlock.

9.4. Estrategias para manejar Deadlocks

Existen cuatro enfoques principales, ordenados de mayor a menor overhead. La elección depende del tipo de sistema.

Estrategia	Concepto	Overhead	Riesgo	Aplicación típica
Prevención	Romper alguna condición necesaria	Alto	Bajo (deadlock imposible)	Sistemas críticos (aviónica)
Evasión	Evitar entrar en estado inseguro	Medio-Alto	Bajo (si se conoce futuro)	Sistemas con recursos predecibles
Detección + Recuperación	Permitir deadlock pero detectarlo y revertirlo	Medio	Medio (ventana de riesgo)	Bases de datos
Ignorar (Ostrich)	Asumir que deadlock es muy improbable	Cero	Alto (el sistema cuelga)	Windows, Linux, macOS

9.4.1. El algoritmo del avestruz: “Do nothing”

La mayoría de los sistemas operativos modernos **no hacen nada** para prevenir, evitar o detectar deadlocks entre procesos de usuario. La filosofía es: “deadlocks son tan raros que es más barato reiniciar la máquina que implementar mecanismos complejos”.

El algoritmo del avestruz consiste en meter la cabeza en la arena y pretender que no pasa nada. En sistemas de propósito general, funciona porque los deadlocks ocurren con una frecuencia menor al 0.001

Estadísticas clave (datos agregados de centros de cómputo): - En servidores Windows/Linux típicos, menos de 1 deadlock por cada 1000 años de operación continua. - En sistemas de bases de datos (que sí implementan detección), la frecuencia sube a 1 deadlock cada meses. - En sistemas mal diseñados (aplicaciones que adquieren locks en orden arbitrario), pueden ocurrir diariamente.

¿Por qué es tan raro? 1. Los procesos usualmente no retienen recursos mientras esperan otros. 2. Las aplicaciones bien escritas usan timeouts o liberan recursos rápidamente. 3. El scheduler preemptivo del CPU no genera deadlock por sí solo (sólo recursos no expropiables).

”Ostrich” es válido para tu PC y servidores web. **No es válido** para:

- Sistemas de control de misiles
- Marcapasos artificiales
- Centrales nucleares
- Controladores de trenes de alta velocidad

En esos sistemas, el deadlock es inaceptable y se usan prevención o evasión.

¿Qué hacen Windows, Linux y macOS realmente?

SO	Estrategia principal	Excepciones
Windows	Ostrich para recursos de usuario; proporciona Wait Chain Traversal API para detección por aplicaciones	El kernel no previene/evita deadlock automáticamente, pero debuggers y aplicaciones (SQL Server) pueden detectarlos
Linux	Ostrich para procesos de usuario	lockdep en kernels de desarrollo (detección estática de posibles deadlocks en código del kernel); en producción, off por defecto
macOS	Ostrich puro	No hay detector general; se confía en la calidad de las aplicaciones

Todos asumen que el deadlock es un bug de la aplicación, no del SO. Si ocurre, el usuario force-quits el programa (o reinicia).

9.5. Prevención de Deadlocks

La prevención ataca directamente las cuatro condiciones necesarias. Si rompemos **una sola**, el deadlock se vuelve matemáticamente imposible.

9.5.1. Rompiendo la exclusión mutua

Consiste en hacer que todos los recursos sean compartibles (spooling). Ejemplo: impresoras virtuales. En lugar de que un proceso tome la impresora física, escribe en un archivo en disco (recurso

compartible), y un demonio (spooler) se encarga de imprimir en orden.

Funciona para recursos que pueden ser spooled (impresoras, colas de correo). No funciona para recursos intrínsecamente no compartibles: archivos abiertos en modo exclusivo, buffers de memoria, dispositivos de entrada (teclado/mouse).

9.5.2. Rompiendo retención y espera

Dos aproximaciones:

1. **Asignación global inicial:** El proceso debe solicitar *todos* los recursos que necesitará antes de comenzar. Si no están todos disponibles, no ejecuta.
2. **Liberar antes de pedir:** El proceso debe liberar todos sus recursos actuales antes de solicitar nuevos.

Desventajas:

- Baja utilización de recursos (proceso retiene recursos que no está usando)
- Riesgo de starvation (proceso largo nunca obtiene todos sus recursos simultáneamente)
- Imposible si el proceso no sabe de antemano qué recursos necesitará

9.5.3. Rompiendo no expropiación

Permitir que el SO expropie recursos. Si un proceso pide un recurso que no está disponible, el SO puede:

- Tomar el recurso de otro proceso que lo tiene (si ese proceso está esperando otro recurso).
- Forzar a procesos a liberar recursos voluntariamente (enviándoles una señal).

Implementación práctica:

Sólo funciona para recursos cuyo estado puede guardarse y restaurarse fácilmente (CPU, memoria). No funciona para impresoras (no podés “des-imprimir” una página) o archivos en mitad de escritura.

9.5.4. Rompiendo espera circular

La solución más práctica: Imponer un orden global de solicitud de recursos.

Ordenación de recursos: Asignar un número único a cada tipo de recurso ($R_1, R_2, \dots, R_n$). Todo proceso debe solicitar los recursos en orden estrictamente creciente. Si necesita R_3 y R_5 , debe pedir R_3 antes que R_5 .

¿Por qué funciona? Si todos los procesos siguen el orden, no puede formarse un ciclo. Para tener ciclo, algún proceso debería pedir un recurso de menor número después de tener uno mayor, lo que está prohibido.

Recursos: Impresora (I=1), Escáner (E=2), Disco (D=3).

Proceso A necesita escáner y luego impresora. Como $2 > 1$, debe pedir impresora (1) antes que escáner (2).

Proceso B necesita impresora y luego disco: $1 < 3$, ok.

No hay forma de que A tenga escáner y espere impresora mientras B tiene impresora y espera escáner; el orden lo impide.

Ventajas:

- Sencillo de implementar (sólo requiere que el SO verifique el orden)
- Sin starvation si el orden es justo
- Overhead bajo

Desventajas:

- Requiere consenso global sobre el orden de recursos (difícil en sistemas heterogéneos)
- Algunos programas necesitan recursos en orden natural diferente al impuesto

9.6. Evasión de Deadlocks (Algoritmo del Banquero)

La evasión es más permisiva que la prevención. Permite que se cumplan las cuatro condiciones, pero el sistema operativo **simula** cada asignación de recursos antes de concederla. Si la asignación llevara a un **estado inseguro** (del cual podría derivarse deadlock), el SO la rechaza y hace esperar al proceso.

9.6.1. Estado seguro vs. estado inseguro

Un estado es **seguro** si existe al menos una secuencia de ejecución de todos los procesos (una “secuencia segura”) que permite que terminen sin deadlock, aunque todos soliciten sus recursos máximos.

Secuencia segura: Una ordenación $\langle P_1, P_2, \dots, P_n \rangle$ tal que para cada P_i , los recursos que aún puede necesitar pueden ser satisfechos por los recursos disponibles más los recursos retenidos por todos los P_j con $j < i$.

Si el sistema está en estado seguro → no hay deadlock.

Si está en estado inseguro → **podría** haber deadlock (no es seguro, pero puede que no ocurra si los

procesos liberan antes).

9.6.2. Estructuras de datos del algoritmo

Para n procesos y m tipos de recursos, el banquero mantiene 4 estructuras:

- **Disponible[1..m]**: Vector de recursos disponibles actualmente. $Disponible[j] = k$ significa que hay k instancias del recurso R_j libres.
- **Máximo[1..n, 1..m]**: Matriz donde $Mximo[i][j]$ es la cantidad máxima de instancias de R_j que el proceso P_i puede solicitar.
- **Asignado[1..n, 1..m]**: Recursos actualmente asignados a cada proceso.
- **Necesidad[1..n, 1..m]**: $Necesidad[i][j] = Mximo[i][j] - Asignado[i][j]$ (lo que P_i todavía podría pedir).

9.6.3. Inicialización y verificación de seguridad

Al iniciar el sistema, todos los procesos declaran su $Mximo$. $Disponible$ inicial tiene todos los recursos. $Asignado$ en cero.

Algoritmo de verificación de seguridad (Safety Algorithm):

1. Inicializar Trabajo = Disponible, y Fin[n] = falso para todo proceso.
2. Buscar un proceso i tal que: Fin[i] == falso y Necesidad[i] ≤ Trabajo.
3. Si existe, agregar sus recursos a Trabajo (Trabajo = Trabajo + Asignado[i]), marcar Fin[i] = verdadero, repetir desde paso 2.
4. Si al final Fin[i] == verdadero para todos los procesos, el estado es seguro.

9.6.4. Solicitud de recursos

Cuando P_i pide recursos (vector $Peticin[1..m]$):

1. Si $Peticin > Necesidad[i] \rightarrow$ error (pidió más de lo declarado).
2. Si $Peticin > Disponible \rightarrow$ proceso debe esperar.
3. **Simular** la asignación: $Disponible' = Disponible - Peticin$, $Asignado'[i] = Asignado[i] + Peticin$, $Necesidad'[i] = Necesidad[i] - Peticin$.
4. Ejecutar Safety Algorithm con $Disponible'$, $Asignado'$, $Necesidad'$.
5. Si el nuevo estado es seguro $\rightarrow$ conceder la solicitud (actualizar las matrices reales).
6. Si es inseguro $\rightarrow$ denegar y hacer esperar a P_i , restaurando las matrices originales.

Limitación fundamental: El algoritmo del banquero requiere que cada proceso conozca su *Máximo* de antemano, y que ese máximo nunca cambie. En la práctica, los procesos no saben cuántos recursos necesitarán (ej: un editor de texto puede abrir archivos arbitrarios). Por eso, el algoritmo del banquero es didáctico pero casi no se usa en SO reales.

Limitación adicional: El algoritmo del banquero asume que los procesos son **independientes** (no se comunican ni sincronizan). Si los procesos cooperan (productor-consumidor, barreras de sincronización), la secuencia segura calculada puede ser incorrecta, porque el orden real de ejecución está limitado por la sincronización, no solo por disponibilidad de recursos.

9.6.5. Ejemplo 1: Estado seguro

Sistema con 3 procesos (P_1, P_2, P_3) y 3 tipos de recursos (A, B, C) con instancias: (10, 5, 7).

Estado actual:

Proceso	Asignado (A,B,C)	Máximo (A,B,C)	Necesidad (A,B,C)
P_1	(0,1,0)	(7,5,3)	(7,4,3)
P_2	(2,0,0)	(3,2,2)	(1,2,2)
P_3	(3,0,2)	(9,0,2)	(6,0,0)

Disponible = (5,3,2)

Verificación de seguridad: - Paso 1: Trabajo = (5,3,2), Fin = (falso,falso,falso)

- P_1 : Necesidad (7,4,3) > Trabajo (5,3,2) → no
- P_2 : Necesidad (1,2,2) ≤ Trabajo (5,3,2) → sí. Trabajo = (5,3,2)+(2,0,0) = (7,3,2), Fin[2]=true
- P_3 : Necesidad (6,0,0) ≤ Trabajo (7,3,2) → sí. Trabajo = (7,3,2)+(3,0,2) = (10,3,4), Fin[3]=true
- P_1 : Ahora Necesidad (7,4,3) ≤ Trabajo (10,3,4)? NO porque $4 \leq 3$ es falso → P_1 no puede ejecutar.

¡Estado inseguro! Aunque no hay deadlock ahora, el banquero no concedería ninguna petición que lleve a este estado.

9.6.6. Ejemplo 2: Solicitud que lleva a estado seguro

Partimos del mismo estado anterior pero **sin** P_3 **aún**. P_2 pide (1,0,1).

Verificación:

Disponible actual = (5,3,2)

$Peticin_{P_2} = (1, 0, 1) \leq Necesidad_{P_2} = (1, 2, 2)$ (ok)
 $Peticin \leq Disponible = (5, 3, 2)? (1, 0, 1) \leq (5, 3, 2)$ (ok)

Simulación:

$Disponible' = (5, 3, 2) - (1, 0, 1) = (4, 3, 1)$

$Asignado'\{P_2\} = (2, 0, 0) + (1, 0, 1) = (3, 0, 1)$

$Necesidad'\{P_2\} = (1, 2, 2) - (1, 0, 1) = (0, 2, 1)$

Safety Algorithm con nuevo estado:

Trabajo = (4, 3, 1)

P_2 necesita $(0, 2, 1) \leq (4, 3, 1)$? sí. Trabajo = $(4, 3, 1) + (3, 0, 1) = (7, 3, 2)$, Fin[2]=true

P_1 necesita $(7, 4, 3) \leq (7, 3, 2)$? NO

P_3 necesita $(6, 0, 0) \leq (7, 3, 2)$? sí. Trabajo = $(7, 3, 2) + (3, 0, 2) = (10, 3, 4)$, Fin[3]=true

P_1 ahora necesita $(7, 4, 3) \leq (10, 3, 4)$? NO $\rightarrow P_1$ no puede.

Sigue siendo inseguro. El banquero **denegaría** la petición de P_2 , aunque los recursos estuvieran disponibles físicamente.

9.7. Detección y Recuperación

Si el sistema no previene ni evita deadlocks, debe al menos detectarlos cuando ocurren y recuperarse. Esta estrategia es común en sistemas de bases de datos y sistemas transaccionales.

9.7.1. Algoritmo de detección (similar al Safety Algorithm)

Mantiene:

- **Disponible[1..m]**: recursos libres.
- **Asignado[1..n, 1..m]**: recursos asignados.
- **Petición[1..n, 1..m]**: recursos que cada proceso está *actualmente esperando* (a diferencia de Necesidad, que es el máximo futuro).

Algoritmo de detección: \ 1. Inicializar Trabajo = Disponible. \ 2. Marcar como “no terminados” todos los procesos con *Asignado* distinto de cero. \ 3. Buscar un proceso no terminado P_i tal que $Peticin[i] \leq Trabajo$. \ 4. Si existe, agregar sus recursos a Trabajo ($Trabajo+ = Asignado[i]$), marcarlo terminado, repetir. \ 5. Si al final hay procesos no terminados $\rightarrow$ están en deadlock. \

La complejidad es $O(m \times n^2)$ en el peor caso. En sistemas con miles de procesos, se ejecuta periódicamente (ej: cada 5 minutos) o cuando la utilización del CPU cae abruptamente (síntoma de deadlock).

Nota: A diferencia del algoritmo del banquero (que usa Necesidad = máximo futuro), aquí usamos Petición = lo que el proceso está *actualmente esperando*. Esto es más realista porque los procesos no necesitan conocer su consumo máximo de antemano.

9.7.2. Recuperación de deadlocks

Una vez detectado, hay que salir del deadlock. Opciones:

1. **Matar procesos** (recuperación drástica):

- Matar todos los procesos en deadlock: simple pero costoso (pérdida de trabajo).
- Matar uno por uno hasta romper el ciclo: más fino, pero requiere ejecutar detección cada vez.

2. **Retroceso (Rollback)**:

- Llevar los procesos a un estado anterior seguro (checkpoint) y reiniciarlos desde allí.
- El proceso retrocedido puede reintentar la operación que causó el deadlock.
- Requiere que el sistema guarde checkpoints periódicamente (overhead alto).

3. **Expropiación selectiva**:

- Forzar a un proceso a liberar un recurso (si es posible salvar estado).
- El proceso expropiado debe reiniciarse desde un checkpoint.

Advertencia práctica:

En sistemas de propósito general (Windows, Linux), la recuperación casi nunca se implementa. Si tu PC entra en deadlock, presionás Ctrl+Alt+Supr o reiniciás. Los administradores de bases de datos sí usan detección + rollback de transacciones.

9.8. Deadlock en Propósito General vs. Sistemas Críticos

La elección de estrategia no es técnica sino **económica y de ingeniería de riesgos**.

9.8.1. Sistemas de propósito general (PCs, servidores web)

- **Frecuencia de deadlock esperada**: extremadamente baja (menos de 1 vez por año en miles de máquinas).
- **Costo de prevención/evasión**: alto (complejidad extra en el kernel, overhead en cada asignación de recurso).

- **Costo de un deadlock cuando ocurre:** bajo (el usuario reinicia la aplicación o la máquina).
- **Estrategia elegida: Ostrich** (ignorar). Incluso si ocurre, el costo de implementar solución supera el costo del problema.

9.8.2. Sistemas críticos (aeronáutica, medicina, industria)

- **Frecuencia esperada:** debe ser cero por diseño.
- **Costo de deadlock si ocurre:** vidas humanas, daños millonarios.
- **Estrategia elegida: Prevención** (rompiendo espera circular mediante orden global de recursos) o **Evasión** (si es posible conocer Máximos).
- **Regulaciones:** DO-178C (aviación), IEC 62304 (software médico) exigen análisis formal de deadlock.



Diagrama de Decisión para el Manejo de Deadlocks

9.9. Ejercicios Tipo Parcial

Ejercicio 1 (Deadlock detection con múltiples instancias)

Dado el siguiente estado de recursos (instancias totales: A=4, B=3, C=2):

Proceso	Asignado (A,B,C)	Petición (A,B,C)
P1	(1,0,1)	(2,0,1)
P2	(2,1,0)	(0,1,1)
P3	(0,0,1)	(1,0,0)
P4	(0,2,0)	(1,2,0)

Disponible = (1,0,0)

Verificar que la suma de asignados más disponible es igual al total de instancias: - A: $1+2+0+0+1 = 4$ ✓ - B: $0+1+0+2+0 = 3$ ✓ - C: $1+0+1+0+0 = 2$ ✓

Determinar si hay deadlock. En caso afirmativo, ¿qué procesos están involucrados?

Solución paso a paso en próxima página...

Ejercicio 2 (Algoritmo del banquero - estado seguro)

Un sistema tiene 4 procesos y 3 recursos (A:8, B:6, C:5). La matriz Máximo es: Máximo = (3,2,2) \ (4,1,3) \ (2,3,1) \ (4,2,2)

Asignado actual = [(1, 0, 0), (2, 1, 1), (0, 1, 0), (1, 0, 1)]. Calcular Necesidad y Disponible. ¿Hay secuencia segura? Si P3 pide (1,0,0), ¿se la concede el banquero?

Ejercicio 3 (Prevención - orden de recursos)

Un sistema tiene 5 tipos de recursos numerados 1 a 5. Se sabe que los procesos A, B, C siguen las siguientes secuencias de solicitud: - A: pide 3, luego 5, luego 1 - B: pide 2, luego 4, luego 3 - C: pide 1, luego 3, luego 2

¿Qué orden global de solicitud (asignación de números a recursos) permitiría evitar deadlock por espera circular? ¿Existe alguna asignación que lo logre?

Ejercicio 4 (Deadlock vs Starvation vs Livelock)

Clasificar cada situación como Deadlock (D), Starvation (S), Livelock (L) o Ninguno (N): - Dos procesos intentan entrar a una sección crítica. El scheduler siempre elige el mismo proceso, el otro nunca avanza. ____ - Procesos P1 tiene recurso R1 y espera R2; P2 tiene R2 y espera R3; P3 tiene R3 y espera R1. ____ - Dos procesos envían paquetes de red, detectan colisión, esperan tiempos aleatorios, colisionan nuevamente una y otra vez sin que ningún paquete llegue a destino. ____ - Un proceso solicita una impresora. No hay disponible, pero el proceso no se bloquea; en su lugar, reintenta cada 10ms, pero siempre ve la impresora ocupada. **Ninguno (espera activa sin deadlock/starvation/livelock)**

9.10. Síntesis

Resumen conceptual: - Deadlock requiere 4 condiciones: exclusión mutua, retención y espera, no expropiación, espera circular. - Prevención → rompe una condición (más efectiva: orden de recursos). - Evasión → algoritmo del banquero (requiere conocer Máximo). - Detección + recuperación → útil en bases de datos. - Ostrich → Windows, Linux, macOS (deadlock es bug de aplicación).

Fórmulas clave: - Necesidad[i][j] = Máximo[i][j] - Asignado[i][j] - Condición de seguridad: $\exists$ secuencia $\langle P_1 \dots P_n \rangle$ tal que $\forall i, Necesidad[P_i] \leq Disponible + \sum_{k=1}^{i-1} Asignado[P_k]$

Tips para el parcial: 1. Grafos de asignación: ciclo + recursos de instancia única → deadlock seguro. 2. Algoritmo del banquero: siempre verificar $Peticin \leq Necesidad$ antes de simular. 3. En ejercicios de prevención por orden, busca asignar números crecientes según el orden real de uso. 4. Distinguir claramente: deadlock es “todos esperan a todos”, starvation es “uno nunca avanza”, livelock es “todos avanzan pero no progresan”.

10 Gestión de Memoria Real

10.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Distinguir entre direcciones lógicas, relativas y físicas
- Explicar los momentos de binding de direcciones (compilación, carga, ejecución)
- Identificar fragmentación interna vs externa y sus causas
- Analizar ventajas y desventajas de particiones fijas y dinámicas
- Aplicar algoritmos de asignación: First Fit, Best Fit, Worst Fit, Next Fit
- Calcular direcciones físicas a partir de direcciones lógicas en paginación
- Determinar el formato de dirección lógica (bits de página y offset)
- Explicar el funcionamiento de MMU, TLB y registros base/límite
- Comparar paginación simple, multinivel y segmentación
- Resolver ejercicios de traducción de direcciones con tablas de páginas
- Comprender el rol del Buddy System en asignación de memoria
- Evaluar cuándo aplicar compactación y sus costos

10.2. Introducción y Contexto

10.2.1. ¿Por qué necesitamos gestionar la memoria?

Imaginemos una biblioteca con espacio limitado para libros. Si cada estudiante llega y toma el espacio que necesita sin control alguno, pronto tendremos espacios desaprovechados entre libros, imposibilidad de ubicar libros nuevos aunque haya espacio total suficiente, estudiantes accediendo a libros que no les pertenecen, y caos al intentar encontrar un libro específico.

Lo mismo sucede con la memoria RAM en un sistema operativo multiprogramado. Con múltiples procesos ejecutándose simultáneamente, el SO debe asignar memoria de manera eficiente, proteger la memoria de cada proceso, permitir compartir memoria cuando sea apropiado, y traducir direcciones para que cada proceso “crea” que tiene toda la memoria disponible.

10.2.2. El problema fundamental

En los primeros sistemas, un programa accedía directamente a direcciones físicas de memoria. Esto presentaba problemas críticos: un proceso podía sobrescribir memoria del SO, era imposible reubicar un programa una vez cargado, no se podía ejecutar más de un programa simultáneamente, y errores de programación podían corromper todo el sistema.

El direccionamiento directo a memoria física es incompatible con sistemas multiprogramados seguros. Sin una capa de abstracción, cualquier error de programación puede destruir el sistema completo.

La solución fue introducir una capa de abstracción: el concepto de espacio de direcciones lógicas. Esta abstracción permite que cada proceso opere en su propio espacio virtual, completamente aislado de otros procesos y del sistema operativo.

10.2.3. Evolución histórica

La gestión de memoria ha evolucionado siguiendo un patrón de “problema → solución → nuevo problema”. La memoria compartida sin protección permitía que un programa destruyera todo el sistema, lo que llevó a las particiones fijas. Estas eliminaron el caos pero introdujeron desperdicio de memoria por fragmentación interna. Las particiones dinámicas resolvieron este problema, pero generaron fragmentación externa severa. La paginación eliminó la fragmentación externa al costo de agregar overhead de traducción. La segmentación ofreció un mejor modelo lógico pero con mayor complejidad. Finalmente, los sistemas híbridos combinaron ventajas de múltiples técnicas, aunque con complejidad adicional.

Este capítulo recorre esta evolución histórica no solo por interés académico, sino porque entender por qué cada técnica surgió nos ayuda a comprender las decisiones de diseño de los sistemas modernos.

10.3. Conceptos Fundamentales

10.3.1. Espacios de Direcciones

Un espacio de direcciones es el conjunto de direcciones que una entidad puede usar para referenciar memoria. Existen tres tipos fundamentales, cada uno con su propósito específico en la jerarquía de traducción de direcciones.

10.3.1.1. Dirección Lógica (Virtual)

La dirección lógica es generada por el CPU durante la ejecución de un programa. Es la dirección que “ve” el proceso. Por ejemplo, cuando un programa en C hace:

```
1 int x = 42;  
2 printf("Dirección de x: %p\n", &x);
```

La dirección mostrada es una dirección lógica. El proceso no sabe (ni le importa) dónde está físicamente en RAM. Esta independencia es fundamental: permite que el sistema operativo reubique el proceso en memoria sin que este se entere, facilita la protección entre procesos, y hace posible la memoria virtual.

Las direcciones lógicas son independientes de la ubicación física, permiten reubicación del proceso, cada proceso tiene su propio espacio lógico aislado, y típicamente van desde 0 hasta el límite del proceso.

10.3.1.2. Dirección Relativa

Es una dirección expresada como desplazamiento desde un punto de referencia, típicamente el inicio del programa. Si un programa se compila y la variable `x` está en el offset 100 desde el inicio del código, su dirección relativa es 100, sin importar dónde se cargue el programa en memoria. Este concepto es clave para generar código reubicable.

10.3.1.3. Dirección Física (Real)

Es la dirección real en los módulos de RAM. El hardware usa estas direcciones para acceder a la memoria física. Hay algo crucial que debes entender: el proceso NUNCA ve direcciones físicas. La traducción la hace el hardware (MMU) de forma transparente, y el SO solo configura los parámetros de traducción.

La separación entre direcciones lógicas y físicas es el fundamento de todos los sistemas operativos modernos. Sin esta abstracción, no existirían la multiprogramación segura, la memoria virtual, ni la protección entre procesos.

10.3.2. Binding de Direcciones

El binding es el proceso de asignar direcciones de programa a direcciones reales de memoria. El momento en que esto ocurre tiene implicaciones profundas en la flexibilidad y eficiencia del sistema. Puede ocurrir en tres momentos diferentes, cada uno con sus ventajas y limitaciones.

10.3.2.1. En Tiempo de Compilación

El compilador genera direcciones físicas absolutas directamente en el código ejecutable.

```
1 // El compilador coloca 'x' en la dirección física 0x1000
2 int x = 10; // Compilado como: MOV [0x1000], 10
```

Este enfoque tiene desventajas severas: el programa solo funciona en esa ubicación específica de memoria, es imposible ejecutar múltiples instancias del mismo programa, no hay protección entre procesos, y hay que recompilar si se cambia la ubicación. Su uso histórico se limitó a sistemas embebidos antiguos y programas únicos en memoria.

10.3.2.2. En Tiempo de Carga

El loader (cargador) ajusta las direcciones cuando carga el programa en memoria. El compilador genera código reubicable, y el loader determina la base al momento de cargar.

```
1 // El compilador genera código reubicable
2 int x = 10; // Compilado como: MOV [BASE+100], 10
3 // El loader determina BASE al cargar
```

Aunque mejor que el binding en compilación, tiene limitaciones importantes: una vez cargado, no se puede mover el proceso en memoria, el tiempo de carga aumenta porque hay que ajustar todas las direcciones, y no permite compactación de memoria. Se usó históricamente en sistemas batch y con overlays.

10.3.2.3. En Tiempo de Ejecución

Las direcciones se traducen dinámicamente durante la ejecución usando hardware especial (MMU). El compilador genera direcciones lógicas, y la MMU traduce cada acceso a memoria en tiempo real.

```
1 // El compilador genera direcciones lógicas
2 int x = 10; // Genera: MOV [100], 10 (dirección lógica)
3 // La MMU traduce 100 -> dirección física en cada acceso
```

Esta técnica ofrece ventajas fundamentales: el proceso puede moverse en memoria mediante compactación, la protección entre procesos es automática, soporta memoria virtual con swap, y permite compartir memoria entre procesos de forma controlada.

El binding dinámico es la ÚNICA forma de soportar multiprogramación con protección, memoria virtual, compactación dinámica y espacios de direcciones independientes. Sin binding dinámico, no existirían los sistemas operativos modernos tal como los conocemos.

10.3.3. Componentes Hardware

10.3.3.1. Memory Management Unit (MMU)

La MMU es un circuito hardware que traduce direcciones lógicas a físicas en tiempo de ejecución. Opera a velocidad del CPU sin intervención del SO, lo cual es absolutamente necesario para mantener el rendimiento del sistema.

El funcionamiento básico es simple en concepto pero crítico en implementación: el CPU genera una dirección lógica, la MMU calcula la dirección física aplicando una función con ciertos parámetros, y la RAM recibe la dirección física resultante. El SO configura los parámetros (registros base/-límite, tablas de páginas), pero la traducción es 100 % hardware.

¿Por qué la MMU debe ser hardware y no software? La respuesta está en los números: se ejecuta en CADA acceso a memoria, un programa hace millones de accesos por segundo, si fuera software el sistema sería inutilizable, y el overhead debe ser menor a 10 nanosegundos por traducción. A esa velocidad, solo el hardware puede operar.

10.3.3.2. Translation Lookaside Buffer (TLB)

La MMU necesita consultar tablas de páginas en RAM para traducir direcciones. Como esto es lento (más de 100 nanosegundos), existe una caché especial dentro del CPU llamada TLB.

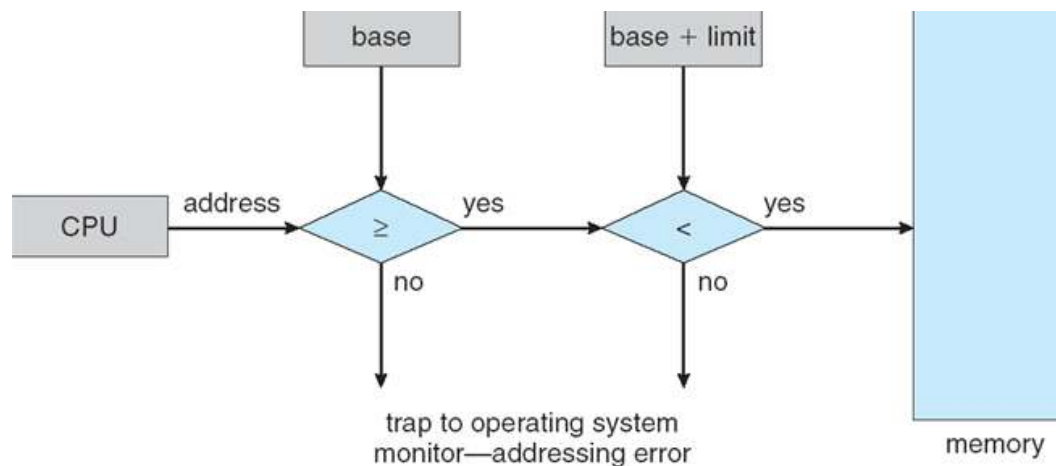
El TLB es una caché hardware de alta velocidad que almacena traducciones recientes de páginas. Típicamente contiene entre 64 y 512 entradas, con tiempo de acceso del orden de 1 nanosegundo (típicamente 1-2 ciclos de CPU). El proceso de traducción con TLB funciona así: el CPU genera una dirección lógica, la MMU busca en TLB en menos de 1 nanosegundo. Si hay un *TLB hit*, usa la traducción cacheada y el acceso total toma alrededor de 10 nanosegundos. Si hay un *TLB miss*, busca en la tabla de páginas en RAM, lo que toma alrededor de 100 nanosegundos. Si fue miss, la entrada se cachea en TLB para futuros accesos.

La efectividad del TLB es impresionante: el hit rate típico es de 98-99 %, gracias a la localidad espacial (los procesos acceden memoria cercana) y temporal (mismas páginas repetidamente). Una aplicación bien escrita puede tener un hit rate mayor al 99 %, lo que hace que el overhead de traducción sea casi imperceptible.

10.3.3.3. Registros Base y Límite

En los esquemas más simples de gestión de memoria, la MMU usa solo dos registros especiales. El **registro base** contiene la dirección física donde comienza el proceso, y el **registro límite** especifica el tamaño máximo del espacio del proceso.

La traducción es directa: $\text{Dirección Física} = \text{Dirección Lógica} + \text{Base}$. Pero hay una verificación crítica: Si $(\text{Dirección Lógica} \geq \text{Límite})$: Generar TRAP (Segmentation Fault).



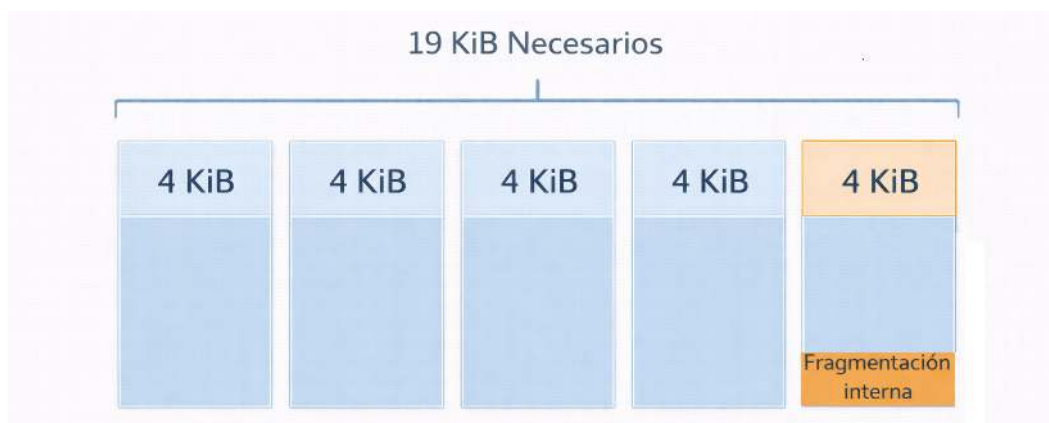
La verificación de límites ocurre en HARDWARE mediante un circuito comparador. El SO carga Base y Límite al hacer context switch. Si un proceso intenta acceder fuera de su espacio, el hardware genera un TRAP automáticamente, y el SO maneja el TRAP (típicamente matando el proceso).

10.3.4. Fragmentación

La fragmentación es el desperdicio de memoria que no puede usarse eficientemente. Entender la diferencia entre sus dos tipos es fundamental para comprender las ventajas y desventajas de cada técnica de gestión de memoria.

10.3.4.1. Fragmentación Interna

La fragmentación interna es memoria desperdiciada DENTRO de una región asignada. Ocurre cuando se asigna más memoria de la necesitada. Imaginá que un proceso necesita 19 KiB pero el sistema asigna bloques de 4 KiB. Se asignan 5 bloques (20 KiB), desperdiciando 1 KiB.



El último "bloque" asignado queda con un sector inutilizable, en el ejemplo 1KiB.

Las causas principales son la asignación en bloques de tamaño fijo, políticas de alineación de memoria, y overhead de estructuras administrativas. Ocurre típicamente en particiones fijas, paginación (desperdicio en última página), y Buddy System.

10.3.4.2. Fragmentación Externa

La fragmentación externa es memoria desperdiciada ENTRE regiones asignadas. Hay suficiente memoria libre total, pero no es contigua. Por ejemplo, si tenés memoria total de 64 MiB con 16 MiB libres, pero en distintos bloques no contiguos, no podés asignar un proceso de 10 MiB a pesar de tener espacio suficiente.

Las causas son la asignación y liberación de bloques de tamaño variable, los procesos que terminan dejan huecos, y con el tiempo la memoria se "perfora". Ocurre en particiones dinámicas, segmentación, y cualquier esquema de asignación variable. La solución es la compactación (mover procesos para consolidar memoria libre), pero es costosa.

La fragmentación externa es uno de los problemas más insidiosos en gestión de memoria. Puede hacer que un sistema con 50 % de memoria libre sea incapaz de asignar nuevos procesos. La paginación fue inventada específicamente para resolver este problema.

10.4. Técnicas de Asignación Contigua

Las primeras técnicas de gestión de memoria asignaban espacios contiguos a cada proceso. Aunque simples, estas técnicas nos enseñan lecciones importantes sobre los trade-offs en diseño de sistemas.

10.4.1. Particiones Fijas

En los primeros sistemas multiprogramados, la memoria se dividía en particiones de tamaño fijo al inicio del sistema. Esta decisión de diseño priorizaba la simplicidad sobre la eficiencia. Imaginá un esquema de memoria con particiones fijas donde después del SO hay varias particiones de diferentes tamaños: 128 KiB, 256 KiB, 512 KiB, y 64 KiB, ocupando todo el espacio hasta 1024 KiB.

El mecanismo de asignación es extremadamente simple: cuando llega un proceso, se busca una partición libre que lo contenga, el proceso ocupa toda la partición aunque no la use completamente, y al terminar, la partición queda libre para el próximo proceso. Las ventajas son tentadoras: implementación extremadamente simple, asignación y liberación en $O(1)$, sin fragmentación externa, y protección fácil porque cada partición tiene base y límite fijos. Sin embargo, las desventajas son severas: fragmentación interna que puede ser brutal, número limitado de procesos fijado al inicio del sistema, procesos grandes pueden simplemente no caber, y memoria desaprovechada si hay particiones vacías.

El problema crítico es evidente con un ejemplo: un proceso de 50 KiB en una partición de 256 KiB desperdicia 206 KiB (lo que representa un 80 %). En un sistema real, este desperdicio es inaceptable.



Memoria Física



Memoria Física

10.4.2. Particiones Dinámicas

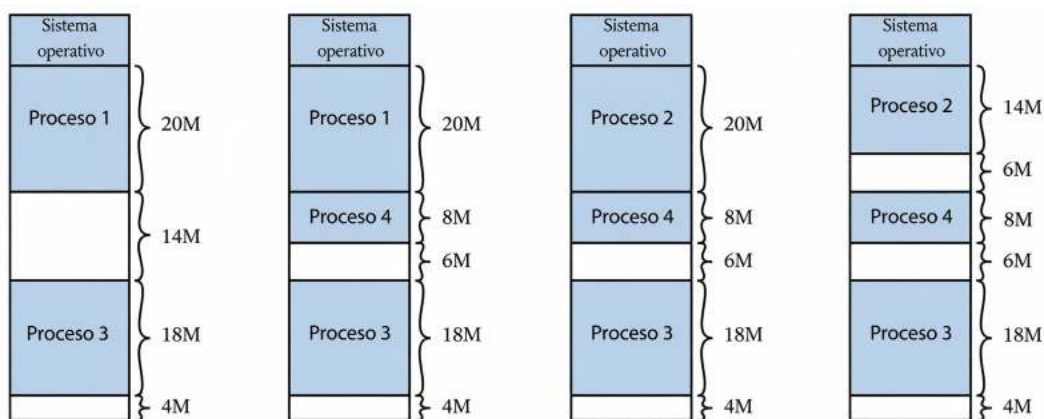
Para resolver la fragmentación interna de las particiones fijas, se desarrollaron las particiones dinámicas: cada proceso recibe exactamente la cantidad de memoria que necesita. Esto parece la solución perfecta, pero como veremos, introduce nuevos problemas.

La evolución de la memoria con particiones dinámicas muestra el problema claramente. Al inicio, el sistema arranca con todo el espacio libre. Cuando llega el primer proceso (P1 de 100 KiB), se le

asigna exactamente ese espacio. Luego llegan P2 (200 KiB) y P3 (150 KiB), ocupando sus espacios precisos. Hasta acá todo perfecto: no hay desperdicio.

El problema aparece cuando P1 termina. Queda un hueco de 100 KiB entre el SO y P2. Luego P2 termina, dejando otro hueco de 200 KiB. Ahora tenemos memoria libre total de 810 KiB, pero fragmentada en tres bloques separados. Un proceso que necesite 400 KiB no puede ejecutarse, a pesar de que hay 810 KiB libres en total. Esta es la esencia de la fragmentación externa.

Las ventajas iniciales son claras: sin fragmentación interna, número dinámico de procesos, y uso eficiente de memoria al principio. Pero las desventajas son significativas: fragmentación externa severa con el tiempo, algoritmo de asignación más complejo, requiere compactación periódica que es costosa, y estructuras de datos para rastrear bloques libres. \vfill



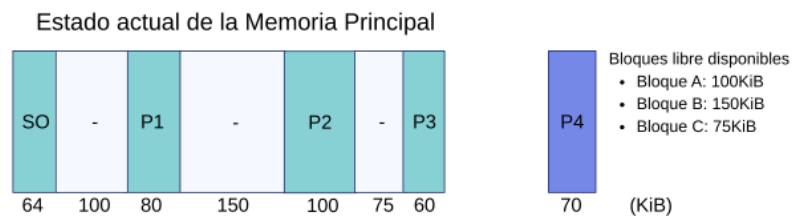
Debido a la fragmentación externa, existe la posibilidad de no poder alojar un proceso aún contando con memoria disponible.

La fragmentación externa es progresiva: empeora con el tiempo de ejecución del sistema. Un sistema que funciona bien al arrancar puede volverse ineficiente después de horas de operación.

10.4.3. Algoritmos de Asignación

Cuando llega un proceso que necesita memoria, el SO debe decidir **en qué bloque libre ubicarlo**. Esta decisión aparentemente simple tiene implicaciones profundas en el rendimiento del sistema. Para entender las diferencias, analicemos cada algoritmo usando el mismo escenario.

Escenario común para todos los algoritmos:



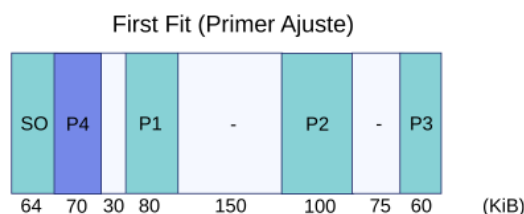
Veamos cómo cada algoritmo maneja esta situación.

10.4.3.1. First Fit (Primer Ajuste)

El algoritmo busca secuencialmente en la lista de bloques libres y asigna el **primer bloque** suficientemente grande.

Decisión de First Fit: - Examina Bloque A (100 KiB): ¿ $100 \geq 70$? Sí, **asigna aquí** - No examina Bloques B ni C (la búsqueda terminó)

Resultado: P4 se asigna en el Bloque A (100 KiB), quedando un fragmento de 30 KiB.



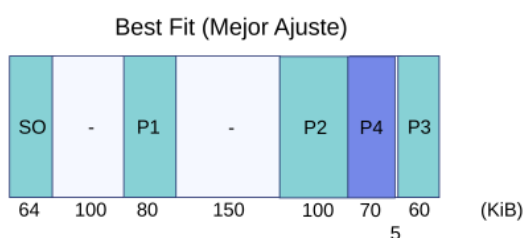
La complejidad es $O(n)$ en el peor caso, pero rápido en promedio. Tiende a dejar bloques pequeños al inicio de la lista, lo que puede generar acumulación de fragmentos inútiles en esa zona.

10.4.3.2. Best Fit (Mejor Ajuste)

Este algoritmo busca en **toda** la lista de bloques libres y asigna el **bloque más pequeño** que sea suficiente.

Decisión de Best Fit: - Examina Bloque A (100 KiB): ¿ $100 \geq 70$? Sí, candidato - Examina Bloque B (150 KiB): ¿ $150 \geq 70$? Sí, candidato (pero peor que A) - Examina Bloque C (75 KiB): ¿ $75 \geq 70$? Sí, candidato **mejor** ($75 < 100 < 150$) - **Asigna en Bloque C** (el más pequeño que alcanza)

Resultado: P4 se asigna en el Bloque C (75 KiB), quedando un fragmento de 5 KiB.



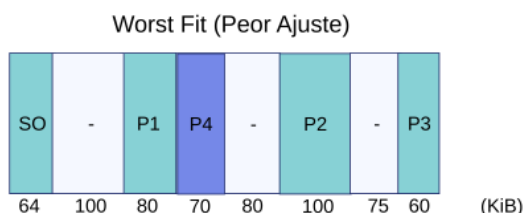
La complejidad es $O(n)$ siempre porque debe recorrer toda la lista. Minimiza el desperdicio por asignación individual, pero genera muchos bloques muy pequeños (como ese fragmento de 5 KiB) que terminan siendo inútiles.

10.4.3.3. Worst Fit (Peor Ajuste)

Contraintuitivamente, este algoritmo busca en toda la lista y asigna el **bloque más grande** disponible.

Decisión de Worst Fit: - Examina Bloque A (100 KiB): Candidato - Examina Bloque B (150 KiB): Candidato **mejor** (el más grande) - Examina Bloque C (75 KiB): No supera a B - **Asigna en Bloque B** (el más grande disponible)

Resultado: P4 se asigna en el Bloque B (150 KiB), quedando un fragmento de 80 KiB.



La complejidad es $O(n)$ siempre, pero deja bloques grandes que son más útiles que los pequeños. En simulaciones, suele tener mejor rendimiento que Best Fit.

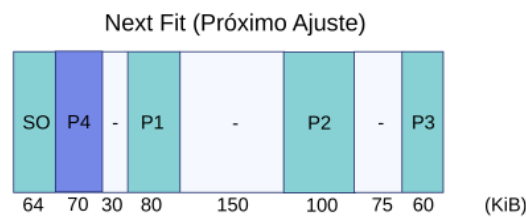
10.4.3.4. Next Fit (Siguiente Ajuste)

Similar a First Fit, pero continúa la búsqueda desde donde terminó la última asignación en forma circular.

Decisión de Next Fit: - Supongamos que la última asignación fue el Proceso P3 - Comienza búsqueda desde "el inicio" - Examina Bloque A (100 KiB): ¿ $100 \geq 70$? Sí, **asigna aquí** - La próxima búsqueda comenzará después de este punto

Resultado: P4 se asigna en el Bloque A (100 KiB), quedando un fragmento de 30 KiB.

Nota: En este ejemplo coincide con First Fit, simplemente porque se utilizó como ultimo proceso asignado P3, en cambio si el ultimo asignado hubiera sido P2, el asignado sería Bloque C.



Tiene complejidad $O(n)$ en el peor caso, distribuye asignaciones más uniformemente, y evita la concentración de bloques pequeños al inicio de la memoria.

¿Por qué Worst Fit puede ser más eficiente que Best Fit? Best Fit genera muchos bloques MUY pequeños (como ese fragmento de 5 KB) que son prácticamente inútiles. Worst Fit deja bloques grandes (80 KB en nuestro ejemplo) que tienen más probabilidad de ser utilizables para procesos futuros. First Fit deja un fragmento de 30 KB, intermedio entre ambos. Esto demuestra que la intuición puede fallar en sistemas complejos.

En la práctica, los sistemas modernos usan variantes de First Fit con optimizaciones como listas ordenadas y segregación por tamaño. La elección del algoritmo depende del patrón de uso esperado del sistema.

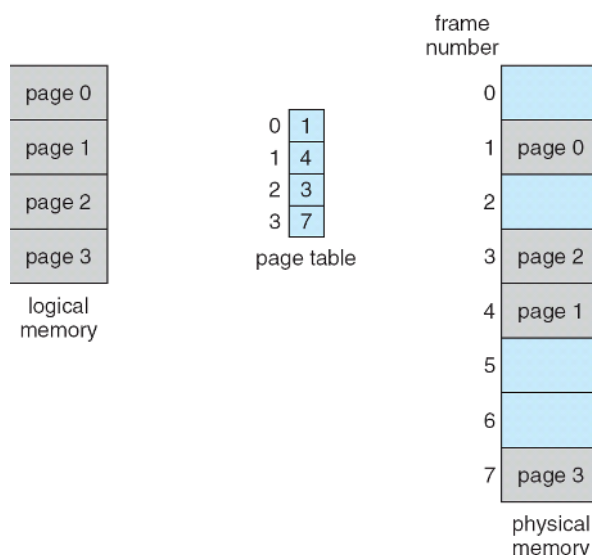
10.5. Paginación Simple

La idea central de la paginación es dividir el espacio de direcciones lógicas y la memoria física en bloques de tamaño fijo llamados páginas y marcos (frames). Una página es un bloque de memoria lógica, típicamente de 4 KiB. Un marco es un bloque de memoria física del mismo tamaño que una página. La tabla de páginas es la estructura que mapea páginas a marcos.

La ventaja fundamental es que las páginas de un proceso NO necesitan estar contiguas en memoria física. Esto resuelve completamente el problema de fragmentación externa. Imaginá el espacio lógico de un proceso con páginas 0, 1, 2 y 3. En memoria física, la página 0 puede estar en el marco 5, la página 1 en el marco 2, la página 2 en el marco 7, y la página 3 en el marco 4. No importa que estén dispersas: la MMU traduce cada acceso correctamente.

Las ventajas de la paginación son significativas: elimina fragmentación externa completamente, la asignación y liberación es simple (solo buscar marcos libres), permite compartir páginas entre procesos, facilita la implementación de memoria virtual, y ofrece protección a nivel de página. Las desventajas incluyen fragmentación interna en la última página, overhead de la tabla de páginas, cada acceso a memoria requiere traducción, y complejidad adicional en hardware.

La paginación es el fundamento de prácticamente todos los sistemas operativos modernos. Aunque tiene costos, los beneficios en términos de flexibilidad y protección son indispensables.

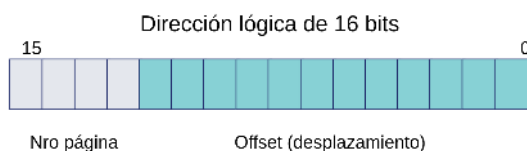


10.5.1. Formato de Dirección Lógica

Una dirección lógica en paginación se divide en dos campos: número de página y offset dentro de la página. Si el tamaño de página es 2^d bytes y el espacio lógico es 2^m bytes , entonces una dirección lógica tiene $m \text{ bits}$ divididos en: $p = m - d \text{ bits}$ para número de página, y $d \text{ bits}$ para offset dentro de la página.

Veamos un ejemplo concreto. Con un espacio de 64 KiB con páginas de 4 KiB, tenemos espacio lógico de 2^{16} bytes (64 KiB) requiriendo 16 bits de dirección. - Espacio lógico: 2^{16} bytes (64 KiB) -> 16 bits de dirección - Tamaño de página: 2^{12} bytes (4 KiB) -> 12 bits de offset - Bits para número de página: $16 - 12 = 4 \text{ bits}$ - Número de páginas: $2^4 = 16 \text{ páginas}$

El formato de dirección de 16 bits se divide en 4 bits para el número de página (permitiendo páginas 0-15) y 12 bits para el offset (permitiendo offset 0-4095).



Representación de dirección lógica de 16 bits, utilizando 4 bits (orden superior) para número de página (0-15) y los restantes 12 bits para el desplazamiento (0-4095).

Formato de Dirección Lógica: Si el tamaño de página es 2^d bytes y el espacio lógico es 2^m bytes, entonces una dirección lógica tiene m bits divididos en: $p = m - d$ bits para número de página, d bits para offset dentro de la página.

10.5.2. Traducción de Direcciones

El proceso de traducción usa la tabla de páginas del proceso. El algoritmo es directo: extraés el número de página p de los bits más significativos, extraés el offset d de los bits menos significativos, buscás en la tabla de páginas $\text{marco} = \text{tabla_paginas}[p]$, y calculás la dirección física como $DF = \text{marco} * \text{tamao_pagina} + d$.

Veamos un ejemplo numérico completo. Con tamaño de página de 1 KiB ($1024\text{bytes} = 2^{10}$), espacio lógico de 8 KiB ($8192\text{bytes} = 2^{13}$), tenemos 13 bits de dirección total, 3 bits de página (8 páginas), y 10 bits de offset (1024 posiciones).

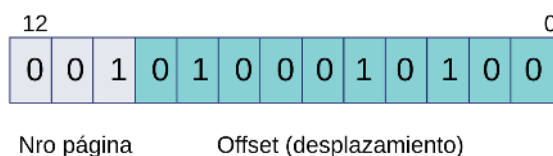
```
1 Configuración:
2 - Tamaño de página: 1 KiB (1024 bytes = 2^10)
3 - Espacio lógico: 8 KiB (8192 bytes = 2^13)
4 - Bits de dirección: 13 bits
5 - Bits de página: 13 - 10 = 3 bits (8 páginas)
6 - Bits de offset: 10 bits (1024 posiciones)
```

Supongamos una tabla donde la página 0 mapea al marco 5, la página 1 al marco 2, la página 2 al marco 7, y la página 3 al marco 0.

Página	Marco
0	5
1	2
2	7
3	0

Para traducir la dirección lógica 1300, primero la convertimos a binario: $1300_{10} = 10100010100_2$. Separamos en página ($001 = 1$) y offset ($0100010100 = 276$).

Nota que hay que completar con la cantidad de bits la dirección lógica (completar con 0 a la izquierda), de otro modo nos quedaría número de páginas 101 (5).



Consultar tabla: $tabla[1] = marco2$

Finalmente calculamos la dirección física:

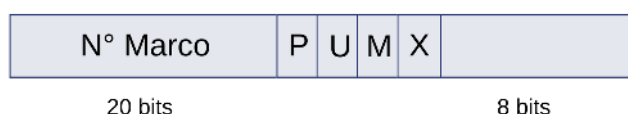
$$DF = 2 * 1024 + 276 = 2048 + 276 = 2324$$

El proceso de traducción es completamente transparente para el proceso. El programa genera la dirección lógica 1300, pero el hardware accede a la dirección física 2324. El proceso nunca sabe dónde está realmente en memoria.

10.5.3. Tabla de Páginas

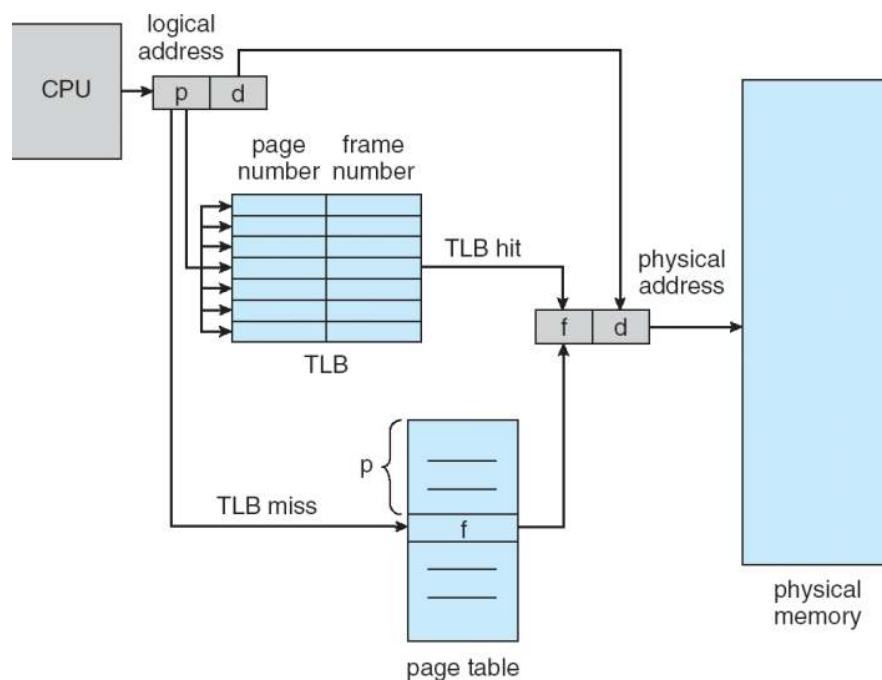
La tabla de páginas es una estructura de datos mantenida por el SO que mapea números de página lógica a números de marco físico. Cada proceso tiene su propia tabla de páginas independiente. Cada entrada de la tabla (PTE - Page Table Entry) contiene más que solo el número de marco. Incluye el marco (número de marco físico donde está la página), el bit P o V (Presencia/Validate, indica si la página está en memoria o en disco), el bit U o R (Uso/Referenced, para algoritmos de reemplazo), el bit M o W (Modificada/Write/Dirty, indica si la página fue modificada), el bit X (Execute, permiso de ejecución), y otros campos para protección, compartición, etc.

Entrada de Tabla de páginas



Un aspecto crucial es la ubicación: la tabla de páginas está en memoria RAM, no en registros del CPU (son demasiadas entradas). El SO mantiene un registro especial llamado PTBR (Page Table Base Register) que apunta al inicio de la tabla. En cada context switch, el SO actualiza el PTBR con la tabla del nuevo proceso.

Esto introduce un problema de rendimiento: cada acceso a memoria requiere 2 accesos reales. Primero hay que leer la entrada de tabla de páginas (en RAM), luego leer el dato solicitado (en RAM). Esto duplica el tiempo de acceso a memoria. La solución es el TLB, una caché de traducciones.



10.5.4. Fragmentación Interna en Paginación

Aunque la paginación elimina fragmentación externa, tiene fragmentación interna en la última página de cada proceso. Si un proceso necesita 13.5 KiB con páginas de 4 KiB, se le asignan 4 páginas (16 KiB), desperdiciando 2.5 KiB (15.6 % de fragmentación interna).

- 1 Proceso necesita: 13.5 KiB
- 2 Tamaño de página: 4 KiB
- 3 Páginas asignadas: 4 páginas (16 KiB)
- 4 Fragmentación interna: $16 - 13.5 = 2.5$ KiB (15.6%)

La fragmentación promedio es de 0.5 páginas por proceso. Si la página es de 4 KiB, el desperdicio promedio es 2 KiB por proceso. Con 100 procesos, se desperdician 200 KiB. Existe un trade-off: páginas más pequeñas reducen fragmentación interna pero aumentan el overhead de las tablas.

10.6. Segmentación

La paginación resuelve problemas técnicos brillantemente, pero no refleja la estructura lógica del programa. La segmentación aborda este aspecto desde una perspectiva completamente diferente.

10.6.1. Concepto y Motivación

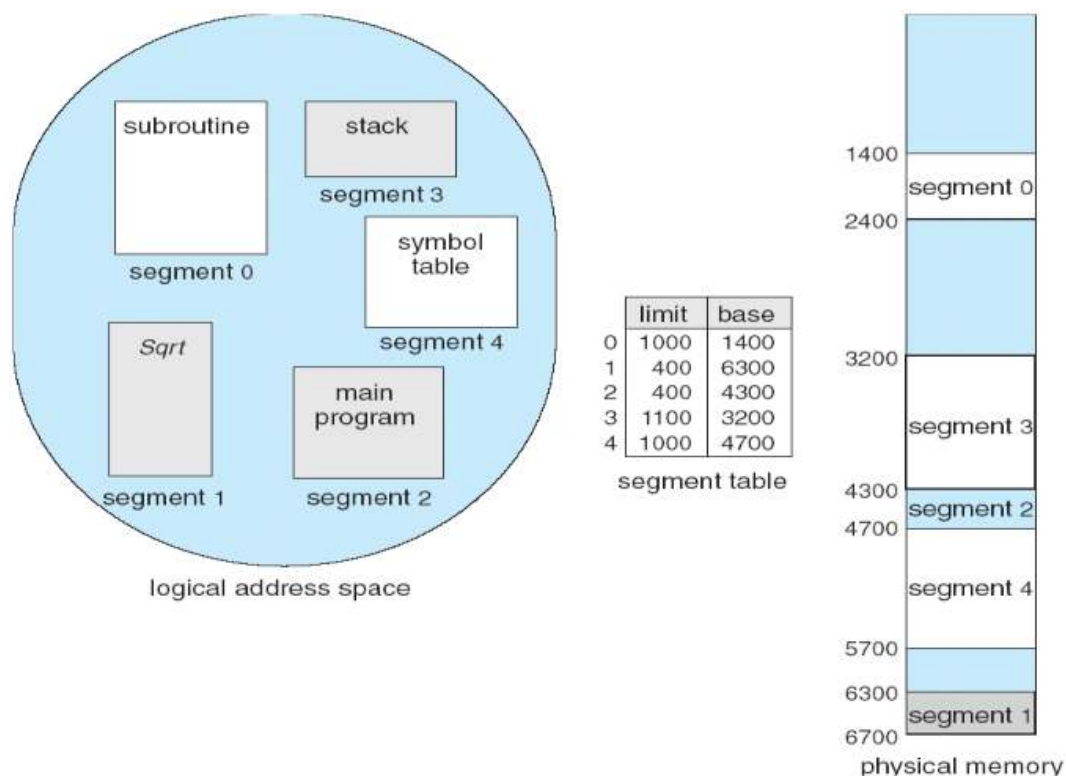
Desde la perspectiva del programador, un programa NO es un arreglo lineal de bytes, sino una colección de unidades lógicas: segmento de código (instrucciones), segmento de datos globales, segmento de heap (memoria dinámica), segmento de stack (variables locales), y segmentos de librerías compartidas.

La segmentación divide el espacio de direcciones en segmentos de tamaño variable, donde cada segmento representa una unidad lógica del programa. La diferencia clave con paginación es fundamental: la paginación divide por tamaño fijo usando un criterio técnico (hardware), mientras que la segmentación divide por tamaño variable usando un criterio lógico (programador). La paginación es invisible al programador, la segmentación es visible. La paginación genera fragmentación interna, la segmentación genera fragmentación externa. La protección en paginación es por página, en segmentación es por segmento (más natural). La compartición en paginación es complicada, en segmentación es natural.

Un proceso puede tener un segmento 0 de código (2000 bytes) con base en 1000 y límite 2000, un segmento 1 de datos (500 bytes) con base en 5000 y límite 5500, y un segmento 2 de stack (1000 bytes) con base en 8000 y límite 9000. Cada segmento puede estar en cualquier parte de la memoria física.

Diferencia clave con paginación:

Aspecto	Paginación	Segmentación
División	Tamaño fijo (4 KiB)	Tamaño variable
Criterio	Técnico (hardware)	Lógico (programador)
Visible al programador	No	Sí
Fragmentación	Interna	Externa
Protección	Por página	Por segmento (más natural)
Compartición	Complicada	Natural



10.6.2. Formato de Dirección Lógica en Segmentación

Una dirección lógica es un par (s, d) donde s es el número de segmento y d es el desplazamiento dentro del segmento. La traducción sigue estos pasos: extraés s y d de la dirección lógica, consultás la tabla de segmentos $entrada = tabla_segmentos[s]$, verificás si $d \geq entrada.limite \rightarrow$ **Segmentation Fault**, y calculás $DF = entrada.base + d$.

Con una tabla donde el segmento 0 tiene base 1000 y límite 2000, el segmento 1 tiene base 5000 y límite 500, y el segmento 2 tiene base 8000 y límite 1000, la dirección $(1, 250)$ se traduce así: $s = 1, d = 250, base = 5000, limite = 500$. Como $250 < 500$ es válido, calculamos $DF = 5000 + 250 = 5250$. Si intentamos traducir $(1, 600)$, como 600 no es menor que 500, el hardware genera un **TRAP** (Segmentation Fault).

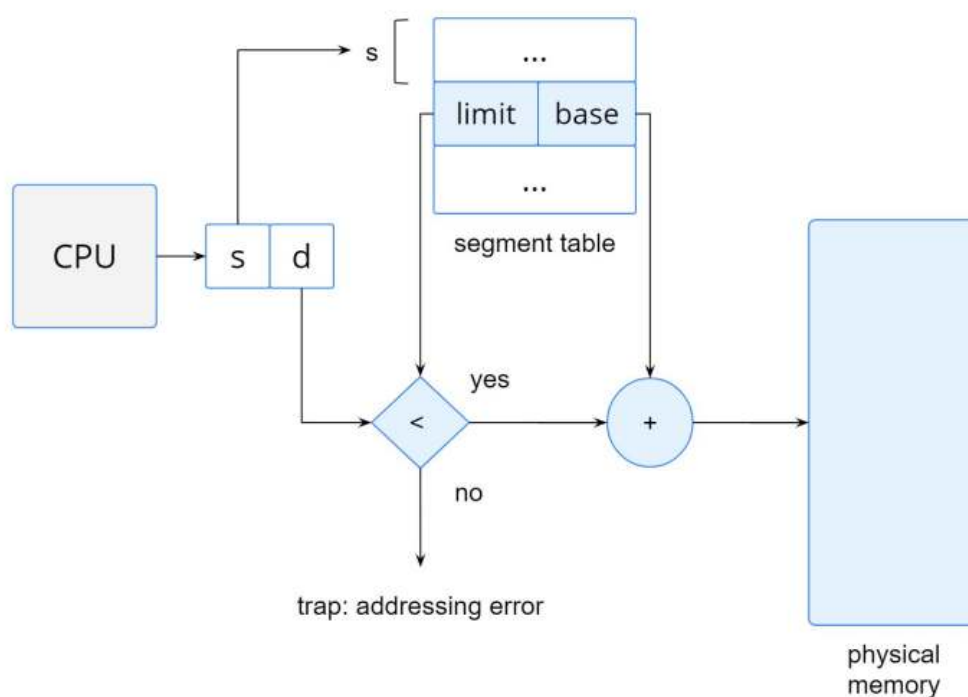
Segmento	Base	Límite
0	1000	2000
1	5000	500
2	8000	1000

¹ Traducir: $(1, 250)$

```

2  1. s=1, d=250
3  2. Base=5000, Límite=500
4  3. ?`250 < 500? Sí -> válido
5  4. DF = 5000 + 250 = 5250
6
7  Traducir: (1, 600)
8  1. s=1, d=600
9  2. Base=5000, Límite=500
10 3. ?`600 < 500? No -> TRAP (Segmentation Fault)

```



10.6.3. Ventajas de Segmentación

La segmentación refleja la estructura lógica del programa naturalmente, ofrece protección natural (cada segmento tiene sus propios permisos), facilita la compartición (código compartido = mismo segmento), permite crecimiento dinámico de segmentos (heap, stack), y facilita modularidad y librerías compartidas.

La compartición de código es elegante en segmentación. Si los procesos A y B ejecutan el mismo programa, ambos pueden apuntar al mismo segmento 0 de código (read-only y compartido), mientras mantienen sus propios segmentos 1 de datos privados. Esto es conceptualmente simple y eficiente.

Las desventajas son que vuelve a aparecer la fragmentación externa (como en particiones dinámicas), la complejidad de asignación requiere algoritmos First/Best/Worst Fit, eventualmente re-

quiere compactación, y la tabla de segmentos es más compleja que la tabla de páginas.

10.6.4. Segmentación con Paginación

Los sistemas modernos combinan ambas técnicas para obtener ventajas de cada una. En la segmentación paginada, cada segmento se divide en páginas. El espacio lógico está segmentado (perspectiva lógica), pero cada segmento se implementa con paginación (evitando fragmentación externa).

El proceso de traducción ocurre en dos niveles. Una dirección lógica es (s, p, d) donde s es el número de segmento, p es el número de página dentro del segmento, y d es el offset dentro de la página. Primero consultas la tabla de segmentos para obtener la tabla de páginas del segmento, luego consultas la tabla de páginas del segmento para obtener el marco, y finalmente calculas la dirección física como $\text{marco} * \text{tamaño_página} + d$.

```
1 Dirección lógica: (s, p, d)
2 - s = número de segmento
3 - p = número de página dentro del segmento
4 - d = offset dentro de la página
5
6 1. Consultar tabla de segmentos -> obtener tabla de páginas del segmento
7 2. Consultar tabla de páginas del segmento -> obtener marco
8 3. Calcular dirección física: marco * tamaño_página + d
```

Intel x86 (arquitectura IA-32) implementa este esquema. Una dirección tiene un selector de segmento de 16 bits que incluye un índice en la GDT (Global Descriptor Table) y un offset. El descriptor de segmento en GDT/LDT proporciona la base del segmento. Esto se combina con el offset que contiene el número de página y el offset dentro de la página. La tabla de páginas del segmento mapea a marcos físicos.

Las ventajas del esquema híbrido son claras: combina la protección y compartición natural de segmentación con la ausencia de fragmentación externa de paginación. Los segmentos pueden crecer dinámicamente agregando páginas, y el uso de memoria es mejor que segmentación pura.

10.7. Técnicas Avanzadas

10.7.1. Buddy System

El Buddy System es un algoritmo de asignación que busca balancear la velocidad de asignación con la fragmentación. Su elegancia está en su simplicidad.

El sistema funciona así: la memoria total es una potencia de 2 (por ejemplo, 256 KiB). Cuando se solicita memoria, se busca el bloque más pequeño (potencia de 2) que lo contenga. Si no existe

ese tamaño, se divide un bloque mayor recursivamente (splitting). Al liberar, se intenta fusionar con el buddy si también está libre (coalescing).

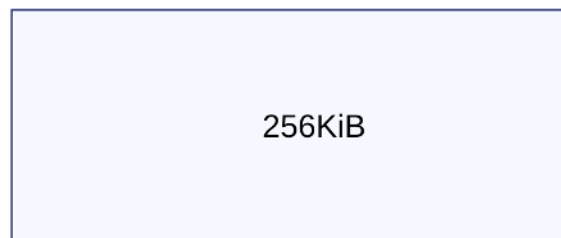
La regla del buddy es matemática, dos bloques de tamaño 2^k en direcciones $addr1$ y $addr2$ son buddies si:

$$addr1 XOR addr2 == 2^k$$

Veamos la operación completa.

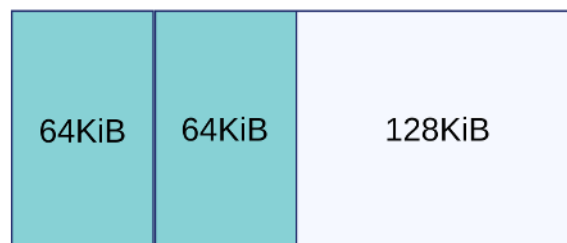
Estado inicial: 256 KiB libre

Estado inicial - 256 KiB libres



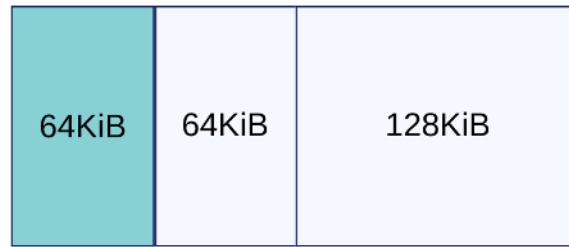
```
1 Solicitud: 40 KiB
2 -> Necesita bloque de 64 KiB (2^6)
3 -> Dividir 256 -> 128 + 128
4 -> Dividir 128 -> 64 + 64
5 -> Asignar primer 64 KiB
```

Estado después de asignar 35KiB



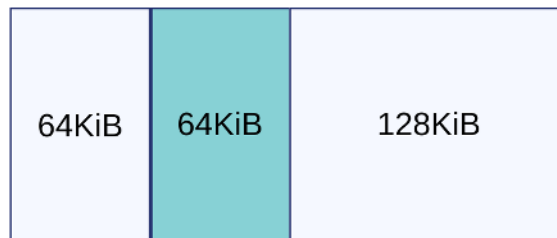
```
1 Solicitud: 35 KiB
2 -> Necesita bloque de 64 KiB
3 -> Ya hay uno libre, asignar
```

Estado después de asignar 40KiB



- ```
1 Liberar primer bloque (64 KiB):
2 -> Su buddy (segundo 64 KiB) está ocupado
3 -> No se puede fusionar
```

Estado después de liberar primer proceso



- ```
1 Liberar segundo bloque (64 KiB):  
2 -> Su buddy (primer 64 KiB) está libre  
3 -> Fusionar en 128 KiB  
4 -> El nuevo 128 tiene buddy libre (otro 128)  
5 -> Fusionar en 256 KiB
```

Luego de liberar proceso 2



Las ventajas son asignación y liberación rápidas en $O(\log n)$, coalescing automático sin escanear toda la memoria, reduce fragmentación externa comparado con particiones dinámicas, e implementación simple con listas por tamaño. Las desventajas son la fragmentación interna (siempre se asigna potencia de 2), por ejemplo un proceso de 65 KiB recibe 128 KiB desperdiciando 63 KiB, y no es tan eficiente como paginación pura.

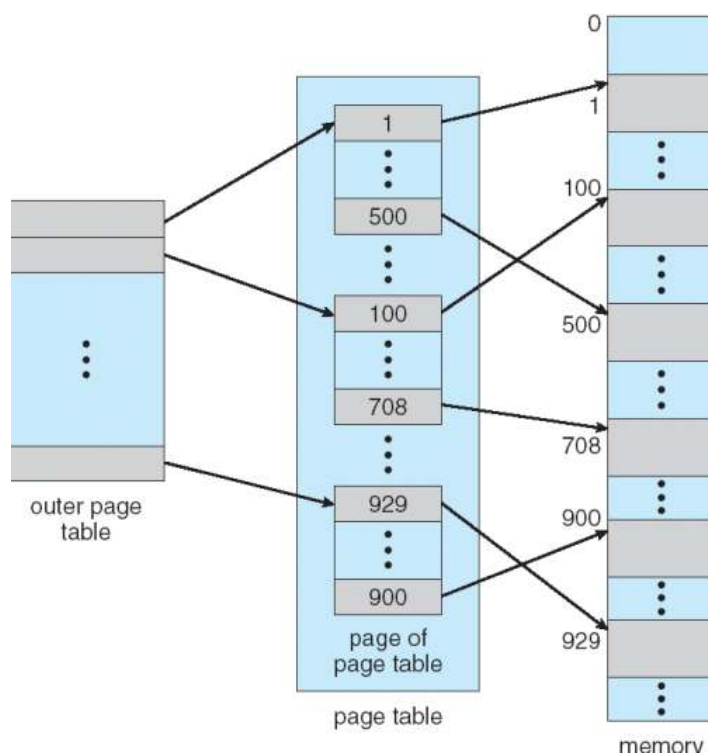
Linux usa una variante del Buddy System para asignar páginas físicas en el kernel, con bloques de hasta orden 11 (es decir, 2^{11} páginas). Es un buen balance entre eficiencia y complejidad.

10.7.2. Paginación Multinivel

Cuando el espacio de direcciones es muy grande, la tabla de páginas se vuelve enorme. Este es un problema serio en sistemas modernos.

En un sistema de 32 bits con páginas de 4 KiB, hay $2^{32} = 4GiB$ de direcciones posibles, lo que significa $2^{32}/2^{12} = 2^{20}$ 1 millón de páginas posibles. Si cada entrada ocupa 4 bytes, el tamaño de la tabla es 4 MiB por proceso. Con 100 procesos, necesitaríamos 400 MiB solo en tablas de páginas, lo cual es inaceptable.

La solución es paginar la tabla de páginas misma, creando una jerarquía de múltiples niveles.

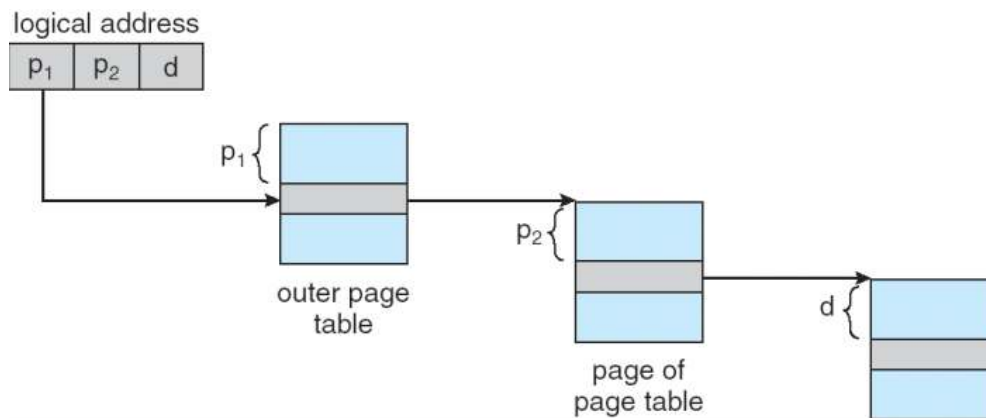


10.7.2.1. Paginación de Dos Niveles

En paginación de dos niveles, la tabla de páginas se divide en páginas. Se mantiene un directorio de páginas que apunta a las tablas de páginas de segundo nivel.

El formato de dirección lógica se divide en tres partes: directorio (p1), página (p2), y offset (d). La traducción ocurre así: usas p1 para indexar el directorio de páginas y obtener la tabla de nivel 2,

usás p_2 para indexar la tabla de nivel 2 y obtener el marco, y usás d como offset dentro del marco. La ventaja es enorme: si un proceso no usa ciertas regiones de memoria, las tablas de nivel 2 correspondientes NO se crean, ahorrando memoria significativamente.



En un sistema de 32 bits con páginas de 4 KiB, podés usar 10 bits para directorio (1024 entradas), 10 bits para página (1024 entradas por tabla nivel 2), y 12 bits para offset (4096 bytes). Si un proceso usa solo 4 MiB, requiere 1 entrada en directorio y 1 tabla de nivel 2 (1024 entradas), totalizando $(1024 + 1024) * 4bytes = 8KiB$, versus 4 MiB en tabla plana.

10.7.2.2. Paginación de Tres Niveles

Para espacios de direcciones de 64 bits, se requieren más niveles. La dirección se divide en p_1 , p_2 , p_3 , y offset.

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

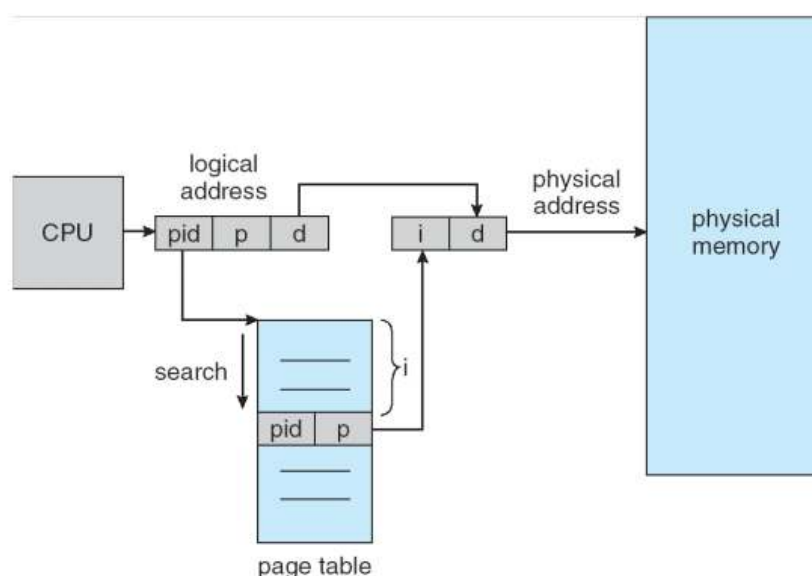
Por ejemplo, x86-64 con páginas de 4 KiB usa direcciones de 48 bits (no se usan los 64 completos), divididas en 9 bits para cada uno de cuatro niveles (PML4, PDPT, PD, PT), más 12 bits de offset. Esto crea cuatro niveles de traducción: Page Map Level 4 (PML4), Page Directory Pointer Table (PDPT), Page Directory (PD), y Page Table (PT).

El costo de traducción es significativo: con 3 niveles se necesitan 4 accesos a memoria (3 niveles más el dato). Sin TLB esto sería devastador para el rendimiento. Un hit rate del TLB del 99 % es esencial para mantener el sistema usable.

10.7.3. Tabla de Páginas Invertida

Un enfoque radicalmente diferente: en lugar de una tabla por proceso, una tabla global única para todo el sistema. La tabla de páginas invertida tiene una entrada por cada marco físico (no por página lógica). Cada entrada indica qué proceso y qué página está en ese marco.

La estructura tiene una entrada por marco físico, conteniendo el PID del proceso dueño, el número de página lógica, y flags de protección y estado. La traducción requiere extraer página p y offset d de la dirección lógica, buscar en la tabla invertida la entrada donde $(PID == actual) \text{ AND } (P \text{ página} == p)$, usar el índice de esa entrada como el marco, y calcular $DF = marco * tamaño_página + d$.



El problema crítico es que la búsqueda es $O(n)$ donde n es la cantidad de marcos. Cada acceso a memoria requiere escanear toda la tabla, lo cual es INACEPTABLE sin optimización. La solución es usar una tabla hash para acelerar la búsqueda:

```
1 Hash(PID, página) -> índice en tabla hash -> cadena de colisiones -> entrada
```

Las ventajas son que el tamaño de tabla es proporcional a memoria física (no a lógica). Un sistema con 4 GiB de RAM y páginas de 4 KiB tiene 1M marcos, entonces 1M entradas, versus potencialmente millones por proceso. Es un ahorro masivo en sistemas con muchos procesos. Las desventajas son que la búsqueda es más lenta incluso con hash, la compartición de páginas es complicada, y no es totalmente compatible con memoria virtual tradicional.

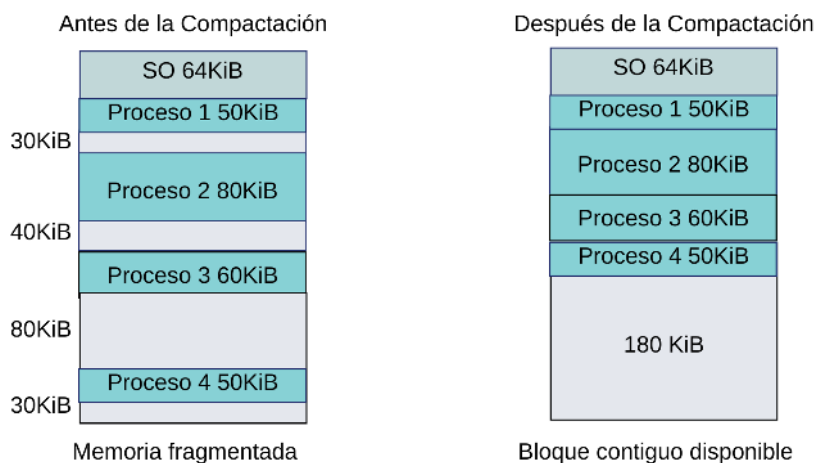
Este esquema se usó en PowerPC, IA-64 (Itanium), y algunas versiones de AIX.

10.8. Compactación y Defragmentación

La compactación es el proceso de mover procesos en memoria para consolidar los espacios libres. Es una solución directa a la fragmentación externa, pero con costos significativos.

El proceso reorganiza la memoria moviendo procesos activos para eliminar fragmentación externa, creando un único bloque contiguo de memoria libre. Antes de compactación, la memoria puede estar fragmentada con procesos y huecos libres intercalados. Después de compactación, todos los procesos están juntos y hay un único bloque libre grande al final.

El algoritmo implica identificar todos los bloques libres, mover procesos hacia direcciones bajas, actualizar tablas de asignación, y actualizar todas las referencias (registros, punteros, tablas de páginas).



Los costos son considerables: copiar todos los procesos en memoria es muy lento, hay que detener la ejecución durante la compactación, actualizar estructuras del SO, y en un sistema con 1 GiB ocupado, esto puede tomar varios segundos.

La compactación solo es factible si se usa binding en tiempo de ejecución. Con binding en compilación o carga, es imposible mover procesos. Con registros base/límite o paginación, solo hay que actualizar los registros y la MMU hace transparente el movimiento para el proceso.

Las estrategias varían: compactación completa mueve todos los procesos al inicio dejando todo el espacio libre al final, compactación parcial solo elimina huecos más pequeños que cierto umbral, y compactación selectiva solo mueve procesos que no están ejecutando activamente.

En paginación, no se necesita compactación tradicional, pero ocasionalmente se hace “defragmentación” moviendo páginas para mejorar localidad, aunque esto es raro en la práctica.

10.9. Protección y Compartición

10.9.1. Mecanismos de Protección

Los sistemas de gestión de memoria incluyen mecanismos de protección para evitar que un proceso acceda memoria de otro, evitar que un proceso acceda memoria del SO, y controlar operaciones permitidas (lectura, escritura, ejecución).

Los bits de protección en la tabla de páginas incluyen **R** (Read, página legible), **W** (Write, página escribible), y **X** (Execute, página ejecutable). Las combinaciones típicas son **R--** para solo lectura (constantes, código compartido), **RW-** para lectura/escritura (datos, heap, stack), **R-X** para solo lectura y ejecución (código), y **RWX** que es peligroso porque permite ataques de data execution.

```

1 R (Read):    Página legible
2 W (Write):   Página escribible
3 X (Execute): Página ejecutable
4
5 Combinaciones típicas:
6 R--: Solo lectura (constantes, código compartido)
7 RW-: Lectura/escritura (datos, heap, stack)
8 R-X: Solo lectura y ejecución (código)
9 RWX: Peligroso (permite data execution attacks)
```

Verificación por hardware: Cuando el CPU intenta acceder a una página, la MMU verifica automáticamente: ¿la página es válida (bit V=1)? Si no, genera Page Fault. ¿El acceso es de lectura y bit R=1? Si no, genera Protection Fault. ¿El acceso es de escritura y bit W=1? Si no, genera Protection Fault. ¿El acceso es de ejecución y bit X=1? Si no, genera Protection Fault. Si todo está bien, permite el acceso.

```

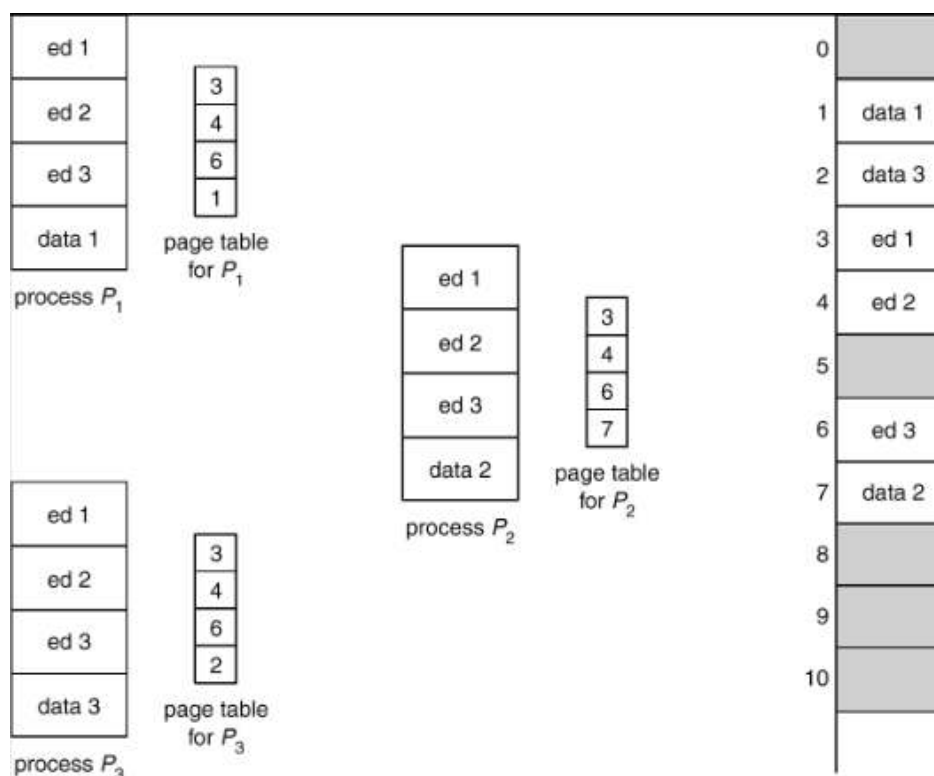
1 1. ¿La página es válida (bit V=1)?
2 NO -> Page Fault (TRAP al SO)
3
4 2. ¿El acceso es de lectura y bit R=1?
5 NO -> Protection Fault (TRAP al SO)
6
7 3. ¿El acceso es de escritura y bit W=1?
8 NO -> Protection Fault (TRAP al SO)
9
10 4. ¿El acceso es de ejecución y bit X=1?
11 NO -> Protection Fault (TRAP al SO)
12
13 5. Todo OK -> Permitir acceso
```

El bit NX (No-eXecute) es fundamental para seguridad moderna. Previene ataques de buffer overflow porque el stack y heap NO deben ser ejecutables. Si un atacante inyecta código en el stack, el CPU rechaza ejecutarlo automáticamente. Este mecanismo es una defensa crítica en sistemas actuales.

10.9.2. Compartición de Memoria

Los sistemas modernos permiten que múltiples procesos compartan páginas de memoria para código compartido (múltiples procesos ejecutando el mismo programa), librerías compartidas (libc.so, libpthread.so, etc.), y comunicación entre procesos mediante segmentos de memoria compartida.

En código compartido, dos procesos ejecutando el mismo programa pueden apuntar al mismo marco físico para su segmento de código (con permisos read-only), mientras mantienen datos y stack privados en marcos separados. Si 100 procesos ejecutan bash (1 MiB de código), sin compartición se necesitarían 100 MiB de código en RAM. Con compartición, se necesita 1 MiB de código más 100 MiB de datos privados, ahorrando 99 MiB.



Los requisitos para compartir código son que el código debe ser reentrante (no se modifica a sí mismo), las páginas compartidas deben tener permisos R-X (no escribibles), y cada proceso tiene sus propios datos y stack privados.

10.10. Código en C

10.10.1. Conceptos Básicos de Memoria y Punteros (Bonus)

Los punteros son la herramienta fundamental para trabajar con memoria en C.

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 42;           // Variable en stack
5     int *ptr = &x;       // ptr apunta a x
6
7     printf("Valor de x: %d\n", x);           // 42
8     printf("Dirección de x: %p\n", &x);     // Dirección lógica
9     printf("Valor de ptr: %p\n", ptr);       // Igual que &x
10    printf("Valor apuntado: %d\n", *ptr);    // 42
11
12    *ptr = 100;           // Modificar x a través del puntero
13    printf("Nuevo valor de x: %d\n", x);     // 100
14
15    return 0;
16 }
```

Salida típica:

```
1 Valor de x: 42
2 Dirección de x: 0x7ffd8c5e3a9c
3 Valor de ptr: 0x7ffd8c5e3a9c
4 Valor apuntado: 42
5 Nuevo valor de x: 100
```

La dirección mostrada es una dirección LÓGICA. La MMU la traduce a una dirección física que el programa nunca ve. El programa opera completamente en su espacio virtual.

10.10.2. Aritmética de Punteros

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[] = {10, 20, 30, 40, 50};
5     int *ptr = arr; // ptr apunta al primer elemento
6
7     printf("ptr apunta a: %p, valor: %d\n", ptr, *ptr);
8     // ptr = dirección base, *ptr = 10
9
10    ptr++; // Avanza sizeof(int) bytes
11    printf("Después de ptr++: %p, valor: %d\n", ptr, *ptr);
12    // ptr = base + 4, *ptr = 20
13
14    ptr += 2; // Avanza 2 * sizeof(int) bytes
15    printf("Después de ptr+=2: %p, valor: %d\n", ptr, *ptr);
16    // ptr = base + 12, *ptr = 40
17 }
```

```
18 // Acceso con índice (equivalente a aritmética)
19 int *base = arr;
20 printf("base[3] = %d\n", base[3]); // 40
21 printf("*(base + 3) = %d\n", *(base + 3)); // 40 (equivalente)
22
23 return 0;
24 }
```

Un punto crítico es que `ptr++` avanza `sizeof(tipo)` bytes, no 1 byte. `int*` avanza 4 bytes, `char*` avanza 1 byte, `double*` avanza 8 bytes. El compilador maneja esto automáticamente.

10.10.3. Asignación Dinámica con malloc

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     // Asignar memoria para 5 enteros
7     int *arr = (int *)malloc(5 * sizeof(int));
8
9     if (arr == NULL) {
10         fprintf(stderr, "Error: malloc falló\n");
11         return 1;
12     }
13
14     // Inicializar el arreglo
15     for (int i = 0; i < 5; i++) {
16         arr[i] = i * 10;
17     }
18
19     // Imprimir valores
20     printf("Valores: ");
21     for (int i = 0; i < 5; i++) {
22         printf("%d ", arr[i]);
23     }
24     printf("\n");
25
26     // Redimensionar con realloc
27     arr = (int *)realloc(arr, 10 * sizeof(int));
28
29     if (arr == NULL) {
30         fprintf(stderr, "Error: realloc falló\n");
31         return 1;
32     }
33
34     // Inicializar nuevos elementos
35     for (int i = 5; i < 10; i++) {
36         arr[i] = i * 10;
37     }
38
39     // Liberar memoria
40     free(arr);
41     arr = NULL; // Buena práctica: evitar dangling pointer
42 }
```

```
43     return 0;
44 }
```

Los errores comunes incluyen no verificar si malloc devuelve NULL, olvidar liberar memoria (memory leak), usar memoria después de free (use-after-free), doble free (undefined behavior), y buffer overflow (escribir fuera del bloque asignado).

10.10.4. Mapeo de Memoria con mmap

`mmap()` es una syscall que mapea archivos o memoria directamente al espacio de direcciones del proceso.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/mman.h>
4  #include <fcntl.h>
5  #include <unistd.h>
6  #include <string.h>
7
8  int main() {
9      // Crear memoria anónima compartida
10     size_t size = 4096; // 1 página
11
12     void *addr = mmap(
13         NULL, // Dirección (NULL = el SO elige)
14         size, // Tamaño
15         PROT_READ | PROT_WRITE, // Protección
16         MAP_ANONYMOUS | MAP_PRIVATE, // Flags
17         -1, // File descriptor (no hay archivo)
18         0, // Offset
19     );
20
21     if (addr == MAP_FAILED) {
22         perror("mmap");
23         return 1;
24     }
25
26     printf("Memoria mapeada en: %p\n", addr);
27
28     // Usar la memoria como un array
29     int *data = (int *)addr;
30     for (int i = 0; i < 10; i++) {
31         data[i] = i * i;
32     }
33
34     // Verificar
35     printf("Valores: ");
36     for (int i = 0; i < 10; i++) {
37         printf("%d ", data[i]);
38     }
39     printf("\n");
40
41     // Liberar memoria
42     if (munmap(addr, size) == -1) {
```



```

43     perror("munmap");
44     return 1;
45 }
46
47     return 0;
48 }

```

Las *ventajas de mmap* son control fino de protecciones de memoria, mapeo de archivos eficiente (I/O mapeado a memoria), memoria compartida entre procesos (MAP_SHARED), y asignación de grandes bloques sin fragmentar heap.

10.10.5. Ejemplo Integrador: Simulación de Tabla de Páginas

Este ejemplo muestra cómo simular una tabla de páginas simple en C.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5
6  #define PAGE_SIZE 1024      // 1 KiB por página
7  #define PAGE_BITS 10       // log2(1024) = 10 bits
8  #define NUM_PAGES 16       // 16 páginas lógicas
9  #define NUM_FRAMES 32      // 32 marcos físicos
10
11 // Entrada de tabla de páginas
12 typedef struct {
13     uint32_t frame;          // Número de marco físico
14     bool valid;              // ¿Página en memoria?
15     bool read;               // Permiso de lectura
16     bool write;              // Permiso de escritura
17     bool execute;            // Permiso de ejecución
18 } PageTableEntry;
19
20 // Tabla de páginas
21 typedef struct {
22     PageTableEntry entries[NUM_PAGES];
23     uint32_t pid;            // ID del proceso
24 } PageTable;
25
26 // Memoria física simulada
27 uint8_t physical_memory[NUM_FRAMES * PAGE_SIZE];
28
29 // Crear tabla de páginas vacía
30 PageTable* create_page_table(uint32_t pid) {
31     PageTable *pt = (PageTable *)malloc(sizeof(PageTable));
32     pt->pid = pid;
33
34     // Inicializar todas las entradas como inválidas
35     for (int i = 0; i < NUM_PAGES; i++) {
36         pt->entries[i].valid = false;
37         pt->entries[i].read = false;
38         pt->entries[i].write = false;
39         pt->entries[i].execute = false;
40         pt->entries[i].frame = 0;

```

```

41     }
42
43     return pt;
44 }
45
46 // Mapear una página a un marco
47 void map_page(PageTable *pt, uint32_t page, uint32_t frame,
48               bool r, bool w, bool x) {
49     if (page >= NUM_PAGES || frame >= NUM_FRAMES) {
50         fprintf(stderr, "Error: página o marco fuera de rango\n");
51         return;
52     }
53
54     pt->entries[page].frame = frame;
55     pt->entries[page].valid = true;
56     pt->entries[page].read = r;
57     pt->entries[page].write = w;
58     pt->entries[page].execute = x;
59
60     printf("Mapeada página %u -> marco %u (R:%d W:%d X:%d)\n",
61           page, frame, r, w, x);
62 }
63
64 // Traducir dirección lógica a física
65 int translate_address(PageTable *pt, uint32_t logical_addr,
66                      uint32_t *physical_addr, bool is_write) {
67     // Extraer número de página y offset
68     uint32_t page = logical_addr >> PAGE_BITS;
69     uint32_t offset = logical_addr & ((1 << PAGE_BITS) - 1);
70
71     printf("Dirección lógica: 0x%X\n", logical_addr);
72     printf(" -> Página: %u, Offset: %u\n", page, offset);
73
74     // Verificar que la página existe
75     if (page >= NUM_PAGES) {
76         fprintf(stderr, "ERROR: Página %u fuera de rango\n", page);
77         return -1;
78     }
79
80     // Verificar que la página es válida
81     if (!pt->entries[page].valid) {
82         fprintf(stderr, "ERROR: Page Fault - página %u no válida\n", page);
83         return -2;
84     }
85
86     // Verificar permisos
87     if (is_write && !pt->entries[page].write) {
88         fprintf(stderr, "ERROR: Protection Fault - página %u no escribible\n",
89               ,
90               page);
91         return -3;
92     }
93
94     if (!is_write && !pt->entries[page].read) {
95         fprintf(stderr, "ERROR: Protection Fault - página %u no legible\n",
96               ,
97               page);
98         return -4;
99     }

```

```

98
99     // Calcular dirección física
100     uint32_t frame = pt->entries[page].frame;
101     *physical_addr = (frame << PAGE_BITS) | offset;
102
103     printf("  -> Marco: %u\n", frame);
104     printf("  -> Dirección física: 0x%X\n", *physical_addr);
105
106     return 0; // Éxito
107 }
108
109 // Función principal de demostración
110 int main() {
111     printf("=== Simulador de Tabla de Páginas ===\n");
112     printf("Tamaño de página: %d bytes\n", PAGE_SIZE);
113     printf("Número de páginas: %d\n", NUM_PAGES);
114     printf("Número de marcos: %d\n\n", NUM_FRAMES);
115
116     // Crear tabla de páginas para proceso 42
117     PageTable *pt = create_page_table(42);
118
119     // Mapear algunas páginas
120     printf("---- Configurando mapeos ----\n");
121     map_page(pt, 0, 5, true, false, true); // Código: R-X
122     map_page(pt, 1, 8, true, false); // Datos: RW-
123     map_page(pt, 2, 3, true, true, false); // Stack: RW-
124     map_page(pt, 5, 12, true, false, false); // Constantes: R--
125     printf("\n");
126
127     // Probar traducciones
128     printf("---- Probando traducciones ----\n");
129     uint32_t phys_addr;
130     int result;
131
132     // Traducción exitosa (lectura en página 0)
133     result = translate_address(pt, 0x0100, &phys_addr, false);
134     if (result == 0) {
135         printf("✓ Traducción exitosa\n");
136     }
137     printf("\n");
138
139     // Traducción exitosa (escritura en página 1)
140     result = translate_address(pt, 0x0500, &phys_addr, true);
141     if (result == 0) {
142         printf("✓ Traducción exitosa\n");
143     }
144     printf("\n");
145
146     // Page Fault (página 3 no mapeada)
147     printf("Intentando acceder página no mapeada:\n");
148     result = translate_address(pt, 0x0C00, &phys_addr, false);
149     printf("\n");
150
151     // Protection Fault (intento de escritura en código)
152     printf("Intentando escribir en página de código:\n");
153     result = translate_address(pt, 0x0100, &phys_addr, true);
154     printf("\n");
155

```

```

156 // Calcular fragmentación interna
157 printf("--- Análisis de fragmentación ---\n");
158 uint32_t logical_size = 3500; // Proceso necesita 3.5 KiB
159 uint32_t pages_needed = (logical_size + PAGE_SIZE - 1) / PAGE_SIZE;
160 uint32_t allocated = pages_needed * PAGE_SIZE;
161 uint32_t internal_frag = allocated - logical_size;
162
163 printf("Tamaño lógico del proceso: %u bytes\n", logical_size);
164 printf("Páginas necesarias: %u\n", pages_needed);
165 printf("Memoria asignada: %u bytes\n", allocated);
166 printf("Fragmentación interna: %u bytes (%.1f%%)\n",
167        internal_frag, (internal_frag * 100.0) / allocated);
168
169 // Liberar memoria
170 free(pt);
171
172 return 0;
173 }

```

Compilación y ejecución:

```

1 gcc -o page_table_sim page_table_sim.c -Wall
2 ./page_table_sim

```

Salida esperada:

```

1 === Simulador de Tabla de Páginas ===
2 Tamaño de página: 1024 bytes
3 Número de páginas: 16
4 Número de marcos: 32
5
6 --- Configurando mapeos ---
7 Mapeada página 0 -> marco 5 (R:1 W:0 X:1)
8 Mapeada página 1 -> marco 8 (R:1 W:1 X:0)
9 Mapeada página 2 -> marco 3 (R:1 W:1 X:0)
10 Mapeada página 5 -> marco 12 (R:1 W:0 X:0)
11
12 --- Probando traducciones ---
13 Dirección lógica: 0x100
14   -> Página: 0, Offset: 256
15   -> Marco: 5
16   -> Dirección física: 0x1500
17   Traducción exitosa
18
19 Dirección lógica: 0x500
20   -> Página: 1, Offset: 256
21   -> Marco: 8
22   -> Dirección física: 0x2100
23   Traducción exitosa
24
25 Intentando acceder página no mapeada:
26 Dirección lógica: 0xC00
27   -> Página: 3, Offset: 0
28 ERROR: Page Fault - página 3 no válida
29
30 Intentando escribir en página de código:
31 Dirección lógica: 0x100
32   -> Página: 0, Offset: 256

```

```

33 ERROR: Protection Fault - página 0 no escribible
34
35 --- Análisis de fragmentación ---
36 Tamaño lógico del proceso: 3500 bytes
37 Páginas necesarias: 4
38 Memoria asignada: 4096 bytes
39 Fragmentación interna: 596 bytes (14.6%)

```

Este ejemplo demuestra la estructura de tabla de páginas, la traducción de direcciones (extracción de página y offset), verificación de permisos (R/W/X), manejo de Page Fault y Protection Fault, y cálculo de fragmentación interna.

10.11. Casos de Estudio

10.11.1. Ejercicio Simple: Traducción de Dirección Lógica

Un sistema usa paginación simple con las siguientes características: - Tamaño de memoria lógica: 32 KiB - Tamaño de página: 2 KiB

Tabla de páginas del proceso:

```

1  Página 0 -> Marco 3
2  Página 1 -> Marco 7
3  Página 2 -> Marco 1
4  Página 3 -> Marco 4

```

Preguntas: 1. ¿Cuántos bits se usan para el número de página? 2. ¿Cuántos bits se usan para el offset? 3. Traducir la dirección lógica 5000 a dirección física

Solución:

Calculamos la cantidad de bits

```

1  Memoria lógica: 32 KiB = 32 * 1024 = 32768 bytes = 2^15 bytes
2  -> Se necesitan 15 bits para direccionar toda la memoria lógica
3
4  Tamaño de página: 2 KiB = 2 * 1024 = 2048 bytes = 2^11 bytes
5  -> Se necesitan 11 bits para el offset
6
7  Bits para página = Bits totales - Bits de offset
8                    = 15 - 11 = 4 bits
9
10 Número de páginas = 2^4 = 16 páginas (0-15)

```

El formato de dirección de 15 bits se divide en 4 bits para página (permitiendo páginas 0-15) y 11 bits para offset (permitiendo offset 0-2047).

Para la traducción de 5000:

```

1 Paso 1: Convertir 5000 a binario10

```

```

2  5000 =  $2_{1001110001000}$  (necesitamos 15 bits) $_{10}$ 
3  5000 =  $2_{001001110001000}$  (padding con ceros)
4
5  Paso 2: Separar página y offset
6
7      0010  01110001000
8      (p=2)  (d=904)
9
10 Verificación:
11 - Página:  $2_{0010} = 10_2$ 
12 - Offset:  $2_{01110001000} = 10_904$ 
13 - Total:  $2 * 2048 + 904 = 4096 + 904 = 5000$ 
14
15 Paso 3: Consultar tabla de páginas
16 tabla[2] = marco 1
17
18 Paso 4: Calcular dirección física
19 DF = marco * tamaño_página + offset
20 DF = 1 * 2048 + 904
21 DF = 2048 + 904
22 DF = 2952 bytes

```

La verificación adicional muestra que el marco 1 ocupa direcciones físicas [2048, 4095], y la dirección calculada 2952 está efectivamente en ese rango. Esta verificación es importante para confirmar que la traducción es correcta.

```

1  Marco 1 ocupa direcciones físicas: [2048, 4095]
2  Dirección calculada: 2952
3  ? 2048 ≤ 2952 ≤ 4095? SÍ
4
5  En binario:
6  DF =  $10_{2952} = 2_{101110001000}$ 

```

Respuestas finales: 1. Bits para página: 4 bits 2. Bits para offset: 11 bits 3. Dirección física: 2952 bytes

10.11.2. Ejercicio Complejo: Deducción y Traducción

Se sabe que la dirección lógica 12345 se traduce a la dirección física 28729, y que la página donde está 12345 se mapea al marco 7.

1. ¿Cuál es el tamaño de página del sistema? 2. ¿Cuántos bits se usan para el número de página si el espacio lógico es de 64 KiB? 3. ¿A qué dirección física se traduce la dirección lógica 15000 sabiendo que su página se mapea al marco 5?

Solución:

Parte 1: Deducir tamaño de página

Sabemos que: - Dirección lógica (DL) = 12345 - Dirección física (DF) = 28729 - Marco = 7

La fórmula de traducción es:

```
1 DF = marco * tamaño_página + offset
```

donde *offset* es el mismo en DL y DF (los bits menos significativos).

Si dividimos DL por el tamaño de página:

```
1 DL = número_página * tamaño_página + offset
```

Como el tamaño de página siempre es potencia de 2, probamos candidatos típicos:

```
1 Hipótesis 1: tamaño_página = 1024 bytes (2^10)
2 DL = 12345 = 12 * 1024 + 57
3   página = 12, offset = 57
4 DF debería ser = 7 * 1024 + 57 = 7168 + 57 = 7225
5 Pero DF real = 28729 -> no coincide
6
7 Hipótesis 2: tamaño_página = 2048 bytes (2^11)
8 DL = 12345 = 6 * 2048 + 57
9   página = 6, offset = 57
10 DF debería ser = 7 * 2048 + 57 = 14336 + 57 = 14393
11 Pero DF real = 28729 -> no coincide
12
13 Hipótesis 3: tamaño_página = 4096 bytes (2^12)
14 DL = 12345 = 3 * 4096 + 57
15   página = 3, offset = 57
16 DF debería ser = 7 * 4096 + 57 = 28672 + 57 = 28729 ok!
17
18 ¡Coincide!
```

Verificación:

```
1 Tamaño de página = 4096 bytes = 4 KiB = 2^12 bytes
2
3 DL = 12345
4   = 12345 ÷ 4096 = 3 con resto 57
5   = página 3, offset 57
6
7 DF = 7 * 4096 + 57 = 28672 + 57 = 28729
```

Parte 2: Bits para número de página

```
1 Espacio lógico: 64 KiB = 65536 bytes = 2^16 bytes
2 -> Se necesitan 16 bits de dirección total
3
4 Tamaño de página: 4096 bytes = 2^12 bytes
5 -> Se necesitan 12 bits para offset
6
7 Bits para página = 16 - 12 = 4 bits
8 Número de páginas = 2^4 = 16 páginas (0-15)
```

Parte 3: Traducir dirección lógica 15000

En este inciso, el enunciado indica explícitamente que la página de 15000 mapea al marco 5. Eso debe tomarse como dato del inciso (puede corresponder a otro proceso o a otro estado de la tabla). No contradice el dato previo de 12345 -> marco 7, porque ese dato se usó para deducir el tamaño de página.

```

1 Paso 1: Extraer página y offset
2 15000 ÷ 4096 = 3.66...
3 3 * 4096 = 12288
4 15000 - 12288 = 2712 (este es el offset correcto)
5
6 -> página = 3, offset = 2712
7
8 Paso 2: Consultar tabla de páginas
9 Se nos dice que esta página se mapea al marco 5
10
11 Paso 3: Calcular dirección física
12 DF = marco * tamaño_página + offset
13 DF = 5 * 4096 + 2712
14 DF = 20480 + 2712
15 DF = 23192 bytes

```

Error típico de parcial: equivocarse en el resto de la división y arrastrar ese error hasta el final.
Regla práctica: siempre verificar que

$$DL = pagina * tamaño_pagina + offset$$

si no cierra exactamente, hay que rehacer la cuenta.

Verificación en binario:

```

1 DL = 1015000 = 211101010011000 (necesitamos 16 bits)
2 DL = 20011101010011000
3
4 Página: 20011 = 103 ✓
5 Offset: 2101010011000 = 102712 ✓
6
7 DF = 1023192 = 2101101010011000

```

Respuestas finales: 1. Tamaño de página: 4096 bytes (4 KiB) 2. Bits para número de página: 4 bits (permite 16 páginas) 3. Dirección física de 15000: 23192 bytes

10.12. Síntesis

10.12.1. Puntos Clave del Capítulo

La evolución de las técnicas de gestión de memoria muestra una progresión clara. Las particiones fijas eran simples pero con fragmentación interna severa. Las particiones dinámicas eliminaron la fragmentación interna pero crearon fragmentación externa. La paginación eliminó la fragmentación externa pero agregó overhead de traducción. La segmentación ofreció mejor modelo lógico pero volvió a introducir fragmentación externa. Finalmente, los sistemas híbridos de paginación más segmentación combinan ventajas de ambos enfoques.

Técnica	Fragm. Interna	Fragm. Externa	Overhead	Complejidad
Particiones Fijas	Alta	No	Mínimo	Baja
Particiones Dinámicas	No	Alta	Medio	Media
Paginación Simple	Baja	No	Medio	Media
Segmentación	No	Alta	Medio	Alta
Seg. + Paginación	Baja	No	Alto	Alta
Buddy System	Media	Media	Medio	Media

Para dominar este capítulo, debés entender la diferencia entre dirección lógica, relativa y física, por qué el binding en tiempo de ejecución es esencial para sistemas modernos, cómo calcular bits de página y offset dado el tamaño de página, el proceso completo de traducción (página → tabla → marco → dirección física), cuándo ocurre fragmentación interna versus externa, las ventajas y desventajas de cada técnica, y el rol fundamental de MMU, TLB y registros base/límite.

Los algoritmos de asignación tienen características distintivas.

- 1 **First Fit:** Rápido, genera pequeños bloques al inicio
- 2 **Best Fit:** Minimiza desperdicio, genera bloques muy pequeños
- 3 **Worst Fit:** Deja bloques grandes utilizables (mejor en simulaciones)
- 4 **Next Fit:** Distribuye asignaciones uniformemente

10.12.2. Conexiones con Otros Temas

Este capítulo se conecta profundamente con otros aspectos del sistema operativo. En relación con procesos, el PCB contiene el puntero a la tabla de páginas del proceso, en cada context switch se actualiza el PTBR con la tabla del nuevo proceso, y la memoria de un proceso incluye código, datos,

heap y stack.

La conexión con *planificación* es directa: un proceso puede bloquearse por Page Fault esperando carga desde disco, el scheduler debe considerar procesos bloqueados por I/O de paginación, y los algoritmos NUMA-aware consideran la localidad de memoria.

Este capítulo prepara el terreno para *memoria virtual*. Todo lo visto aquí es base para memoria virtual, que combina paginación con disco como extensión de RAM. El bit V (válido) indica si la página está en RAM o en disco, y el Page Fault es manejado por el SO para traer páginas desde disco.

También se relaciona con el *sistema de archivos*. La función `mmap()` permite mapear archivos a memoria, el I/O mapeado a memoria usa las mismas técnicas de traducción, y la caché de bloques del filesystem usa páginas de memoria.

10.12.3. Errores Comunes en Parciales

Los errores frecuentes incluyen confundir bits de página con número de páginas (son conceptos diferentes), olvidar que el offset se mantiene igual en la traducción, sumar mal la fórmula ($DF = \text{marco} * \text{tamaño} + \text{offset}$, no página), no verificar que la dirección calculada sea válida, confundir fragmentación interna con externa, decir que la paginación tiene fragmentación externa (no la tiene), y no considerar que la tabla de páginas está en RAM, no en el CPU.

Un checklist para ejercicios de traducción debe incluir: identificar el tamaño de página (dato o deducir), calcular bits de offset ($\log_2$ del tamaño de página), calcular bits de página (bits totales menos bits de offset), extraer página de dirección lógica, extraer offset, buscar marco en tabla de páginas, calcular $DF = \text{marco} * \text{tamaño_pagina} + \text{offset}$, y verificar que DF esté en rango válido.

10.12.4. Preguntas de Reflexión

Algunas preguntas para profundizar tu comprensión: ¿Por qué los sistemas modernos NO usan particiones dinámicas a pesar de no tener fragmentación interna? Si la paginación elimina fragmentación externa, ¿por qué no se usan siempre páginas de 256 bytes para minimizar fragmentación interna? ¿Cómo afecta el tamaño de página al rendimiento del TLB? ¿Por qué la tabla de páginas invertida no se popularizó a pesar de ahorrar memoria? En un sistema con 100 procesos, ¿cuál es más eficiente: 100 tablas de páginas o una tabla invertida con hash?

10.12.5. Ejercicios Propuestos

Ejercicio 1: Un sistema tiene páginas de 8 KiB y espacio lógico de 256 KiB. Si la dirección lógica 50000 se traduce a la dirección física 90000, ¿en qué marco está mapeada la página correspondiente?

Ejercicio 2: Calculá la fragmentación interna promedio en un sistema con páginas de 4 KiB si los procesos tienen tamaños aleatorios uniformemente distribuidos entre 1 KiB y 100 KiB.

Ejercicio 3: Compará el overhead de memoria para tablas de páginas en paginación simple de 1 nivel, paginación de 2 niveles, y tabla invertida. Asumí espacio lógico de 4 GiB, páginas de 4 KiB, y entrada de tabla de 4 bytes.

Ejercicio 4: Diseñá la estructura de una tabla de páginas que soporte protección R/W/X, páginas compartidas entre procesos, copy-on-write, y páginas en disco (memoria virtual).

10.12.6. Material para Profundizar

Las lecturas recomendadas incluyen Silberschatz (Capítulo 8: “Memory Management”), Stallings (Capítulo 7: “Memory Management”), y Tanenbaum (Capítulo 3: “Memory Management”). Entre los papers clásicos están Denning, P. J. (1970) sobre “Virtual Memory” en ACM Computing Surveys, y Corbató, F. J. et al. (1962) sobre “An Experimental Time-Sharing System”, el primer sistema con memoria virtual.

Para experimentar con estos conceptos, podés usar herramientas como pmap para ver el mapeo de memoria de un proceso, valgrind para detectar errores de memoria, `/proc/[pid]/maps` para ver regiones de memoria de un proceso, y gdb con comandos `info proc mappings`. La documentación del kernel de Linux en `Documentation/vm/` es invaluable para entender implementaciones reales.

Este capítulo ha cubierto los fundamentos de la gestión de memoria real. El próximo paso es entender cómo estos mecanismos se extienden para soportar memoria virtual, permitiendo ejecutar programas más grandes que la RAM física disponible.

11 Gestión de Memoria Virtual

11.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- Comprender la diferencia entre espacio de direcciones virtual y físico
- Explicar el concepto de page fault como evento normal del sistema
- Describir el flujo completo de manejo de un page fault
- Entender el principio de localidad y su importancia crítica
- Diferenciar entre demand paging y prepaging
- Explicar Copy-on-Write y memory-mapped files
- Analizar y comparar algoritmos de reemplazo de páginas
- Identificar thrashing y sus causas
- Aplicar algoritmos FIFO, LRU, Clock y Clock-M a secuencias de referencias
- Implementar un simulador de algoritmo de reemplazo
- Evaluar el rendimiento de diferentes políticas de reemplazo

11.2. Introducción y Contexto

11.2.1. El Problema Fundamental

Imaginemos que estamos desarrollando un sistema operativo en 1960. La situación es desafiante: tenemos apenas 64 KiB de memoria RAM (extremadamente cara en esa época), pero nuestros programas necesitan 128 KiB, 256 KiB o incluso más. Para empeorar las cosas, queremos ejecutar múltiples procesos simultáneamente.

La pregunta fundamental es inevitable: *¿cómo ejecutamos un programa más grande que la RAM disponible?*

Las primeras soluciones fueron manuales y requirieron un esfuerzo considerable por parte de los programadores. La técnica dominante en la década de 1960 se llamaba **overlays**, y era tan tediosa como efectiva.

Overlays (década de 1960):

El programador debía dividir manualmente el código en secciones. Solo una sección se car-

gaba en memoria a la vez, y el programador tenía que escribir código explícito para cargar y descargar secciones según fuera necesario. Este enfoque era extremadamente tedioso y propenso a errores. Por ejemplo, al terminar la fase de input, el programador debía escribir código para cargar la fase de procesamiento.

Un ejemplo conceptual de overlays muestra cómo el programador manejaba esto:

```
1 // Ejemplo conceptual de overlays (código del programador)
2 void fase1_input() {
3     // ... leer datos ...
4     // Termina fase 1, descargar de memoria
5 }
6
7 void cargar_fase2() {
8     // Cargar manualmente fase2 desde disco
9     read_from_disk("fase2.code", memoria);
10 }
11
12 void fase2_procesamiento() {
13     // ... procesar datos ...
14 }
```

Esta carga manual de código era frustrante y limitaba severamente la productividad del desarrollo de software. Necesitábamos una solución mejor.

La solución moderna: Memoria Virtual

La memoria virtual representa uno de los avances más significativos en el diseño de sistemas operativos. Es una técnica que permite ejecutar procesos cuyo espacio de direcciones total excede la memoria física disponible, creando la ilusión de que cada proceso tiene acceso a un espacio de direcciones enorme y contiguo. Todo esto se gestiona automáticamente por el hardware y el sistema operativo, sin intervención del programador.

La memoria virtual permite ejecutar procesos cuyo espacio de direcciones total excede la memoria física disponible, mediante la ilusión de que cada proceso tiene acceso a un espacio de direcciones enorme y contiguo, gestionado automáticamente por el hardware y el sistema operativo.

Las ventajas de este enfoque fueron revolucionarias. Primero, es completamente transparente al programador: no más overlays manuales. Cada proceso opera bajo la creencia de que tiene toda la memoria para sí, lo que simplifica enormemente la programación. Podemos ejecutar programas más grandes que la RAM disponible, y obtenemos protección automática entre procesos. Además, el código puede compartirse eficientemente entre múltiples procesos.

11.2.2. ¿Cómo es posible?

La viabilidad de la memoria virtual se basa en dos observaciones fundamentales. Primero, no todo el programa necesita estar en RAM simultáneamente (esto se conoce como el *principio de localidad*). Segundo, podemos usar el disco como extensión de la RAM, con traducción automática de direcciones.



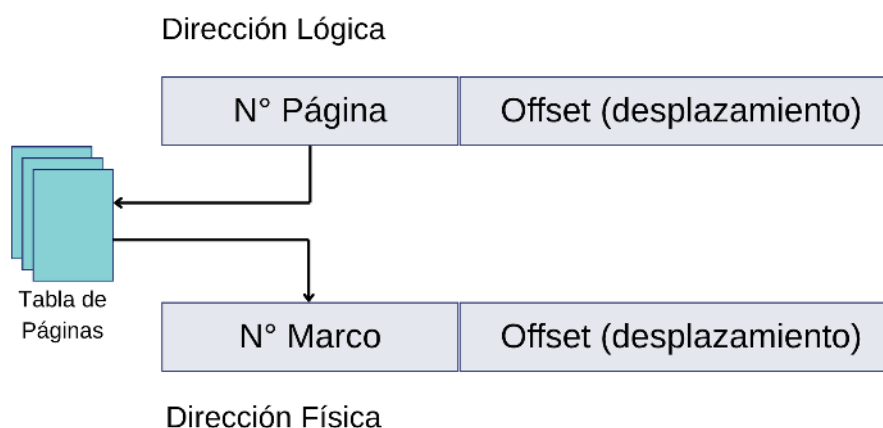
Un proceso de 1 GiB puede correr con aprox. 37 MiB en RAM, y el resto en almacenamiento secundario (swap).

11.3. Recap: Conceptos del Capítulo 7

Antes de sumergirnos en los detalles de la memoria virtual, necesitamos repasar brevemente los conceptos de paginación que estudiamos en el Capítulo 7. Estos conceptos son la base sobre la cual construiremos nuestra comprensión de la memoria virtual.

11.3.1. Paginación Básica (Recap)

En el Capítulo 7 aprendimos que el espacio lógico se divide en *páginas* de tamaño fijo (típicamente 4 KiB), mientras que la memoria física se divide en *marcos* del mismo tamaño. Una tabla de páginas mapea páginas lógicas a marcos físicos, y la MMU (Memory Management Unit) traduce direcciones automáticamente en hardware.



11.3.2. TLB (Recap)

El Translation Lookaside Buffer es una caché hardware que acelera dramáticamente la traducción de direcciones. Almacena traducciones recientes (típicamente entre 64 y 512 entradas) y puede accederlas en menos de 1 nanosegundo, comparado con aproximadamente 100 nanosegundos si hay que buscar en la tabla de páginas. El hit rate típico es de 98-99 %, lo que hace que este mecanismo sea extremadamente efectivo.

El TLB es crítico en memoria virtual, donde cada acceso requiere traducción. Sin él, el overhead de traducción de direcciones sería inaceptable.

11.3.3. Lo Nuevo en Memoria Virtual

En el Capítulo 7, todas las páginas del proceso estaban en RAM. Esta es la diferencia fundamental con memoria virtual: ahora, no todas las páginas están en RAM simultáneamente. Algunas páginas residen en disco (en el swap space), y el *bit presente* en la tabla de páginas indica dónde está cada página. Cuando se intenta acceder a una página no presente, se produce un **page fault**.

Diferencia clave:

Capítulo 7: La tabla de páginas mapea TODAS las páginas a marcos físicos.

Capítulo 8: La tabla de páginas puede indicar "esta página NO está en RAM".

Para más detalles sobre paginación básica, tablas multinivel, TLB y segmentación, podés consultar el Capítulo 7.

11.4. Conceptos Fundamentales

11.4.1. Espacio de Direcciones Virtual vs Físico

El espacio de direcciones virtual es el rango completo de direcciones que un proceso puede generar, completamente independiente de la cantidad de memoria física disponible. En un sistema de 32 bits, este espacio va de 0 a 4 GiB. En sistemas de 64 bits, el rango teórico es de 0 a 16 exabytes, aunque en la práctica se usa un subconjunto más pequeño.

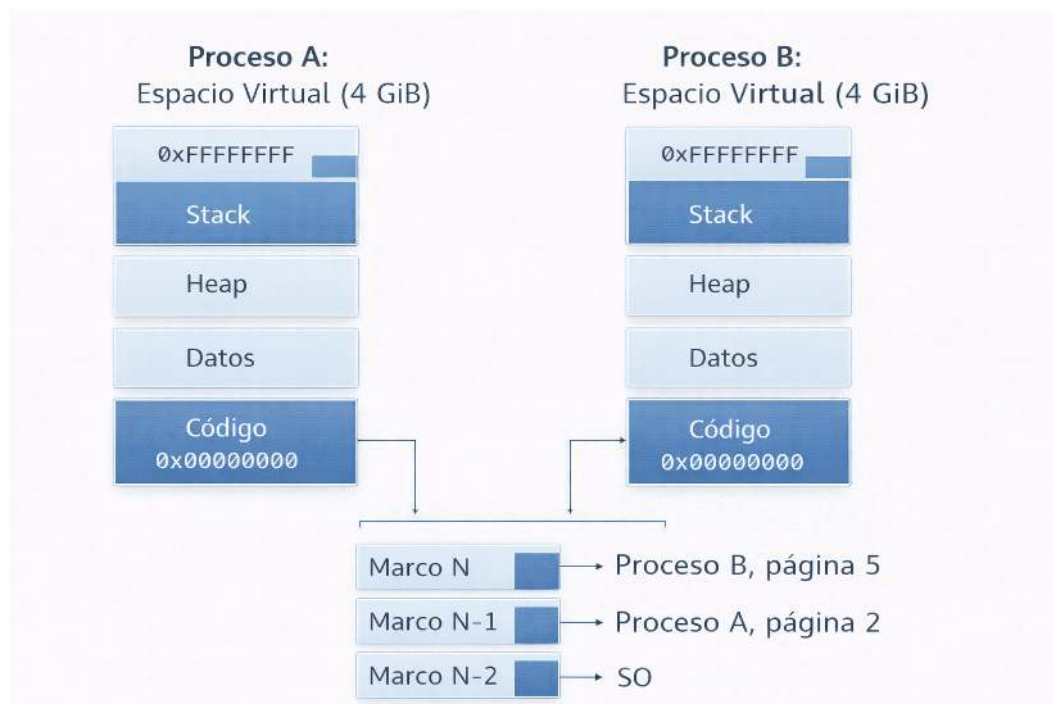
El espacio de direcciones virtual es el rango completo de direcciones que un proceso puede generar, independiente de la cantidad de memoria física disponible.

Veamos un ejemplo práctico de cómo funciona esto:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     // En un sistema de 64 bits, este proceso "ve" un espacio
6     // de direcciones de ~16 exabytes, aunque la máquina
7     // solo tenga 8 GiB de RAM física
8
9     void *ptr = malloc(1024 * 1024 * 1024); // 1 GiB
10    printf("Dirección virtual: %p\n", ptr);
11
12    // Esta dirección es VIRTUAL
13    // Puede ser 0x7f8a3c000000 (ejemplo)
14    // Pero físicamente puede estar en marco 2847 de RAM
15    // O ni siquiera estar en RAM (en disco)
16
17    free(ptr);
18    return 0;
19 }
```

Este programa ilustra un punto crucial: la dirección que vemos (como `0x7f8a3c000000`) es completamente virtual. Físicamente, esos datos pueden estar en el marco 2847 de RAM, o incluso no estar en RAM en absoluto, sino en disco.

La separación entre espacios virtual y físico tiene consecuencias profundas. Ambos procesos pueden usar la misma dirección virtual (digamos, `0x1000`), pero estas se mapean a marcos físicos completamente diferentes. Esto proporciona protección automática: un proceso no puede acceder al marco de otro. Además, simplifica enormemente la programación, ya que cada proceso comienza en `0x0` y el programador no tiene que preocuparse de dónde están otros procesos en memoria.



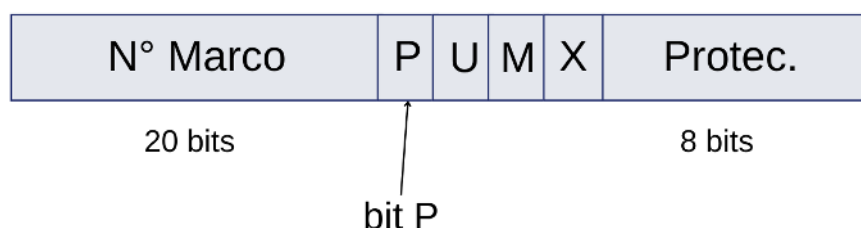
Ambos procesos pueden usar la misma dirección virtual 0x1000, se mapean a marcos físicos diferentes.

11.4.2. El Bit Presente (Present bit): El Héroe de Memoria Virtual

En el Capítulo 7, cada entrada de tabla de páginas era simplemente un número de marco. Ahora, agregamos información crucial que hace posible la memoria virtual. La estructura expandida incluye varios bits de control, siendo el más importante el *bit presente*.

En la mayoría de arquitecturas (x86, ARM), el bit se llama bit válido (Valid), para mantener relación con la cátedra, en el presente capítulo lo llamaremos bit P (presente).

Entrada de Tabla de páginas (expandida)



El bit presente (P) indica si la página está presente en memoria RAM (P=1) o está en disco/swap (P=0). Es la base del mecanismo de memoria virtual.

Los bits de control tienen diferentes significados según el estado de la página. Cuando $P=1$, la página está en RAM, y los otros bits indican si fue accedida recientemente ($U=1$), si fue modificada ($M=1$, también llamada *dirty*), y permisos de ejecución (X). Cuando $P=0$, la página no está en RAM y reside en disco, haciendo que los otros bits sean irrelevantes en ese momento.

Consideremos un ejemplo concreto de una tabla de páginas con memoria virtual:

Proceso con 8 páginas:

Pág	Marco	P	U	M	Ubicación
0	5	1	1	0	RAM (marco 5)
1	-	0	-	-	Disco blq 10
2	8	1	0	1	RAM (marco 8)
3	-	0	-	-	Disco blq 15
4	2	1	1	1	RAM (marco 2)
5	-	0	-	-	Disco blq 20
6	-	0	-	-	Nunca usada
7	1	1	0	0	RAM (marco 1)

Solo 4 de 8 páginas en RAM Proceso usa $8 \times 4 \text{ KiB} = 32 \text{ KiB}$ virtual Pero solo $4 \times 4 \text{ KiB} = 16 \text{ KiB}$ físicos

La ventaja es crítica: un proceso de 1 GiB puede ejecutarse con solo 50 MiB en RAM. El resto permanece en disco hasta que se necesite, permitiendo una utilización mucho más eficiente de la memoria física.

11.4.3. Page Fault: Evento Normal, NO Error

Este es un punto que genera mucha confusión en estudiantes, y es importante aclararlo desde el principio. Un page fault es una interrupción generada por la MMU cuando el CPU intenta acceder a una página cuyo bit presente está en 0 (página no presente en RAM). Contrario a lo que el nombre podría sugerir, es un mecanismo normal y esperado del sistema de memoria virtual, no un error de programación.

Un page fault es una interrupción generada por la MMU cuando el CPU intenta acceder a una página cuyo bit presente está en 0. Es un mecanismo normal y esperado del sistema de memoria virtual, NO un error de programación.

¡IMPORTANTE!

Page Fault $\neq$ Segmentation Fault

- Page Fault: Evento normal que el SO maneja transparentemente
- Segmentation Fault: Error de acceso a memoria inválida

La distinción es crucial. Un page fault ocurre cuando se accede a una página con $P=0$, y el SO lo maneja transparentemente cargando la página desde disco. El proceso ni siquiera se entera de que ocurrió. Por otro lado, un segmentation fault es un acceso fuera del espacio válido del proceso, lo que causa que el SO termine el proceso enviando la señal SIGSEGV.

Nota avanzada: En algunos sistemas, es posible capturar SIGSEGV y recuperarse (ej: lenguajes con manejo de memoria seguro). Sin embargo, para propósitos educativos, page fault = página no está en RAM (recuperable por SO), segmentation fault = acceso fuera del espacio de direcciones del proceso (típicamente fatal).

11.4.4. Swap Space (Backing Store)

El swap space es el área del disco duro reservada para almacenar páginas que no están en RAM. Actúa como una extensión de la memoria física, permitiendo que el sistema ejecute más procesos o procesos más grandes de lo que la RAM física permitiría por sí sola.

El swap space es el área del disco duro reservada para almacenar páginas que no están en RAM. Actúa como extensión de la memoria física.

En Linux, podemos ver la configuración del swap con el comando `swapon --show`. Una configuración típica podría mostrar una partición de 8 GiB y un archivo de swap de 4 GiB. El swap puede implementarse tanto como una partición dedicada como un archivo regular en el sistema de archivos.

La estructura del disco duro típicamente incluye una o más particiones para el sistema de archivos, y una partición o archivo dedicado al swap. Esta área almacena las páginas que no están actualmente en RAM, organizadas en bloques que corresponden a las páginas de los diferentes procesos.

El swap es aproximadamente 1000 veces más lento que la RAM cuando se usa un disco duro tradicional (HDD). Con SSDs, la diferencia se reduce a unas 100 veces, pero sigue siendo significativa. Por eso es crítico minimizar los page faults. El sistema operativo intenta mantener las páginas "calientes" (frecuentemente accedidas) en RAM.

La relación entre el tamaño de la RAM y el swap ha evolucionado con el tiempo. En sistemas con 8 GiB de RAM, es común configurar entre 8 y 16 GiB de swap. Con 16 GiB de RAM, típicamente se usa entre 8 y 16 GiB de swap. En sistemas con 32 GiB o más de RAM, el swap puede ser de solo 4 a 8 GiB, o incluso menos. La razón es simple: con más RAM disponible, hay menos page faults y, por lo tanto, se necesita menos espacio de swap.

11.5. Page Fault: Anatomía Completa

Ahora que entendemos qué es un page fault y por qué es necesario, veamos en detalle el flujo completo de lo que sucede cuando ocurre uno. Este proceso involucra una coordinación compleja entre el hardware y el sistema operativo.

11.5.1. Flujo Paso a Paso

El proceso comienza cuando el CPU ejecuta una instrucción como `MOV R1, [0x2000]`. La MMU intenta traducir la dirección virtual `0x2000` a una dirección física, determinando que corresponde a la página 2. Al consultar la tabla de páginas, descubre que el bit presente está en 0, lo que indica que la página no está en RAM.

En este punto, la MMU genera una interrupción especial llamada *trap*, específicamente un trap de page fault. El hardware automáticamente guarda el estado completo del proceso, incluyendo todos los registros y el contador de programa (PC). El control pasa entonces al sistema operativo, específicamente al manejador de page faults.

El sistema operativo tiene varias responsabilidades críticas. Primero, debe verificar si el acceso es válido. Si el acceso es a una dirección completamente fuera del espacio válido del proceso, el SO envía una señal SIGSEGV (segmentation fault) y termina el proceso. Si el acceso es válido, el proceso continúa.

El manejador de page faults debe distinguir entre tres situaciones:

1. Acceso válido a página que está en disco (page fault normal)
2. Acceso a página que nunca ha sido asignada (primer acceso)
3. Acceso inválido a memoria fuera del espacio del proceso (segmentation fault)

Solo en el tercer caso se termina el proceso. Los primeros dos son situaciones normales que el SO maneja transparentemente.

El siguiente paso es buscar un marco libre en RAM. Si hay un marco disponible, se usa directamente. Si no hay marcos libres, el SO debe ejecutar un algoritmo de reemplazo para seleccionar una página víctima que será desalojada. Si esta página víctima tiene el bit modificado ($M=1$), lo que

significa que fue modificada desde que se cargó, debe escribirse a disco antes de ser reemplazada. Esta escritura toma entre 5 y 10 milisegundos en un disco duro tradicional.

Una vez que tenemos un marco disponible (ya sea porque estaba libre o porque desalojamos una página), el SO lee la página necesaria desde disco. Esta operación también toma entre 5 y 10 milisegundos en HDD. Después de cargar la página en el marco seleccionado, el SO actualiza la tabla de páginas, estableciendo el número de marco, poniendo $P=1$, y reiniciando los bits U y M a 0.

Un detalle importante es la invalidación del TLB. Si existía una entrada anterior para esta página en el TLB (por ejemplo, de cuando la página estaba en RAM anteriormente), debe invalidarse para forzar una nueva traducción que use la tabla de páginas actualizada.

Finalmente, el SO retorna de la interrupción. El hardware restaura automáticamente los registros del proceso, y el CPU reintenta la instrucción `MOV R1, [0x2000]`. Esta vez, como $P=1$, la traducción es exitosa y el acceso a RAM se completa normalmente.

11.5.2. Resumen de Casos de Page Fault

Tipo	Causa	Bit P	Acción del SO	¿Se escribe a disco?
Falta de página normal	Página en swap/disco	0	Cargar desde disco	No (solo lectura)
Primer acceso	Página nunca asignada	0	Asignar nueva página (ceros)	No
Página dirty (víctima)	Se necesita reemplazar página con $M=1$	1 (pero será reemplazada)	Escribir a disco antes de descartar	Sí
Copy-on-Write	Escritura en página compartida (R/O)	1	Copiar página a nuevo marco	No (se copia en RAM)

11.5.3. Tiempos Aproximados

Los tiempos involucrados en un page fault son significativos y vale la pena analizarlos en detalle. Estos valores están basados en mediciones de sistemas reales y literatura técnica estándar.

Tiempos típicos de page fault:

La detección por hardware (MMU) toma aproximadamente 1 nanosegundo. El context switch al manejador del SO requiere entre 1 y 5 microsegundos. La búsqueda de marco libre o ejecución del algoritmo de reemplazo toma entre 1 y 10 microsegundos.

Las operaciones de disco son las más costosas. La escritura de una página dirty a disco HDD toma entre 5 y 10 milisegundos, mientras que la lectura desde disco HDD también requiere 5 a 10 milisegundos. Con SSDs SATA, estos tiempos se reducen dramáticamente: entre 0.1 y 0.5 milisegundos para escritura, y entre 0.1 y 0.5 milisegundos para lectura. Con NVMe, los tiempos son aún más rápidos: entre 0.03 y 0.1 milisegundos para escritura, y entre 0.03 y 0.1 milisegundos para lectura.

La actualización de estructuras del SO requiere entre 1 y 5 microsegundos, y el context switch de vuelta al proceso toma otros 1 a 5 microsegundos.

Total con HDD: 10-20 milisegundos

Total con SSD SATA: 0.2-1 milisegundo

Total con NVMe: 0.06-0.2 milisegundos

Para poner estos números en perspectiva, comparémoslos con un acceso normal a RAM. Un acceso con TLB hit toma aproximadamente 10 nanosegundos (0.00001 ms). Un acceso con TLB miss requiere unos 100 nanosegundos (0.0001 ms). Un page fault con NVMe toma alrededor de 100 microsegundos (0.1 ms), lo que es aproximadamente 10,000 veces más lento. Un page fault con SSD SATA toma alrededor de 300 microsegundos (0.3 ms), lo que es aproximadamente 30,000 veces más lento. Un page fault con HDD toma unos 10 milisegundos, lo que es aproximadamente 1,000,000 veces más lento que un acceso normal a RAM.

Consecuencia crítica:

Un solo page fault equivale a aproximadamente 100,000 accesos normales a RAM. Por eso el principio de localidad es fundamental para el rendimiento del sistema. Minimizar page faults es esencial para mantener un rendimiento aceptable.

En un CPU de 3 GHz, un ciclo de reloj dura aproximadamente 0.33 ns.

Un acceso a RAM de 10 ns equivale a unos 30 ciclos de CPU, mientras que un page fault con SSD puede costar más de 1.5 millones de ciclos, y con HDD más de **30 millones de ciclos**.

11.5.4. Casos Especiales

Existen varios casos especiales de page faults que vale la pena examinar, cada uno con características únicas.

11.5.4.1. Caso 1: Primera Referencia (Demand Paging)

Cuando se crea un proceso nuevo, todas sus páginas tienen inicialmente el bit presente en 0. La primera referencia a la página 0 (típicamente código) genera un page fault, y el SO carga esa página desde el archivo ejecutable. La primera referencia a la página 1 genera otro page fault, y así sucesivamente.

Este enfoque se llama *demand paging* (paginación bajo demanda) porque las páginas se cargan solo cuando se necesitan. Es fundamentalmente diferente de cargar todo el programa en memoria antes de ejecutarlo, lo que sería más lento y desperdiciaría memoria en páginas que nunca se usan.

11.5.4.2. Caso 2: Copy-on-Write

Copy-on-Write es una optimización elegante que exploraremos en detalle más adelante. Cuando un proceso hace `fork()`, el proceso hijo no copia inmediatamente todas las páginas del padre. En su lugar, ambos procesos comparten las mismas páginas físicas, pero marcadas como solo lectura.

Si el hijo intenta escribir en una página compartida, se genera un page fault (por intento de escritura en una página de solo lectura). El SO detecta que es un caso de Copy-on-Write, copia la página a un nuevo marco, y actualiza la tabla del hijo para apuntar a esta copia privada. Ahora el hijo puede escribir en su propia copia sin afectar al padre.

11.5.4.3. Caso 3: Memory-Mapped Files

Los archivos mapeados en memoria son otro caso especial fascinante. Cuando llamamos a `mmap()` para mapear un archivo de 1 MiB, la llamada no carga el archivo completo inmediatamente. Solo mapea direcciones virtuales al archivo.

La primera lectura de la dirección mapeada genera un page fault. El SO carga la página correspondiente desde el archivo (no desde el swap). Si escribimos en esta dirección y la página no está en RAM, también habrá un page fault. Una vez en RAM, la página se marca como dirty ($M=1$), y el SO eventualmente escribirá los cambios de vuelta al archivo.

11.6. Principio de Localidad

El principio de localidad es la razón fundamental por la cual la memoria virtual funciona eficientemente. Sin este principio, cada proceso necesitaría todas sus páginas en RAM todo el tiempo, haciendo que la memoria virtual fuera impráctica.

11.6.1. ¿Qué es el Principio de Localidad?

El principio de localidad es una observación empírica sobre el comportamiento de los programas. Los programas tienden a acceder un subconjunto relativamente pequeño de su espacio de direcciones en cualquier intervalo de tiempo dado. Este principio se divide en dos tipos fundamentales: localidad temporal y localidad espacial.

El principio de localidad es la observación de que los programas tienden a acceder un subconjunto relativamente pequeño de su espacio de direcciones en cualquier intervalo de tiempo dado. Se divide en localidad temporal y localidad espacial.

La importancia de este principio para la memoria virtual no puede subestimarse. Si un proceso de 1 GiB necesitara acceder aleatoriamente todas sus páginas constantemente, tendríamos page faults continuos, el disco trabajando sin parar, y un rendimiento catastrófico. Pero gracias a la localidad, el proceso típicamente solo accede activamente a unos 50 MiB en cualquier momento, permitiendo que funcione eficientemente con poca RAM física.

11.6.2. Localidad Temporal

La localidad temporal se refiere al fenómeno de que si se referencia una ubicación de memoria en el tiempo t , es muy probable que se reference nuevamente en un futuro cercano ($t + \Delta t$).

El ejemplo clásico de localidad temporal son las variables en un bucle:

```
1  #include <stdio.h>
2
3  int main() {
4      int suma = 0;          // Variable local en stack
5      int contador = 0;      // Variable local en stack
6
7      // Estas dos variables se acceden repetidamente
8      // en cada iteración del bucle
9      for (contador = 0; contador < 1000000; contador++) {
10         suma += contador;
11         // 'suma' y 'contador' se referencian una y otra vez
12         // → Localidad temporal
13         // → Muy probable que estén en caché y en RAM
14     }
15
16     printf("Suma: %d\n", suma);
17     return 0;
18 }
```

¿Qué está pasando con las páginas en este ejemplo? El proceso tiene la página 0 conteniendo el código (las instrucciones del bucle), y la página 1 conteniendo el stack (las variables `suma` y

`contador`). Durante la ejecución del bucle, se accede a la página 0 un millón de veces, y a la página 1 otro millón de veces. En total, solo 2 páginas están activas de quizás 100 páginas totales del proceso.

Otros ejemplos comunes de localidad temporal incluyen funciones que se llaman frecuentemente (permanecen “calientes” en memoria), variables globales usadas en múltiples funciones, y especialmente código en bucles internos (*hot loops*) que se ejecutan millones de veces.

11.6.3. Localidad Espacial

La localidad espacial describe el fenómeno de que si se referencia una ubicación de memoria en la dirección X , es muy probable que se referencien ubicaciones cercanas ($X \pm \delta$) en un futuro próximo.

El ejemplo clásico es el recorrido secuencial de un array:

```
1  #include <stdio.h>
2  #define SIZE 1000000
3
4  int main() {
5      int array[SIZE]; // 4 MiB en heap
6      int suma = 0;
7
8      // Inicializar array
9      for (int i = 0; i < SIZE; i++) {
10         array[i] = i;
11         // Acceso secuencial: array[0], array[1], array[2], ...
12         // → Localidad espacial
13     }
14
15     // Sumar elementos
16     for (int i = 0; i < SIZE; i++) {
17         suma += array[i];
18         // Nuevamente acceso secuencial
19         // → Localidad espacial
20     }
21
22     printf("Suma: %d\n", suma);
23     return 0;
24 }
```

Para entender el impacto, consideremos cómo se organizan estos datos en páginas. Un array de 4 MiB con páginas de 4 KiB significa que los elementos `array[0..1023]` están en la página 0, `array[1024..2047]` en la página 1, y así sucesivamente.

Cuando accedemos `array[0]`, traemos la página 0 completa (que contiene `array[0..1023]`). Los próximos 1023 accesos no generan page faults porque todos esos elementos ya están en RAM, en la misma página.

El impacto en page faults es dramático. Para un array de 1,000,000 elementos (4 MiB total), necesitamos 1024 páginas. Con acceso secuencial (que tiene localidad espacial), tenemos exactamente 1024 page faults: uno por página, la primera vez que la accedemos. Los restantes 998,976 accesos no generan page faults.

Con acceso aleatorio (sin localidad espacial), cada acceso potencialmente va a una página diferente. Podríamos tener cientos de miles de page faults, resultando en un rendimiento desastroso.

Impacto medible de la localidad espacial:

Array de 1,000,000 elementos (int): 4 MiB de tamaño y 1024 páginas necesarias.

Acceso secuencial: 1024 page faults (uno por página), 998,976 accesos sin PF con una duración estimada de 10 ms + tiempo de procesamiento.

Acceso aleatorio: potencialmente cientos de miles de page faults, rendimiento de 100-1000 veces peor

11.6.4. Localidad en Programas Reales

Para ver cómo el principio de localidad funciona en la práctica, consideremos un navegador web real. El código del navegador puede ocupar 50 MiB, y las páginas web cargadas otros 500 MiB, para un total de 550 MiB de espacio de direcciones virtual.

Sin embargo, en cualquier momento de 10 segundos, solo está activo aproximadamente 5 MiB de código (el motor de renderizado, la máquina virtual de JavaScript) y unos 20 MiB de datos (la página web actual y las imágenes visibles). El total activo es de solo 25 MiB.

El ratio es revelador: $25/550 = 4.5\%$ del espacio total está en uso activo. La localidad hace viable ejecutar este navegador con mucha menos RAM de la que su tamaño total sugeriría.

11.6.5. Working Set

Este concepto de “conjunto activo” se formaliza en la noción de *working set*.

El **working set** (conjunto de trabajo) es el conjunto de páginas que un proceso está usando activamente en un intervalo de tiempo dado. Representa el “tamaño mínimo de RAM” que el proceso necesita para ejecutar eficientemente.

Formalmente, el working set en el instante t es el conjunto de páginas referenciadas durante el intervalo $(t - \Delta, t]$, donde Δ es un parámetro (típicamente la longitud de la ventana de tiempo). Si Δ es muy pequeño, el working set no captura toda la localidad; si es muy grande, se acerca al tamaño total del proceso.

El working set cambia con el tiempo a medida que el proceso pasa por diferentes fases de ejecución. Por ejemplo, en los primeros 5 segundos, un proceso podría tener un working set de las páginas $\{0,1,2,5,7\}$. En los siguientes 5 segundos, el working set podría ser $\{0,1,2,3,8\}$. Y en los siguientes 5 segundos, $\{0,1,10,11,12\}$.

La clave está en el tamaño del working set. Si le damos al proceso menos páginas de RAM que su working set, tendremos page faults continuos, una condición conocida como *thrashing* que estudiaremos más adelante. Si le damos al proceso suficientes páginas para cubrir su working set (en este ejemplo, 5 páginas o más), funcionará eficientemente.

Importancia del working set:

El sistema operativo intenta estimar continuamente el working set de cada proceso. Asigna suficientes marcos de RAM para cubrirlo, permitiendo que el proceso ejecute sin thrashing. Si el sistema no puede satisfacer los working sets de todos los procesos activos, debe suspender algunos procesos temporalmente para prevenir el colapso del rendimiento. Algoritmos avanzados como Working Set y PFF (Page Fault Frequency) se basan explícitamente en esta idea.

11.6.6. ¿Por Qué Funciona en Práctica?

El principio de localidad no es un accidente: surge naturalmente de cómo escribimos y estructuramos programas. Existen varias razones fundamentales por las cuales los programas exhiben localidad.

Primero, la estructura misma de los programas promueve localidad. El código se organiza en funciones, y las funciones activas en cualquier momento dado son relativamente pocas (localidad temporal de funciones). Los datos se organizan en arrays y estructuras, promoviendo accesos cercanos en memoria (localidad espacial).

Los bucles son particularmente importantes para la localidad. Las instrucciones dentro del bucle se ejecutan repetidamente, permaneciendo “calientes” en las cachés y en RAM. Las variables utilizadas en el bucle se acceden miles o millones de veces sin moverse de su ubicación en memoria.

Las llamadas a funciones también contribuyen a la localidad. Mientras una función está activa, usa repetidamente su stack frame, que permanece en la misma región de memoria. Cuando la función retorna al caller, volvemos a código que ya estaba en memoria.

El acceso secuencial a datos es omnipresente en programación. Los archivos típicamente se leen o escriben secuencialmente, y los arrays se procesan elemento por elemento. La organización natural de datos también ayuda: las estructuras agrupan datos relacionados que se acceden juntos, y objetos relacionados tienden a alocarse cerca en memoria.

11.6.6.1. Contraejemplo: Programa SIN localidad

Consideremos un programa patológico que accede memoria completamente al azar:

```
1 // Programa patológico: acceso completamente aleatorio
2 #include <stdlib.h>
3 #include <time.h>
4
5 #define PAGES 10000
6
7 int main() {
8     int *huge_array = malloc(PAGES * 4096); // 40 MiB
9     srand(time(NULL));
10
11     // Acceso completamente aleatorio a páginas
12     for (int i = 0; i < 1000000; i++) {
13         int random_page = rand() % PAGES;
14         int random_offset = rand() % 1024;
15         int index = random_page * 1024 + random_offset;
16         huge_array[index] = i;
17         // → NO hay localidad
18         // → Page faults constantes
19         // → Rendimiento terrible
20     }
21
22     free(huge_array);
23     return 0;
24 }
```

Este programa tendría un rendimiento ~1000 veces peor que uno con acceso secuencial, porque viola completamente el principio de localidad. Cada acceso aleatorio tiene alta probabilidad de ir a una página diferente, generando page faults constantes.

11.7. Demand Paging vs Prepaging

Ahora que entendemos el principio de localidad, podemos explorar las estrategias que usa el sistema operativo para decidir cuándo cargar páginas en memoria. La pregunta fundamental es: cuando se carga un proceso, ¿cargamos todas sus páginas inmediatamente o esperamos a que se necesiten?

11.7.1. Demand Paging (Paginación Bajo Demanda)

La estrategia más común en sistemas modernos es *demand paging*, donde las páginas se cargan en memoria solo cuando se referencian por primera vez, generando un page fault.

En demand paging, las páginas se cargan en memoria SOLO cuando se referencian por primera vez, generando un page fault. Es la estrategia más común en sistemas modernos.

Cuando se crea un proceso nuevo, su tabla de páginas se inicializa con todas las entradas marcadas como no presentes ($P=0$). Ninguna página está en RAM inicialmente. La primera instrucción del programa intentará acceder a la página 0 (código), generando un page fault. El sistema operativo maneja este fault cargando la página 0 desde el archivo ejecutable y marcándola como presente ($P=1$).

A medida que el programa continúa ejecutando, cada nueva página que accede genera su propio page fault inicial. El acceso a datos genera page faults para las páginas del heap y el stack. Gradualmente, el working set del proceso se carga en memoria, pero solo las páginas realmente necesarias.

Las ventajas de demand paging son significativas. Solo se cargan páginas realmente necesarias, ahorrando tanto memoria como tiempo de I/O. El startup es rápido porque no esperamos a cargar todo el programa. Las páginas que nunca se usan (como código de manejo de errores raros) nunca se cargan, ahorrando recursos valiosos. Esto permite ejecutar programas más grandes que la RAM disponible.

Desventajas de demand paging:

El inicio del programa genera muchos page faults (cold start), causando latencia inicial. La latencia en la primera referencia a cada página es impredecible, lo que puede ser problemático para aplicaciones de tiempo real. Cada página nueva implica el overhead completo de un page fault, incluyendo la interrupción, el context switch, y la I/O de disco.

Un ejemplo práctico ilustra cómo funciona esto.

```
1 // Programa de 100 páginas
2 int main() {
3     // Al iniciar: 0 páginas en RAM
4
5     funcion_principal(); // Carga páginas 0-5 (código)
6     // Ahora: 6 páginas en RAM
7
8     if (condicion_rara) {
9         // Este código casi nunca se ejecuta
10        funcion_rara(); // Páginas 50-55 nunca se cargan
11    }
12
13    // Final: solo ~10-20 páginas cargadas de 100 totales
```

```
14     return 0;
15 }
```

Cuando comienza a ejecutar, llama a `funcion_principal()`, lo que carga las páginas 0-5 de código. Ahora tenemos 6 páginas en RAM. Si hay un `if` (`condicion_rara`) que casi nunca se evalúa como verdadero, las páginas 50-55 que contienen `funcion_rara()` nunca se cargan. Al final de la ejecución, solo unas 10-20 páginas fueron cargadas de las 100 totales.

11.7.2. Prepaging (Paginación Anticipada)

Una alternativa es *prepaging*, donde el sistema operativo intenta predecir qué páginas se necesitarán pronto y las carga anticipadamente, antes de que se generen page faults.

En prepaging, el SO intenta predecir qué páginas se necesitarán pronto y las carga anticipadamente, antes de que se generen page faults.

Existen varias estrategias de prepaging. Una es cargar páginas contiguas: cuando se accede a la página 5 y se genera un page fault, el SO carga no solo la página 5 sino también las páginas 6, 7 y 8, anticipando acceso secuencial debido a la localidad espacial.

Otra estrategia usa el histórico de acceso. Si el SO observa que un proceso siempre accede las páginas en el orden $0 \rightarrow 1 \rightarrow 5 \rightarrow 10$, al cargar el proceso puede traer ese conjunto completo desde el principio.

Una tercera estrategia aprovecha el working set previo. Cuando un proceso se suspende con las páginas {0,1,2,5,7} en RAM, al reanudarlo el SO puede cargar ese working set completo, asumiendo que será necesario nuevamente.

Las ventajas de prepaging son claras cuando la predicción es correcta. Se reducen los page faults durante la ejecución, mejorando el rendimiento para patrones predecibles. Es particularmente útil al reanudar procesos que fueron suspendidos, ya que podemos restaurar su working set previo.

Desventajas de prepaging:

Si la predicción falla, desperdiciamos I/O y memoria cargando páginas que nunca se usan. La implementación es más compleja que demand paging simple, requiriendo mantener estadísticas y hacer predicciones. El trade-off fundamental es: prepaging solo vale la pena si la predicción es correcta más del 80 % del tiempo.

En sistemas operativos reales, vemos mayormente demand paging con prepaging limitado en casos específicos. Linux usa principalmente demand paging, pero implementa *readahead* para archivos que se acceden secuencialmente, y carga páginas de código contiguas cuando detecta ejecución secuencial. Windows es similar, usando demand paging como base pero con un sistema de

prefetch que almacena patrones de inicio de aplicaciones y precarga páginas según el histórico de arranques previos.

11.8. Shared Pages y Técnicas Avanzadas

Hasta ahora hemos considerado principalmente las páginas como recursos privados de cada proceso. Sin embargo, algunos de los usos más poderosos de la memoria virtual involucran compartir páginas entre múltiples procesos de manera eficiente y segura.

11.8.1. Copy-on-Write (COW)

Copy-on-Write es una optimización crucial que hace viable la operación `fork()` en sistemas Unix/Linux. Sin esta técnica, crear un proceso hijo sería prohibitivamente costoso.

Copy-on-Write (COW) es una técnica donde múltiples procesos comparten las mismas páginas físicas de memoria mientras sean solo lectura. Al intentar escribir, se crea una copia privada de la página para ese proceso.

Para entender el problema que resuelve COW, consideremos qué pasaría sin esta optimización.

```
1  pid_t pid = fork();
2
3  // Sin COW: fork() debe copiar TODAS las páginas del padre
4  // Padre tiene 100 MiB en RAM
5  // → Copiar 100 MiB toma ~100 ms
6  // → Desperdicio si el hijo hace exec() inmediatamente
```

Cuando un proceso hace `fork()`, el sistema operativo tendría que copiar todas las páginas del padre. Si el padre tiene 100 MiB en RAM, copiar todo tomaría aproximadamente 100 milisegundos. Esto sería un desperdicio enorme, especialmente considerando que el caso más común es que el hijo haga `exec()` inmediatamente, reemplazando todo su espacio de direcciones.

La solución con COW es elegante. Antes del `fork()`, el proceso padre tiene sus páginas marcadas como lectura-escritura (RW). Después del `fork()`, tanto el padre como el hijo tienen tablas de páginas que apuntan a los mismos marcos físicos, pero ahora marcados como solo lectura (R/O). Ambos procesos comparten la memoria física sin copiarla.



El momento crítico ocurre cuando uno de los procesos intenta escribir. Supongamos que el hijo ejecuta `x = 20`, intentando modificar una variable.

```

1 // Proceso hijo
2 int main() {
3     int x = 10; // Variable en página 1 (marco 8)
4     x = 20;    // INTENTO DE ESCRITURA
5
6     // → Page fault (intento de escritura en página R/O)
7
8     return 0;
9 }

```

Esto genera un page fault porque la página está marcada como solo lectura. El sistema operativo detecta que no es un error sino un caso de COW. En respuesta, asigna un nuevo marco (digamos, el marco 12), copia el contenido del marco compartido a este nuevo marco, actualiza la tabla del hijo para apuntar al nuevo marco con permisos de escritura, y finalmente reintenta la instrucción, que ahora tiene éxito.

Proceso Padre:

Pág	Marco	P	R	W
0	5	1	1	0
1	8	1	1	0

Proceso Hijo:

Pág	Marco	P	R	W
0	5	1	1	0
1	12	1	1	1

Aún compartida
Ahora privada!

Estadísticas reales de COW:

En el caso común de 'fork()' seguido de 'exec()': - Sin COW: copiar 100 MiB → 100 ms - Con COW: copiar 0 MiB → 1 ms (solo estructuras del SO)

Cuando el hijo modifica el 10- Sin COW: copiar 100 MiB → 100 ms - Con COW: copiar 10 MiB bajo demanda → 10 ms total

El ahorro es dramático en ambos casos.

11.8.2. Memory-Mapped Files

Memory-mapped files representan otra técnica poderosa que aprovecha la memoria virtual para simplificar el I/O y la compartición de datos.

Memory-mapped files mapean un archivo del disco directamente al espacio de direcciones virtual de un proceso, permitiendo acceder al archivo como si fuera un array en memoria.

La syscall principal es `mmap()`. Cuando mapeamos un archivo de 1 MiB, la llamada no carga el archivo completo inmediatamente. Solo configura las estructuras del sistema operativo para que las direcciones virtuales correspondan a bloques del archivo. Es otra forma de demand paging, pero usando el archivo como backing store en lugar del swap.

```

1 #include <sys/mman.h>
2 #include <fcntl.h>
3 #include <stdio.h>
4 #include <unistd.h>
5
6 int main() {
7     // Abrir archivo
8     int fd = open("datos.bin", O_RDWR);
9
10    // Mapear archivo a memoria (1 MiB)
11    void *addr = mmap(
12        NULL,                      // SO elige dirección
13        1024*1024,                  // 1 MiB
14        PROT_READ | PROT_WRITE,    // Permisos
15        MAP_SHARED,                 // Cambios visibles en archivo

```

```

16     fd,                // File descriptor
17     0                  // Offset en archivo
18 );
19
20 // Ahora 'addr' apunta a las páginas mapeadas
21 // NO se cargó nada en RAM todavía (demand paging)
22
23 // Primera lectura
24 char c = ((char*)addr)[0];
25 // → Page fault
26 // → SO carga página desde archivo (no desde swap)
27 // → Ahora página está en RAM
28
29 // Escritura
30 ((char*)addr)[100] = 'X';
31 // → Marca página como dirty (M=1)
32 // → Cambio se escribirá al archivo eventualmente
33
34 // Deshacer mapeo
35 munmap(addr, 1024*1024);
36 close(fd);
37
38 return 0;
39 }

```

Internamente, después de `mmap()`, la tabla de páginas tiene todas las entradas con `P=0`, pero en lugar de apuntar al swap, cada entrada indica qué offset del archivo contiene los datos para esa página.

Memory-Mapped Files (Archivos mapeados en memoria)



Las ventajas de memory-mapped files son sustanciales. El I/O se simplifica dramáticamente: los archivos se tratan como arrays en memoria, eliminando las llamadas explícitas a `read()` y `write()`. La compartición entre procesos es fácil usando `MAP_SHARED`. El sistema operativo maneja el

caché automáticamente, y se evitan copias entre user space y kernel space, mejorando el rendimiento.

Casos de uso de memory-mapped files:

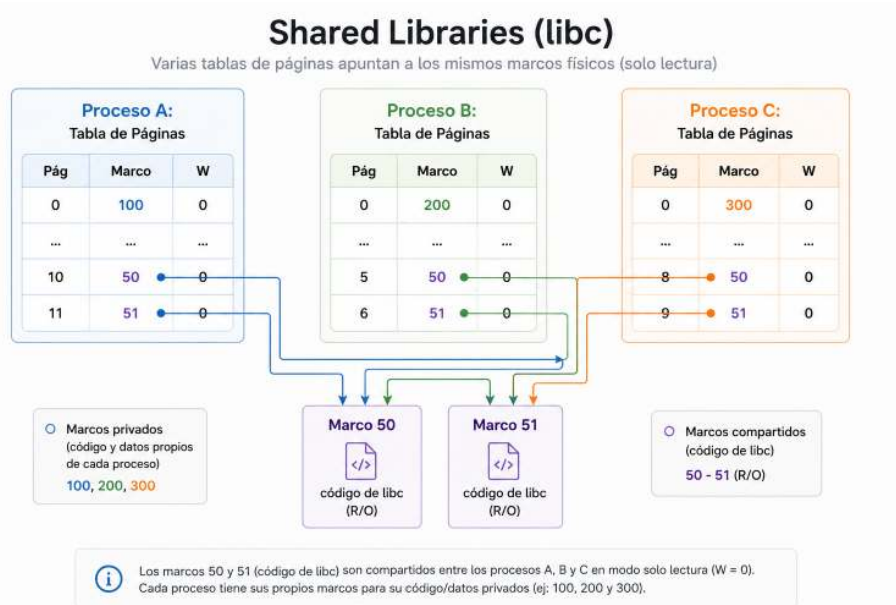
Bases de datos: Mapean el archivo de BD completo y acceden a registros directamente sin 'read/write', permitiendo índices eficientes y acceso aleatorio rápido.

Librerías compartidas (.so, .dll): Múltiples procesos mapean 'libc.so', compartiendo el código en RAM y ahorrando masivamente memoria.

IPC (Inter-Process Communication): El proceso A y B mapean el mismo archivo con MAP_SHARED, permitiendo que ambos vean los cambios del otro en tiempo real.

11.8.3. Shared Libraries

Las librerías compartidas son un caso especial particularmente importante de shared pages. Cuando tres procesos ejecutan programas que usan `libc`, sin compartición necesitaríamos tres copias completas de la librería en RAM. Con shared pages, los tres procesos apuntan a los mismos marcos físicos para el código de la librería.



Cada proceso tiene su propia tabla de páginas. Las páginas que contienen código privado del proceso apuntan a marcos únicos. Pero las páginas que contienen código de `libc` (marcadas como solo lectura) apuntan a los mismos marcos compartidos. Los marcos 50-51, por ejemplo, contienen código de `libc` y son compartidos por todos los procesos.

El ahorro de memoria es considerable. Sin compartición, tres procesos cada uno usando 2 MiB de `libc` consumirían 6 MiB de RAM. Con compartición, los tres procesos comparten una sola copia

de 2 MiB, ahorrando 4 MiB (66 %). En un servidor con 100 procesos, el ahorro puede alcanzar varios cientos de megabytes.

11.8.4. TLB Reach

Un último concepto relacionado con el rendimiento de la memoria virtual es el *TLB reach*.

El TLB reach es la cantidad de memoria que puede ser mapeada por todas las entradas del TLB. Se calcula como: $\text{TLB Reach} = (\text{número de entradas TLB}) \times (\text{tamaño de página})$.

Un TLB con 64 entradas y páginas de 4 KiB tiene un reach de 256 KiB. Esto significa que si el working set del proceso es menor a 256 KiB, todas las traducciones estarán en el TLB con un hit rate cercano al 100 %.

El problema surge cuando el working set excede el TLB reach. Un proceso con working set de 1 MiB pero TLB reach de solo 256 KiB experimentará muchos TLB misses, degradando el rendimiento. La solución es usar páginas más grandes. Con páginas de 2 MiB (llamadas *huge pages*), el mismo TLB de 64 entradas tiene un reach de 128 MiB, suficiente para que el working set completo quepa en el TLB.

Uso en sistemas reales:

Las bases de datos modernas usan huge pages de 2 MiB o 1 GiB para reducir la presión sobre el TLB. Las máquinas virtuales usan páginas grandes para las Extended Page Tables (EPT), reduciendo el overhead de virtualización anidada. La computación de alto rendimiento (HPC) frecuentemente usa páginas de 1 GiB para grandes datasets científicos.

11.9. Algoritmos de Reemplazo de Páginas

Cuando ocurre un page fault y no hay marcos libres en RAM, el sistema operativo enfrenta una decisión crítica: debe seleccionar una página “víctima” para reemplazar. Esta decisión puede tener un impacto dramático en el rendimiento del sistema. Los algoritmos de reemplazo son las políticas que guían esta decisión.

11.9.1. Objetivo del Algoritmo de Reemplazo

El propósito fundamental de cualquier algoritmo de reemplazo es minimizar la tasa de page faults. Cuantos menos page faults ocurran, menos veces tendremos que acceder al disco, y mejor será el rendimiento del sistema.

Un algoritmo de reemplazo es una política que selecciona cuál página residente en RAM será reemplazada cuando se necesita cargar una nueva página y no hay marcos libres. El objetivo es minimizar la tasa de page faults.

La métrica clave para evaluar estos algoritmos es la *page fault rate*, que se calcula como el número de page faults dividido por el número total de referencias a memoria.

```
1 Page Fault Rate = (Número de page faults) / (Número total de referencias)
2
3 Ejemplo:
4 Secuencia de 20 referencias, 5 page faults
5 PF Rate = 5/20 = 0.25 = 25%
```

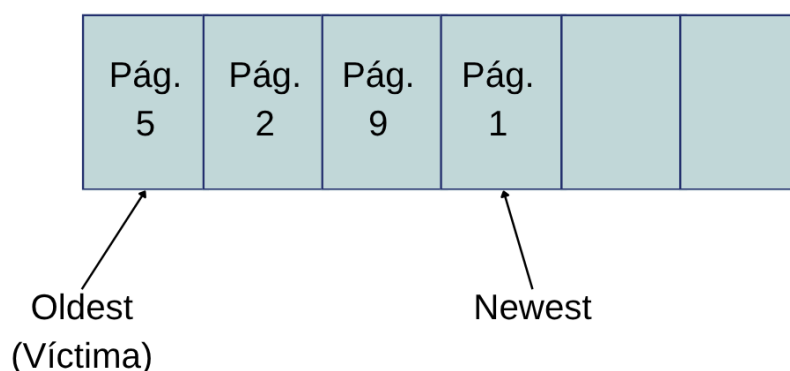
La métrica clave para evaluar estos algoritmos es la *page fault rate*, que se calcula como el número de page faults dividido por el número total de referencias a memoria. Por ejemplo, si en una secuencia de 20 referencias ocurren 5 page faults, la tasa es $5/20 = 0.25$ o 25 %. Cuanto menor sea este número, mejor está funcionando el algoritmo.

11.9.2. FIFO (First-In-First-Out)

El algoritmo más simple es FIFO: reemplazar la página que lleva más tiempo en memoria. La intuición es que las páginas “viejas” probablemente ya no se necesitan.

La implementación usa una cola circular donde la página más antigua está al frente. Cuando se necesita una víctima, simplemente se toma la página del frente de la cola. Cuando se carga una nueva página, se agrega al final de la cola.

Estructura



Las ventajas de FIFO son claras: es muy simple de implementar, encontrar la víctima tiene complejidad $O(1)$, y el overhead es mínimo. Sin embargo, tiene desventajas significativas. No considera

la frecuencia de uso de las páginas, por lo que puede reemplazar páginas que se usan constantemente. Peor aún, sufre de la *anomalía de Belady*, un fenómeno contraintuitivo donde agregar más marcos puede aumentar los page faults en lugar de reducirlos.

Desventajas de FIFO:

FIFO no considera cuán frecuentemente se usa una página. Puede reemplazar páginas críticas que se acceden constantemente, simplemente porque fueron cargadas hace tiempo. Además, sufre de la anomalía de Belady: en ciertos patrones de acceso, ¡agregar más marcos aumenta los page faults en lugar de reducirlos!

11.9.3. Óptimo (OPT o MIN)

El algoritmo óptimo fue propuesto por Belady como una referencia teórica. La idea es reemplazar la página cuya próxima referencia está más lejana en el futuro, o que nunca será referenciada nuevamente.

Este algoritmo tiene una propiedad importante: garantiza la menor cantidad posible de page faults para cualquier secuencia de referencias dada. Es, por definición, el mejor algoritmo posible.

Pero hay un problema fundamental: este algoritmo requiere conocer el futuro, es decir, necesita saber de antemano toda la secuencia de referencias que el programa hará. Esto es imposible en sistemas reales. Entonces, ¿para qué sirve? El algoritmo óptimo se usa como referencia para comparar otros algoritmos. Nos permite responder: “¿cuán cerca está mi algoritmo práctico del mejor algoritmo posible?”

¿Por qué estudiamos el algoritmo óptimo si es imposible implementarlo?

Aunque no podemos usarlo en la práctica, el algoritmo óptimo es invaluable como punto de referencia. Nos da una cota inferior: sabemos que ningún algoritmo puede tener menos page faults que el óptimo para una secuencia dada. Esto nos permite evaluar cuán buenos son los algoritmos prácticos, midiendo qué tan cerca están del óptimo.

También es útil en sistemas con patrones de acceso predecibles (ej: procesamiento de streams).

11.9.4. LRU (Least Recently Used)

LRU es una aproximación práctica al algoritmo óptimo. La idea se basa en la localidad temporal: si el óptimo mira hacia el futuro, LRU mira hacia el pasado. Reemplaza la página cuyo último acceso fue el más lejano en el tiempo.

LRU (Least Recently Used) selecciona como víctima la página cuyo último acceso fue el más lejano en el pasado. Se basa en la observación de que páginas recientemente usadas probablemente se usarán pronto (localidad temporal).

En una implementación ideal, cada marco tendría un timestamp que se actualiza en cada acceso. Para encontrar la víctima, buscamos el marco con el timestamp más antiguo. Por ejemplo, si tenemos páginas con timestamps $t=100$, $t=250$, $t=180$, y $t=300$, la víctima sería la página con $t=100$.

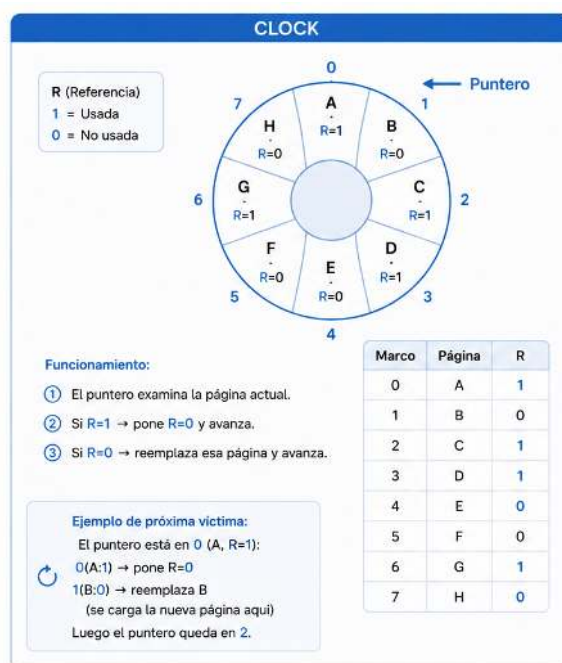
LRU tiene ventajas significativas. Su rendimiento es bueno en la práctica, explota efectivamente la localidad temporal, y en muchos casos está muy cerca del algoritmo óptimo. Sin embargo, la implementación exacta es costosa. Necesitamos actualizar un timestamp en cada acceso a memoria, no solo en page faults. Buscar el mínimo tiene complejidad $O(n)$ donde n es el número de marcos. Peor aún, cada acceso a memoria generaría una escritura adicional para actualizar el timestamp, duplicando efectivamente el tráfico de memoria.

Problema de implementar LRU exacto:

Cada acceso a memoria requiere tres operaciones: traducir la dirección (MMU), actualizar el timestamp (¡escritura adicional!), y acceder al dato real. Esto significa que cada lectura genera también una escritura para el timestamp. El resultado es que duplicamos el tráfico de memoria, lo cual es inaceptable. Por eso se usan aproximaciones de LRU como Clock y Clock-M.

11.9.5. Clock (Second Chance)

Clock es una aproximación eficiente de LRU que usa un solo bit de referencia en lugar de timestamps. Los marcos se organizan conceptualmente en un anillo circular con un puntero (la “manecilla del reloj”) que se mueve alrededor del anillo.



El algoritmo funciona de la siguiente manera. Cuando se busca una víctima, examinamos la página actual (donde apunta el puntero). Si su bit U (uso/referencia) está en 1, le damos una “segunda oportunidad”: ponemos U=0, avanzamos el puntero, y continuamos. Si U está en 0, esta página es la víctima: la reemplazamos y avanzamos el puntero.

El bit U es manejado por el hardware y el software cooperativamente. El hardware lo pone en 1 automáticamente cada vez que se accede a la página (ya sea lectura o escritura). El algoritmo Clock lo pone en 0 cuando da la segunda oportunidad.

La intuición es simple pero efectiva. Una página con U=1 está diciendo “he sido usada recientemente, dame otra oportunidad”. Una página con U=0 está diciendo “no he sido usada desde la última inspección”. Esto aproxima LRU: las páginas no usadas recientemente tienen más probabilidad de tener U=0 y ser eliminadas primero.

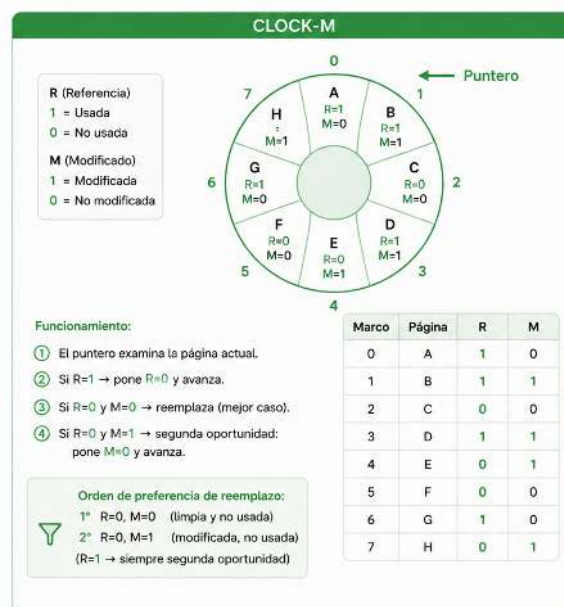
Funcionamiento de Clock:

Supongamos que necesitamos una víctima y el puntero está en una página con U=1. Le damos segunda oportunidad (U=0), avanzamos el puntero a la siguiente página. Si esta también tiene U=1, repetimos el proceso. Eventualmente encontraremos una página con U=0, que será nuestra víctima. En el peor caso, damos la vuelta completa al anillo, reseteando todos los bits U a 0, y la segunda vuelta garantiza encontrar una víctima.

11.9.6. Clock-M (Clock Mejorado / Enhanced Second Chance)

Clock-M es una versión mejorada del algoritmo Clock que considera tanto el bit U (Referenced) como el bit M (Modified/Dirty). Esta consideración adicional puede reducir significativamente el overhead de I/O.

Clock-M (Clock Mejorado) es una extensión del algoritmo Clock que usa los bits U y M para clasificar páginas en 4 categorías de prioridad para reemplazo. Prefiere reemplazar páginas no modificadas para evitar escrituras a disco.



Las páginas se clasifican en cuatro clases según sus bits:

- **Clase 0 ($U=0, M=0$):** No usada recientemente y no modificada. Esta es la mejor víctima posible.
- **Clase 1 ($U=0, M=1$):** No usada recientemente pero modificada. Segunda mejor opción.
- **Clase 2 ($U=1, M=0$):** Usada recientemente pero no modificada. Tercera opción.
- **Clase 3 ($U=1, M=1$):** Usada recientemente y modificada. La peor víctima.

La razón para esta clasificación está en el costo de reemplazo. Reemplazar una página dirty ($M=1$) requiere escribirla a disco antes de descartarla, lo que toma aproximadamente 10 ms en un disco duro tradicional. Luego debemos leer la nueva página, otros 10 ms, para un total de 20 ms. Por otro lado, reemplazar una página clean ($M=0$) no requiere escritura: simplemente la descartamos y leemos la nueva página (10 ms). Es 50 % más rápido.

El algoritmo Clock-M puede hacer hasta cuatro vueltas alrededor del anillo:

Vuelta 1: Buscar clase 0 ($U=0$, $M=0$) sin modificar ningún bit. Si existe una página no usada y no modificada, la encontraremos rápidamente.

Vuelta 2: Buscar clase 1 ($U=0$, $M=1$) sin modificar bits. Si no había clase 0, buscamos páginas no usadas pero dirty.

Vuelta 3: Buscar clase 0 nuevamente, pero ahora SÍ modificando U . En cada página con $U=1$, la ponemos en $U=0$ (segunda oportunidad). Si encontramos $U=0$, $M=0$, la tomamos.

Vuelta 4: Buscar clase 1 modificando U . En este punto, todas las páginas que tenían $U=1$ ahora tienen $U=0$. Tomamos la primera página con $M=1$ que encontremos.

Lógica de las cuatro vueltas de Clock-M:

Las vueltas 1 y 2 intentan encontrar una víctima sin dar segundas oportunidades. Si existe una página de clase 0 o 1, la encontraremos rápidamente sin resetear bits U . Esto es eficiente para el caso común.

Las vueltas 3 y 4 dan segundas oportunidades. Si todas las páginas tenían $U=1$, ahora las reseteamos a $U=0$. Esto garantiza que eventualmente encontraremos una víctima, pero también da a las páginas recientemente usadas otra oportunidad antes de ser reemplazadas.

11.9.7. Ejercicio Comparativo de Algoritmos

Para comprender realmente las diferencias entre algoritmos, analicemos una secuencia de referencias con tres algoritmos diferentes. Usaremos 3 marcos disponibles y la siguiente secuencia:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

11.9.7.1. Algoritmo FIFO

Con FIFO, simplemente reemplazamos la página más antigua en cada page fault:

Referencia	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Marco 0	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	7	7	7
Marco 1	0	0	0	0	3	3	3	2	2	2	2	2	2	2	0	0	0	0	0
Marco 2			1	1	1	1	1	1	3	3	3	3	3	3	3	1	1	1	1
PF	F	F	F	F		F		F	F	F	F			F		F	F	F	

Total Page Faults: 15

FIFO genera 15 page faults en esta secuencia. Podemos ver que a veces reemplaza páginas que se usarán pronto, como cuando reemplaza la página 0 justo antes de que se necesite nuevamente.

11.9.7.2. Algoritmo Óptimo (OPT)

El algoritmo óptimo, conociendo todo el futuro, toma decisiones perfectas:

Referencia	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Marco 0	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Marco 1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Marco 2			1	1	1	3	3	4	4	3	3	3	3	1	1	1	1	1	1
PF	F	F	F	F		F		F		F				F				F	

Total Page Faults: 9

OPT logra solo 9 page faults. Las decisiones clave incluyen: en la referencia 6 (página 3), reemplaza la página 7 porque su próxima referencia está muy lejos (posición 18). En la referencia 8 (página 4), reemplaza la página 1 cuya próxima referencia está en la posición 14. En la referencia 10 (página 3), reemplaza la página 4 que nunca se vuelve a usar.

11.9.7.3. Algoritmo Clock-M

Ahora veamos Clock-M con el detalle completo de los bits U y M. Para este ejemplo, asumiremos que algunas referencias son lecturas (R) y otras escrituras (W):

7R, 0R, 1W, 2R, 0R, 3W, 0W, 4R, 2W, 3R, 0R, 3R, 2R, 1R, 2W, 0R, 1R, 7W, 0R, 1R

Las páginas cargadas desde disco inician con U=0, M=0. Al acceder una página, se pone U=1. Al escribir una página, se pone M=1 (además de U=1).

Analicemos algunos momentos clave del algoritmo:

```

1 Ref 1: Acceso 7 (lectura)
2 - Page fault, cargar en marco 0
3 - Marcos: [7(0,0)] [ ] [ ]
4 - Puntero: marco 1
5 PF: 1
6
7 Ref 4: Acceso 2 (lectura)
8 - Page fault, necesita reemplazar
9 - Puntero en marco 0: [7(0,0)] ← U=0,M=0 (clase 0)→
10   Víctima encontrada en vuelta 1→
11   Reemplazar 7 por 2
12 - Marcos: [2(0,0)] [0(0,0)] [1(0,1)]
13 PF: 4
14
15 Ref 6: Acceso 3 (escritura)
16 - Page fault, necesita reemplazar
17 - Estado actual: [2(0,0)] [0(1,0)] [1(0,1)]
18 - Puntero en marco 1:
19
20   Vuelta 1: Buscar (U=0,M=0)
```

```

21 - Marco 1: [0(1,0)] → U=1, siguiente
22 - Marco 2: [1(0,1)] → M=1, siguiente
23 - Marco 0: [2(0,0)] → U=0,M=0 - ¡Víctima!
24
25 - Reemplazar página 2 por página 3
26 - Marcos: [3(0,1)] [0(1,0)] [1(0,1)]
27 PF: 5
28
29 Ref 8: Acceso 4 (lectura)
30 - Page fault, necesita reemplazar
31 - Estado actual: [3(0,1)] [0(1,1)] [1(0,1)]
32 - Todas las páginas tienen M=1 o U=1
33
34 Vuelta 1: No encuentra clase 0
35 Vuelta 2: Buscar (U=0,M=1)
36 - Marco 2: [1(0,1)] → U=0,M=1 - ¡Víctima!
37
38 - Debe escribir página 1 a disco (dirty)
39 - Reemplazar página 1 por página 4
40 - Marcos: [3(0,1)] [0(1,1)] [4(0,0)]
41 PF: 6

```

El algoritmo continúa de esta manera, clasificando páginas en las cuatro clases y prefiriendo víctimas clean sobre dirty cuando es posible. Al final de la secuencia, Clock-M logra 10 page faults.

Observaciones del ejercicio comparativo:

Clock-M realizó dos vueltas completas en las referencias 6 y 8, demostrando el mecanismo de búsqueda en múltiples vueltas. En la referencia 8, no encontró páginas de clase 0 en la vuelta 1, así que buscó clase 1 en la vuelta 2. Las escrituras incrementan M, haciendo a las páginas menos deseables como víctimas, lo cual es correcto porque reemplazarlas requiere I/O adicional. El algoritmo evitó escribir páginas dirty cuando pudo encontrar víctimas clean.

11.9.8. Comparación Final

Resumiendo los resultados de todos los algoritmos en esta secuencia:

Algoritmo	Page Faults	Complejidad	Hardware Requerido
FIFO	15	O(1)	Ninguno
Óptimo (OPT)	9	O(n)	Conocer futuro
LRU	12	O(n)	Timestamp
Clock	11	O(n)	Bit U
Clock-M	10	O(n)	Bits U y M

Las conclusiones son claras. El algoritmo óptimo es inalcanzable en la práctica pero nos da una

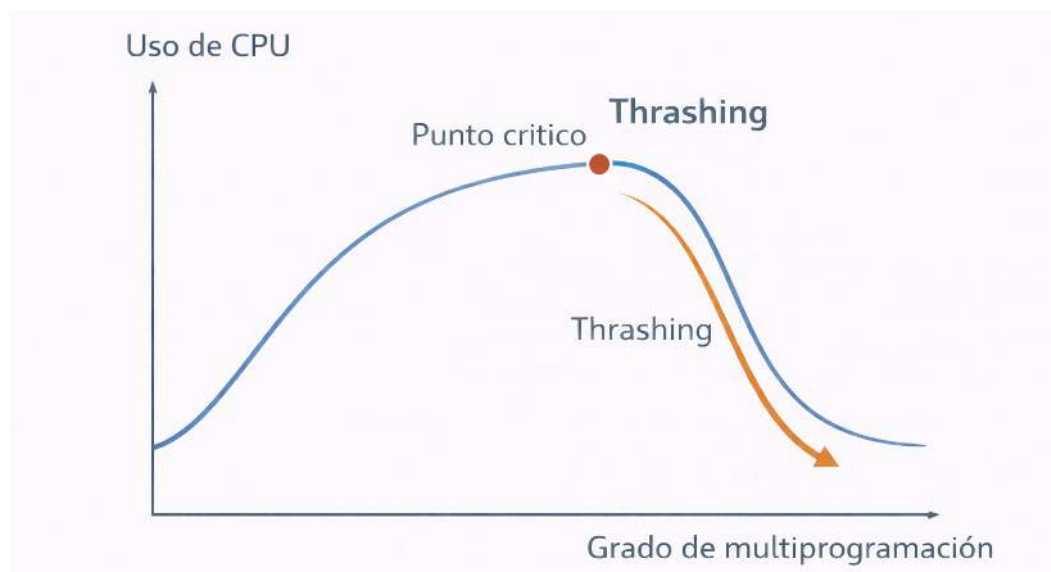
cota inferior: sabemos que ningún algoritmo real puede hacer mejor que 9 page faults en esta secuencia. Clock-M se acerca bastante al óptimo (10 vs 9), logrando solo un page fault más. FIFO es el peor, con 15 page faults (67 % más que el óptimo). El bit M realmente ayuda: Clock-M es mejor que Clock simple gracias a evitar escrituras a disco cuando es posible.

11.10. Thrashing

11.10.1. ¿Qué es Thrashing?

Thrashing es uno de los problemas más severos que puede enfrentar un sistema de memoria virtual. Es un estado donde el sistema dedica más tiempo a manejar page faults (cargando y descargando páginas) que a ejecutar instrucciones útiles.

Thrashing es un estado del sistema donde se dedica más tiempo a manejar page faults (cargar/descargar páginas) que a ejecutar instrucciones útiles. Ocurre cuando la suma de los working sets de todos los procesos excede la memoria física disponible.



<!-- Thrashing caption -->

Los síntomas son inconfundibles y dramáticos. La utilización de CPU cae a niveles muy bajos (menos del 20 %), mientras el disco trabaja constantemente al 100 % de su capacidad. Los procesos avanzan extremadamente lento, y el sistema se vuelve prácticamente inutilizable para los usuarios.

Veamos un ejemplo numérico concreto. Consideremos un sistema con 1 GiB de RAM y 10 procesos ejecutando simultáneamente. Cada proceso necesita un working set de 150 MiB para funcionar eficientemente. El total necesario sería $10 \times 150 \text{ MiB} = 1500 \text{ MiB}$, pero solo tenemos 1000 MiB disponibles. Hay un déficit de 500 MiB.

¿Qué sucede en esta situación? Solo aproximadamente 6 procesos caben cómodamente en memoria ($6 \times 150 = 900 \text{ MiB}$). Los otros 4 procesos generan page faults constantemente. Cuando cargamos páginas del proceso A, debemos quitar páginas del proceso B. Entonces el proceso B genera page faults, quitando páginas del proceso C. El proceso C genera page faults, quitando páginas del proceso A. El resultado es un ciclo vicioso sin fin.

La diferencia en rendimiento es catastrófica. En condiciones normales sin thrashing, la CPU opera al 80 % de utilización, el disco al 20 %, y el tiempo promedio por instrucción es de unos 10 nanosegundos. Con thrashing, la CPU cae al 5 % de utilización, el disco sube al 95 %, y el tiempo promedio por instrucción aumenta a aproximadamente 10 milisegundos. Esto representa una degradación de un millón de veces en el rendimiento.

El impacto del thrashing:

Durante thrashing, el sistema puede ser aproximadamente un millón de veces más lento que en condiciones normales. La CPU está mayormente idle esperando que el disco complete las operaciones de paginación. Para el usuario, el sistema parece "congelado", con el disco sonando constantemente pero las aplicaciones sin responder.

11.10.2. ¿Por Qué Ocurre?

La causa fundamental del thrashing es la sobrecarga de multiprogramación: intentar ejecutar demasiados procesos simultáneamente sin suficiente memoria física.

Existe una relación entre el grado de multiprogramación y el throughput del sistema que puede graficarse como una curva. Con pocos procesos (0-6), la CPU está subutilizada porque no hay suficiente trabajo. En el punto óptimo (alrededor de 6-8 procesos), alcanzamos el máximo throughput. Pero con demasiados procesos (más de 10), el sistema cae en thrashing y el throughput colapsa.

El ciclo del thrashing es pernicioso y se auto-refuerza. Comienza cuando el sistema tiene poca carga y la CPU está idle. El sistema operativo, viendo CPU ociosa, decide agregar más procesos para aumentar la utilización. La memoria se llena y los procesos comienzan a generar page faults. Los procesos se bloquean esperando I/O de disco. El sistema operativo, viendo procesos bloqueados, interpreta que hay poca carga y agrega aún más procesos. Más procesos significan más page faults y menos memoria por proceso. El resultado final es thrashing: el disco saturado y la CPU idle.

La paradoja del thrashing:

La baja utilización de CPU lleva al SO a agregar más procesos. Esto empeora el thrashing, reduciendo aún más la CPU. El SO responde agregando todavía más procesos. Este ciclo de retroalimentación positiva puede llevar al colapso completo del sistema si no hay mecanismos de prevención.

11.10.3. Detección de Thrashing

Para prevenir o recuperarse del thrashing, primero debemos detectarlo. Existen varias métricas que podemos monitorear.

La primera es la *page fault rate*. Si la tasa de page faults por segundo excede un umbral (por ejemplo, 10 page faults por segundo por proceso), es una señal clara de thrashing.

La segunda es la relación entre actividad de disco y CPU. Si $(\text{Disk I/O \%}) / (\text{CPU \%})$ es mayor que 5, el thrashing es probable. En condiciones normales, con 20 % de disco y 80 % de CPU, el ratio es 0.25. Durante thrashing, con 95 % de disco y 5 % de CPU, el ratio es 19, indicando claramente el problema.

La tercera métrica es el tiempo promedio de servicio de page faults. Si el tiempo promedio para resolver un page fault excede 50 milisegundos, es señal de que el disco no da abasto con la carga de trabajo.

11.10.4. Prevención y Recuperación

Existen varias estrategias para prevenir el thrashing antes de que ocurra.

El *Working Set Model* es un enfoque donde estimamos el working set de cada proceso. Solo ejecutamos un proceso si la suma de todos los working sets es menor o igual a la RAM disponible. Si no hay suficiente memoria, suspendemos procesos hasta que haya recursos suficientes.

El enfoque de *Page Fault Frequency (PFF)* monitorea cada proceso individualmente. Si la tasa de page faults de un proceso excede un umbral alto, le damos más marcos. Si está por debajo de un umbral bajo, le quitamos marcos (que pueden ir a otros procesos). Si no hay marcos disponibles para un proceso que los necesita, lo suspendemos.

Una tercera estrategia es simplemente limitar el grado de multiprogramación. Establecemos un máximo de procesos activos basado en la memoria disponible. Por ejemplo, con 8 GiB de RAM, podríamos limitar a 20 procesos máximo, basándonos en estimaciones del working set promedio.

Cuando ya estamos en thrashing, necesitamos estrategias de recuperación. La más efectiva es suspender procesos: swapear un proceso completo a disco, liberando toda su memoria, reduciendo así el grado de multiprogramación. También podemos priorizar procesos críticos, garantizando su

working set a expensas de procesos de baja prioridad que pueden suspenderse. La solución obvia, aunque no siempre viable, es aumentar la memoria RAM física del sistema o reducir el memory footprint de los procesos.

11.10.5. Preguntas Típicas de Examen

Para consolidar la comprensión del thrashing, analicemos algunas preguntas típicas que aparecen en exámenes.

Pregunta 1: *Si un sistema está en thrashing, ¿mejora el rendimiento agregando más procesos?*

La respuesta es un rotundo no. Agregar más procesos empeora el thrashing, no lo mejora. La razón es que habrá menos memoria disponible por proceso, lo que genera más page faults, más I/O de disco, y peor rendimiento general. La solución correcta es reducir la cantidad de procesos, no agregarlos.

Pregunta 2: *¿Agregar más RAM siempre soluciona el thrashing?*

La respuesta depende del contexto. Si el thrashing es causado por una genuina falta de RAM física, entonces agregar RAM ayudará. Si el problema es que los procesos tienen working sets extremadamente grandes, agregar RAM ayudará parcialmente pero puede no ser suficiente. Si el thrashing es causado por un algoritmo de reemplazo inadecuado, más RAM no resolverá el problema subyacente. Y si el sistema operativo continúa agregando procesos sin límite, más RAM solo postergará el thrashing pero no lo evitará, porque el problema es de diseño del sistema.

Pregunta 3: *Un sistema tiene 20 % de utilización de CPU y 90 % de utilización de disco. ¿Está en thrashing? ¿Qué hacer?*

Este es un caso claro de thrashing. El ratio Disk/CPU es $90/20 = 4.5$, que es muy alto (lo normal sería menos de 1). Las acciones inmediatas incluyen identificar los procesos con alta tasa de page faults, suspender procesos menos críticos, verificar que los working sets no excedan la RAM disponible, y definitivamente no agregar más procesos al sistema.

Pregunta 4: *¿Puede ocurrir thrashing con mucha RAM libre?*

Sorprendentemente, la respuesta es sí, aunque en casos raros. Esto puede ocurrir si el algoritmo de reemplazo es muy malo. Por ejemplo, FIFO podría reemplazar constantemente páginas que están siendo usadas, si todas las páginas del working set son “viejas” según el criterio de FIFO. El resultado serían page faults continuos a pesar de tener RAM libre. Sin embargo, este escenario es muy poco común; el thrashing típico es causado por falta genuina de memoria física.

Prevención vs. Recuperación:

Es mucho mejor prevenir el thrashing que intentar recuperarse de él. Los mecanismos de prevención como Working Set y PFF pueden detectar las condiciones que llevarían al thrashing

antes de que ocurra, suspendiendo procesos proactivamente. Una vez que el sistema está en thrashing profundo, la recuperación es más difícil y puede requerir intervención manual del administrador del sistema.

11.11. Código en C: Simulador de Clock-M

Este simulador implementa el algoritmo Clock-M (Clock Mejorado) y permite visualizar el proceso de reemplazo paso a paso.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4  #include <string.h>
5
6  // Configuración del sistema
7  #define NUM_FRAMES 4          // Número de marcos en RAM
8  #define MAX_REFS 50          // Máximo de referencias a simular
9
10 // Estructura de un marco
11 typedef struct {
12     int page;                  // Número de página (-1 si vacío)
13     bool referenced;           // Bit U (referenced)
14     bool modified;             // Bit M (modified)
15 } Frame;
16
17 // Estado del sistema
18 typedef struct {
19     Frame frames[NUM_FRAMES];
20     int clock_hand;            // Puntero del reloj
21     int page_faults;           // Contador de page faults
22 } SystemState;
23
24 // Tipo de acceso
25 typedef enum {
26     ACCESS_READ,
27     ACCESS_WRITE
28 } AccessType;
29
30 // Inicializar sistema
31 void init_system(SystemState *sys) {
32     for (int i = 0; i < NUM_FRAMES; i++) {
33         sys->frames[i].page = -1;        // Marco vacío
34         sys->frames[i].referenced = false;
35         sys->frames[i].modified = false;
36     }
37     sys->clock_hand = 0;
38     sys->page_faults = 0;
39 }
40
41 // Imprimir estado de marcos
42 void print_frames(SystemState *sys) {
43     printf("Marcos: ");

```

```

44     for (int i = 0; i < NUM_FRAMES; i++) {
45         if (sys->frames[i].page == -1) {
46             printf("[ - ] ");
47         } else {
48             printf("[%d(%d,%d)] ",
49                 sys->frames[i].page,
50                 sys->frames[i].referenced ? 1 : 0,
51                 sys->frames[i].modified ? 1 : 0);
52         }
53     }
54     printf(" →Clock%d\n", sys->clock_hand);
55 }
56
57 // Buscar página en marcos
58 int find_page(SystemState *sys, int page) {
59     for (int i = 0; i < NUM_FRAMES; i++) {
60         if (sys->frames[i].page == page) {
61             return i; // Encontrada
62         }
63     }
64     return -1; // No encontrada
65 }
66
67 // Buscar marco vacío
68 int find_empty_frame(SystemState *sys) {
69     for (int i = 0; i < NUM_FRAMES; i++) {
70         if (sys->frames[i].page == -1) {
71             return i;
72         }
73     }
74     return -1; // No hay marcos vacíos
75 }
76
77 // Algoritmo Clock-M: buscar víctima
78 // Retorna índice del marco a reemplazar
79 int clock_m_find_victim(SystemState *sys) {
80     printf(" Buscando víctima con Clock-M...\n");
81
82     int start = sys->clock_hand;
83     int victim = -1;
84
85     // Vuelta 1: Buscar clase 0 (U=0, M=0) sin modificar bits
86     printf(" Vuelta 1: Buscando (U=0, M=0)\n");
87     for (int i = 0; i < NUM_FRAMES; i++) {
88         int idx = (start + i) % NUM_FRAMES;
89         Frame *f = &sys->frames[idx];
90
91         printf(" Marco %d: pág %d (U=%d, M=%d) → ",
92             idx, f->page, f->referenced, f->modified);
93
94         if (!f->referenced && !f->modified) {
95             printf("Clase 0 - Víctima encontrada!\n");
96             victim = idx;
97             sys->clock_hand = (idx + 1) % NUM_FRAMES;
98             return victim;
99         }
100     }
101     printf("Siguiente\n");

```

```

102
103 // Vuelta 2: Buscar clase 1 (U=0, M=1) sin modificar bits
104 printf("  Vuelta 2: Buscando (U=0, M=1)\n");
105 for (int i = 0; i < NUM_FRAMES; i++) {
106     int idx = (start + i) % NUM_FRAMES;
107     Frame *f = &sys->frames[idx];
108
109     printf("    Marco %d: pág %d (U=%d, M=%d) → ",
110           idx, f->page, f->referenced, f->modified);
111
112     if (!f->referenced && f->modified) {
113         printf("Clase 1 - Víctima encontrada!\n");
114         victim = idx;
115         sys->clock_hand = (idx + 1) % NUM_FRAMES;
116         return victim;
117     }
118     printf("Siguiente\n");
119 }
120
121 // Vuelta 3: Buscar clase 0 (U=0, M=0) reseteando U
122 printf("  Vuelta 3: Buscando (U=0, M=0) y reseteando U\n");
123 for (int i = 0; i < NUM_FRAMES; i++) {
124     int idx = (start + i) % NUM_FRAMES;
125     Frame *f = &sys->frames[idx];
126
127     printf("    Marco %d: pág %d (U=%d, M=%d) → ",
128           idx, f->page, f->referenced, f->modified);
129
130     if (f->referenced) {
131         printf("Resetear U=0, ");
132         f->referenced = false;
133     }
134
135     if (!f->referenced && !f->modified) {
136         printf("Clase 0 - Víctima encontrada!\n");
137         victim = idx;
138         sys->clock_hand = (idx + 1) % NUM_FRAMES;
139         return victim;
140     }
141     printf("Siguiente\n");
142 }
143
144 // Vuelta 4: Buscar clase 1 (U=0, M=1) con U ya reseteado
145 printf("  Vuelta 4: Buscando (U=0, M=1)\n");
146 for (int i = 0; i < NUM_FRAMES; i++) {
147     int idx = (start + i) % NUM_FRAMES;
148     Frame *f = &sys->frames[idx];
149
150     printf("    Marco %d: pág %d (U=%d, M=%d) → ",
151           idx, f->page, f->referenced, f->modified);
152
153     if (!f->referenced && f->modified) {
154         printf("Clase 1 - Víctima encontrada!\n");
155         victim = idx;
156         sys->clock_hand = (idx + 1) % NUM_FRAMES;
157         return victim;
158     }
159     printf("Siguiente\n");

```

```

160     }
161
162     // No debería llegar aquí (al menos una página debe ser víctima)
163     // En caso extremo, tomar la primera
164     printf(" [ERROR] No se encontró víctima, tomando marco 0\n");
165     return 0;
166 }
167
168 // Acceder a una página
169 void access_page(SystemState *sys, int page, AccessType type) {
170     char *access_str = (type == ACCESS_READ) ? "LECTURA" : "ESCRITURA";
171
172     printf("\n\n");
173     printf("Acceso %d: Página %d (%s)\n",
174           sys->page_faults + 1, page, access_str);
175
176     // Buscar página en marcos
177     int frame_idx = find_page(sys, page);
178
179     if (frame_idx != -1) {
180         // HIT: página ya está en RAM
181         printf(" - HIT: Página %d en marco %d\n", page, frame_idx);
182         sys->frames[frame_idx].referenced = true;
183         if (type == ACCESS_WRITE) {
184             sys->frames[frame_idx].modified = true;
185         }
186     } else {
187         // MISS: page fault
188         printf("PAGE FAULT: Página %d no está en RAM\n", page);
189         sys->page_faults++;
190
191         // Buscar marco vacío
192         frame_idx = find_empty_frame(sys);
193
194         if (frame_idx != -1) {
195             // Hay marco vacío, cargar directamente
196             printf(" Marco %d está vacío, cargar página %d\n",
197                   frame_idx, page);
198         } else {
199             // No hay marcos vacíos, ejecutar algoritmo de reemplazo
200             printf(" No hay marcos vacíos, ejecutar Clock-M\n");
201             frame_idx = clock_m_find_victim(sys);
202
203             Frame *victim = &sys->frames[frame_idx];
204             printf(" Reemplazar página %d (en marco %d)\n",
205                   victim->page, frame_idx);
206
207             // Si la víctima está modificada, hay que escribirla a disco
208             if (victim->modified) {
209                 printf(" [I/O] Escribir página %d a disco (dirty)\n",
210                       victim->page);
211             }
212         }
213
214         // Cargar nueva página
215         printf(" [I/O] Leer página %d desde disco\n", page);
216         sys->frames[frame_idx].page = page;
217         sys->frames[frame_idx].referenced = true; // El acceso que causó el

```

```

218         fault
219         sys->frames[frame_idx].modified = (type == ACCESS_WRITE);
220     }
221     print_frames(sys);
222 }
223
224 // Función principal
225 int main() {
226     SystemState sys;
227     init_system(&sys);
228
229     printf("=====\\n");
230     printf("  SIMULADOR DE CLOCK-M (Clock Mejorado)\\n");
231     printf("=====\\n");
232     printf("Configuración: %d marcos en RAM\\n", NUM_FRAMES);
233     printf("Formato: [pág(U,M)] donde U=Referenced, M=Modified\\n\\n");
234
235     // Secuencia de referencias (página, tipo de acceso)
236     // R = READ, W = WRITE
237     typedef struct {
238         int page;
239         AccessType type;
240     } Reference;
241
242     Reference refs[] = {
243         {7, ACCESS_READ},    // 1
244         {0, ACCESS_READ},    // 2
245         {1, ACCESS_WRITE},   // 3
246         {2, ACCESS_READ},    // 4
247         {0, ACCESS_READ},    // 5
248         {3, ACCESS_WRITE},   // 6
249         {0, ACCESS_WRITE},   // 7
250         {4, ACCESS_READ},    // 8
251         {2, ACCESS_WRITE},   // 9
252         {3, ACCESS_READ},    // 10
253         {0, ACCESS_READ},    // 11
254         {3, ACCESS_READ},    // 12
255         {2, ACCESS_READ},    // 13
256         {1, ACCESS_READ},    // 14
257         {2, ACCESS_WRITE},   // 15
258     };
259
260     int num_refs = sizeof(refs) / sizeof(refs[0]);
261
262     // Ejecutar simulación
263     for (int i = 0; i < num_refs; i++) {
264         access_page(&sys, refs[i].page, refs[i].type);
265     }
266
267     // Resumen final
268     printf("\\n=====\\n");
269     printf("  RESUMEN FINAL\\n");
270     printf("=====\\n");
271     printf("Total de referencias: %d\\n", num_refs);
272     printf("Total de page faults: %d\\n", sys.page_faults);
273     printf("Hit rate: %.2f%%\\n",
274         ((num_refs - sys.page_faults) * 100.0) / num_refs);

```

```

275     printf("Page fault rate: %.2f%%\n",
276           (sys.page_faults * 100.0) / num_refs);
277
278     printf("\nEstado final de marcos:\n");
279     print_frames(&sys);
280
281     return 0;
282 }

```

Para compilar y ejecutar el simulador:

```

1 gcc -o clock_m_sim clock_m_sim.c -Wall -std=c99
2 ./clock_m_sim

```

El simulador define estructuras para representar marcos de memoria, cada uno con un número de página, un bit referenced (U), y un bit modified (M). El estado del sistema incluye el array de marcos, el puntero del reloj (clock hand), y un contador de page faults.

Las funciones clave implementan la lógica del algoritmo. `find_page()` busca si una página está en RAM. `find_empty_frame()` busca marcos vacíos. La función más importante es `clock_m_find_victim()`, que implementa las cuatro vueltas del algoritmo Clock-M con salida detallada para propósitos educativos.

La vuelta 1 busca páginas de clase 0 (U=0, M=0) sin modificar bits. Si encuentra una, la selecciona inmediatamente como víctima. La vuelta 2 busca páginas de clase 1 (U=0, M=1), también sin modificar bits. La vuelta 3 busca nuevamente clase 0, pero ahora resetea los bits U de las páginas con U=1, dándoles una segunda oportunidad. La vuelta 4 busca clase 1 con los bits U ya reseteados por la vuelta 3, garantizando que eventualmente encontrará una víctima.

La función `access_page()` maneja tanto hits como page faults. En un hit, simplemente actualiza los bits referenced y modified según el tipo de acceso. En un page fault, primero busca un marco vacío; si no hay, llama al algoritmo de reemplazo. Si la víctima está modificada, simula escribirla a disco antes de reemplazarla.

Salida esperada del simulador (fragmento):

```

1  =====
2
3  SIMULADOR DE CLOCK-M (Clock Mejorado)
4  =====
5  Configuración: 4 marcos en RAM
6  Formato: [pág(U,M)] donde U=Referenced, M=Modified
7
8
9  Acceso 1: Página 7 (LECTURA)
10 PAGE FAULT: Página 7 no está en RAM
11 Marco 0 está vacío, cargar página 7
12 [I/O] Leer página 7 desde disco
13 Marcos: [7(0,0)] [ - ] [ - ] [ - ] → Clock1

```

```

14
15
16 Acceso 8: Página 4 (LECTURA)
17 PAGE FAULT: Página 4 no está en RAM
18 No hay marcos vacíos, ejecutar Clock-M
19 Buscando víctima con Clock-M...
20 Vuelta 1: Buscando (U=0, M=0)
21 Marco 0: pág 7 (U=0, M=0) → Clase 0 - Víctima encontrada!
22 Reemplazar página 7 (en marco 0)
23 [I/O] Leer página 4 desde disco
24 Marcos: [4(0,0)] [0(1,1)] [1(0,1)] [3(0,1)] → Clock1
        
25
26
27 RESUMEN FINAL=====
28
29 Total de referencias: 15
30 Total de page faults: 9
31 Hit rate: 40.00%
32 Page fault rate: 60.00%

```

El simulador es una herramienta educativa valiosa. Permite ver exactamente cómo el algoritmo toma decisiones, cómo maneja los bits U y M, cuándo da segundas oportunidades, y por qué ciertas páginas son seleccionadas como víctimas. Modificar la secuencia de referencias en el código permite experimentar con diferentes patrones de acceso y observar cómo responde el algoritmo.

Conceptos demostrados en el código:

El simulador ilustra varios conceptos clave: la estructura de marcos con bits U y M, el funcionamiento del puntero circular (clock hand), la lógica completa de las cuatro vueltas del algoritmo Clock-M, la detección y manejo de páginas dirty que requieren escritura a disco, la diferencia entre hits y page faults en términos de rendimiento, y el cálculo de métricas de rendimiento como hit rate y page fault rate.

11.12. Conclusiones del Capítulo

La memoria virtual es uno de los logros más significativos en la historia de los sistemas operativos. Permite ejecutar procesos más grandes que la memoria física disponible, simplifica enormemente la programación al darle a cada proceso la ilusión de un espacio de direcciones enorme y privado, y permite una multiprogramación eficiente al compartir la memoria física entre múltiples procesos.

El éxito de la memoria virtual depende críticamente del principio de localidad. Sin localidad temporal y espacial, cada proceso necesitaría todas sus páginas en RAM todo el tiempo, haciendo la memoria virtual impráctica. Gracias a la localidad, los procesos típicamente solo necesitan una pequeña fracción de su espacio de direcciones en cualquier momento dado, el working set, que

puede caber en la memoria física disponible.

Los algoritmos de reemplazo de páginas son fundamentales para el rendimiento del sistema. FIFO es simple pero puede tener rendimiento pobre. El algoritmo óptimo es inalcanzable pero provee una cota inferior útil. LRU es efectivo pero costoso de implementar exactamente. Clock y Clock-M son aproximaciones prácticas de LRU que ofrecen buen rendimiento con overhead razonable, siendo Clock-M particularmente efectivo al considerar tanto el uso reciente como si las páginas están modificadas.

El thrashing representa el límite de lo que la memoria virtual puede lograr. Ocurre cuando intentamos ejecutar demasiados procesos simultáneamente, excediendo la capacidad de la memoria física. La prevención del thrashing requiere controlar el grado de multiprogramación, estimar working sets, y estar dispuesto a suspender procesos cuando la memoria es insuficiente.

Los sistemas operativos modernos combinan todas estas técnicas: demand paging para cargar páginas solo cuando se necesitan, algoritmos sofisticados de reemplazo como Clock-M, monitoreo continuo para detectar condiciones de thrashing, y mecanismos de prevención basados en working sets o page fault frequency. El resultado es un sistema que puede ejecutar eficientemente muchos procesos grandes en hardware con memoria física limitada, todo de manera transparente para las aplicaciones.

La memoria virtual es un ejemplo perfecto de cómo la abstracción en sistemas operativos puede simplificar la programación mientras mejora la utilización de recursos. Los programadores escriben código como si tuvieran acceso ilimitado a memoria contigua, mientras el sistema operativo y el hardware colaboran detrás de escena para hacer realidad esta ilusión, manejando la complejidad de la paginación, el reemplazo, y la gestión de memoria física de manera completamente transparente.

12 Sistema de Archivos (File System)

12.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

- Comprender la necesidad y función de los sistemas de archivos en un sistema operativo
- Distinguir entre las abstracciones de archivo y directorio desde la perspectiva del usuario
- Identificar las operaciones básicas sobre archivos y directorios
- Comprender las estructuras administrativas del SO para gestión de archivos (tablas, locks)
- Analizar el modelo de permisos y protección de Unix
- Diferenciar bloques lógicos de sectores físicos en el nivel de hardware
- Identificar y comparar los métodos de asignación de espacio en disco
- Analizar la estructura y funcionamiento del sistema de archivos FAT (12/16/32)
- Comprender en profundidad la estructura de EXT2/UFS, especialmente el concepto de inodo
- Calcular accesos a disco en operaciones de lectura y escritura considerando bloques de datos y punteros
- Diferenciar entre hard links y soft links
- Resolver ejercicios tipo parcial sobre estructuras de file systems

12.2. File System: Definición y Objetivos

12.2.1. ¿Por qué necesitamos File Systems?

Imaginemos por un momento que no existieran los sistemas de archivos. Cada vez que queremos guardar información en el disco duro, tendríamos que recordar exactamente en qué sectores físicos guardamos cada dato. Si queremos acceder a un documento de texto, necesitaríamos saber que está almacenado en los sectores 1024 al 1536 del cilindro 45, cabeza 2. Para recuperar una foto, deberíamos recordar que ocupa los sectores 8192 al 12288 del cilindro 103.

Este escenario es completamente impracticable. Los usuarios no pueden recordar direcciones físicas de miles de archivos, ni existe forma de dar nombres significativos a los datos. Sería imposible organizar la información de manera jerárquica, y el riesgo de sobrescribir datos accidentalmente sería altísimo. Sin un mecanismo de control, no existiría forma de implementar permisos o restricciones de acceso. Además, la pérdida de datos ante la corrupción de una pequeña región del disco sería total y catastrófica.

Sistema de Archivos (File System): Componente del sistema operativo que proporciona mecanismos para el almacenamiento, organización, manipulación, recuperación y administración de información en dispositivos de almacenamiento secundario.

12.2.2. Objetivos del File System

El sistema de archivos surge como una capa de abstracción entre el hardware de almacenamiento y el usuario o las aplicaciones. Esta capa de abstracción transforma el disco —que en realidad es solo un conjunto lineal de sectores físicos— en una estructura lógica organizada que podemos entender y manipular de forma intuitiva.

El primer objetivo fundamental es el *naming*: permitir dar nombres significativos a conjuntos de datos. En lugar de recordar “sectores 1024-1536”, podemos simplemente referirnos a “informe_final.pdf”. Este concepto aparentemente simple revoluciona la forma en que interactuamos con el almacenamiento.

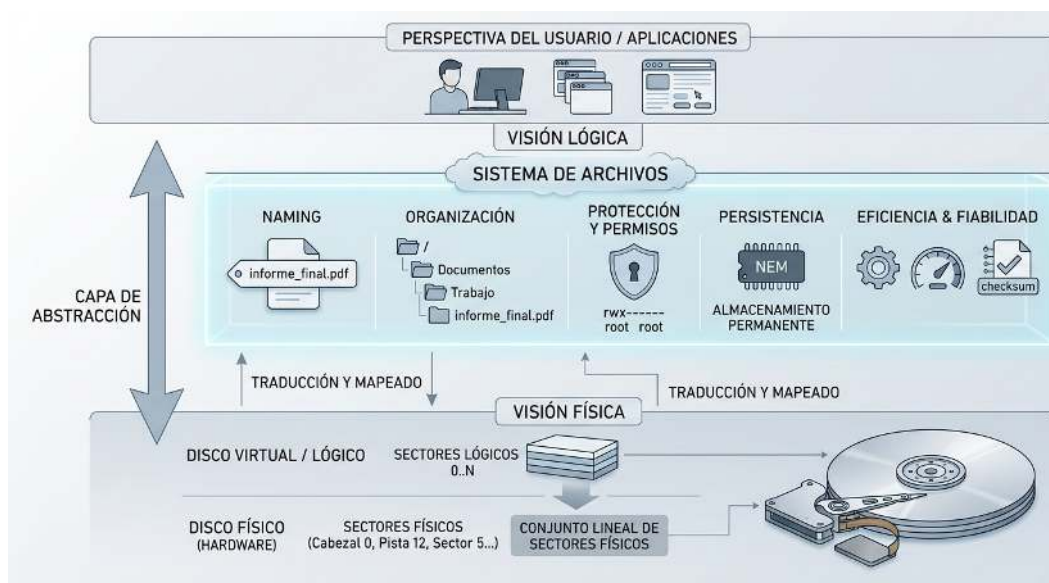
La *organización* es otro pilar esencial. Los archivos se estructuran en jerarquías mediante directorios y subdirectorios, lo que nos permite agrupar información relacionada de manera lógica. Esta organización jerárquica refleja naturalmente cómo pensamos sobre nuestros datos.

La *protección* garantiza que podamos controlar quién puede acceder a qué información. Mediante sistemas de permisos, establecemos barreras de seguridad que protegen datos sensibles y previenen modificaciones no autorizadas.

La *persistencia* asegura que los datos sobrevivan al apagado del sistema. A diferencia de la memoria RAM, el almacenamiento secundario mantiene la información incluso sin energía eléctrica. El file system gestiona esta persistencia de manera transparente.

La eficiencia optimiza tanto el uso del espacio disponible como la velocidad de acceso. El sistema debe minimizar el desperdicio de espacio y maximizar la velocidad con la que podemos leer y escribir información.

Finalmente, la confiabilidad garantiza la *integridad de los datos ante fallas*. Mecanismos como journaling y verificaciones de consistencia protegen contra corrupción y pérdidas de información.



12.3. Archivos: Abstracción de Usuario

12.3.1. Concepto de Archivo

En bases de datos y sistemas de información, existe una jerarquía conceptual bien definida. En el nivel más bajo encontramos el **campo**: una unidad mínima de información con significado propio, como “nombre”, “edad” o “dirección”. Un conjunto de campos relacionados que describen una entidad forma un **registro**. Por ejemplo, un registro de estudiante contiene campos como nombre, legajo, carrera y email. Finalmente, un **archivo** es una colección de registros del mismo tipo —el archivo “estudiantes.dat” contiene todos los registros de estudiantes de la institución.

Archivo (desde la perspectiva del SO): Secuencia nombrada de bytes almacenada en dispositivo de almacenamiento secundario. El sistema operativo NO interpreta el contenido del archivo; esa es responsabilidad de las aplicaciones.

Esta definición es crucial: para el sistema operativo, un archivo es simplemente una secuencia de bytes sin estructura interna. No existe diferencia entre un documento de texto, una imagen o un ejecutable a nivel del SO. La interpretación y estructura del contenido es responsabilidad exclusiva de las aplicaciones que usan esos archivos.

12.3.2. Atributos de un Archivo

Cada archivo posee **metadatos** que el sistema operativo almacena y gestiona. Estos metadatos son información sobre el archivo, no el contenido del archivo en sí. El *nombre* es el identificador

legible por humanos que usamos para referirnos al archivo. El *tipo* indica si es un archivo regular, un directorio, un enlace simbólico o un dispositivo. La *ubicación* contiene punteros a los bloques de datos físicos en el disco donde realmente se almacena el contenido.

El *tamaño* registra la cantidad de bytes que ocupa el archivo. Los *permisos* especifican quién puede leer, escribir o ejecutar el archivo, divididos entre propietario, grupo y otros usuarios. Los *timestamps* mantienen tres marcas temporales: la creación (ctime), la última modificación del contenido (mtime) y el último acceso de lectura (atime). El *propietario* se identifica mediante el UID (User ID) del usuario dueño del archivo, mientras que el *grupo* se identifica con el GID (Group ID) asociado. Finalmente, el *contador de enlaces* registra cuántos hard links apuntan al mismo archivo físico —concepto que exploraremos en detalle más adelante.

12.3.3. File Control Block (FCB)

File Control Block (FCB): Estructura de datos que almacena todos los metadatos de un archivo. En sistemas Unix se denomina **inodo (inode)**. En FAT, la entrada de directorio cumple esta función.

El FCB es fundamental porque separa la *identidad del archivo* —sus metadatos— de su *contenido* —los datos en bloques físicos. Esta separación permite que un archivo pueda tener múltiples nombres (hard links) pero mantener un solo FCB. Cuando creamos un segundo nombre para un archivo existente, no duplicamos sus datos ni sus metadatos completos; simplemente creamos otra entrada de directorio que apunta al mismo FCB.

12.3.4. Operaciones sobre Archivos

El sistema operativo provee system calls para manipular archivos. Estas operaciones constituyen la interfaz entre las aplicaciones y el file system.

12.3.4.1. Operaciones Básicas

La operación `create(nombre)` crea un archivo nuevo. Internamente, asigna un FCB o inodo, crea una entrada en el directorio padre y inicializa los metadatos con valores por defecto: permisos según el umask, timestamps con la hora actual y tamaño igual a cero.

Para trabajar con un archivo, primero debemos abrirlo mediante `open(nombre, modo)`. Esta operación busca el archivo en el directorio, verifica que tenemos los permisos necesarios para el modo de acceso solicitado (solo lectura, solo escritura o lectura-escritura), carga el FCB en memoria y retorna un file descriptor —un número entero que usaremos en operaciones posteriores.

La lectura se realiza con `read(fd, buffer, cantidad)`. El sistema usa el file descriptor para ubicar el archivo abierto, copia los datos desde los bloques del disco al buffer en memoria que le indicamos, y actualiza internamente el puntero de lectura/escritura para que la próxima operación continúe desde donde terminó esta.

De manera simétrica, `write(fd, buffer, cantidad)` escribe bytes al archivo. Si es necesario, el sistema asigna bloques nuevos para almacenar la información. Los datos se copian desde el buffer en memoria a los bloques en disco, y se actualizan el tamaño del archivo y sus timestamps.

La operación `seek(fd, offset, whence)` permite mover el puntero de lectura/escritura sin leer ni escribir datos. Podemos posicionarnos desde el inicio del archivo (`SEEK_SET`), desde la posición actual (`SEEK_CUR`) o desde el final (`SEEK_END`). Esta capacidad es esencial para el acceso aleatorio a archivos.

Cuando terminamos de trabajar con un archivo, `close(fd)` libera las estructuras en memoria asociadas, escribe a disco cualquier buffer pendiente (flush) y actualiza los metadatos finales. Es importante cerrar archivos para evitar pérdida de datos y agotamiento de file descriptors.

Finalmente, `delete(nombre)` elimina un archivo. Más precisamente, decrementa el contador de enlaces. Solo cuando este contador llega a cero —cuando no quedan nombres apuntando al archivo— se liberan los bloques de datos y el FCB, y se elimina la entrada del directorio.

12.3.4.2. Operaciones Adicionales

Existen operaciones complementarias que permiten manipular archivos sin modificar su contenido.

`rename(viejo, nuevo)`: Cambia el nombre del archivo `truncate(nombre, tamaño)`: Reduce el tamaño del archivo `stat(nombre)`: Obtiene metadatos sin abrir el archivo `chmod(nombre, permisos)`: Cambia permisos `chown(nombre, uid, gid)`: Cambia propietario

12.3.5. Métodos de Acceso

Método de Acceso: Forma en que los procesos leen y escriben datos en un archivo. Define el orden y la forma de acceso a la información.

El método de acceso determina cómo una aplicación interactúa con el contenido del archivo. La elección del método correcto puede impactar dramáticamente el rendimiento.

12.3.5.1. Acceso Secuencial

En el acceso secuencial, los bytes se leen o escriben uno después del otro, desde el inicio hasta el final del archivo. Este patrón es simple de implementar y óptimo cuando necesitamos procesar archivos completos. El sistema operativo puede anticiparse a nuestras necesidades mediante pre-fetching, cargando bloques adicionales en caché antes de que los solicitemos explícitamente.

```
1 // Lectura secuencial
2 int fd = open("datos.txt", O_RDONLY);
3 char buffer[1024];
4 while (read(fd, buffer, 1024) > 0) {
5     // Procesar buffer
6 }
7 close(fd);
```

Los archivos de log, los archivos de texto y el procesamiento por lotes son candidatos ideales para acceso secuencial. La localidad temporal y espacial de este patrón permite al sistema optimizar agresivamente las operaciones de I/O.

12.3.5.2. Acceso Directo (Random Access)

El acceso directo permite leer o escribir cualquier byte del archivo sin recorrer los bytes anteriores. Simplemente calculamos la posición deseada y nos movemos allí con `seek()`.

```
1 // Acceso directo
2 int fd = open("base_datos.dat", O_RDWR);
3 // Leer registro 100 (cada registro = 256 bytes)
4 lseek(fd, 100 * 256, SEEK_SET);
5 read(fd, &registro, 256);
6 close(fd);
```

Este patrón es ideal para bases de datos y estructuras indexadas donde necesitamos acceso instantáneo a registros específicos sin procesar todo el archivo. No hay penalidad por “saltar” a posiciones arbitrarias.

12.3.5.3. Acceso Indexado

El acceso indexado mantiene un índice separado que mapea claves lógicas a posiciones físicas en el archivo de datos. El índice actúa como un directorio que nos dice exactamente dónde buscar.

```
1 Archivo de índice:
2   "Juan" → byte 0
3   "María" → byte 256
4   "Pedro" → byte 512
5
6 Archivo de datos:
7   byte 0-255: datos de Juan
8   byte 256-511: datos de María
```

9 **byte** 512-767: `datos de Pedro`

Las búsquedas por clave son extremadamente rápidas. Podemos mantener múltiples índices sobre el mismo archivo —por ejemplo, uno por nombre y otro por ID— permitiendo consultas flexibles y eficientes. Este método es la base de los sistemas de bases de datos modernos y las bibliotecas digitales.

12.4. Directorios: Organización Jerárquica

12.4.1. Concepto de Directorio

Directorio: Archivo especial que contiene una tabla de entradas, donde cada entrada asocia un nombre de archivo con su FCB/inodo. Permite organización jerárquica del file system.

Aunque los directorios aparecen como carpetas en nuestra interfaz gráfica, en realidad son archivos con un propósito especial: contienen una tabla que mapea nombres a referencias de FCB. Cada entrada en un directorio es simplemente un par (nombre, referencia_a_FCB).

```
1 struct directory_entry {
2     char name[256];           // Nombre del archivo
3     uint32_t inode_number;    // Referencia al FCB (en Unix)
4     uint8_t file_type;        // Regular, directorio, link, etc.
5 };
```

Esta estructura tan simple permite construir jerarquías complejas. Un directorio puede contener referencias a archivos regulares y a otros directorios, formando un árbol de organización.

12.4.2. Estructuras de Directorios

12.4.2.1. Directorio de Un Nivel (Single-Level)

La estructura más simple coloca todos los archivos en un solo directorio raíz. No hay subdirectorios ni organización alguna.

```
1 / (raíz)
2 |-- programa1.c
3 |-- programa2.c
4 |-- datos.txt
5 `-- imagen.jpg
```

Este diseño es impracticable para sistemas reales. Sin capacidad de organizar archivos por categorías, los conflictos de nombres entre usuarios son inevitables. La estructura no escala cuando

tenemos miles de archivos, y encontrar un archivo específico se convierte en una tarea imposible.

12.4.2.2. Directorio de Dos Niveles

Una mejora asigna a cada usuario su propio directorio personal. Esto resuelve los conflictos de nombres, ya que dos usuarios pueden tener archivos con el mismo nombre en sus espacios separados.

```
1 /
2 |-- user1/
3 |   |-- programa.c
4 |   |-- datos.txt
5 |-- user2/
6 |   |-- programa.c (mismo nombre, diferente archivo)
7 |   |-- imagen.jpg
```

Sin embargo, sigue siendo limitado: no permite subdivisiones dentro del espacio de cada usuario. No podemos crear una estructura lógica de proyectos, documentos y código dentro de nuestro directorio personal.

12.4.2.3. Estructura Jerárquica (Árbol)

La solución moderna permite que los directorios contengan otros directorios, formando un árbol jerárquico arbitrariamente profundo. Esta estructura refleja naturalmente cómo organizamos información en el mundo real.

```
1 /
2 |-- home/
3 |   |-- alumno/
4 |       |-- documentos/
5 |           |-- apuntes.txt
6 |           |-- ejercicios.pdf
7 |       |-- proyectos/
8 |           |-- tp1.c
9 |   |-- profesor/
10 |       |-- examen.txt
11 |-- etc/
12 |   |-- config.cfg
13 |-- tmp/
14 |   |-- temporal.dat
```

Esta organización es intuitiva, escalable a millones de archivos, y permite aplicar permisos de manera jerárquica. Los namespaces quedan naturalmente separados por directorio, evitando conflictos de nombres.

12.4.2.4. Grafo Acíclico Dirigido (con Links)

Cuando permitimos hard links y soft links, la estructura deja de ser un árbol puro y se convierte en un grafo acíclico dirigido. Un mismo archivo puede ser accesible desde múltiples ubicaciones con diferentes nombres.

```
1 /home/alumno/  
2 |-- proyecto/  
3 |   |-- datos.txt (inodo 1234)  
4 |-- backup/  
5     |-- datos_respaldo.txt (hard link a inodo 1234)
```

Ambos nombres apuntan al mismo archivo físico —mismo inodo, mismos bloques de datos. No hay duplicación, simplemente múltiples caminos para llegar al mismo contenido.

Los hard links a directorios NO se permiten (excepto . y ..) porque crearían ciclos en el grafo. Un ciclo haría imposible realizar operaciones como eliminación recursiva, que quedarían atrapadas en un bucle infinito.

12.4.3. Rutas: Absolutas y Relativas

12.4.3.1. Ruta Absoluta

Ruta Absoluta: Especifica la ubicación de un archivo desde el directorio raíz. Siempre comienza con / en Unix o C : \ en Windows.

Una ruta absoluta es inequívoca: identifica exactamente un archivo sin importar desde dónde la invoquemos. `/home/alumno/documentos/apuntes.txt` siempre se refiere al mismo archivo, independientemente del directorio de trabajo actual.

Ejemplos:

```
1 /home/alumno/documentos/apuntes.txt  
2 /etc/passwd  
3 /var/log/syslog
```

Las rutas absolutas son esenciales en scripts y programas que pueden ejecutarse desde cualquier ubicación, donde no podemos asumir un directorio de trabajo específico.

12.4.3.2. Ruta Relativa

Ruta Relativa: Especifica la ubicación desde el directorio de trabajo actual. No comienza con `/`.

Las rutas relativas son más convenientes para trabajo interactivo. En lugar de escribir la ruta completa, nos movemos relativamente a nuestra posición actual. El símbolo `.` representa el directorio actual, `..` representa el directorio padre, y `~` (expandido por el shell) representa nuestro directorio home.

Ejemplos:

```
1 # Si estoy en /home/alumno/proyectos/  
2 ./datos.txt # /home/alumno/proyectos/datos.txt  
3 ../documentos/apuntes.txt # /home/alumno/documentos/apuntes.txt  
4 ../../profesor/examen.txt # /home/profesor/examen.txt
```

Notar cómo `..` nos permite “subir” en la jerarquía. Podemos encadenar múltiples `..` para navegar a directorios ancestros más lejanos.

12.4.4. Operaciones sobre Directorios

Las operaciones sobre directorios son análogas a las de archivos, pero con semántica específica. `mkdir(nombre)` crea un directorio nuevo: asigna un FCB de tipo directorio, lo inicializa con las entradas especiales `.` (referencia a sí mismo) y `..` (referencia al padre), y añade una entrada en el directorio padre.

La operación `rmdir(nombre)` elimina un directorio, pero solo si está vacío —es decir, solo contiene `.` y `..`. Esta restricción previene la pérdida accidental de contenido. Para eliminar directorios con contenido, debemos hacerlo recursivamente, eliminando primero todos los archivos y subdirectorios.

Para leer el contenido de un directorio, usamos `opendir(nombre)`, `readdir(dir)` para obtener cada entrada sucesivamente, y `closedir(dir)` para finalizar. Finalmente, `chdir(nombre)` cambia el directorio de trabajo actual del proceso, afectando cómo se resuelven las rutas relativas.

12.5. Estructuras Administrativas del SO

El sistema operativo mantiene varias estructuras en memoria para gestionar archivos abiertos eficientemente. Estas estructuras forman una jerarquía de tres niveles que separa las preocupaciones de cada proceso de las preocupaciones globales del sistema.

12.5.1. Arquitectura de Tres Niveles

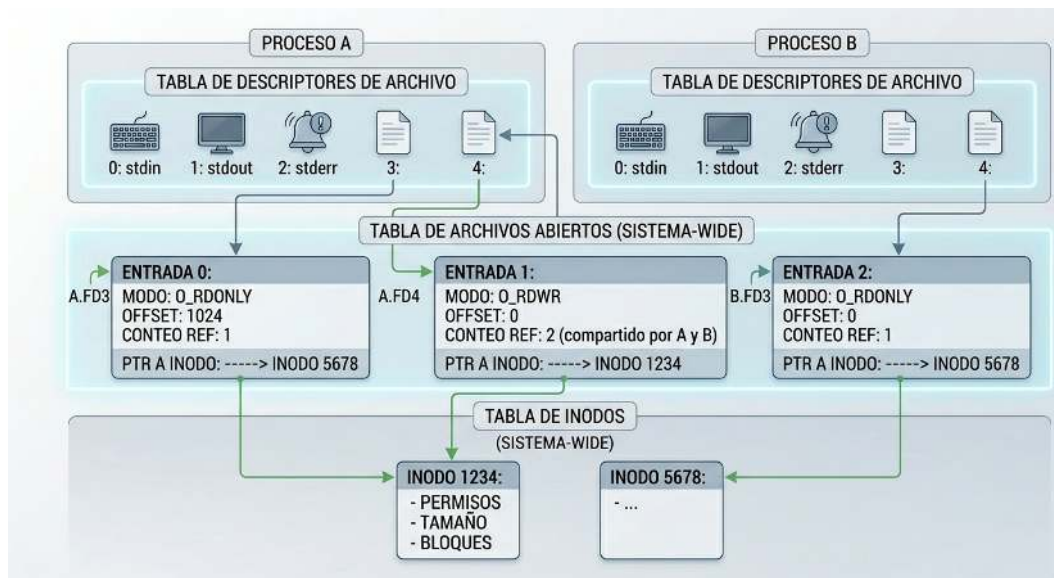


Diagrama de arquitectura de tres niveles. File descriptor, system-wide, inode table.

12.5.2. Nivel 1: File Descriptor Table (por proceso)

Cada proceso mantiene su propia tabla de descriptors de archivo. Esta tabla es privada —no compartida con otros procesos— y contiene típicamente 1024 entradas, aunque este límite puede configurarse.

```
1 struct file_descriptor_table {
2     struct file *entries[OPEN_MAX]; // Típicamente 1024 entradas
3 };
```

El índice en esta tabla es el file descriptor: el número entero que retorna `open()` y que usamos en operaciones posteriores. Cada entrada apunta a una posición en la Open File Table global. Los primeros tres descriptors están reservados: 0 para stdin (entrada estándar), 1 para stdout (salida estándar) y 2 para stderr (error estándar). Estos descriptors preconfigurados permiten que los programas lean de la consola y escriban mensajes sin necesidad de abrir archivos explícitamente.

12.5.3. Nivel 2: Open File Table (system-wide)

La Open File Table es una estructura global del sistema que mantiene información de todos los archivos abiertos por todos los procesos. A diferencia de la File Descriptor Table, esta tabla es compartida.

```

1 struct open_file_entry {
2     int mode;                // O_RDONLY, O_WRONLY, O_RDWR
3     off_t offset;            // Posición actual de lectura/escritura
4     int ref_count;           // Cantidad de descriptores apuntando aquí
5     struct inode *inode;     // Puntero al inodo del archivo
6     int flags;               // O_APPEND, O_NONBLOCK, etc.
7 };

```

Cada entrada mantiene el modo de apertura, el offset actual de lectura/escritura, un contador de referencias que indica cuántos file descriptors apuntan a esta entrada, y un puntero al inodo del archivo. Este nivel intermedio es crucial porque permite que múltiples procesos compartan el mismo offset —algo que ocurre cuando un proceso hace `fork()` después de abrir un archivo, heredando el file descriptor y compartiendo el offset con el padre. Sin embargo, diferentes procesos pueden abrir el mismo archivo con diferentes modos y offsets independientes simplemente creando entradas separadas en esta tabla.

Múltiples procesos pueden compartir la misma entrada en Open File Table si se hizo `fork()` después de abrir el archivo, o si se pasó el descriptor via `dup()` o sockets UNIX. En estos casos, el offset es verdaderamente compartido: cuando un proceso lee, el offset avanza para todos los procesos que comparten la entrada.

12.5.4. Nivel 3: Inode Table

La Inode Table es una caché en memoria de los inodos almacenados en disco. El sistema operativo mantiene aquí los inodos de todos los archivos actualmente abiertos.

```

1 struct inode {
2     mode_t mode;             // Tipo y permisos
3     uid_t uid;               // Propietario
4     gid_t gid;               // Grupo
5     off_t size;              // Tamaño en bytes
6     time_t atime, mtime, ctime; // Timestamps
7     blkcnt_t blocks;         // Bloques asignados
8     uint32_t block_ptrs[15]; // Punteros a bloques
9     int ref_count;           // Cantidad de open file entries
10    // ... más campos
11 };

```

Este nivel es global: existe un solo inodo en memoria por cada archivo, sin importar cuántos procesos lo tengan abierto o cuántas entradas existan en la Open File Table. El inodo contiene todos los metadatos persistentes del archivo y los punteros a sus bloques de datos. Los cambios en el inodo —como actualizaciones de tamaño o timestamps— eventualmente se escriben de vuelta al disco.

12.5.5. Ejemplo de Interacción

Veamos cómo interactúan estos tres niveles en un escenario concreto:

```
1 // Proceso A
2 int fd1 = open("/home/alumno/datos.txt", O_RDONLY); // fd1 = 3
3 read(fd1, buffer, 100); // offset avanza a 100
4
5 // Proceso B (independiente)
6 int fd2 = open("/home/alumno/datos.txt", O_RDONLY); // fd2 = 3
7 read(fd2, buffer, 50); // offset avanza a 50 (INDEPENDIENTE de A)
```

Aquí, ambos procesos abrieron el mismo archivo, pero sus offsets son independientes. Cada proceso tiene su propia entrada en la Open File Table, aunque ambas entradas apuntan al mismo inodo. El proceso A puede estar leyendo desde el byte 100 mientras el proceso B lee desde el byte 50. Ahora consideremos un escenario diferente:

```
1 // Proceso A hace fork()
2 pid_t pid = fork();
3 if (pid == 0) {
4     // Hijo: heredó fd1, pero COMPARTE el offset con el padre
5     read(fd1, buffer, 50); // Lee desde byte 100, offset ahora en 150
6 }
7 // Padre: el offset también está en 150 (compartido)
```

Después del `fork()`, tanto el padre como el hijo tienen un file descriptor 3 que apunta a la misma entrada en la Open File Table. Por lo tanto, comparten el offset. Cuando el hijo lee 50 bytes, el offset avanza a 150, y el padre ve este cambio porque están mirando la misma estructura.

12.5.6. File Locking: Sincronización de Acceso

Cuando múltiples procesos acceden al mismo archivo, pueden ocurrir race conditions. Si dos procesos intentan escribir simultáneamente en la misma posición, los datos podrían corromperse. El sistema operativo provee mecanismos de locking para coordinar el acceso.

12.5.6.1. Advisory Locking (flock)

El advisory locking es un mecanismo cooperativo. Los procesos voluntariamente solicitan locks antes de acceder al archivo.

```
1 #include <sys/file.h>
2
3 int fd = open("archivo.dat", O_RDWR);
4
5 // Adquirir lock exclusivo
6 if (flock(fd, LOCK_EX) == 0) {
7     // Sección crítica: solo este proceso puede escribir
8     write(fd, datos, tamaño);
```

```
9     flock(fd, LOCK_UN); // Liberar lock
10 }
```

Este tipo de lock es "cooperativo". Un proceso malicioso o mal programado puede IGNORAR el lock y escribir sin solicitarlo. Se confía en que todos los procesos respeten el protocolo. Es responsabilidad de los desarrolladores usar estos locks consistentemente.

12.5.6.2. Mandatory Locking (fcntl)

El mandatory locking es más robusto: el kernel impone los locks, bloqueando operaciones que violen restricciones activas.

```
1  #include <fcntl.h>
2
3  struct flock lock;
4  lock.l_type = F_WRLCK; // Write lock
5  lock.l_whence = SEEK_SET;
6  lock.l_start = 0; // Desde byte 0
7  lock.l_len = 1024; // Lockear 1024 bytes
8
9  // Intentar adquirir lock
10 if (fcntl(fd, F_SETLK, &lock) == -1) {
11     perror("No se pudo obtener lock");
12 } else {
13     // Tenemos el lock, trabajar con el archivo
14     write(fd, datos, 1024);
15
16     // Liberar
17     lock.l_type = F_UNLCK;
18     fcntl(fd, F_SETLK, &lock);
19 }
```

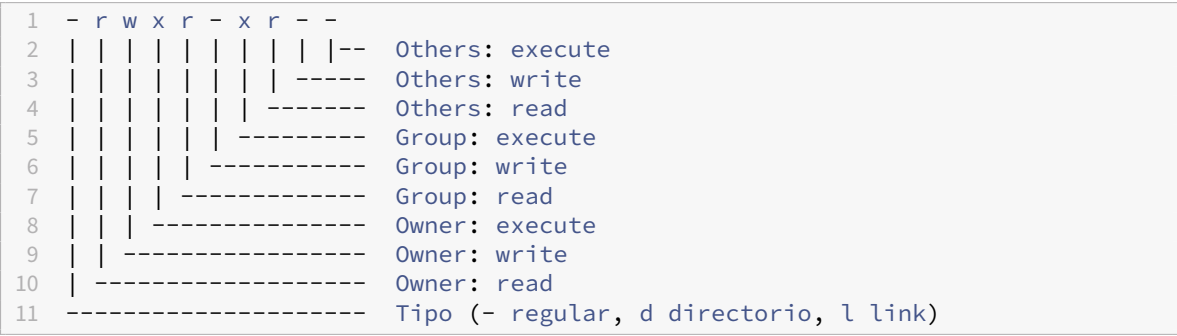
Existen dos tipos de locks: shared locks (F_RDLCK) permiten que múltiples procesos lean simultáneamente, mientras que exclusive locks (F_WRLCK) garantizan acceso exclusivo para escritura, bloqueando tanto lecturas como escrituras de otros procesos.

Los locks son esenciales en sistemas de bases de datos para lockear registros específicos, en archivos de configuración para evitar escrituras concurrentes que podrían dejar el archivo en estado inconsistente, y en logs para coordinar appends de múltiples procesos sin sobrescribir datos.

12.6. Protección y Permisos

12.6.1. Modelo de Permisos Unix

Unix implementa un modelo simple pero efectivo de control de acceso basado en 9 bits que especifican permisos para tres categorías de usuarios: el propietario (owner), el grupo (group) y todos los demás (others).



Los permisos también pueden expresarse en notación octal, donde cada grupo de tres bits forma un dígito:

Owner	Group	Others	Octal
rwX	rwX	rwX	777
rwX	r-X	r-	754
rw-	r-	r-	644
rwX	-	-	700

12.6.1.1. Significado de los Permisos

Para archivos regulares, el permiso *read* permite leer el contenido, *write* permite modificar el contenido (pero no necesariamente eliminar el archivo), y *execute* permite ejecutar el archivo como programa.

Para directorios, la semántica es diferente. El permiso *read* permite listar el contenido del directorio mediante `readdir()`, pero no acceder a los archivos dentro. El permiso *write* permite crear o eliminar archivos en el directorio —esta es una distinción crucial. El permiso *execute* permite “atravesar” el directorio, es decir, acceder a archivos dentro de él aunque no podamos listar el contenido.

Para eliminar un archivo, NO se necesita permiso de escritura sobre el archivo mismo, sino sobre el DIRECTORIO que lo contiene. El directorio es quien mantiene la lista de nombres, y eliminar un archivo es simplemente remover su entrada de esa lista.

12.6.2. Verificación de Permisos

Cuando un proceso intenta abrir un archivo, el kernel ejecuta un algoritmo de verificación que considera la identidad del proceso (UID y GID efectivos) y los permisos del archivo.

La verificación procede en orden: primero se verifica si el proceso es el propietario del archivo. Si lo es, se evalúan únicamente los bits de permiso del owner, ignorando los bits de group y others. Si el proceso no es el propietario pero pertenece al grupo del archivo, se evalúan únicamente los bits de group. Finalmente, si el proceso no es owner ni miembro del group, se evalúan los bits de others.

```

1  // Pseudocódigo simplificado
2  int check_permission(struct inode *inode, int operation) {
3      uid_t process_uid = current->uid;
4      gid_t process_gid = current->gid;
5
6      // 1. ¿El proceso es el owner?
7      if (process_uid == inode->uid) {
8          if (operation == READ && (inode->mode & 0400)) return OK;
9          if (operation == WRITE && (inode->mode & 0200)) return OK;
10         if (operation == EXEC && (inode->mode & 0100)) return OK;
11         return -EACCES; // Owner pero sin permiso específico
12     }
13
14     // 2. ¿El proceso pertenece al group?
15     if (process_gid == inode->gid) {
16         if (operation == READ && (inode->mode & 0040)) return OK;
17         if (operation == WRITE && (inode->mode & 0020)) return OK;
18         if (operation == EXEC && (inode->mode & 0010)) return OK;
19         return -EACCES;
20     }
21
22     // 3. Verificar permisos de others
23     if (operation == READ && (inode->mode & 0004)) return OK;
24     if (operation == WRITE && (inode->mode & 0002)) return OK;
25     if (operation == EXEC && (inode->mode & 0001)) return OK;
26
27     return -EACCES; // Sin permisos
28 }

```

Es importante notar que esta verificación es en cascada: si somos owner, los permisos de group y others no importan. Esto puede crear situaciones contraintuitivas donde ser el propietario de un archivo nos da *menos* acceso que si fuéramos un usuario cualquiera, si configuramos los permisos de manera inadecuada.

El usuario root (UID 0) bypassa todas las verificaciones de permisos. Root puede leer, escribir y ejecutar cualquier archivo del sistema, sin importar los permisos configurados. Esta es una de las razones por las que trabajar como root es peligroso: no hay red de seguridad que prevenga errores.

12.6.3. Umask: Permisos por Defecto

Cuando creamos un archivo nuevo, el sistema debe decidir qué permisos asignarle. Aquí entra en juego el **umask**.

Umask: Máscara que especifica qué permisos NO otorgar al crear archivos nuevos. Se resta de los permisos solicitados mediante una operación AND con el complemento.

```
1 // Si umask = 0022 (octal)
2 // Permisos solicitados: 0666 (rw-rw-rw-)
3 // Permisos finales: 0666 & ~0022 = 0644 (rw-r--r--)
4
5 int fd = open("nuevo.txt", O_CREAT | O_WRONLY, 0666);
6 // El archivo se crea con permisos 0644 debido al umask
```

Los umask típicos son 0022 (usuario puede escribir, grupo y others solo leer, resultando en archivos 644 y directorios 755), 0002 (usuario y grupo pueden escribir, others solo leer, resultando en archivos 664 y directorios 775), y 0077 (solo el usuario tiene acceso, resultando en archivos 600 y directorios 700, máxima privacidad).

12.6.4. Cambio de Permisos

Los permisos pueden modificarse después de crear un archivo mediante las system calls `chmod()` y `chown()`.

```
1 #include <sys/stat.h>
2
3 // Cambiar permisos de un archivo
4 chmod("archivo.txt", 0644); // rw-r--r--
5
6 // Cambiar propietario (requiere privilegios)
7 chown("archivo.txt", 1000, 1000); // UID 1000, GID 1000
```

Desde la shell, estos comandos tienen sintaxis conveniente:

```
1 chmod 755 script.sh      # rwxr-xr-x
2 chmod u+x programa.c     # Agregar ejecución para owner
3 chmod go-w archivo.txt   # Quitar escritura a group y others
4 chown alumno:estudiantes datos.txt
```

La notación simbólica (`u+x`, `go-w`) es más legible para cambios incrementales, mientras que la notación octal (`755`) es más directa para configurar permisos completos de una vez.

12.7. Implementación Física: Disco

Hasta ahora vimos la abstracción lógica de archivos y directorios. Ahora bajamos al nivel físico para entender cómo se almacenan realmente los datos en el disco y cómo el file system traduce operaciones lógicas en accesos físicos.

12.7.1. Hardware: Sectores vs Bloques

12.7.1.1. Sectores (Físico)

Sector: Unidad mínima de transferencia de datos en el hardware del disco. Es una característica física impuesta por la controladora, no por el software.

Los discos históricos usaban sectores de 512 bytes, un estándar que dominó desde los años 1980 hasta 2010. Los discos modernos y SSD han migrado al formato Advanced Format con sectores de 4096 bytes (4 KiB), mejorando eficiencia y reduciendo overhead.

El hardware del disco solo puede leer o escribir sectores completos. No existe forma de acceder a un byte individual sin leer todo el sector que lo contiene. Esta es una restricción fundamental del hardware que el file system debe manejar.

12.7.1.2. Bloques (Lógico)

Bloque (o Cluster): Unidad mínima de asignación utilizada por el file system. Agrupa uno o más sectores consecutivos. Es una abstracción del software, configurable al formatear el disco.

El tamaño del bloque se elige al formatear el sistema de archivos. Si tenemos sectores de 512 bytes y elegimos bloques de 4 KiB, cada bloque agrupa 8 sectores consecutivos. Si tenemos sectores de 4 KiB y elegimos bloques de 4 KiB, hay correspondencia uno a uno.

Aspecto	Sector	Bloque
Naturaleza	Física (hardware)	Lógica (software)
Tamaño	Fijo por el disco	Configurable al formatear

Aspecto	Sector	Bloque
Usado por	Controladora de disco	Sistema de archivos
Granularidad	Operaciones de bajo nivel	Asignación de archivos

12.7.2. ¿Por qué usar bloques en vez de sectores directamente?

La abstracción de bloques ofrece varias ventajas. Reduce la fragmentación al trabajar con unidades más grandes, disminuye la cantidad de entradas en estructuras administrativas (menos punteros que mantener), mejora el rendimiento al leer o escribir múltiples sectores de una sola vez, y simplifica la gestión de espacio libre.

Un archivo de 100 KiB con bloques de 4 KiB requiere $100 / 4 = 25$ bloques. Si usáramos sectores de 512 bytes directamente, necesitaríamos $100 \times 1024 / 512 = 200$ sectores. El sistema debe mantener 25 punteros en vez de 200, reduciendo significativamente el overhead de metadatos.

12.7.3. Fragmentación Interna

El uso de bloques introduce un problema conocido como fragmentación interna. Si un archivo ocupa 5 KiB pero los bloques son de 4 KiB, debemos asignar 2 bloques completos (8 KiB), desperdiciando 3 KiB que no pueden ser usados por otros archivos. Este espacio queda atrapado dentro del bloque asignado.

El desperdicio promedio es la mitad del tamaño del bloque por archivo. Con bloques de 4 KiB, un archivo de 1 byte desperdicia aproximadamente 4095 bytes, y un archivo de 4097 bytes también desperdicia aproximadamente 4095 bytes. En promedio, cada archivo desperdicia unos 2 KiB.

Existe un trade-off fundamental en la elección del tamaño de bloque:

Tamaño	Fragmentación interna	Overhead metadatos	Rendimiento
1 KiB	Baja	Alto (muchos punteros)	Bajo
4 KiB	Media	Medio	Medio
8 KiB	Alta	Bajo	Alto (archivos grandes)
64 KiB	Muy alta	Muy bajo	Muy alto (streaming)

La elección típica de 4 KiB representa un buen balance para uso general, alineándose además con el tamaño de página de memoria en arquitecturas modernas.

12.8. Métodos de Asignación de Espacio

El file system debe decidir cómo asignar bloques físicos del disco a los archivos lógicos. Esta decisión afecta el rendimiento, la fragmentación y la complejidad del sistema. Existen tres enfoques principales, cada uno con sus ventajas y desventajas.

12.8.1. Método 1: Asignación Contigua

En la asignación contigua, todos los bloques de un archivo se almacenan en posiciones consecutivas del disco, uno detrás del otro sin interrupciones.

Para representar un archivo solo necesitamos dos números: el bloque inicial y la cantidad de bloques. Esta simplicidad es atractiva y hace que el acceso secuencial sea extremadamente rápido —el brazo del disco puede leer todos los bloques sin moverse. El acceso aleatorio también es eficiente porque podemos calcular directamente la posición física de cualquier byte.

Sin embargo, los problemas aparecen con el uso. La fragmentación externa se vuelve severa con el tiempo: después de crear y eliminar muchos archivos, el disco queda lleno de huecos pequeños que no pueden utilizarse eficientemente. Hacer crecer un archivo es difícil o imposible si no hay espacio contiguo después del último bloque —podríamos necesitar mover TODO el archivo a otra ubicación. Esto requiere compactación periódica del disco, una operación costosa que mueve archivos para eliminar los huecos.

La asignación contigua es ideal para medios de solo lectura como CD-ROM y DVD, donde los archivos nunca crecen ni se eliminan. En estos contextos, sus ventajas de rendimiento brillan sin sufrir sus desventajas de fragmentación.

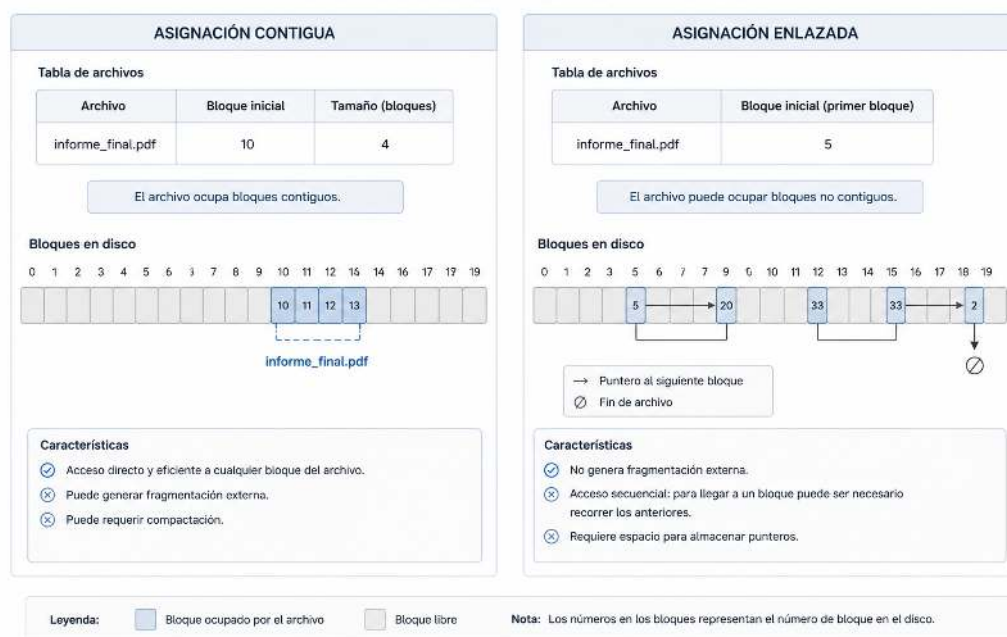
12.8.2. Método 2: Asignación Enlazada

En la asignación enlazada, cada bloque contiene datos y un puntero al siguiente bloque, formando una lista enlazada dispersa por el disco.

Este método elimina completamente la fragmentación externa —cualquier bloque libre puede ser usado— y los archivos pueden crecer dinámicamente sin necesidad de mover datos existentes. Solo necesitamos guardar el bloque inicial en los metadatos del archivo; el resto de la información de ubicación está distribuida en los punteros.

Los problemas surgen con el acceso aleatorio. Para leer el byte 10,000 de un archivo con bloques de 4 KiB, debemos calcular que está en el bloque número 2 ($10,000 / 4096$). Pero para llegar al bloque 2, debemos leer el bloque 0, seguir su puntero al bloque 1, seguir ese puntero al bloque 2, y finalmente leer los datos. Son 3 accesos a disco en vez de 1. La pérdida de un puntero puede corromper el resto del archivo, y parte del espacio de cada bloque se desperdicia almacenando el puntero en vez de datos.

MÉTODOS DE ASIGNACIÓN DE ARCHIVOS



12.8.2.1. Mejora: FAT (File Allocation Table)

FAT mejora la asignación enlazada moviendo todos los punteros a una tabla centralizada en memoria.

```

1  Tabla FAT:
2  Índice  Valor
3      45 →   78
4      78 →   12
5      12 → EOF

```

Ahora los bloques de datos quedan completos —no contienen punteros— y la tabla FAT se cachea en memoria para acceso rápido. La pérdida de datos en un bloque solo afecta su contenido, no la cadena de punteros completa. La recuperación ante corrupción es más fácil porque la estructura crítica está separada de los datos.

Este es el método usado por el File System FAT de Microsoft, que veremos en detalle en secciones posteriores.

12.8.3. Método 3: Asignación Indexada

La asignación indexada usa un bloque especial (bloque de índice) que contiene punteros a todos los bloques de datos del archivo.



El acceso aleatorio es eficiente: para leer el byte 10,000 con bloques de 4 KiB, calculamos que está en el bloque 2, leemos el bloque de índice, obtenemos el puntero en la posición 2 (que es 789), y leemos el bloque 789. Son solo 2 accesos a disco. No hay fragmentación externa, y toda la información de ubicación está consolidada en un solo lugar, facilitando operaciones como truncar o eliminar el archivo.

El costo es el overhead del bloque de índice —archivos pequeños desperdician la mayor parte de este bloque— y el tamaño máximo de archivo está limitado por cuántos punteros caben en un bloque. Con bloques de 4 KiB y punteros de 4 bytes, caben 1024 punteros, limitando el tamaño máximo a $1024 \times 4 \text{ KiB} = 4 \text{ MiB}$. Para archivos más grandes, se usan índices multinivel, una técnica que veremos al estudiar EXT2.

12.8.4. Comparación de Métodos

Aspecto	Contigua	Enlazada (FAT)	Indexada
Fragmentación externa	Alta	Ninguna	Ninguna
Tamaño dinámico	No	Sí	Sí
Acceso secuencial	Excelente	Bueno	Bueno
Acceso aleatorio	Excelente	Muy malo	Excelente
Overhead metadatos	Mínimo	Medio (tabla FAT)	Alto (bloques índice)
Confiabilidad	Alta	Media	Alta

Aspecto	Contigua	Enlazada (FAT)	Indexada
Complejidad	Muy simple	Simple	Moderada

12.9. Manejo de Espacio Libre

Además de rastrear qué bloques pertenecen a qué archivos, el file system debe mantener información sobre qué bloques están libres y disponibles para asignación. La eficiencia de esta estructura impacta directamente el rendimiento de creación de archivos y escrituras que requieren bloques adicionales.

12.9.1. Método 1: Bitmap de Bloques

Un bitmap es un array de bits donde cada bit representa un bloque del disco. Si el bit está en 1, el bloque está ocupado; si está en 0, está libre.

```

1 Bitmap (1 = ocupado, 0 = libre):
2 [1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 0 ...]
3  0 1 2 3 4 5 6 7 8 9...
4
5 Bloque 0: ocupado
6 Bloque 4: libre
7 Bloque 5: libre

```

Para buscar un bloque libre, recorremos el bitmap buscando el primer bit en 0. Cuando lo encontramos, lo cambiamos a 1 para marcarlo como ocupado. Para liberar un bloque, simplemente cambiamos su bit a 0.

```

1 // Buscar primer bit en 0
2 for (int i = 0; i < total_bloques; i++) {
3     if (bitmap[i / 8] & (1 << (i % 8)) == 0) {
4         // Bloque i está libre
5         bitmap[i / 8] |= (1 << (i % 8)); // Marcar como ocupado
6         return i;
7     }
8 }
9
10 // Liberar bloque
11 bitmap[bloque / 8] &= ~(1 << (bloque % 8));

```

El bitmap es extremadamente compacto —solo 1 bit por bloque— y buscar bloques contiguos es relativamente fácil (buscamos secuencias de bits en 0). Para un disco de 500 GiB con bloques de 4 KiB, tenemos 131.072.000 bloques, y el bitmap ocupa aproximadamente 16 MiB. Este overhead es razonable y el bitmap típicamente se mantiene cacheado en memoria.

12.9.2. Método 2: Lista Enlazada de Bloques Libres

En este método, los bloques libres forman una lista enlazada donde cada bloque libre contiene el número del siguiente bloque libre.

```
1 Head → Bloque 45 → Bloque 78 → Bloque 12 → NULL
```

Este enfoque es ineficiente para encontrar bloques contiguos y requiere accesos a disco para explorar la lista. Parte del espacio en los bloques libres se desperdicia almacenando punteros. Este método era común en sistemas antiguos pero ha sido mayormente abandonado en favor de bitmaps.

12.9.3. Método 3: Agrupamiento

El agrupamiento combina ideas de bitmap y lista: cada entrada de la lista apunta a un grupo de bloques contiguos.

```
1 Head → [45-48] → [78-82] → [100-103] → NULL
```

Esto reduce la cantidad de punteros necesarios y facilita encontrar bloques contiguos cuando los necesitamos.

12.9.4. Método Usado en Práctica

Los file systems modernos han convergido en soluciones eficientes. EXT2/3/4 usa bitmap de bloques por su simplicidad y eficiencia. FAT reutiliza la misma tabla FAT (una entrada con valor 0x0000 indica bloque libre). NTFS usa bitmap de bloques junto con su Master File Table (MFT).

12.10. Seguridad y Recuperación

12.10.1. Journaling: Registro de Transacciones

Journaling: Técnica que registra cambios ANTES de aplicarlos al file system. Si ocurre un fallo durante una operación, el sistema puede usar el journal para completar o deshacer operaciones incompletas, manteniendo consistencia.

Sin journaling, un corte de energía durante una operación puede dejar el file system en estado inconsistente. Supongamos que estamos escribiendo un archivo nuevo. El proceso requiere actualizar el bitmap (marcar bloques como usados), crear o actualizar el inodo, actualizar el directorio padre y escribir los datos. Si el sistema se apaga después de actualizar el bitmap pero antes de

escribir el inodo, tenemos bloques marcados como usados que no pertenecen a ningún archivo —espacio perdido que ni siquiera podemos recuperar fácilmente.

El journaling resuelve este problema registrando nuestras intenciones antes de ejecutarlas. Primero escribimos al journal: “Voy a asignar bloque 500 a inodo 1234, actualizar directorio /home/user”. Luego marcamos esta operación como COMMIT —lista para aplicar. Después aplicamos los cambios reales al bitmap, inodo, directorio y datos. Finalmente, marcamos la operación como CLEANUP —completada exitosamente.

Si el sistema se apaga durante el paso de aplicar cambios, al reiniciar el sistema operativo lee el journal. Ve que hay una operación con COMMIT pero sin CLEANUP, lo que indica que comenzó pero no terminó. El sistema vuelve a aplicar los cambios —las operaciones son idempotentes, aplicarlas dos veces produce el mismo resultado que una vez. El file system queda consistente sin pérdida de datos.

Existen diferentes niveles de journaling con diferentes garantías de seguridad y costos de rendimiento. El modo *metadata only* solo registra cambios en inodos, directorios y bitmaps, ofreciendo alto rendimiento pero protección media. El modo *ordered* registra metadata y además garantiza que los datos se escriben ANTES del commit, evitando que un archivo con tamaño actualizado contenga basura en los bloques recién asignados. El modo *full data* registra tanto metadata como el contenido completo de los archivos, ofreciendo máxima seguridad a costa de rendimiento significativamente menor.

La ventaja principal del journaling es la recuperación casi instantánea después de un crash. En lugar de escanear todo el disco con fsck (lo que puede tardar horas en discos grandes), simplemente reproducimos las entradas del journal (operación que toma segundos).

Los file systems modernos implementan journaling: EXT3, EXT4, XFS, NTFS, HFS+ y Btrfs todos proveen esta protección.

12.10.2. fsck / chkdsk: File System Consistency Check

fsck (File System Consistency Check): Herramienta que verifica y repara inconsistencias en un file system NO montado. Escanea inodos, directorios, bitmaps y bloques de datos buscando y corrigiendo errores estructurales.

fsck se ejecuta después de un apagado incorrecto en sistemas sin journaling, periódicamente cada N montajes o M días como mantenimiento preventivo, o manualmente cuando se detecta corrupción. La herramienta debe ejecutarse con el file system desmontado para evitar modificaciones durante la verificación.

La operación de `fsck` en EXT2 procede en cinco fases. La Fase 1 verifica inodos: que la estructura de cada inodo sea válida, que los bloques referenciados estén dentro del rango del disco, y que el tipo de archivo sea válido. La Fase 2 verifica directorios: que las entradas apunten a inodos válidos, que no existan ciclos en la estructura, y que cada directorio tenga las entradas especiales `.` y `..` correctas.

La Fase 3 verifica conectividad: todos los inodos marcados como usados deben ser accesibles desde el directorio raíz. Los inodos huérfanos —que existen pero no tienen entrada de directorio— se mueven a `/lost+found` donde el usuario puede inspeccionarlos. La Fase 4 verifica contadores: el campo `links_count` de cada inodo debe coincidir con la cantidad real de entradas de directorio que apuntan a él. Si no coincide, se corrige.

Finalmente, la Fase 5 verifica bitmaps: el bitmap de bloques debe coincidir exactamente con qué bloques están realmente en uso, el bitmap de inodos debe coincidir con qué inodos están usados, y los contadores de espacio libre en el superbloque deben ser correctos.

Problemas comunes que `fsck` detecta y repara:

- "Bloque X reclamado por múltiples inodos" → hay que elegir a cuál pertenece
- "Inodo Y no tiene entrada de directorio" → mover a `/lost+found`
- "Directorio Z referencia inodo inexistente" → eliminar entrada corrupta
- "links_count incorrecto en inodo W" → corregir contador
- "Bloques marcados como libres pero en uso" → corregir bitmap

Las desventajas de `fsck` son significativas. En discos grandes de varios terabytes, la verificación completa puede tardar HORAS. El file system debe desmontarse durante la operación, dejándolo offline. Además, `fsck` solo detecta y repara inconsistencias estructurales; no puede recuperar datos que se perdieron físicamente.

En Windows, el equivalente es `chkdsk` (Check Disk):

```
1 # Linux
2 fsck /dev/sda1      # Verificar partición
3 fsck -y /dev/sda1   # Auto-reparar sin preguntar
4
5 # Windows
6 chkdsk C:           # Verificar
7 chkdsk C: /F        # Reparar
```

12.11. Caso de Estudio 1: FAT (File Allocation Table)

FAT es uno de los sistemas de archivos más simples y ampliamente soportados. Creado por Microsoft para MS-DOS entre 1977 y 1980, sigue siendo el estándar de facto en dispositivos portátiles como pendrives y tarjetas SD.

12.11.1. Características Generales

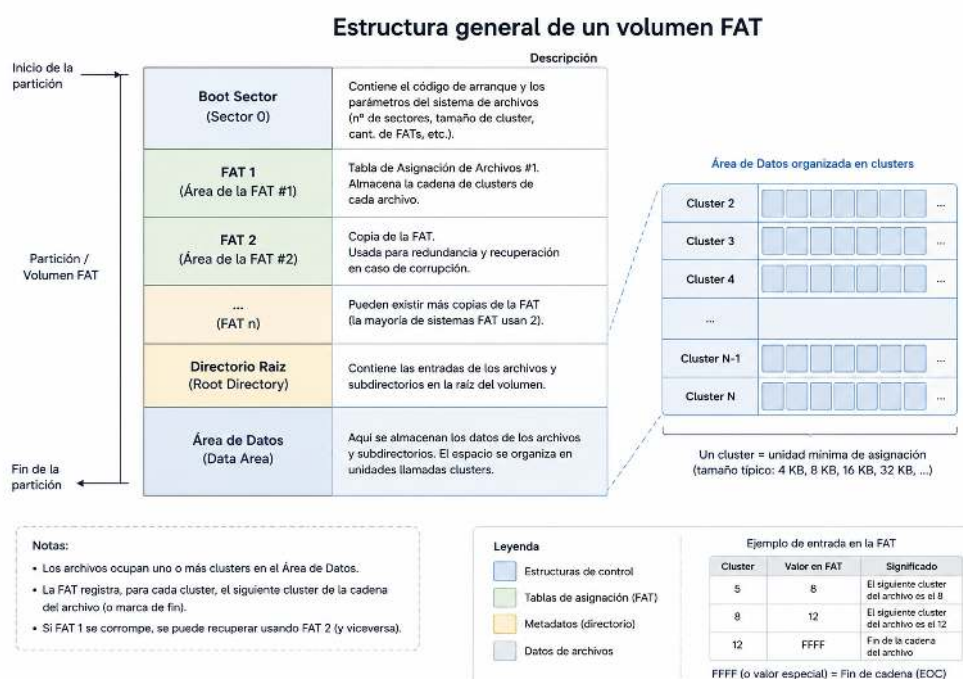
Antes de sumergirnos en los detalles técnicos, es crucial entender una diferencia conceptual fundamental que distingue a FAT de sistemas más sofisticados como EXT2.

En FAT NO existe el concepto de FCB/inodo como estructura separada. La entrada del directorio contiene DIRECTAMENTE todos los metadatos del archivo: nombre, tamaño, primer cluster, atributos y timestamps. La tabla FAT solo contiene información de enlazamiento entre clusters, no metadatos completos.

Esta arquitectura unificada hace que FAT sea más simple de implementar, pero también más limitado en funcionalidad. No hay separación entre la identidad del archivo y su ubicación en el directorio, lo que imposibilita características como hard links.

12.11.2. Componentes de FAT

El volumen FAT se estructura en cinco regiones distintas, cada una con un propósito específico:



12.11.2.1. 1. Boot Sector

El primer sector del volumen contiene los parámetros fundamentales del file system. Aquí se especifican los bytes por sector (típicamente 512 o 4096), los sectores por cluster (siempre una potencia

de 2), la cantidad de tablas FAT (típicamente 2 para redundancia), el tamaño de las tablas FAT y el tipo específico de FAT (12, 16 o 32 bits).

12.11.2.2. 2. Tabla FAT

La tabla FAT es esencialmente un array de entradas donde cada índice representa un cluster del disco.

Esta tabla implementa el método de asignación enlazada que estudiamos anteriormente.

1	Índice	Valor	Significado
2	-----	-----	-----
3	0	Reservado	Media descriptor
4	1	Reservado	Marcador de limpieza
5	2	0x0003	Cluster 2 apunta a cluster 3
6	3	0x0004	Cluster 3 apunta a cluster 4
7	4	0xFFFF	Fin de cadena (último cluster)
8	5	0x0000	Cluster libre
9	6	0x0008	Cluster 6 apunta a cluster 8
10	7	0xFFFF7	Bad cluster (sector defectuoso)
11	8	0xFFFF	Fin de cadena

Los valores en la tabla tienen significados específicos. El valor 0x0000 indica un cluster libre disponible para asignación. Valores entre 0x0002 y 0xFFEF son punteros que indican el siguiente cluster del archivo. El valor 0xFFF7 marca un bad cluster —un sector físicamente defectuoso que debe evitarse— y valores entre 0xFFF8 y 0xFFFF marcan el final de una cadena de clusters.

La tabla FAT se mantiene duplicada (FAT 1 y FAT 2) para proporcionar redundancia básica. Si la primera copia se corrompe, el sistema puede recuperar usando la segunda. Esta es una de las pocas características de confiabilidad de FAT.

12.11.2.3. 3. Entrada de Directorio

Cada archivo o subdirectorio tiene una entrada de exactamente 32 bytes en su directorio padre. Esta estructura compacta contiene toda la información necesaria para acceder al archivo.

```

1 struct fat_directory_entry {
2     char name[8];           // Nombre (sin extensión)
3     char extension[3];      // Extensión (sin el punto)
4     uint8_t attributes;     // Ver atributos abajo
5     uint8_t reserved[10];   // Reservado (case, timestamps extendidos)
6     uint16_t time;          // Hora de última modificación
7     uint16_t date;          // Fecha de última modificación
8     uint16_t first_cluster; // Número del primer cluster (FAT16)
9     uint32_t file_size;      // Tamaño en bytes
10 }; // Total: 32 bytes

```

El campo `attributes` es un byte de flags donde cada bit tiene un significado específico:

Bit	Nombre	Significado
0	READ_ONLY	Archivo de solo lectura
1	HIDDEN	Archivo oculto
2	SYSTEM	Archivo de sistema
3	VOLUME_LABEL	Etiqueta del volumen (no es archivo)
4	DIRECTORY	Es un subdirectorio
5	ARCHIVE	Modificado desde último backup

La limitación histórica más notoria de FAT es el esquema de nombres 8.3: ocho caracteres para el nombre más tres para la extensión. Nombres más largos se truncan usando una notación con tilde: “documento.txt” se convierte en “DOCUMENT~1.TXT”, y “mi archivo largo.doc” en “MIARCH~1.DOC”.

FAT32 y las extensiones VFAT agregaron soporte para nombres largos usando entradas de directorio adicionales especialmente codificadas. Sin embargo, internamente el sistema sigue siendo 8.3, y cada nombre largo requiere múltiples entradas de 32 bytes encadenadas.

12.11.3. Concepto de Cluster

Cluster (en FAT): Unidad mínima de asignación. Agrupa varios sectores consecutivos. El tamaño del cluster depende del tamaño de la partición y se elige al formatear.

Por ejemplo, si los sectores son de 512 bytes y elegimos clusters de 4 KiB, cada cluster agrupa 8 sectores consecutivos. Esta agrupación reduce el overhead de la tabla FAT —menos entradas que mantener— pero aumenta la fragmentación interna, especialmente en archivos pequeños.

12.11.4. FAT12, FAT16, FAT32: Evolución

Las tres variantes de FAT difieren principalmente en el tamaño de las entradas de la tabla FAT, lo que determina cuántos clusters pueden direccionar:

Versión	Bits/entrada	Clusters máximos	Tamaño máx partición
FAT12	12 bits	4.096 (2^{12})	~32 MiB (clusters de 8 KiB)
FAT16	16 bits	65.536 (2^{16})	~2 GiB (clusters de 32 KiB)

Versión	Bits/entrada	Clusters máximos	Tamaño máx partición
FAT32	32 bits (28 usados)	~268 millones	~2 TiB (clusters de 32 KiB)

FAT12 dominó la era de los disquetes de 1.44 MiB en los años 1980 y 1990. FAT16 fue el estándar para discos duros pequeños hasta 2 GiB en MS-DOS y Windows 95. FAT32, introducido con Windows 95 OSR2 en 1996, sigue siendo el estándar actual en pendrives y tarjetas SD por su compatibilidad universal.

Aunque una partición FAT32 puede ser de 2 TiB, el sistema NO puede almacenar archivos individuales mayores a 4 GiB. Esta limitación proviene del campo `file_size` de 32 bits en la entrada de directorio: 2^{32} bytes = 4 GiB exactos. Intentar copiar un archivo de 5 GiB a un pendrive FAT32 fallará, independientemente del espacio libre disponible.

12.11.5. Ejemplo de Operación: Lectura de Archivo

Veamos paso a paso cómo FAT lee un archivo, siguiendo la cadena de clusters. Consideremos un archivo “documento.txt” de 12 KiB, con clusters de 4 KiB y primer cluster en la posición 245.

El proceso comienza buscando en el directorio padre. Leemos el directorio, encontramos la entrada con `name="DOCUMENT~1"` y `extension="TXT"`, y obtenemos `first_cluster = 245`.

Ahora leemos el primer cluster de 4 KiB accediendo al cluster 245 en el área de datos y copiando 4 KiB a memoria. Consultamos la tabla FAT: `FAT[245] = 246`, lo que indica que hay más datos en el cluster 246.

Leemos el segundo cluster accediendo al cluster 246, acumulando 8 KiB. Consultamos nuevamente: `FAT[246] = 247`, todavía hay más datos.

Leemos el tercer cluster en la posición 247, completando los 12 KiB del archivo. La consulta final `FAT[247] = 0xFFFF` indica EOF (end of file), señalando que hemos llegado al final.

El costo total es aproximadamente 4 accesos reales a disco: uno para el directorio más tres para los clusters de datos. Las tres consultas a la tabla FAT típicamente se resuelven desde memoria, ya que el sistema operativo mantiene la tabla FAT cacheada en RAM para mejorar el rendimiento.

Para un archivo de 100 KiB con clusters de 4 KiB, necesitaríamos seguir 25 punteros en la tabla FAT. Si todos estos punteros están en memoria (lo cual es típico), el costo es solo 25 lecturas de clusters de datos, no 50 operaciones de I/O.

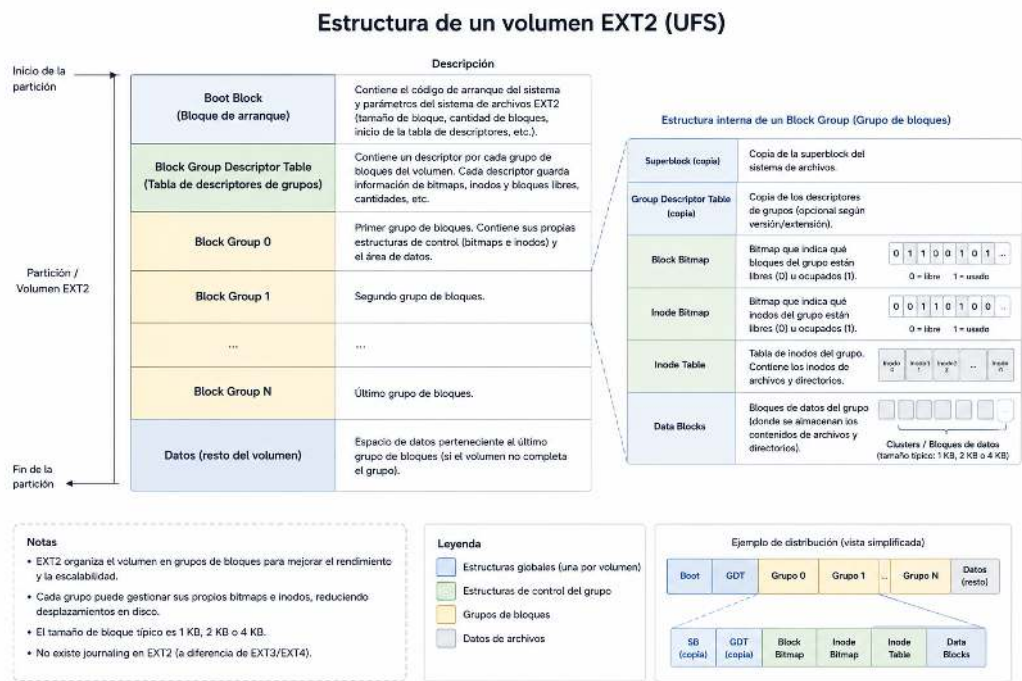
12.12. Caso de Estudio 2: EXT2 / UFS (Unix File System)

EXT2 (Second Extended File System) es el sistema de archivos clásico de Linux, basado conceptualmente en UFS de BSD Unix. Representa una evolución significativa sobre FAT, introduciendo características que soportan entornos multiusuario robustos y archivos de gran tamaño.

12.12.1. Filosofía de Diseño

La diferencia arquitectural fundamental con FAT es la separación clara entre metadatos y nombres. Los **inodos** almacenan todos los metadatos del archivo excepto su nombre: permisos, propietario, tamaños, timestamps y punteros a bloques de datos. Los **directorios** solo almacenan pares simples de (nombre, número_de_inodo).

Esta separación elegante permite funcionalidades que FAT no puede ofrecer. Los hard links se vuelven triviales: múltiples nombres en diferentes directorios pueden referenciar el mismo número de inodo. Los permisos robustos de Unix (owner, group, others con lectura, escritura y ejecución) se almacenan naturalmente en el inodo. La información extendida puede agregarse al inodo sin modificar las entradas de directorio, manteniendo compatibilidad hacia atrás.



12.12.2. Estructura General: Block Groups

EXT2 divide el volumen completo en **grupos de bloques** (block groups) para mejorar la localidad de los datos.

La organización en block groups no es arbitraria. Esta estructura mejora la localidad manteniendo el inodo de un archivo y sus bloques de datos cerca físicamente, típicamente en el mismo grupo. Proporciona escalabilidad mediante una estructura repetida que permite volúmenes enormes sin overhead excesivo. Aumenta la confiabilidad replicando el superbloque crítico en múltiples grupos. Y reduce la fragmentación manteniendo juntos los archivos de un mismo directorio.

12.12.3. Componentes de un Block Group

Cada grupo de bloques contiene una estructura idéntica con cinco componentes:

12.12.3.1. 1. Superbloque

Superbloque: Estructura crítica que contiene metadatos globales del file system. Se replica en varios block groups para redundancia, permitiendo recuperación si el superbloque principal se corrompe.

La estructura del superbloque mantiene información esencial del sistema de archivos:

```
1 struct ext2_superblock {
2     uint32_t s_inodes_count;      // Total de inodos
3     uint32_t s_blocks_count;     // Total de bloques
4     uint32_t s_free_blocks_count; // Bloques libres
5     uint32_t s_free_inodes_count; // Inodos libres
6     uint32_t s_first_data_block; // Primer bloque de datos (0 o 1)
7     uint32_t s_log_block_size;   // Tamaño de bloque (log2(size) - 10)
8     uint32_t s_blocks_per_group; // Bloques por grupo
9     uint32_t s_inodes_per_group; // Inodos por grupo
10    uint16_t s_magic;             // Magic number: 0xEF53
11    // ... más campos (mtime, state, creator_os, etc.)
12 };
```

El magic number 0xEF53 identifica inequívocamente un volumen EXT2, permitiendo al sistema operativo detectar el tipo de file system al montar. El superbloque se copia estratégicamente en los grupos 0, 1 y en grupos cuyo número es potencia de 3, 5 o 7, balanceando redundancia con overhead de espacio.

12.12.3.2. 2. Group Descriptor Table (GDT)

La GDT es un array que describe cada block group del sistema. Cada entrada contiene punteros a las estructuras administrativas del grupo:

```
1 struct ext2_group_desc {
2     uint32_t bg_block_bitmap;    // Bloque que contiene bitmap de bloques
3     uint32_t bg_inode_bitmap;    // Bloque que contiene bitmap de inodos
4     uint32_t bg_inode_table;     // Primer bloque de la inode table
```

```

5     uint16_t bg_free_blocks_count; // Bloques libres en este grupo
6     uint16_t bg_free_inodes_count; // Inodos libres en este grupo
7     uint16_t bg_used_dirs_count;   // Cantidad de directorios en el grupo
8     uint16_t bg_pad;               // Padding para alineación
9 };

```

12.12.3.3. 3. Bitmaps

EXT2 usa bitmaps para rastrear espacio libre, una técnica eficiente que vimos anteriormente. El **block bitmap** dedica 1 bit por bloque del grupo: 1 indica ocupado, 0 indica libre. El **inode bitmap** usa la misma técnica para los inodos del grupo.

```

1 Ejemplo: [1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 0]
2           Bloques 0-3: ocupados
3           Bloques 4-6: libres
4           Bloque 7: ocupado

```

Las operaciones son simples y eficientes. Para buscar un bloque o inodo libre, escaneamos el bitmap hasta encontrar un bit en 0. Para asignar, cambiamos el bit a 1 y decrementamos el contador correspondiente en el superbloque y el GDT. Para liberar, cambiamos el bit a 0 e incrementamos los contadores.

12.12.3.4. 4. Inode Table

La tabla de inodos es un array de estructuras inodo. Cada grupo tiene su propia tabla con `inodes_per_group` entradas. El tamaño típico de cada inodo es 128 bytes, aunque puede configurarse a 256 bytes para soportar características extendidas.

12.12.3.5. 5. Data Blocks

El resto del grupo está dedicado a data blocks: los bloques donde realmente se almacena el contenido de archivos y directorios. Esta es la porción más grande de cada grupo, representando la mayor parte del espacio utilizable.

12.12.4. El Inodo: Corazón de EXT2

Inodo (Index Node): Estructura de datos que almacena todos los metadatos de un archivo EXCEPTO su nombre. Contiene permisos, tamaños, timestamps y los punteros cruciales a los bloques de datos.

La estructura completa del inodo es:

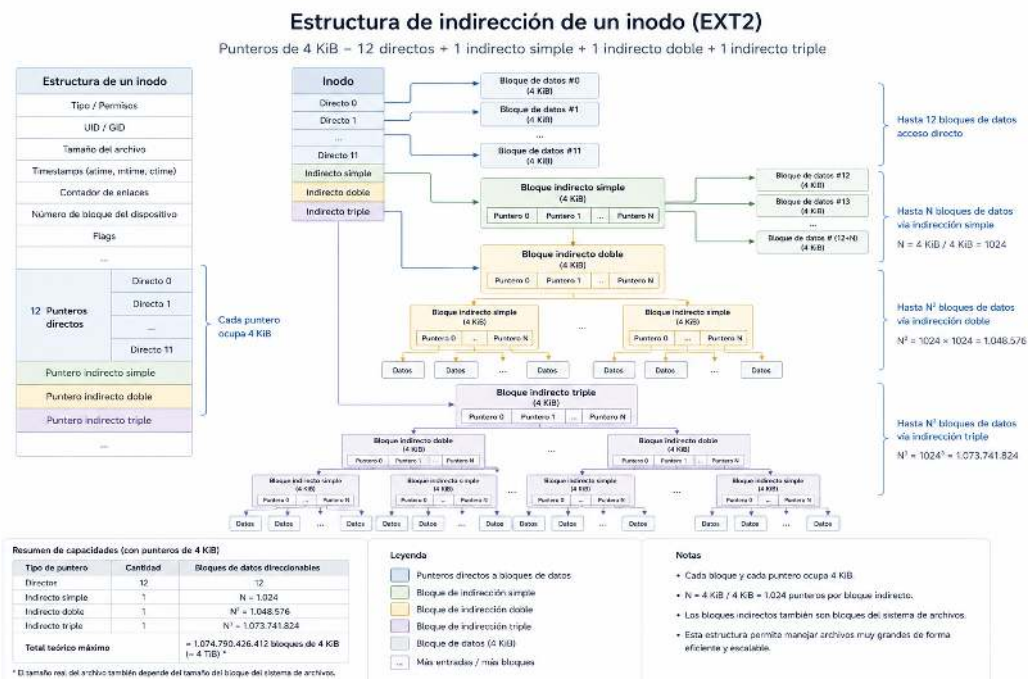
```

1 struct ext2_inode {
2     uint16_t i_mode;           // Tipo de archivo y permisos
3     uint16_t i_uid;           // User ID del propietario
4     uint32_t i_size;          // Tamaño en bytes (límite 4 GiB en EXT2)
5     uint32_t i_atime;         // Tiempo de último acceso
6     uint32_t i_ctime;         // Tiempo de creación/cambio de metadatos
7     uint32_t i_mtime;         // Tiempo de última modificación de datos
8     uint32_t i_dtime;         // Tiempo de eliminación
9     uint16_t i_gid;           // Group ID
10    uint16_t i_links_count;    // Cantidad de hard links
11    uint32_t i_blocks;         // Cantidad de bloques (en unidades de 512
                                // bytes)
12    uint32_t i_flags;          // Flags del archivo
13
14    // Punteros a bloques de datos (15 punteros)
15    uint32_t i_block[15];      // [0-11] directos, [12] ind. simple,
                                // [13] ind. doble, [14] ind. triple
16
17
18    uint32_t i_generation;      // Número de versión (para NFS)
19    uint32_t i_file_acl;        // Access Control List extendida
20    uint32_t i_dir_acl;         // (EXT2) o i_size_high (EXT4)
21    // ... más campos
22 }; // Total: 128 bytes típicamente

```

12.12.5. Esquema de Punteros del Inodo

El inodo contiene **15 punteros** que permiten acceder a los bloques de datos:



Los primeros 12 punteros son directos: apuntan inmediatamente a bloques de datos. Esto hace

que archivos pequeños (hasta 48 KiB con bloques de 4 KiB) sean extremadamente eficientes —solo se necesita leer el inodo y luego directamente los bloques de datos.

El puntero 13 (indirecto simple) apunta a un bloque que contiene punteros. Con bloques de 4 KiB y punteros de 4 bytes, caben 1024 punteros, lo que permite direccionar $1024 \times 4KiB = 4MiB$ adicionales.

El puntero 14 (indirecto doble) apunta a un bloque de punteros a bloques de punteros. Esto permite $1024 \times 1024 \times 4KiB = 4GiB$ adicionales.

El puntero 15 (indirecto triple) agrega un nivel más de indirección, permitiendo teóricamente $1024 \times 1024 \times 1024 \times 4KiB = 4TiB$.

12.12.6. Cálculo de Tamaño Máximo Teórico

Consideremos un sistema con bloques de 4 KiB y punteros de 4 bytes. El número de punteros por bloque es $4096/4 = 1024$.

Los 12 bloques directos proporcionan $12 \times 4KiB = 48KiB$. El indirecto simple agrega $1024 \times 4KiB = 4MiB$. El indirecto doble contribuye $1024^2 \times 4KiB = 4GiB$. El indirecto triple permitiría $1024^3 \times 4KiB = 4TiB$.

Sumando todo: $48KiB + 4MiB + 4GiB + 4TiB \approx 4TiB$ de capacidad teórica.

EXT2 usa un campo `i_size` de 32 bits para el tamaño del archivo. Por lo tanto, el límite REAL es $2^{32}bytes = 4GiB$, independientemente de que la estructura de punteros podría soportar 4 TiB. Esta es una limitación del diseño original que EXT4 corrige usando 64 bits para el tamaño.

Si tenemos un disco de 512 GiB con bloques de 4 KiB, tenemos $(512 \times 1024 \times 1024)/4 = 134.217.728$ bloques totales. Cada puntero de 4 bytes puede direccionar hasta $2^{32} = 4.294.967.296$ bloques posibles, más que suficiente. El límite real sigue siendo el campo `i_size` de 32 bits: 4 GiB máximo por archivo.

12.12.7. Ejemplo de Acceso a Archivo en EXT2

Veamos dos casos que ilustran la eficiencia del esquema de punteros.

Caso 1: Archivo pequeño (40 KiB) - Solo bloques directos

El archivo ocupa 10 bloques ($40KiB/4KiB = 10bloques$).

Para leer los primeros 8 KiB necesitamos leer el inodo (1 acceso) para obtener `i_block[0]` e `i_block[1]`, luego leer el bloque de datos apuntado por `i_block[0]` (1 acceso) y el bloque apuntado por `i_block[1]` (1 acceso). Total: 3 accesos a disco, sin ningún bloque de punteros intermedio.

Caso 2: Archivo mediano (5 MiB) - Requiere indirecto simple

El archivo ocupa 1280 bloques ($5 \text{ MiB} / 4 \text{ KiB} = 1280$). - Primeros 12 bloques en punteros directos (48 KiB) - Restantes 1268 bloques en indirecto simple

Para leer desde el byte 200.000, primero calculamos el bloque objetivo: $200.000 / 4096 = \text{bloque}48$. Como $48 > 11$, está fuera de los punteros directos. Su posición en el indirecto simple es $48 - 12 = \text{ndice}36$.

Los pasos son: leer el inodo (1 acceso) para obtener `i_block[12]`, leer el bloque indirecto simple (1 acceso) para obtener el puntero en el índice 36, y finalmente leer el bloque de datos (1 acceso).

Total: 3 accesos a disco, comparable al caso anterior a pesar de que el archivo es 128 veces más grande.

12.13. Comparación: FAT vs EXT2

Característica	FAT	EXT2/UFS
Separación metadatos/nombre	No (todo en entrada dir.)	Sí (inodo separado)
Método de asignación	Lista enlazada (tabla FAT)	Indexación multinivel
Hard links	No soporta	Soportado (contador <code>links_count</code>)
Permisos	Atributos básicos (R/O, hidden)	Completo (owner/group/others, rwx)
Acceso aleatorio	Lento (recorrer cadena FAT)	Rápido (cálculo directo)
Tamaño máx archivo	4 GiB (FAT32)	4 GiB (por <code>i_size</code> de 32 bits)
Tamaño máx volumen	2 TiB (FAT32)	32 TiB (en práctica)
Fragmentación	Alta (clusters dispersos)	Baja (block groups, localidad)
Confiabilidad	Baja (tabla FAT es punto único de falla)	Alta (superbloque replicado)

Característica	FAT	EXT2/UFS
Journaling	No (exFAT añade algo básico)	No en EXT2, sí en EXT3/4
Complejidad	Muy simple	Moderada
Overhead metadatos	Bajo	Medio (inodos preasignados)
Compatibilidad	Universal (todos los SO)	Linux nativo, soportado en otros
Uso típico	Pendrives, SD cards, cámaras	Discos Linux, servidores

FAT sigue usándose porque ofrece compatibilidad universal—todos los sistemas operativos lo soportan nativamente sin drivers adicionales. Su simplicidad hace que sea ideal para dispositivos embebidos con recursos limitados. No requiere el overhead de permisos complejos en dispositivos portátiles que se comparten entre múltiples usuarios y sistemas. Y tiene menor overhead de metadatos en dispositivos pequeños donde cada byte cuenta.

EXT2 es superior para sistemas operativos porque proporciona soporte robusto de permisos necesario en entornos multiusuario. Los hard links permiten una estructura de directorios más flexible y eficiente. El acceso aleatorio eficiente es crítico para bases de datos y aplicaciones que requieren seek frecuente. Y la escalabilidad a volúmenes grandes lo hace apropiado para servidores y estaciones de trabajo modernas.

12.14. Links: Hard Links y Soft Links

Ahora que entendemos cómo funcionan FAT (sin inodos) y EXT2 (con inodos), podemos comprender los links.

12.14.1. Hard Links

Hard Link: Entrada de directorio adicional que apunta al MISMO inodo de un archivo existente. Ambos nombres son completamente equivalentes—no existe concepto de "original" versus "copia".

Veamos un ejemplo práctico:

```
1 $ ls -li archivo.txt
2 1234567 -rw-r--r-- 1 alumno alumno 5000 Dec 29 10:00 archivo.txt
3
4 $ ln archivo.txt copia.txt
5
6 $ ls -li
7 1234567 -rw-r--r-- 2 alumno alumno 5000 Dec 29 10:00 archivo.txt
8 1234567 -rw-r--r-- 2 alumno alumno 5000 Dec 29 10:00 copia.txt
```

Observemos los detalles: ambos archivos muestran el mismo número de inodo (1234567), confirmando que apuntan a la misma estructura. El contador `links_count` ha incrementado a 2, indicando que el inodo tiene dos nombres. Cualquier modificación por cualquier nombre afecta al mismo contenido físico. Y eliminar un nombre no elimina el archivo mientras `links_count` sea mayor que cero.

La estructura interna es reveladora:

```
1 Directorio /home/alumno/:
2 +-----+-----+
3 | Nombre           | Inodo |
4 +-----+-----+
5 | archivo.txt      | 1234567 |
6 | copia.txt        | 1234567 | ← Mismo inodo
7 +-----+-----+
8
9 Inodo 1234567:
10 - i_links_count = 2
11 - i_size = 5000
12 - i_block[0] = 8945 ← Datos reales
```

Los hard links tienen tres limitaciones fundamentales. Primero, no pueden cruzar file systems—cada FS tiene su propia tabla de inodos numerada independientemente. Segundo, no pueden apuntar a directorios (excepto las entradas especiales `.` y `..`) porque permitirlo crearía ciclos que romperían algoritmos de traversal recursivo. Tercero, solo funcionan en sistemas con inodos, lo que excluye a FAT.

El archivo se elimina realmente solo cuando el último nombre desaparece:

```
1 $ rm archivo.txt # links_count: 2 → 1 (archivo SIGUE existiendo)
2 $ rm copia.txt   # links_count: 1 → 0 (AHORA se liberan los bloques)
```

12.14.2. Soft Links (Symbolic Links)

Soft Link (Symbolic Link): Archivo especial cuyo contenido es la RUTA (path) textual a otro archivo. Es una redirección a nivel de nombres, no de inodos—el sistema operativo debe “seguir” el link para llegar al destino.

Un ejemplo ilustrativo:

```
1 $ ln -s /home/alumno/proyecto/datos.txt acceso_datos.txt
2
3 $ ls -l
4 lrwxrwxrwx 1 alumno alumno 30 Dec 29 10:05 acceso_datos.txt -> /home/alumno/
  proyecto/datos.txt
5 -rw-r--r-- 1 alumno alumno 5000 Dec 29 10:00 datos.txt
```

El soft link tiene su propio inodo, completamente separado del archivo destino. El primer carácter `l` en los permisos indica que es un link. Los permisos aparecen como `lrwxrwxrwx` pero son ignorados—el control de acceso real está en el archivo destino. El “contenido” del soft link es simplemente el path como string: `“/home/alumno/proyecto/datos.txt”`.

La estructura interna muestra la separación:

```
1 Directorio /home/alumno/:
2 +-----+-----+
3 | Nombre           | Inodo |
4 +-----+-----+
5 | acceso_datos.txt | 9876543 | ← Inodo del LINK
6
7 Inodo 9876543 (el soft link):
8 - i_mode = S_IFLNK | 0777 (tipo = link)
9 - i_size = 30 (longitud del path)
10 - contenido: "/home/alumno/proyecto/datos.txt"
11
12 Directorio /home/alumno/proyecto/:
13 +-----+-----+
14 | Nombre           | Inodo |
15 +-----+-----+
16 | datos.txt        | 1234567 | ← Inodo del ARCHIVO REAL
17
18 Inodo 1234567 (el archivo):
19 - i_mode = S_IFREG | 0644
20 - i_size = 5000
21 - i_block[0] = datos reales
```

Si eliminamos el archivo original, el soft link queda “roto” (dangling link):

```
1 $ rm /home/alumno/proyecto/datos.txt
2 $ cat acceso_datos.txt
3 cat: acceso_datos.txt: No such file or directory
```

El link sigue existiendo pero apunta a un archivo inexistente. Interesantemente, si luego creamos un archivo nuevo con el mismo nombre y ubicación, el link vuelve a funcionar automáticamente.

12.14.3. Comparación Hard Link vs Soft Link

Aspecto	Hard Link	Soft Link
Inodo	Mismo que el original	Propio inodo
Contenido	Apunta directamente a datos	Contiene path como string
Si se borra original	Datos siguen accesibles	Link queda roto (dangling)
Cruza file systems	No	Sí
Apunta a directorios	No (excepto . y ..)	Sí
Overhead	Mínimo (solo entrada dir.)	Un inodo extra
Transparencia	Totalmente transparente	Se nota que es link
Uso típico	Backups, deduplicación	Atajos, referencias flexibles

Los hard links son ideales para sistemas de backups incrementales. Los archivos sin cambios se hard-linkean en vez de copiarse, ahorrando espacio masivamente:

```
1 backup/2024-12-01/archivo.txt ← Inodo 12345
2 backup/2024-12-02/archivo.txt ← Inodo 12345 (mismo, no usa espacio)
3 backup/2024-12-03/archivo.txt ← Inodo 67890 (cambió, nueva copia)
```

Los soft links son perfectos para gestión de versiones de software:

```
1 /usr/bin/python3 -> /usr/bin/python3.11 (soft link)
2 # Actualizar Python solo requiere cambiar el link, no recompilar programas
```

12.15. Ejercicios integradores

12.15.1. Ejercicio 1: lectura simple en EXT2

Se tiene un sistema EXT2 con bloques de 4 KiB y punteros de 4 bytes. Un archivo ocupa 20 KiB y está almacenado íntegramente en la zona de punteros directos del inodo. Se desea leer el archivo completo, del byte 0 al byte 20.479.

¿Cuántos accesos a disco se requieren? Diferenciar entre accesos a bloques de datos y accesos a bloques de punteros.

12.15.1.1. Solución

El archivo ocupa 20.480 bytes. Como cada bloque contiene 4.096 bytes, se necesitan exactamente cinco bloques:

$$\left\lceil \frac{20.480}{4.096} \right\rceil = 5 \text{ bloques}$$

El inodo EXT2 dispone de 12 punteros directos. Como $5 < 12$, todos los bloques caben en la zona directa y no se requiere ningún bloque indirecto.

Para leer el archivo, el sistema operativo realiza los siguientes accesos a disco:

1. Leer el inodo, para obtener `i_block[0]` a `i_block[4]` junto con el resto de metadatos.
2. Leer el bloque 0 (bytes 0–4.095).
3. Leer el bloque 1 (bytes 4.096–8.191).
4. Leer el bloque 2 (bytes 8.192–12.287).
5. Leer el bloque 3 (bytes 12.288–16.383).
6. Leer el bloque 4 (bytes 16.384–20.479).

Los cinco punteros viven dentro del propio inodo, de modo que no se accede a ningún bloque de indirección separado.

Resultado: 5 accesos a bloques de datos, 0 accesos a bloques de punteros, 1 acceso al inodo.
Total: 6 accesos a disco.

12.15.2. Ejercicio 2: lectura con múltiples niveles de indirección en EXT2

Se tiene un sistema EXT2 con bloques de 8 KiB y punteros de 64 bits. El inodo dispone de 12 punteros directos, 1 indirecto simple y 2 indirectos dobles. Se pide determinar la cantidad de accesos a disco necesaria para leer un archivo desde el byte 6.553.600 hasta el byte 8.631.975.936, diferenciando accesos a bloques de datos de accesos a bloques de punteros.

12.15.2.1. Solución

12.15.2.2. Paso 1 — Parámetros del sistema

Con los datos del enunciado se obtienen tres valores que se usarán en todo el ejercicio:

$$B = 8.192 \text{ bytes}, \quad P = 8 \text{ bytes}, \quad N = \frac{B}{P} = \frac{8.192}{8} = 1.024 \text{ punteros/bloque}$$

N es la cantidad de punteros que caben en un bloque, y determina cuántos bloques de datos puede referenciar cada nivel de indirección.

12.15.2.3. Paso 2 — Capacidad y rango de bytes de cada nivel

Con B y N se calcula la capacidad de cada nivel y, acumulando, el rango de bytes que le corresponde:

Nivel	Capacidad	Rango de bytes
Directos (12 punteros)	$12 \times B = 96 \text{ KiB}$	0 – 98.303
Indirecto simple	$N \times B = 8 \text{ MiB}$	98.304 – 8.486.911
Indirecto doble ##1	$N^2 \times B = 8 \text{ GiB}$	8.486.912 – 17.076.846.503
Indirecto doble ##2	$N^2 \times B = 8 \text{ GiB}$	17.076.846.504 – 25.666.781.095

12.15.2.4. Paso 3 — Bloques que abarca el rango solicitado

El primer paso es convertir los límites de la lectura a números de bloque lógico:

$$\text{bloque inicial} = \left\lfloor \frac{6.553.600}{8.192} \right\rfloor = 800$$

$$\text{bloque final} = \left\lfloor \frac{8.631.975.936}{8.192} \right\rfloor = 1.053.710$$

En total hay que leer $1.053.710 - 800 + 1 = 1.052.911$ bloques de datos.

12.15.2.5. Paso 4 — Distribución de bloques por nivel

Para saber cuántos de esos bloques caen en cada nivel, se construyen primero los rangos en número de bloque lógico, análogos a la tabla de bytes del paso anterior:

- Directos: bloques 0–11
- Indirecto simple: bloques 12–1.035 (es decir, $12 + 1.024 - 1$)
- Indirecto doble ##1: bloques 1.036–1.049.611

- Indirecto doble ##2: bloques 1.049.612–2.098.187

Cruzando el rango solicitado [800, 1.053.710] con esos intervalos:

Directos (bloques 0–11). El bloque 800 queda fuera de este rango; la lectura no toca ningún bloque directo.

Indirecto simple (bloques 12–1.035). El inicio (800) cae aquí; el final (1.053.710), no. Se leen los bloques 800 a 1.035:

$$1.035 - 800 + 1 = \mathbf{236} \text{ bloques de datos}$$

Indirecto doble ##1 (bloques 1.036–1.049.611). El rango solicitado abarca este nivel por completo, desde el primer bloque hasta el último:

$$1.049.611 - 1.036 + 1 = \mathbf{1.048.576} \text{ bloques de datos}$$

Indirecto doble ##2 (bloques 1.049.612–2.098.187). El final del rango (1.053.710) cae aquí, comenzando desde el primer bloque de este nivel:

$$1.053.710 - 1.049.612 + 1 = \mathbf{4.099} \text{ bloques de datos}$$

Verificación:

$$236 + 1.048.576 + 4.099 = 1.052.911 \checkmark$$

12.15.2.6. Paso 5 — Accesos a bloques de punteros

Para llegar a un bloque de datos que está bajo un nivel de indirección, el sistema operativo debe primero leer los bloques que almacenan los punteros. Cada uno de esos accesos cuenta como un acceso a bloque de punteros.

Indirecto simple. Un único bloque de indirección contiene los 1.024 punteros del nivel. No importa cuántos de esos punteros se usen: el bloque hay que leerlo entero.

$$\text{accesos} = \mathbf{1}$$

Indirecto doble ##1. Un bloque raíz apunta a 1.024 bloques de segundo nivel, cada uno de los cuales contiene 1.024 punteros a bloques de datos. Como se usa el nivel completo, se accede a la raíz más a todos los bloques de segundo nivel:

$$\text{accesos} = 1 (\text{raíz}) + 1.024 (\text{bloques de segundo nivel}) = 1.025$$

Indirecto doble ##2. Se necesitan 4.099 bloques de datos. Cada bloque de segundo nivel cubre 1.024 bloques de datos, así que:

$$\left\lceil \frac{4.099}{1.024} \right\rceil = 5 \text{ bloques de segundo nivel}$$

$$\text{accesos} = 1 (\text{raíz}) + 5 (\text{bloques de segundo nivel}) = 6$$

El techo ($\lceil \cdot \rceil$) es indispensable aquí. Los 4.099 bloques de datos no son múltiplo exacto de 1.024: los primeros cuatro bloques de segundo nivel cubren $4 \times 1.024 = 4.096$ bloques, y queda un remanente de 3 bloques que necesita un quinto bloque de segundo nivel. Usar piso en lugar de techo llevaría a un resultado incorrecto.

Total de accesos a bloques de punteros:

$$1 + 1.025 + 6 = 1.032$$

12.15.2.7. Paso 6 — Resultado final

Accesos a bloques de datos: 1.052.911

Accesos a bloques de punteros: 1.032

Total de accesos a disco: 1.053.943 (sin contar el acceso inicial al inodo).

En la práctica el sistema operativo cachea los bloques de punteros en memoria, de modo que lecturas sucesivas al mismo archivo no repiten esos accesos. El análisis teórico los cuenta todos porque asume un caché frío, lo que representa el peor caso posible.

12.16. Otros sistemas de archivos

La cátedra se enfoca en FAT y EXT2, pero conviene conocer la existencia de otros sistemas de archivos ampliamente usados, porque cada uno toma decisiones de diseño distintas según el problema que resuelve.

12.16.1. EXT3 y EXT4

EXT3 (2001) extiende EXT2 con una sola adición fundamental: journaling. Mantiene el mismo formato de inodos y la misma estructura en disco, lo que permite montar una partición EXT2 directamente como EXT3 sin reformatar. El journaling puede operar en tres modos —journal, ordered y writeback— que intercambian velocidad por garantías de consistencia.

EXT4 (2008) da un paso más grande. En lugar de listas de bloques individuales, introduce los extents: rangos contiguos de bloques descritos por un solo registro. Esto reduce drásticamente la cantidad de metadatos en archivos grandes y la fragmentación. Además agrega delayed allocation (los bloques se asignan justo antes de escribirse a disco, lo que mejora la decisión de ubicación) y multiblock allocation (asigna varios bloques de una sola vez). El soporte de archivos de hasta 16 TiB y volúmenes de hasta 1 EiB lo convierte en el sistema de archivos estándar de la mayoría de distribuciones Linux actuales.

12.16.2. NTFS

NTFS (1993, Microsoft) es el sistema de archivos principal de Windows desde NT 4.0. Ofrece journaling completo de datos y metadatos, listas de control de acceso más expresivas que el modelo Unix rwx, streams alternativos de datos (un mismo archivo puede tener múltiples contenidos asociados), y soporte de compresión y cifrado a nivel de sistema de archivos. Los límites teóricos son de 16 EiB por archivo y 256 TiB por volumen en la práctica.

12.16.3. exFAT

exFAT (2006, Microsoft) fue diseñado para tarjetas SD y pendrives de gran capacidad. Su objetivo era simple: eliminar el límite de 4 GiB por archivo que tiene FAT32, manteniendo un overhead menor que NTFS y un mejor comportamiento sobre memoria flash. Soporta archivos de hasta 16 EiB y volúmenes de hasta 128 PiB teóricos. Es compatible con Windows (Vista SP1+), macOS (10.6.5+) y Linux.

12.16.4. APFS, XFS, Btrfs y ZFS

APFS (2017, Apple) reemplaza a HFS+ en todos los dispositivos Apple. Usa copy-on-write —nunca sobrescribe datos en su lugar— lo que hace que los snapshots sean instantáneos y sin costo de espacio hasta que los datos divergen. Está optimizado para almacenamiento SSD.

XFS (1994, SGI; 2001 en Linux) apunta al alto rendimiento con archivos muy grandes y operaciones de I/O paralelas. Es el sistema de archivos por defecto en RHEL/CentOS desde la versión 7.

Btrfs (2009) lleva las ideas de copy-on-write más lejos: agrega checksums de datos y metadatos para detectar corrupción silenciosa, compresión transparente y RAID integrado a nivel de sistema de archivos.

ZFS (2005, Sun Microsystems) es la opción enterprise de referencia. Combina copy-on-write, checksums end-to-end con autocorrección, RAID-Z, deduplicación y compresión, todo en un diseño donde el volumen lógico y el sistema de archivos son una sola capa. Sus límites teóricos son esencialmente ilimitados para uso práctico.

12.16.5. Cuadro comparativo

Sistema	Año	Journaling	Máx. archivo	Máx. volumen	Uso principal
FAT32	1996	No	4 GiB	2 TiB	Dispositivos portátiles
EXT2	1993	No	2 TiB	32 TiB	Linux legacy
EXT3	2001	Sí	2 TiB	32 TiB	Linux (histórico)
EXT4	2008	Sí	16 TiB	1 EiB	Linux (actual)
NTFS	1993	Sí	16 EiB	256 TiB	Windows
exFAT	2006	Parcial	16 EiB	128 PiB	Flash de gran capacidad
APFS	2017	CoW	8 EiB	8 EiB	macOS / iOS
XFS	1994	Sí	8 EiB	8 EiB	Servidores Linux
Btrfs	2009	CoW	16 EiB	16 EiB	Linux avanzado
ZFS	2005	CoW	16 EiB	256 ZiB	Enterprise

12.17. Síntesis y puntos clave

12.17.1. Conceptos fundamentales

Un sistema de archivos es, ante todo, una abstracción: transforma sectores físicos en una jerarquía de archivos y directorios con nombre, protección y persistencia. El sistema operativo no interpre-

ta el contenido de un archivo; solo gestiona sus metadatos y garantiza el acceso ordenado a los datos.

La distinción entre sector y bloque es importante: el sector es la unidad mínima que maneja el hardware (habitualmente 512 bytes o 4 KiB), mientras que el bloque es la unidad lógica del sistema de archivos y puede configurarse al formatear. Bloques más grandes reducen el overhead de metadatos pero aumentan la fragmentación interna; bloques más pequeños hacen lo contrario.

El FCB (File Control Block), llamado inodo en Unix, concentra todos los metadatos de un archivo —tamaño, permisos, marcas de tiempo, ubicación de los datos— excepto su nombre. El nombre vive en el directorio, que no es más que un archivo especial que mapea nombres a inodos.

12.17.2. Operaciones y estructuras del sistema operativo

Cuando un proceso abre un archivo, el sistema operativo mantiene tres estructuras relacionadas: la tabla de descriptores de archivo (por proceso), la tabla de archivos abiertos (global, compatible entre procesos que hagan `fork`) y la tabla de inodos en memoria. Esta arquitectura de tres niveles permite que dos descriptores distintos compartan o no el mismo offset según cómo se abrió el archivo.

El file locking puede ser advisory —el proceso puede ignorarlo, es meramente informativo— o mandatory, que el sistema operativo impone. Dentro de eso, un lock compartido (shared) permite que varios lectores accedan simultáneamente, mientras que un lock exclusivo bloquea a todos los demás.

Los permisos Unix se codifican en nueve bits: tres para el propietario, tres para el grupo y tres para el resto. Sobre archivos, los bits `r`, `w` y `x` significan leer, escribir y ejecutar. Sobre directorios, `r` permite listar el contenido, `w` permite crear o eliminar entradas, y `x` permite atravesar el directorio para acceder a lo que contiene.

[warning] Para eliminar un archivo se necesita permiso de escritura en el directorio que lo contiene, no en el archivo mismo. Un archivo con permisos `'000'` puede borrarse si el directorio padre tiene bit `'w'` activo.

12.17.3. Métodos de asignación de espacio

La asignación contigua almacena el archivo en bloques consecutivos. El acceso secuencial es muy rápido, pero sufre de fragmentación externa y no permite hacer crecer un archivo sin reubicarlo. Se usa principalmente en medios de solo lectura como CD-ROM y DVD.

La asignación enlazada encadena los bloques mediante punteros: cada bloque apunta al siguiente. Elimina la fragmentación externa y permite archivos dinámicos, pero el acceso aleatorio requiere

recorrer la cadena desde el inicio. La tabla FAT es la mejora clave: centraliza todos los punteros en una estructura que cabe en memoria, evitando los accesos a disco para atravesar la cadena.

La asignación indexada concentra los punteros a los bloques de datos en una estructura separada (el inodo en EXT2). El acceso aleatorio es eficiente porque basta leer el inodo y luego ir directamente al bloque. El esquema multinivel —punteros directos, indirecto simple, indirecto doble, indirecto triple— permite escalar desde archivos pequeños hasta archivos muy grandes sin desperdiciar espacio en metadatos cuando el archivo es chico.

12.17.4. FAT y EXT2 comparados

Aspecto	FAT	EXT2
Metadatos	En la entrada de directorio	En el inodo (separado)
Asignación de espacio	Lista enlazada (tabla FAT)	Indexada multinivel
Hard links	No soportados	Sí
Permisos	Básicos	Completos (rwx, owner/group/others)
Acceso aleatorio	Lento (recorre la FAT)	Rápido (inodo → bloque directo)
Complejidad	Muy baja	Moderada

FAT organiza el espacio en clusters. La tabla FAT es un arreglo donde `FAT[i]` indica el siguiente cluster del archivo o un marcador de fin. FAT12, FAT16 y FAT32 difieren en el ancho de esas entradas, lo que limita la cantidad de clusters direccionables y, en consecuencia, el tamaño máximo de archivo (4 GiB en FAT32, por el campo de tamaño en la entrada de directorio).

EXT2 organiza el disco en block groups que agrupan inodos y datos cercanos entre sí para mejorar la localidad. El inodo tiene 15 entradas en `i_block`: 12 directas, 1 indirecta simple, 1 indirecta doble y 1 indirecta triple (aunque la cátedra trabaja con variantes que pueden diferir en cantidad de indirectos dobles). La capacidad teórica supera los 4 TiB, pero el campo `i_size` de 32 bits limita el tamaño real de un archivo individual a 4 GiB en la versión original de EXT2.

12.17.5. Links

Un hard link es simplemente otra entrada de directorio que apunta al mismo inodo. El archivo existe mientras haya al menos una entrada apuntándolo (`links_count > 0`). No puede cruzar sistemas de archivos porque los inodos son locales a cada partición.

Un soft link (o symlink) es un archivo independiente cuyo contenido es una ruta. Tiene su propio inodo y puede apuntar a directorios o cruzar particiones. Si se elimina el archivo destino, el symlink queda roto.

12.17.6. Consistencia y recuperación

El journaling resuelve el problema de las escrituras parciales: antes de modificar el sistema de archivos, el sistema operativo registra la operación en un journal. Si hay un crash, la recuperación simplemente reproduce o descarta las operaciones pendientes del journal, sin necesidad de escanear el disco completo.

Sin journaling (como en EXT2 y FAT), la única alternativa es `fsck` o `chkdsk`: un escaneo completo que verifica inodos, directorios, conectividad, contadores de referencias y bitmaps. En discos grandes puede tardar horas.

12.18. Guía de resolución para ejercicios numéricos

Esta sección sistematiza el proceso de resolución de los ejercicios típicos de parcial sobre accesos a disco en EXT2.

12.18.1. Procedimiento paso a paso

Paso 1. Anotar los parámetros del sistema:

$$B = \text{tamaño de bloque}, \quad P = \text{tamaño de puntero}, \quad N = \frac{B}{P}$$

Paso 2. Calcular la capacidad y el rango de bytes de cada nivel:

$$\text{Directos: } D \times B \quad \text{Ind. simple: } N \times B \quad \text{Ind. doble: } N^2 \times B \quad \text{Ind. triple: } N^3 \times B$$

Paso 3. Convertir los límites de la operación a números de bloque lógico:

$$\text{bloque} = \left\lfloor \frac{\text{byte}}{B} \right\rfloor$$

Paso 4. Determinar en qué nivel cae cada extremo del rango y cuántos bloques de datos corresponden a cada nivel. Verificar que la suma coincida con el total.

Paso 5. Contar los accesos a bloques de punteros por nivel:

- Indirecto simple: 1 acceso (el bloque que contiene los punteros).
- Indirecto doble: 1 (raíz) + $\lceil \text{bloques de datos} / N \rceil$ (bloques de segundo nivel).
- Indirecto triple: aplicar la misma lógica recursivamente.

Paso 6. Sumar y verificar.

12.18.2. Errores frecuentes

Olvidar el bloque raíz del indirecto doble. El acceso a un indirecto doble requiere primero leer el bloque raíz que apunta a los bloques de segundo nivel. El total no es solo la cantidad de bloques de segundo nivel.

Usar piso en lugar de techo al contar bloques de segundo nivel. Si se necesitan 4.099 bloques de datos y cada bloque de segundo nivel cubre 1.024, la cuenta es $\lceil 4.099 / 1.024 \rceil = 5$, no 4. El remanente siempre ocupa un bloque completo adicional.

Confundir bloques de datos con bloques de punteros. Un bloque indirecto simple no es un bloque de datos; es un bloque que almacena punteros a bloques de datos.

Olvidar que los directos ocupan las primeras posiciones. El bloque lógico 0 corresponde a `i_block[0]`; el indirecto simple empieza recién en el bloque 12 (o en el que indique el enunciado).

Usar la capacidad teórica ignorando límites reales. EXT2 tiene capacidad teórica mayor a 4 TiB, pero el campo `i_size` de 32 bits limita los archivos individuales a 4 GiB.

Confundir FAT con EXT2 en la estructura de metadatos. En FAT los metadatos están en la entrada de directorio; en EXT2 están en el inodo, que es una estructura separada del directorio.

Para permisos Unix: 4 = r, 2 = w, 1 = x. La conversión entre notación octal y simbólica es un cálculo directo: 755 equivale a `rw-r--r--`, y 644 a `rw-r--r--`. El bit x en un directorio es crítico: sin él no es posible acceder a ningún archivo dentro, aunque se tenga permiso r.

12.19. Conexiones con otros capítulos

Estos temas no existen en aislamiento. Cada proceso mantiene su propia tabla de file descriptors, y `fork()` la hereda: padre e hijo comparten el offset en la Open File Table, lo que puede producir comportamientos inesperados si ambos escriben simultáneamente. Los archivos ejecutables se cargan desde el sistema de archivos al espacio de direcciones del proceso (capítulo de gestión de memoria), apoyándose en mecanismos como los archivos mapeados en memoria. El file locking es una instancia concreta del problema de sincronización (capítulo de sincronización) y puede dar

lugar a deadlocks entre procesos que esperan locks sobre distintos archivos (capítulo de interbloqueo). Finalmente, toda operación sobre el sistema de archivos se traduce en operaciones de I/O al disco gestionadas por el driver correspondiente, generalmente con DMA, sin intervención de la CPU en la transferencia de datos (capítulo de interrupciones e I/O).

13 I/O - Gestión de Dispositivos

13.1. Objetivos de Aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

- Comprender la complejidad del subsistema de I/O y sus desafíos
- Identificar los diferentes tipos de dispositivos y sus características
- Comparar las técnicas de I/O: programado (polling), por interrupciones y DMA
- Analizar la estructura física de los discos magnéticos
- Calcular tiempos de acceso a disco (seek, latencia rotacional, transferencia)
- Aplicar y comparar algoritmos de scheduling de disco (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK, FSCAN, N-STEP-SCAN)
- Comprender los niveles básicos de RAID (0, 1, 5) y calcular capacidades
- Explicar el rol de buffering y caching en el subsistema de I/O
- Resolver ejercicios tipo parcial sobre scheduling de disco y tiempos de acceso

13.2. Introducción: ¿Por qué es complejo el I/O?

En los capítulos anteriores estudiamos la gestión de procesos, memoria y file systems. Todos estos componentes dependen críticamente del subsistema de entrada/salida para funcionar: los procesos necesitan leer datos del disco, la memoria virtual requiere swap a disco, y los file systems almacenan archivos en dispositivos físicos. Ahora llegamos a uno de los componentes más fascinantes y complejos del sistema operativo.

El subsistema de I/O enfrenta un desafío único: debe coordinar la comunicación entre la CPU (que opera a gigahertz) y dispositivos que van desde teclados lentos hasta SSDs ultrarrápidos. Esta brecha de velocidad puede ser de seis órdenes de magnitud. Imaginate tratar de coordinar una conversación donde una persona habla a velocidad normal y la otra tarda un año en responder cada palabra.

13.2.1. Desafíos del I/O

Complejidad del I/O: El subsistema de I/O debe manejar una enorme variedad de dispositivos: discos, teclados, impresoras, tarjetas de red, GPUs, sensores. Cada uno tiene interfaces y protocolos diferentes, con rangos de velocidad extremadamente amplios (desde 10 bytes/-seg de un teclado hasta 7 GB/seg de un SSD NVMe). Además, necesita sincronización precisa entre CPU y dispositivos lentos, confiabilidad crítica (la pérdida de datos en disco es catastrófica), y compatibilidad con múltiples dispositivos y versiones de drivers.

Para ilustrar esta diversidad, considerará la siguiente tabla de dispositivos típicos:

1	Dispositivo	Velocidad típica	Características
2	-----	-----	-----
3	Teclado	10-100 bytes/seg	Interactivo, impredecible
4	Mouse	100 bytes/seg	Interactivo, streams
5	Disco HDD	100-200 MiB/seg	Bloques, latencia alta
6	SSD SATA	500-600 MiB/seg	Bloques, latencia media
7	SSD NVMe	3-7 GiB/seg	Bloques, latencia baja
8	Tarjeta de red	100 Mbps - 100 Gbps	Paquetes, tiempo real
9	GPU	100+ GiB/seg	Paralelo masivo

La diferencia entre un teclado y un SSD NVMe es la misma que existe entre caminar (5 km/h) y viajar a 30.000 km/h. El sistema operativo debe hacer que todos estos dispositivos funcionen juntos de manera coordinada.

13.2.2. Objetivos del Subsistema de I/O

Subsistema de I/O: Conjunto de componentes del sistema operativo responsables de gestionar la comunicación entre el CPU/memoria y los dispositivos periféricos, proporcionando abstracción, eficiencia y confiabilidad.

El subsistema de I/O persigue cinco objetivos principales. Primero, la abstracción o independencia de dispositivo: ocultar las diferencias entre dispositivos mediante interfaces uniformes, para que tu programa pueda usar `read()` tanto con un archivo en disco como con datos de red. Segundo, la eficiencia: maximizar el throughput y minimizar la latencia, aprovechando al máximo las capacidades del hardware. Tercero, la compartición: permitir que múltiples procesos accedan concurrentemente a dispositivos compartibles de manera segura y coordinada.

El cuarto objetivo es la protección: prevenir que procesos no autorizados accedan directamente a dispositivos críticos (imagínate si cualquier programa pudiera leer o escribir arbitrariamente en tu disco). Finalmente, el manejo de errores: detectar y recuperarse de fallos de hardware, que son inevitables en dispositivos mecánicos y electrónicos.

13.3. Hardware de I/O

Antes de entender cómo el sistema operativo gestiona el I/O, necesitamos comprender el hardware subyacente. Los dispositivos no son todos iguales, y sus diferencias fundamentales determinan cómo el SO debe interactuar con ellos.

13.3.1. Tipos de Dispositivos

Los dispositivos de I/O se clasifican según varias dimensiones, siendo las más importantes la unidad de transferencia y el modo de acceso.

13.3.1.1. Por Unidad de Transferencia

Los dispositivos de bloque transfieren datos en bloques de tamaño fijo, típicamente entre 512 bytes y 4 KiB. Estos dispositivos soportan acceso aleatorio mediante seek: podés saltar a cualquier bloque sin tener que leer todos los anteriores. Cada bloque tiene una dirección única, lo que permite el direccionamiento directo. Los ejemplos clásicos son discos duros (HDD), SSDs, CD-ROMs y cintas magnéticas. Estos dispositivos son los que típicamente asociamos con “almacenamiento persistente”.

En contraste, los **dispositivos de carácter** transfieren datos como un flujo continuo de bytes individuales. Generalmente no soportan seek, solo acceso secuencial: los datos llegan en orden y deben procesarse en ese orden. Pensá en ellos como un río de información que fluye constantemente.

Ejemplos incluyen teclados, mice, puertos serie e impresoras. Cuando presionás una tecla, ese carácter llega inmediatamente y no tiene sentido “buscar” la tecla anterior.

Existe una categoría especial: los dispositivos de red. Estos no son estrictamente de bloque ni de carácter, sino que transfieren paquetes de tamaño variable. Cada paquete es una unidad independiente con headers, datos y checksums. Las tarjetas Ethernet, WiFi y módems caen en esta categoría.

13.3.1.2. Por Modo de Acceso

Los **dispositivos compartibles (sharables)** pueden ser usados por múltiples procesos concurrentemente, aunque requieren sincronización cuidadosa para evitar condiciones de carrera. Los discos son el ejemplo paradigmático: cientos de procesos pueden leer y escribir simultáneamente, y el sistema operativo coordina estos accesos mediante locks y colas. Las tarjetas de red también son compartibles, multiplexando paquetes de diferentes conexiones.

Los dispositivos dedicados solo permiten un proceso a la vez. Las impresoras (sin spooling) son el ejemplo clásico: si un proceso está imprimiendo, los demás deben esperar. Las cintas magnéticas también requieren acceso exclusivo, ya que son inherentemente secuenciales.

13.3.2. Controladores de Dispositivo

Controlador de Dispositivo (Device Controller): Componente de hardware que actúa como interfaz entre el bus del sistema y el dispositivo físico. Contiene registros y lógica para controlar el dispositivo.

Un concepto crucial que a menudo se malentiende: el sistema operativo no interactúa directamente con el dispositivo físico, sino con su controlador. El controlador es un chip especializado que habla dos idiomas: por un lado, entiende el protocolo del bus del sistema (PCIe, SATA, USB), y por otro, sabe cómo manejar las peculiaridades electrónicas del dispositivo específico.

La arquitectura típica se ve así:

```
1 CPU/Memoria <---> Bus del Sistema <---> Controlador <---> Dispositivo
2                                     (Controller)      (Device)
```

El controlador contiene varios componentes esenciales. Los registros de control son donde la CPU escribe comandos: “leer sector 1234”, “mover cabeza al cilindro 500”, etc. Los registros de estado permiten que la CPU consulte el estado actual: ¿está ocupado el dispositivo? ¿hubo algún error? ¿completó la operación? Los registros de datos funcionan como buffers temporales para la transferencia de datos entre el dispositivo y la memoria. Finalmente, la lógica interna ejecuta las operaciones específicas del dispositivo, traduciendo comandos de alto nivel en secuencias de señales eléctricas.

Un ejemplo concreto de controlador de disco podría tener:

```
1 Registro de Control: [READ|WRITE|SEEK] comando
2 Registro de Estado: [BUSY|READY|ERROR] flags
3 Registro de Datos:   Buffer de sector (512 bytes)
4 Registro de Dirección: LBA (Logical Block Address)
```

El driver del sistema operativo habla con el controlador mediante lectura y escritura de sus registros, no con el dispositivo directamente. Esta abstracción permite que el mismo driver funcione con diferentes dispositivos que usan el mismo tipo de controlador.

13.3.3. Interfaces de I/O

Existen dos formas principales de que la CPU acceda a los registros del controlador. En **Memory-Mapped I/O**, los registros del controlador se mapean a direcciones específicas del espacio de

memoria física.

La CPU usa instrucciones normales de memoria (LOAD/STORE) para interactuar con ellos. Por ejemplo, escribir a la dirección `0xF0000000` podría estar escribiendo al registro de control del controlador de disco. Esta técnica no requiere instrucciones especiales y es la dominante en sistemas modernos por su simplicidad.

En **Port-Mapped I/O** (o I/O Aislado), existe un espacio de direcciones separado específico para I/O. La CPU usa instrucciones especiales como `IN` y `OUT` (en arquitectura x86) para acceder a estos puertos. Esta arquitectura era común en sistemas antiguos pero ha caído en desuso.

La mayoría de los sistemas modernos usan exclusivamente Memory-Mapped I/O porque permite usar toda la potencia del caché de CPU y las optimizaciones del pipeline para accesos a dispositivos.

13.4. Técnicas de I/O

Ahora que entendemos el hardware, podemos explorar cómo el sistema operativo gestiona las operaciones de I/O. Existen tres técnicas principales, cada una con trade-offs significativos en eficiencia, complejidad y uso de CPU. La evolución histórica de estas técnicas refleja la búsqueda constante por minimizar el desperdicio de ciclos de CPU.

13.4.1. Técnica 1: I/O Programado (Polling)

La técnica más simple es el I/O programado o polling. La idea es directa: la CPU ejecuta un bucle que constantemente verifica el estado del dispositivo hasta que esté listo. Es como preguntarle repetidamente a alguien “¿ya terminaste? ¿ya terminaste? ¿ya terminaste?” hasta que responda que sí.

```
1 // Pseudocódigo de I/O programado
2 void write_data(char *data, int size) {
3     for (int i = 0; i < size; i++) {
4         // Busy-wait: esperar hasta que dispositivo esté listo
5         while (status_register & BUSY) {
6             // Hacer nada, solo esperar (polling)
7         }
8
9         // Escribir el byte
10        data_register = data[i];
11
12        // Enviar comando de escritura
13        control_register = WRITE_COMMAND;
14    }
15 }
```

El flujo es simple: la CPU escribe un comando en el registro de control, luego entra en un bucle

verificando constantemente el registro de estado. Cuando el dispositivo finalmente está listo, la CPU lee o escribe los datos. Este ciclo se repite hasta completar la transferencia completa.

La ventaja del polling es su simplicidad extrema: no requiere hardware especial para interrupciones, es fácil de depurar, y tiene latencia muy baja para dispositivos que responden inmediatamente. En sistemas embebidos simples donde la CPU no tiene nada más que hacer, puede ser la solución correcta.

Sin embargo, el polling tiene un problema fundamental: la CPU está completamente ocupada durante todo el I/O. Es un desperdicio catastrófico de ciclos de CPU. Mientras esperas que el disco complete una operación de 10 ms, la CPU podría haber ejecutado millones de instrucciones útiles. Además, no puede hacer multitasking: está atrapada en ese bucle de polling.

El polling solo tiene sentido en escenarios muy específicos: cuando el dispositivo responde en menos de 100 ciclos de CPU, o en sistemas embebidos sin sistema operativo donde la CPU está dedicada a una sola tarea. En sistemas de propósito general modernos, el polling es prácticamente inexistente.

13.4.2. Técnica 2: I/O por Interrupciones

La segunda técnica resuelve el problema del busy-waiting: en lugar de que la CPU pregunte constantemente, el dispositivo le avisa cuando termina mediante una interrupción. Es como darle a alguien tu número de teléfono en vez de llamarlo cada 5 segundos.

```
1 // Pseudocódigo de I/O por interrupciones
2
3 // Función principal
4 void write_data_interrupt(char *data, int size) {
5     buffer = data;
6     count = size;
7     index = 0;
8
9     // Iniciar primera escritura
10    data_register = buffer[index++];
11    control_register = WRITE_COMMAND;
12
13    // CPU retorna y puede hacer otras cosas
14    // El resto lo maneja el interrupt handler
15 }
16
17 // Interrupt Service Routine (ISR)
18 void disk_interrupt_handler() {
19     if (index < count) {
20         // Escribir siguiente byte
21         data_register = buffer[index++];
22         control_register = WRITE_COMMAND;
23     } else {
```

```
24         // Operación completada
25         signal_completion();
26     }
27 }
```

El flujo cambia fundamentalmente. La CPU inicia la operación escribiendo al controlador, pero inmediatamente retorna y puede ejecutar otros procesos. Cuando el dispositivo completa la operación, genera una interrupción de hardware. Esta interrupción fuerza a la CPU a suspender el proceso actual, ejecutar el *Interrupt Service Routine* (ISR) que maneja el evento, y luego retornar al proceso que fue interrumpido.

Las interrupciones son el mecanismo fundamental que permite multitasking efectivo en presencia de I/O. Sin ellas, la CPU estaría constantemente atrapada esperando dispositivos lentos, desperdiciando su capacidad de procesamiento.

Las ventajas son claras: la CPU queda libre para ejecutar otros procesos durante el I/O, la utilización de CPU es mucho mejor que con polling, y permite multitasking real. Un sistema puede tener docenas de operaciones de I/O en progreso simultáneamente, con la CPU alternando entre procesos productivos.

Sin embargo, las interrupciones introducen su propio overhead. Cada interrupción requiere un *context switch*: guardar el estado del proceso actual, cambiar al kernel, ejecutar el ISR, y restaurar el proceso. Este overhead es típicamente de cientos a miles de ciclos. Si las interrupciones llegan muy frecuentemente (por ejemplo, en dispositivos de red de alta velocidad recibiendo miles de paquetes por segundo), puede ocurrir una *interrupt storm* donde la CPU pasa más tiempo manejando interrupciones que ejecutando código útil.

Además, aunque mejor que polling, la CPU sigue involucrada en cada transferencia de datos. Para transferencias grandes (como leer un archivo de varios megabytes), esto significa miles de interrupciones, cada una con su overhead asociado.

Las interrupciones son ideales para dispositivos de velocidad media, eventos esporádicos (como teclas presionadas), y transferencias pequeñas. Son el estándar para teclados, mice, y para señalar la completación de operaciones de disco.

13.4.3. Técnica 3: DMA (Direct Memory Access)

La tercera técnica lleva la eficiencia al siguiente nivel. Con DMA, un controlador especializado maneja la transferencia completa de datos entre el dispositivo y la memoria, sin que la CPU toque ningún byte individual.

DMA (Direct Memory Access): Técnica que permite al controlador de dispositivo transferir datos directamente entre el dispositivo y la memoria RAM sin intervención de la CPU.

La arquitectura con DMA introduce un nuevo componente:



El flujo de operación es elegante. Primero, la CPU configura el DMA controller: le indica la dirección de memoria origen o destino, la cantidad de bytes a transferir, la dirección del dispositivo, y la dirección de control (read/write). Luego, el DMA controller ejecuta la transferencia completa de manera autónoma: “roba” ciclos del bus según necesita (cycle stealing), transfiere datos directamente entre dispositivo y RAM byte por byte o palabra por palabra, e incrementa punteros automáticamente. Finalmente, cuando completa la transferencia completa, el DMA genera una única interrupción para notificar a la CPU. Ejemplo de código:

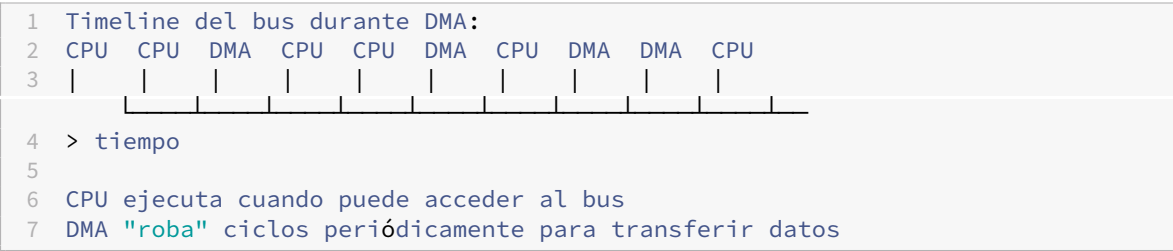
```

1 // Pseudocódigo de lectura con DMA
2 void read_disk_dma(char *buffer, int sector, int count) {
3     // Configurar DMA
4     dma_source_address = DISK_DATA_REGISTER;
5     dma_dest_address = buffer;
6     dma_byte_count = count * SECTOR_SIZE;
7     dma_mode = DEVICE_TO_MEMORY;
8
9     // Configurar controlador de disco
10    disk_sector = sector;
11    disk_command = READ;
12
13    // Iniciar DMA (retorna inmediatamente)
14    dma_control = START;
15
16    // CPU está libre, hacer otras cosas
17    // Cuando termine, llegará una interrupción
18 }
19
20 // ISR cuando DMA completa
21 void dma_complete_interrupt() {
22     // Transferencia completa
23     // buffer ahora contiene los datos del disco
24     wake_up_process_waiting_for_io();
25 }
  
```

Un concepto importante es el *cycle stealing*. El DMA controller necesita acceso al bus para transferir datos, pero la CPU también necesita el bus para ejecutar instrucciones. La solución es que el DMA “roba” temporalmente el control del bus cuando lo necesita. Si la CPU puede ejecutar desde caché (sin acceder a RAM), ni siquiera nota el robo. Si necesita el bus simultáneamente, debe esperar unos pocos ciclos.

Cycle Stealing: Técnica donde el DMA controller temporalmente "roba" el control del bus de sistema para realizar transferencias, mientras la CPU ejecuta desde cache o espera brevemente.

Un timeline típico del bus durante DMA se ve así:



Las ventajas de DMA son dramáticas: la CPU queda completamente libre durante la transferencia, el throughput es muy alto (limitado solo por la velocidad del dispositivo y el bus), solo hay una interrupción por operación completa (no una por byte), y es esencial para dispositivos de alta velocidad como SSDs modernos que transfieren gigabytes por segundo.

Las desventajas son menores en comparación: requiere hardware especializado (que ya viene incluido en todas las computadoras modernas), la configuración es más compleja que con interrupciones simples, el cycle stealing puede ralentizar ligeramente a la CPU (típicamente 1-5 %), y para transferencias muy pequeñas (unos pocos bytes) el overhead de configuración puede ser mayor que simplemente copiar los datos con la CPU.

En sistemas modernos, virtualmente todos los dispositivos de bloque (discos, SSDs, tarjetas de red, GPUs) usan DMA. Es la técnica dominante para cualquier transferencia mayor a 1 KiB.

13.4.4. Comparación de Técnicas

Resumamos las tres técnicas en una tabla comparativa:

Aspecto	Polling	Interrupciones	DMA
CPU durante I/O	100 % ocupada	Interrumpida por evento	Libre
Interrupciones	Ninguna	Una por transferencia	Una por bloque
Overhead	Bajo (simple)	Medio (context switch)	Alto (setup), bajo (ejecución)

Aspecto	Polling	Interrupciones	DMA
Throughput	Bajo	Medio	Alto
Uso de CPU	Muy alto	Medio	Muy bajo
Mejor para	Dispositivos rápidos	Eventos y chars	Transferencias grandes
Escalabilidad	Muy mala	Media	Excelente

Regla práctica para elegir técnica de I/O:

Usá polling si el dispositivo responde en menos de 100 ciclos de CPU (rarísimo en la práctica).

Usá interrupciones para eventos esporádicos y transferencias pequeñas (menos de 1 KiB).

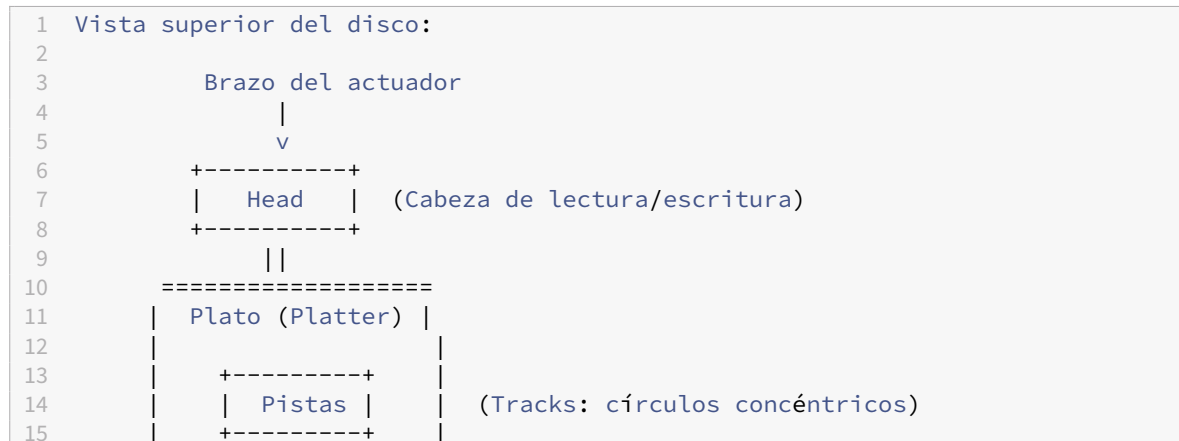
Usá DMA para transferencias mayores a 1 KiB o dispositivos rápidos. Es obligatorio para discos y redes modernas.

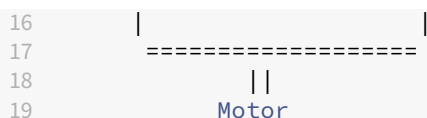
13.5. Discos Magnéticos: Estructura y Rendimiento

Aunque los SSDs han ganado popularidad, los discos duros (HDD) siguen siendo ampliamente usados para almacenamiento masivo debido a su bajo costo por gigabyte. Más importante aún, comprender su estructura física es esencial para entender el rendimiento del I/O y por qué los algoritmos de scheduling de disco son necesarios.

13.5.1. Geometría del Disco

Un disco duro es una maravilla de ingeniería mecánica de precisión. Imagínalo como un tocadiscos moderno pero con múltiples discos girando a miles de revoluciones por minuto.





Componentes del disco: Plato (Platter): Disco circular recubierto de material magnético. Un disco tiene múltiples platos apilados verticalmente.

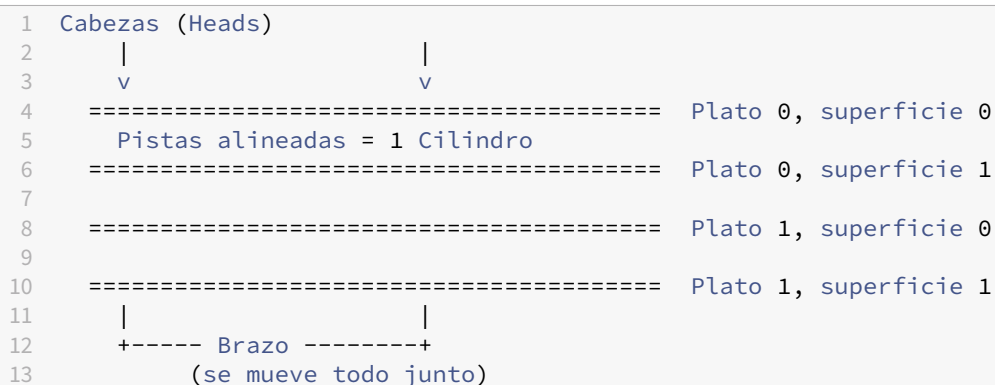
Pista (Track): Círculo concéntrico en la superficie del plato donde se almacenan datos magnéticamente.

Sector: Subdivisión de una pista, unidad mínima de transferencia (típicamente 512 bytes o 4 KiB).

Cilindro (Cylinder): Conjunto de pistas en la misma posición radial en todos los platos.

Cabeza (Head): Dispositivo que lee/escribe datos magnéticamente flotando a nanómetros de la superficie. Hay una cabeza por superficie de plato.

La vista lateral de un disco con múltiples platos revela su estructura tridimensional:



Un detalle crucial: todas las cabezas están montadas en el mismo brazo actuador y se mueven juntas. No pueden moverse independientemente. Esto significa que cuando el brazo se mueve a un cilindro específico, todas las cabezas quedan posicionadas sobre la misma posición radial en sus respectivos platos.

Históricamente, los sectores se direccionaban usando CHS (*Cylinder-Head-Sector*): especificabas exactamente en qué cilindro, qué cabeza, y qué sector querías acceder. Los sistemas modernos usan LBA (*Logical Block Addressing*): cada sector tiene un número secuencial único, y el controlador del disco se encarga de traducirlo a la geometría física. Esto simplifica enormemente la vida del sistema operativo.

Un ejemplo concreto ayuda a visualizar las capacidades. Considera un disco con 1000 cilindros, 4 cabezas (2 platos con dos superficies cada uno), 100 sectores por pista, y 512 bytes por sector:

```
1 Ejemplo de disco:
2 - 1000 cilindros
3 - 4 cabezas (2 platos, 2 superficies c/u)
```



```

4 - 100 sectores por pista
5 - 512 bytes por sector
6
7 Capacidad total = 1000 × 4 × 100 × 512 = 204.800.000 bytes ≈ 195 MiB

```

Este sería un disco muy antiguo, por supuesto. Los discos modernos tienen cientos de miles de cilindros y capacidades de terabytes.

13.5.2. Tiempos de Acceso a Disco

El tiempo total para leer o escribir datos de un disco mecánico se descompone en tres componentes fundamentales. Esta descomposición es crítica para entender el rendimiento de I/O.

Tiempo de acceso total:

$$T_{total} = T_{seek} + T_{rotacional} + T_{transferencia}$$

13.5.2.1. Tiempo de Seek (Búsqueda)

El seek time es el tiempo que tarda el brazo actuador en mover las cabezas desde su posición actual hasta el cilindro destino. Es una operación mecánica: el brazo debe acelerar, recorrer la distancia, y decelerar precisamente en el cilindro correcto.

```

1 T_seek depende de la distancia en cilindros
2
3 Valores típicos para HDD:
4 - Track-to-track (1 cilindro): 0.2 - 0.5 ms
5 - Seek promedio: 8 - 12 ms
6 - Full stroke (máximo): 15 - 20 ms

```

Los modelos físicos del seek time son interesantes. Un modelo simple lineal sería $T_{seek} = a + b \times distancia$, pero la realidad es más compleja debido a la aceleración. Un modelo más realista usa $T_{seek} = a + b \times \sqrt{distancia}$, reflejando que el brazo pasa la mayor parte del tiempo acelerando y frenando.

Para cálculos en este libro, utilizaremos el seek promedio simplificado de aproximadamente 9 ms en discos de 7200 RPM, a menos que el problema especifique otra cosa.

13.5.2.2. Latencia Rotacional

Una vez que las cabezas están posicionadas en el cilindro correcto, debemos esperar a que el sector deseado rote bajo la cabeza. Los platos giran a velocidad constante, medida en RPM (revoluciones por minuto). El tiempo para una revolución completa es simplemente:

$$T_{rotacional_{promedio}} = T_{revolucion}/2$$

Valores típicos:

RPM	Revolución completa	Latencia promedio
5400	11.1 ms	5.55 ms
7200	8.33 ms	4.17 ms
10000	6.0 ms	3.0 ms
15000	4.0 ms	2.0 ms

La latencia rotacional es una limitación física fundamental del disco. No puede mejorarse con algoritmos inteligentes de scheduling. Es simplemente una consecuencia de la velocidad de rotación del motor. La única forma de reducirla es comprar un disco más rápido (o usar un SSD, que no tiene partes móviles y por tanto no tiene latencia rotacional).

13.5.2.3. Tiempo de Transferencia

Una vez que el sector está bajo la cabeza, los datos deben transferirse entre el disco y el controlador. Este tiempo depende de la tasa de transferencia del disco y la cantidad de datos:

$$T_{transferencia} = (\text{bytes a transferir}) / (\text{tasa de transferencia})$$

Tasa de transferencia típica:

- 1 - HDD interno SATA: 100 - 200 MiB/s
- 2 - HDD externo USB 3.0: 80 - 120 MiB/s
- 3 - SSD SATA: 500 - 600 MiB/s
- 4 - SSD NVMe: 3000 - 7000 MiB/s

Ejemplo de cálculo:

Transferir 4 KiB (un bloque) en HDD a 150 MiB/s:

```

1 T_transferencia = 4096 bytes / (150 × 10^6 bytes/seg)
2                 = 0.0000273 seg
3                 = 0.027 ms

```

Una observación crítica: para discos duros, el tiempo de transferencia es completamente despreciable comparado con seek y rotación. Transferir 4 KiB toma 0.027 ms, mientras que seek + rotación combinados toman 13 ms. El tiempo de transferencia es menos del 0.2%. Esta es la razón fundamental por la que la localidad espacial es tan importante para el rendimiento de discos: el costo está en llegar al dato, no en transferirlo una vez que lo encuentre.

13.5.3. Ejercicio: Cálculo de Tiempo de Acceso

Enunciado:

Se tiene un disco duro con las siguientes características: - Velocidad de rotación: 7200 RPM - Seek time promedio: 9 ms - Tasa de transferencia: 150 MiB/s - Tamaño de sector: 4 KiB

Calcular el tiempo total para leer:

- a) Un sector aleatorio del disco
- b) 10 sectores consecutivos en la misma pista
- c) 10 sectores dispersos aleatoriamente en el disco

Solución:

Primero, calculemos los componentes base que usaremos repetidamente:

```

1 Latencia rotacional promedio:
2 T_rot = (60 / 7200) / 2 = 0.00833 / 2 = 0.00417 seg = 4.17 ms
3
4 Tiempo de transferencia por sector:
5 T_trans_sector = 4096 bytes / (150 × 10^6 bytes/seg) = 0.000027 seg ≈ 0.027 ms

```

a) Un sector aleatorio:

```

1 T_total = T_seek + T_rot + T_trans
2         = 9 ms + 4.17 ms + 0.027 ms
3         = 13.197 ms ≈ 13.2 ms

```

b) 10 sectores consecutivos en la misma pista:

Aquí está la parte interesante. Los sectores están contiguos en la misma pista, así que después del primer seek y la espera rotacional inicial, los siguientes 9 sectores pasan bajo la cabeza inmediatamente. Solo pagamos el costo de transferencia para cada uno:

```

1 T_total = T_seek + T_rot + (10 × T_trans_sector)
2         = 9 ms + 4.17 ms + (10 × 0.027 ms)
3         = 9 ms + 4.17 ms + 0.27 ms
4         = 13.44 ms

```

Observación sorprendente: leer 10 sectores consecutivos toma apenas 0.24 ms más que leer 1 sector (13.44 ms vs 13.2 ms). ¿Por qué? Porque el seek y la rotación dominan completamente. Una vez que llegaste al lugar correcto, leer más datos es casi gratis.

Esta es la razón por la que los sistemas operativos modernos implementan prefetching: cuando pedís leer un bloque, el SO aprovecha y lee los bloques siguientes también, porque el costo marginal es despreciable.

c) 10 sectores dispersos aleatoriamente:

Cada sector requiere su propio seek completo, rotación completa, y transferencia:

```
1 T_total = 10 × (T_seek + T_rot + T_trans_sector)
2         = 10 × 13.2 ms
3         = 132 ms
```

Resumen comparativo:

Operación	Tiempo total
1 sector aleatorio	13.2 ms
10 sectores consecutivos	13.44 ms (1.02× más lento)
10 sectores dispersos	132 ms (10× más lento)

Conclusión crítica para el diseño de sistemas: La localidad espacial es fundamental para el rendimiento de discos magnéticos. Leer bloques consecutivos es órdenes de magnitud más rápido que accesos aleatorios dispersos. Los algoritmos de scheduling de disco existen precisamente para maximizar esta localidad.

13.6. Scheduling de I/O: Algoritmos de Disco

Los sistemas operativos mantienen una cola de solicitudes de I/O pendientes. En un sistema activo, pueden llegar docenas o cientos de solicitudes por segundo. El orden en que se atienden estas solicitudes tiene un impacto dramático en el rendimiento total del sistema.

Disk Scheduling: Algoritmos que determinan el orden en que se atienden las solicitudes de I/O a disco, con el objetivo de minimizar el seek time total y maximizar el throughput.

Consideremos un escenario típico que usaremos para todos nuestros ejemplos:

```
1 Cola de solicitudes pendientes: [98, 183, 37, 122, 14, 124, 65, 67]
2 Posición actual de la cabeza: cilindro 53
3 Rango del disco: 0 - 199 cilindros
```

Nuestro objetivo es simple pero crucial: minimizar el movimiento total del brazo, que es la suma de todas las distancias recorridas en cilindros. ¿Por qué? Porque el seek time es el componente dominante del tiempo de acceso, como vimos en la sección anterior.

13.6.1. Algoritmo 1: FCFS (First-Come, First-Served)

El algoritmo más simple: atender las solicitudes en el orden exacto de llegada, como una cola FIFO en el supermercado.

Secuencia:

```

1 Orden: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
2
3 Movimiento total:
4 |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| +
   |67-65|
5 = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2
6 = 640 cilindros

```

Observá cómo el brazo salta de un lado al otro del disco constantemente: 53→98 (derecha), 98→183 (más derecha), 183→37 (salto enorme a la izquierda), 37→122 (derecha otra vez). Es un movimiento caótico e ineficiente.

La ventaja de FCFS es su simplicidad: implementación trivial con una cola FIFO, es perfectamente justo (todas las solicitudes se atienden en orden), y no hay riesgo de starvation (inanición, donde una solicitud espera indefinidamente).

Sin embargo, las desventajas son severas: no optimiza el movimiento del brazo en absoluto, el movimiento total es muy alto con ese zigzag constante, y el rendimiento es pobre comparado con algoritmos más inteligentes. FCFS casi nunca se usa en sistemas reales.

13.6.2. Algoritmo 2: SSTF (Shortest Seek Time First)

Un enfoque greedy: siempre atender la solicitud más cercana a la posición actual. Es el equivalente a optimizar localmente en cada decisión.

Secuencia:

```

1 Inicio en 53:
2 Más cercano: 65 (distancia 12)
3 Desde 65:
4 Más cercano: 67 (distancia 2)
5 Desde 67:
6 Más cercano: 37 (distancia 30)
7 Desde 37:
8 Más cercano: 14 (distancia 23)
9 Desde 14:
10 Más cercano: 98 (distancia 84)
11 Desde 98:
12 Más cercano: 122 (distancia 24)
13 Desde 122:
14 Más cercano: 124 (distancia 2)
15 Desde 124:
16 Más cercano: 183 (distancia 59)
17
18 Orden: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

```

```

19
20 Movimiento total:
21 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 cilindros

```

SSTF reduce el movimiento de 640 a 236 cilindros (63 % de reducción). El throughput mejora significativamente, y la implementación sigue siendo relativamente simple: solo necesitas una lista ordenada o un heap.

Sin embargo, SSTF tiene un problema serio: puede causar starvation. Imaginá que constantemente llegan solicitudes en el centro del disco (cilindro 100). Una solicitud en el cilindro 5 podría esperar indefinidamente, porque siempre habrá solicitudes más cercanas a la posición actual. Este no es un problema teórico. En sistemas con mucha carga de I/O, las solicitudes en los extremos del disco pueden experimentar latencias muy altas o incluso nunca ser atendidas.

SSTF es común en sistemas con baja o media carga de I/O, donde la probabilidad de starvation es baja y se prioriza el throughput.

13.6.3. Algoritmo 3: SCAN (Elevator Algorithm)

SCAN resuelve el problema de starvation de SSTF usando una estrategia inspirada en ascensores: el brazo se mueve en una dirección hasta el final del disco, atendiendo todas las solicitudes en el camino, luego invierte la dirección y hace lo mismo.

Secuencia (asumiendo dirección inicial hacia arriba):

```

1 Inicio en 53, dirección UP:
2 Atender: 65, 67, 98, 122, 124, 183
3 Llegar al final (199)
4 Cambiar dirección a DOWN:
5 Atender: 37, 14
6
7 Orden: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14
8
9 Movimiento total:
10 |65-53| + |67-65| + |98-67| + |122-98| + |124-122| + |183-124| + |199-183| +
    |37-199| + |14-37|
11 = 12 + 2 + 31 + 24 + 2 + 59 + 16 + 162 + 23
12 = 331 cilindros

```

El nombre “Elevator Algorithm” es perfecto: pensá en cómo funciona un ascensor moderno. No va inmediatamente al piso que pediste; primero completa todos los pedidos en su dirección actual, luego da la vuelta.

Las ventajas de SCAN son importantes: no hay starvation porque eventualmente el brazo pasa por todos los cilindros, el movimiento es más predecible que SSTF, y el throughput sigue siendo bueno. Es un buen balance entre eficiencia y fairness.

La desventaja es que hay movimiento innecesario hasta el extremo del disco. En nuestro ejemplo, no había ninguna solicitud entre 183 y 199, pero el brazo se movió hasta allí de todos modos. Además, las solicitudes que acaban de llegar en la dirección opuesta deben esperar todo un barrido completo del disco.

SCAN era usado en ascensores reales y en algunos sistemas operativos antiguos. Las variantes modernas (LOOK y C-LOOK) mejoran sobre esta idea básica.

13.6.4. Algoritmo 4: C-SCAN (Circular SCAN)

C-SCAN mejora la fairness de SCAN tratando al disco como una estructura circular: al llegar al final, el brazo salta inmediatamente al inicio sin atender solicitudes en el camino de vuelta, luego continúa en la dirección original.

Secuencia:

```

1 Inicio en 53, dirección UP:
2   Atender: 65, 67, 98, 122, 124, 183
3   Llegar al final (199)
4   Saltar al inicio (0) sin atender
5   Continuar UP:
6   Atender: 14, 37
7
8 Orden: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37
9
10 Movimiento total:
11 |65-53| + |67-65| + |98-67| + |122-98| + |124-122| + |183-124| + |199-183| +
    |199-0| + |14-0| + |37-14|
12 = 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23
13 = 382 cilindros

```

La idea clave es que C-SCAN trata a todos los cilindros más uniformemente. Las solicitudes en el extremo inferior no tienen que esperar todo el viaje de ida y vuelta; el brazo regresa rápidamente al inicio y las atiende. Esto proporciona una latencia más uniforme y predecible que SCAN.

El costo es que el movimiento total es mayor que SCAN (382 vs 331), porque ese salto de 199 a 0 cuenta como movimiento completo del disco. Sin embargo, en sistemas con alta carga de I/O distribuida uniformemente, esta desventaja se compensa con la mejor fairness.

C-SCAN es usado en sistemas donde se prioriza la consistencia de latencia sobre el throughput puro.

13.6.5. Algoritmo 5: LOOK

LOOK es una optimización obvia de SCAN: ¿para qué ir hasta el extremo del disco si no hay solicitudes ahí? El brazo solo va hasta la última solicitud en esa dirección, luego invierte.

Secuencia:

```
1 Inicio en 53, dirección UP:
2   Atender: 65, 67, 98, 122, 124, 183
3   (183 es la última solicitud UP, no ir hasta 199)
4   Cambiar dirección a DOWN:
5   Atender: 37, 14
6
7 Orden: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14
8
9 Movimiento total:
10 12 + 2 + 31 + 24 + 2 + 59 + 146 + 23 = 299 cilindros
```

La mejora sobre SCAN es clara: 299 cilindros vs 331, una reducción del 10 %. Eliminamos ese movimiento desperdiciado de 183 a 199. El brazo “mira” (look) hacia adelante, ve que no hay más solicitudes, y da la vuelta inteligentemente.

LOOK mantiene todas las ventajas de SCAN (sin starvation, predecible, buen throughput) y elimina su principal desventaja (movimiento innecesario). Es generalmente superior a SCAN en todos los aspectos y es una de las opciones más populares en sistemas operativos modernos.

13.6.6. Algoritmo 6: C-LOOK (Circular LOOK)

C-LOOK combina las ideas de C-SCAN y LOOK: solo va hasta la última solicitud en la dirección actual, luego salta a la primera solicitud pendiente (no necesariamente al extremo del disco), y continúa en la dirección original.

Secuencia:

```
1 Inicio en 53, dirección UP:
2   Atender: 65, 67, 98, 122, 124, 183
3   (183 es la última UP)
4   Saltar a la primera solicitud pendiente: 14
5   Continuar UP:
6   Atender: 14, 37
7
8 Orden: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37
9
10 Movimiento total:
11 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23 = 322 cilindros
```

C-LOOK ofrece un excelente balance. Combina la latencia uniforme de C-SCAN con la eliminación de movimiento innecesario de LOOK. Es más eficiente que C-SCAN (322 vs 382 cilindros) pero mantiene su fairness superior a SCAN/LOOK.

Este algoritmo es una variante muy popular en sistemas Linux modernos, a menudo implementado como parte del scheduler mq-deadline.

13.6.7. Algoritmo 7: FSCAN (Freeze SCAN)

FSCAN introduce un concepto nuevo: divide la cola en dos sublistas y las procesa por separado. Esto previene que solicitudes nuevas “se cuelen” y retrasen solicitudes antiguas.

Funcionamiento:

- 1 1. Al inicio: todas las solicitudes actuales van a Queue1
- 2 2. Ejecutar SCAN sobre Queue1
- 3 3. Nuevas solicitudes que llegan durante SCAN van a Queue2 (congeladas)
- 4 4. Al terminar Queue1, intercambiar: Queue2 se vuelve Queue1
- 5 5. Repetir

Ejemplo:

- 1 Queue1 inicial: [98, 183, 37, 122, 14, 124, 65, 67]
- 2 Ejecutar SCAN sobre Queue1: 53 → 65 → 67 → 98 → ... → 183 → 37 → 14
- 3
- 4 Durante este SCAN, llegan solicitudes [50, 175, 80]
- 5 Estas van a Queue2 (esperan al siguiente ciclo)
- 6
- 7 Al terminar Queue1:
- 8 Queue2 se vuelve Queue1: [50, 175, 80]
- 9 Ejecutar SCAN sobre estos

La idea detrás de FSCAN es prevenir starvation de una manera diferente a SCAN. En SCAN puro, si constantemente llegan nuevas solicitudes, algunas solicitudes antiguas podrían ser “empujadas” continuamente. FSCAN garantiza que cada solicitud se procesa dentro de dos ciclos de SCAN como máximo: en el peor caso, llegás justo después de que tu Queue comenzó a procesarse, así que esperarás ese ciclo completo más el siguiente.

Las ventajas son claras: previene starvation de solicitudes lejanas, el tiempo de espera está acotado (máximo 2 ciclos), y ofrece mejor fairness que SCAN simple. Las desventajas son la mayor complejidad de implementación (dos colas a mantener) y potencialmente mayor latencia para solicitudes nuevas (deben esperar al siguiente ciclo incluso si están cerca de la posición actual). FSCAN es usado en sistemas que requieren garantías de tiempo de respuesta, como sistemas de tiempo real.

13.6.8. Algoritmo 8: N-STEP-SCAN

N-STEP-SCAN es similar a FSCAN pero más granular: la cola se divide en sublistas de máximo N solicitudes cada una.

Funcionamiento:

```

1 1. Cola actual tiene solicitudes: [r1, r2, r3, ..., rK]
2 2. Dividir en grupos de máximo N:
3   - Grupo1: [r1, ..., rN]
4   - Grupo2: [rN+1, ..., r2N]
5   - ...
6 3. Ejecutar SCAN sobre Grupo1
7 4. Nuevas solicitudes van a cola futura
8 5. Al terminar Grupo1, procesar Grupo2
9 6. Repetir

```

Ejemplo con N=5:

```

1 Cola: [98, 183, 37, 122, 14, 124, 65, 67, 50, 175]
2
3 Grupo1: [98, 183, 37, 122, 14] → Ejecutar SCAN
4 Grupo2: [124, 65, 67, 50, 175] → Espera
5
6 Al terminar Grupo1, procesar Grupo2
7 Nuevas solicitudes forman Grupo3

```

El parámetro N permite un trade-off configurable. Con N pequeño (por ejemplo, N=10), el sistema es más responsive: cada batch se procesa rápidamente, pero puede haber más movimiento del brazo porque los grupos son pequeños. Con N grande (por ejemplo, N=100), hay más oportunidad de optimizar el movimiento dentro de cada batch, mejorando throughput, pero las solicitudes pueden esperar más porque los batches son grandes.

Las ventajas incluyen tiempo de espera predecible (máximo $\text{ceil}(K/N)$ ciclos para K solicitudes), el parámetro N es configurable según las necesidades del sistema, y ofrece un balance ajustable entre responsiveness y throughput. Las desventajas son que requiere tuning del parámetro N para cada sistema específico, y la implementación es más compleja que algoritmos simples.

N-STEP-SCAN es usado en sistemas de tiempo real donde se necesita acotar las latencias máximas con precisión.

13.6.9. Comparación de Algoritmos

Algoritmo	Movimiento (ejemplo)	Starvation	Fairness	Complejidad
FCFS	640	No	Perfecta	Muy baja
SSTF	236	Sí (posible)	Mala	Baja
SCAN	331	No	Media	Media
C-SCAN	382	No	Buena	Media

Algoritmo	Movimiento (ejemplo)	Starvation	Fairness	Complejidad
LOOK	299	No	Media	Media
C-LOOK	322	No	Buena	Media
FSCAN	~300	No	Buena	Alta
N-STEP	~300	No	Muy buena	Alta

Recomendaciones prácticas:

SSTF: Sistemas con baja carga de I/O donde la prioridad es throughput puro y starvation es improbable.

LOOK/C-LOOK: Default en muchos sistemas modernos. Ofrecen el mejor balance entre throughput, fairness y complejidad de implementación.

FSCAN/N-STEP: Sistemas de tiempo real o con garantías estrictas de latencia máxima.

FCFS: Prácticamente nunca en la práctica, excepto como componente interno de algoritmos más complejos.

13.7. Ejercicio Integrador: Scheduling de Disco

Enunciado:

Un disco tiene 200 cilindros (0-199). La cola de solicitudes contiene:

```
1 Solicitudes: [95, 180, 34, 119, 11, 123, 62, 64]
2 Posición actual de la cabeza: 50
3 Dirección inicial: hacia arriba (UP)
```

Calcular el **movimiento total del brazo** para cada algoritmo:

- a) FCFS
- b) SSTF
- c) SCAN
- d) C-SCAN
- e) LOOK
- f) C-LOOK

Indicar también cuál algoritmo es más eficiente en este caso.

Solución:

- a) FCFS:

```

1 Secuencia: 50 → 95 → 180 → 34 → 119 → 11 → 123 → 62 → 64
2
3 Movimiento:
4 |95-50| + |180-95| + |34-180| + |119-34| + |11-119| + |123-11| + |62-123| +
   |64-62|
5 = 45 + 85 + 146 + 85 + 108 + 112 + 61 + 2
6 = 644 cilindros

```

b) SSTF:

```

1 Desde 50, elegir más cercano:
2 50 → 62 (12) → 64 (2) → 34 (30) → 11 (23) → 95 (84) → 119 (24) → 123 (4) →
   180 (57)
3
4 Movimiento total:
5 12 + 2 + 30 + 23 + 84 + 24 + 4 + 57 = 236 cilindros

```

c) SCAN:

```

1 Desde 50, dirección UP, atender todas hasta el final, luego DOWN:
2 50 → 62 → 64 → 95 → 119 → 123 → 180 → 199 (fin) → 34 → 11
3
4 Movimiento:
5 |62-50| + |64-62| + |95-64| + |119-95| + |123-119| + |180-123| + |199-180| +
   |34-199| + |11-34|
6 = 12 + 2 + 31 + 24 + 4 + 57 + 19 + 165 + 23
7 = 337 cilindros

```

d) C-SCAN:

```

1 Desde 50, UP hasta el final, saltar al inicio, continuar UP:
2 50 → 62 → 64 → 95 → 119 → 123 → 180 → 199 → 0 (salto) → 11 → 34
3
4 Movimiento:
5 12 + 2 + 31 + 24 + 4 + 57 + 19 + 199 + 11 + 23 = 382 cilindros

```

e) LOOK:

```

1 Desde 50, UP hasta última solicitud (180), luego DOWN:
2 50 → 62 → 64 → 95 → 119 → 123 → 180 → 34 → 11
3
4 Movimiento:
5 12 + 2 + 31 + 24 + 4 + 57 + 146 + 23 = 299 cilindros

```

f) C-LOOK:

```

1 Desde 50, UP hasta 180, saltar a primera solicitud pendiente (11), continuar
   UP:
2 50 → 62 → 64 → 95 → 119 → 123 → 180 → 11 (salto) → 34
3
4 Movimiento:
5 12 + 2 + 31 + 24 + 4 + 57 + 169 + 23 = 322 cilindros

```

Resumen:

Algoritmo	Movimiento total	Observaciones
FCFS	644	Peor caso
SSTF	236	Minimo movimiento
SCAN	337	
C-SCAN	382	
LOOK	299	Buen balance
C-LOOK	322	

Respuesta:

El algoritmo más eficiente en términos de movimiento puro es SSTF con 236 cilindros. Sin embargo, debemos recordar que SSTF puede causar starvation en sistemas con alta carga.

El mejor balance entre eficiencia y fairness lo ofrece LOOK con 299 cilindros: solo 27En un sistema real de producción, típicamente elegiríamos LOOK o C-LOOK sobre SSTF.

13.8. RAID (Redundant Array of Independent Disks)

RAID es una tecnología fascinante que surgió de una pregunta simple: ¿qué pasa si combinamos múltiples discos baratos para crear algo mejor que un disco caro? La respuesta resultó ser compleja y multifacética.

RAID: Tecnología que utiliza múltiples discos trabajando en conjunto para proporcionar mayor rendimiento (mediante paralelismo) y/o mayor confiabilidad (mediante redundancia).

La idea central es que múltiples discos operando en paralelo pueden superar las limitaciones de un disco individual. RAID puede mejorar el rendimiento (leyendo/escribiendo en paralelo), mejorar la confiabilidad (duplicando datos), o ambas cosas. Sin embargo, diferentes “niveles” de RAID hacen diferentes trade-offs.

13.8.1. RAID 0: Striping (sin redundancia)

RAID 0 es el más simple: los datos se dividen en bloques que se distribuyen (stripe) uniformemente entre todos los discos. Es como repartir un mazo de cartas entre varios jugadores.

```
1 Archivo dividido en bloques: A1, A2, A3, A4, A5, A6
2
3 Disco 0: [A1] [A3] [A5]
4 Disco 1: [A2] [A4] [A6]
5
6 Lectura/escritura en paralelo
```

Las ventajas de RAID 0 son impresionantes para ciertas aplicaciones:

- Rendimiento máximo: el throughput se multiplica por N discos (en el caso ideal)
- Capacidad total = suma de todos los discos (no se desperdicia nada)
- Implementación simple y eficiente
- Latencia reducida para archivos grandes

Para edición de video 4K o rendering 3D, donde necesitás throughput sostenido de varios GiB/s, RAID 0 es ideal.

Sin embargo, RAID 0 tiene una desventaja catastrófica: cero redundancia. Si cualquier disco falla, perdés todos los datos del array completo. Peor aún, la probabilidad de fallo aumenta: si cada disco tiene 1RAID 0 hace que tu sistema sea menos confiable que un disco individual. El nombre "RAID" es casi irónico aquí, ya que no hay redundancia en absoluto.

Cálculos para RAID 0

```
1 N discos de C capacidad cada uno:
2 Capacidad útil = N × C
3 Rendimiento lectura/escritura = N × velocidad_disco
```

Ejemplo práctico:

- 4 discos de 1 TiB cada uno - Capacidad útil: 4 TiB - Si 1 disco falla: pérdida total

Uso típico: Aplicaciones que requieren máximo rendimiento y tienen backups robustos externos: edición de video profesional, rendering 3D, caches de alto rendimiento, ambientes de desarrollo donde los datos no son críticos.

13.8.2. RAID 1: Mirroring (espejo completo)

RAID 1 toma el enfoque opuesto: cada dato se duplica completamente en dos (o más) discos. Es la técnica más antigua de redundancia: simplemente mantener dos copias idénticas.

```
1 Archivo: A1, A2, A3, A4
2
```

```
3 Disco 0: [A1] [A2] [A3] [A4]
4 Disco 1: [A1] [A2] [A3] [A4] (copia exacta)
```

Cada escritura va a ambos discos síncronamente. Las lecturas pueden venir de cualquier disco, lo que permite balance de carga.

Las ventajas de RAID 1 son claras para datos críticos:

- Confiabilidad alta: tolera la falla de N-1 discos (con 2 discos, toleras 1 falla).
 - Rendimiento de lectura mejorado: podés leer de cualquier disco, efectivamente duplicando el throughput de lectura.
 - Recuperación trivial: el disco espejo es una copia exacta, simplemente copiás y listo.
 - Sin cálculos complejos: no hay overhead de paridad o reconstrucción
- Para bases de datos críticas o servidores de producción, RAID 1 ofrece la máxima tranquilidad.
- Las desventajas son económicas: capacidad útil = 50 % (la mitad se usa para el espejo), costo alto (necesitás el doble de discos para la misma capacidad útil), y la escritura no es más rápida (debe completarse en ambos discos, aunque en paralelo).

Cálculos:

```
1 N discos de C capacidad cada uno (N par):
2 Capacidad útil = (N / 2) × C = C (con 2 discos)
3 Tolerancia a fallos = N/2 discos pueden fallar
```

Ejemplo:

- 2 discos de 1 TiB cada uno
- Capacidad útil: 1 TiB
- Si 1 disco falla: datos siguen accesibles

Uso típico: Datos críticos donde la pérdida sería catastrófica: bases de datos transaccionales, servidores de archivos corporativos, sistemas operativos de servidores de producción, cualquier escenario donde la confiabilidad es más importante que el costo.

13.8.3. RAID 5: Striping con Paridad Distribuida

RAID 5 es el nivel más interesante conceptualmente: combina el rendimiento de RAID 0 con redundancia, pero de manera más eficiente que RAID 1. Los datos y la paridad se distribuyen entre todos los discos.

```
1 4 discos, bloques A1-A6:
2
3 Disco 0: [A1] [A4] [Parity(A5,A6)]
```

```

4 Disco 1: [A2] [Parity(A3,A4)] [A6]
5 Disco 2: [A3] [A5] [Parity(A1,A2)]
6 Disco 3: [Parity(A1,A2,A3)] [A6] [...]
7
8 Paridad se calcula con XOR

```

La clave está en cómo se calcula la paridad usando la operación XOR (Exclusive OR).

XOR (Exclusive OR): Operación bit a bit donde el resultado es 1 si los bits son diferentes, 0 si son iguales.

```

1 Ejemplo simple:
2 Bloque A1 = 10110101
3 Bloque A2 = 11001100
4 Bloque A3 = 01010011
5
6 Paridad = A1 XOR A2 XOR A3
7           = 10110101 XOR 11001100 XOR 01010011
8           = 00101110
9
10 Si falla disco con A2, recuperar:
11 A2 = A1 XOR A3 XOR Paridad
12     = 10110101 XOR 01010011 XOR 00101110
13     = 11001100 ✓ (recuperado)

```

Propiedad clave del XOR: $A \text{ XOR } B \text{ XOR } C \text{ XOR } \dots \text{ XOR } \text{Parity} = 0$, por lo tanto cualquier elemento puede recuperarse.

Tabla de verdad XOR:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Ejemplo numérico completo:

Supongamos 3 discos de datos en RAID 5:

```

1 Disco 0: 11010110
2 Disco 1: 10100101
3 Disco 2: 01110011
4
5 Calcular paridad:
6 Paridad = D0 XOR D1 XOR D2
7           = 11010110 XOR 10100101 XOR 01110011
8
9 Paso a paso:
10 11010110
11 XOR 10100101
12 -----
13 01110011
14 XOR 01110011

```



```

15 -----
16 00000000 ← Paridad (Disco 3)
17
18 Si falla Disco 1, recuperar:
19 D1 = D0 XOR D2 XOR Paridad
20    = 11010110 XOR 01110011 XOR 00000000
21    = 10100101 ✓

```

Este cálculo se hace a nivel de bloque completo (típicamente 512 bytes o 4 KiB). Cada bit de la paridad protege la posición correspondiente en todos los bloques de datos.

La ventaja de RAID 5 es que distribuye la paridad. A diferencia de tener un disco dedicado solo a paridad (lo que crearía un cuello de botella), la paridad se esparce entre todos los discos. Esto significa que todas las escrituras se distribuyen uniformemente, evitando que un disco se desgaste más rápido que los otros.

Las ventajas de RAID 5 crean un balance excelente:

- Buena capacidad: $(N-1)/N$ de espacio útil (75- Tolerancia a fallos: cualquier disco puede fallar sin pérdida de datos
- Rendimiento de lectura excelente: N discos leyendo en paralelo
- Costo/beneficio superior a RAID 1: más capacidad útil con la misma protección

Para servidores de archivos o NAS domésticos/empresariales, RAID 5 es a menudo la elección óptima.

Las desventajas de RAID 5 son sutiles pero importantes:

- Escritura más lenta: cada escritura requiere: (1) leer bloques viejos, (2) calcular nueva paridad, (3) escribir datos y paridad. Esto se llama "write penalty" y típicamente hace que escrituras sean 4× operaciones en vez de 1×.
- Recuperación lenta: reconstruir un disco requiere leer todos los otros discos y calcular cada bloque faltante con XOR.
- Vulnerable durante reconstrucción: si un segundo disco falla mientras estás reconstruyendo el primero, perdés todo. Con discos grandes (multi-TB), la reconstrucción puede tomar días. NO tolera fallas simultáneas de 2+ discos

Cálculos:

```

1 N discos de C capacidad cada uno ( $N \geq 3$ ):
2 Capacidad útil =  $(N - 1) \times C$ 
3 Espacio para paridad =  $1 \times C$  (distribuido)
4 Tolerancia a fallos = 1 disco

```

Ejemplo práctico:

- 4 discos de 1 TiB cada uno
- Capacidad útil: 3 TiB (75 %)
- Espacio de paridad: 1 TiB
- Si 1 disco falla: datos recuperables
- Si 2 discos fallan: pérdida total

Uso típico: NAS (Network Attached Storage) domésticos y empresariales, servidores de archivos, almacenamiento general donde se busca balance entre rendimiento, capacidad y confiabilidad. Es el “sweet spot” para muchas aplicaciones.

13.8.4. Comparación de RAID

Nivel	Discos mín	Capacidad útil	Tolerancia fallos	Rendimiento lectura	Rendimiento escritura	Uso típico
RAID 0	2	$N \times C$	0	Excelente ($N\times$)	Excelente ($N\times$)	Edición video, cache
RAID 1	2	C (50 %)	$N-1$	Bueno ($N\times$)	Normal ($1\times$)	Datos críticos
RAID 5	3	$(N-1) \times C$	1	Excelente	Medio (paridad)	Servidores, NAS

Nota sobre RAID 6:

RAID 6 extiende RAID 5 con doble paridad (bloques P y Q usando diferentes algoritmos matemáticos). Capacidad útil = $(N-2) \times C$. Tolera la falla de 2 discos simultáneos, lo que es crítico con discos modernos de múltiples TB donde la reconstrucción puede tomar días.

El costo es mayor write penalty (6 operaciones por escritura en vez de 4) y mayor complejidad de cálculo de paridad Q (usa aritmética de campos de Galois en vez de simple XOR).

Uso: Almacenamiento empresarial crítico, arrays grandes (8+ discos) donde la probabilidad de fallo doble durante reconstrucción es significativa.

13.9. Buffering y Caching

El subsistema de I/O no solo maneja dispositivos; también implementa técnicas de software para mejorar el rendimiento y manejar la asincronía entre componentes de diferentes velocidades.

13.9.1. Buffering

Buffer: Área de memoria temporal usada para almacenar datos durante transferencias entre componentes de diferente velocidad o cuando hay desincronización temporal.

Los buffers son esenciales porque los productores y consumidores de datos rara vez operan exactamente a la misma velocidad. Pensá en un buffer como un tanque de agua entre dos tuberías de diferente diámetro: suaviza las variaciones de flujo.

1. Simple Buffer:

```
1 Productor → [Buffer] → Consumidor
2
3 Mientras Productor llena el buffer, Consumidor espera
4 Cuando buffer está lleno, Productor espera mientras Consumidor procesa
```

Este es el esquema más simple: un solo buffer. El problema es obvio: solo uno puede trabajar a la vez. Cuando el productor está llenando el buffer, el consumidor debe esperar. Cuando el consumidor está procesando, el productor debe esperar. No hay concurrencia real.

2. Double Buffering:

```
1 Productor → [Buffer 1] ↔ Consumidor
2               [Buffer 2]
3
4 Mientras Consumidor procesa Buffer 1,
5 Productor llena Buffer 2
6
7 Luego intercambian (swap)
```

Double buffering es una mejora elegante: mientras el consumidor procesa los datos de un buffer, el productor ya está llenando el otro. Cuando ambos terminan, simplemente intercambian roles. Esto permite overlap real: ambos trabajan en paralelo.

Ejemplo numérico:

```
1 Con simple buffer:
2   Tiempo_llenar = 10ms
3   Tiempo_procesar = 15ms
4   Tiempo_total_por_buffer = 25ms
5
6 Con double buffer:
7   Ambos trabajan en paralelo
8   Tiempo_total = max(10ms, 15ms) = 15ms por buffer
9   Mejora = 25/15 = 1.67× más rápido
```

En este ejemplo, double buffering mejora el throughput en 67 %. Para dispositivos reales donde la diferencia de velocidad es mayor, las mejoras pueden ser aún más dramáticas.

3. Circular Buffer (Ring Buffer):

```

1    [B0] → [B1] → [B2] → [B3]↕
2
3    [B7] ← [B6] ← [B5] ← [B4]
4
5    Productor escribe en write_ptr
6    Consumer lee desde read_ptr
7    Ambos avanzan circularmente

```

El ring buffer extiende la idea de double buffering a N buffers organizados circularmente. Es ampliamente usado en drivers de dispositivos de alta velocidad.

```

1  #define BUFFER_SIZE 8
2  char buffer[BUFFER_SIZE];
3  int write_ptr = 0;
4  int read_ptr = 0;
5  int count = 0; // Elementos en buffer
6
7  // Productor escribe
8  void produce(char data) {
9      while (count == BUFFER_SIZE); // Buffer lleno, esperar
10     buffer[write_ptr] = data;
11     write_ptr = (write_ptr + 1) % BUFFER_SIZE;
12     count++;
13 }
14
15 // Consumidor lee
16 char consume() {
17     while (count == 0); // Buffer vacío, esperar
18     char data = buffer[read_ptr];
19     read_ptr = (read_ptr + 1) % BUFFER_SIZE;
20     count--;
21     return data;
22 }

```

Las ventajas del ring buffer son poderosas para I/O de alta velocidad:

- Múltiples buffers permiten mayor concurrencia y paralelismo
- Suaviza ráfagas (bursts) de datos: si llega un burst rápido, se almacena en múltiples buffers mientras el consumidor procesa a su ritmo
- Usado extensivamente en drivers de red (para paquetes entrantes), audio (para evitar stuttering), y video streaming
- x - Permite que productor y consumidor operen a velocidades muy diferentes sin bloqueos frecuentes

Casos de uso reales:

- **Disco → memoria:** Buffer cache mantiene bloques recientemente leídos
- **Tarjeta de red → memoria:** Ring buffers de paquetes entrantes/salientes

- **Teclado → aplicación:** Input buffer para keystrokes (por eso podés seguir escribiendo aunque la app esté ocupada)
- **Audio/video streaming:** Buffers para prevenir stuttering ante variaciones de red

13.9.2. Caching

Cache: Copia de datos frecuentemente accedidos almacenada en un medio más rápido para acelerar accesos futuros.

Un cache es fundamentalmente diferente de un buffer. Un buffer almacena datos *en tránsito* temporalmente (hay una sola copia, moviéndose). Un cache almacena una *copia duplicada* de datos que también existen en su ubicación original (hay dos copias: la rápida en cache, la lenta en origen).

Diferencia conceptual:

```
1 Buffer:
2   Datos en tránsito
3   Almacenamiento temporal
4   Una sola copia (moviéndose)
5   Propósito: suavizar diferencias de velocidad
6
7 Cache:
8   Datos duplicados
9   Almacenamiento persistente (mientras esté en cache)
10  Dos copias (origen + cache)
11  Propósito: acelerar accesos repetidos
```

Ejemplo: Buffer cache del sistema operativo

```
1 Aplicación solicita read(fd, buffer, 4096)
2
3 1. SO verifica buffer cache:
4   - ?`Bloque ya en cache? → Retornar inmediatamente (cache hit)
5     Tiempo: ~100 nanosegundos (acceso a RAM)
6
7   - ?`No en cache? → Leer de disco y cachear (cache miss)
8     Tiempo: ~10 milisegundos (acceso a disco)
9
10 2. Al escribir write(fd, buffer, 4096):
11   - Escribir en cache (write-back)
12   - Marcar bloque como "dirty" (modificado)
13   - Eventualmente flush a disco (asíncrono)
14   - Retornar inmediatamente a la aplicación
```

El buffer cache del sistema operativo es uno de los componentes más críticos para el rendimiento. Un cache hit evita un acceso a disco completo (10ms), reemplazándolo por un acceso a RAM (100ns). Esto es una aceleración de 100.000×.

Con un hit rate típico de 90-95

Beneficios del buffer cache:

- *Reduce dramáticamente accesos a disco:* 100.000× más rápido servir desde RAM
- *Write coalescing:* Múltiples escrituras pequeñas al mismo bloque se agrupan en una sola escritura física
- *Read-ahead:* Al leer un bloque, el SO predice que probablemente querrás los siguientes y los precarga
- *Mejora throughput general:* Menos tiempo esperando I/O = más tiempo ejecutando código útil

Hit rate típico: 85-95 % en sistemas de escritorio con 8-16 GB RAM, puede llegar a 98-99 % en servidores con cientos de GB dedicados a cache.

Políticas de reemplazo de cache:

Cuando el cache está lleno y necesitás agregar un nuevo bloque, ¿cuál removés? Varias estrategias:

- *LRU (Least Recently Used):* Reemplazar el bloque que hace más tiempo no se usó. Asume que lo que usaste recientemente probablemente lo uses pronto. Muy efectivo en práctica.
- *LFU (Least Frequently Used):* Reemplazar el bloque que menos veces se ha usado en total. Asume que algunos bloques son “calientes” (frecuentemente usados) y deben mantenerse.
- *Clock (Second Chance):* Aproximación eficiente de LRU. Usa un bit de “referencia” y una mano de reloj que recorre el cache. Cuando se necesita reemplazo, la mano avanza hasta encontrar un bloque con bit = 0, dándole “segunda oportunidad” a los que tienen bit = 1.

Linux modernos usan variantes sofisticadas como *LRU de dos listas* (activa e inactiva) que balancea entre recencia y frecuencia.

13.9.3. Spooling

Spooling (Simultaneous Peripheral Operations On-Line): Técnica donde se almacena la salida de varios procesos en disco antes de enviarla a un dispositivo lento y dedicado (como una impresora).

Spooling es una técnica clásica para manejar dispositivos dedicados lentos. El ejemplo canónico es la impresión.

13.9.4. Ejemplo - Print spooler:

```

1 Proceso 1 →-- |
2 Proceso 2 →-- | Spool Directory (disco) → Print Daemon → Impresora
3 Proceso 3 →-- |
4
5 Cada proceso escribe su documento al spool inmediatamente (rápido)
6 Print daemon envía a impresora secuencialmente (lento)

```

Sin spooling, cada proceso tendría que esperar a que la impresora terminara completamente antes de continuar. Con spooling, el proceso escribe al spool (típicamente toma segundos) y puede continuar inmediatamente. El print daemon maneja la tarea lenta de imprimir en background.

Ventajas:

- Procesos no bloquean esperando dispositivo lento
- Múltiples procesos pueden “usar” el dispositivo simultáneamente (sus trabajos se encolan)
- Permite priorización y reordenamiento de trabajos
- Facilita cancelación de trabajos antes de que se procesen

Ubicación típica en Linux: `/var/spool/cups/` para impresoras, `/var/spool/mail/` para correo saliente.

Spooling es usado no solo para impresoras, sino también para correo saliente, faxes, y cualquier dispositivo dedicado lento donde querés desacoplar la generación de datos de su procesamiento.

13.10. Caso de Estudio: I/O en Linux

Linux proporciona una implementación completa y sofisticada del subsistema de I/O. Veamos cómo estructura este complejo componente.

13.10.1. Arquitectura de I/O en Linux

Linux implementa un subsistema de I/O en capas:

```

1 Aplicación
2 |
3 syscall (read, write, ioctl)
4 |
5 VFS (Virtual File System)
6 |
7 File System Driver (ext4, FAT, etc.)
8 |
9 Block Layer / Page Cache
10 |
11 Device Driver (SCSI, SATA, NVMe)

```

```

12      |
13  Hardware Controller
14      |
15  Dispositivo Físico

```

Cada capa tiene responsabilidades específicas. El *VFS* (Virtual File System) proporciona la abstracción que permite que todas las operaciones de archivo sean uniformes, sin importar si el archivo está en ext4, NTFS, o incluso NFS remoto. El *Block Layer* maneja el scheduling de I/O, request merging, y coordina el page cache. Los *Device Drivers* específicos traducen comandos genéricos a secuencias específicas de hardware.

Esta arquitectura en capas permite que agregar soporte para un nuevo file system (digamos, BTRFS) no requiera modificar ningún device driver, y viceversa: agregar soporte para un nuevo tipo de disco (digamos, NVMe) no requiere modificar ningún file system.

13.10.2. Archivos Especiales en /dev

Una de las ideas más elegantes de Unix/Linux es “*todo es un archivo*”. Los dispositivos se representan como archivos especiales en `/dev`:

```

1 $ ls -l /dev/
2 brw-rw---- 1 root disk 8, 0 Dec 30 10:00 sda # Disco completo
3 brw-rw---- 1 root disk 8, 1 Dec 30 10:00 sda1 # Partición 1
4 brw-rw---- 1 root disk 8, 2 Dec 30 10:00 sda2 # Partición 2
5 crw-rw-rw- 1 root tty 5, 0 Dec 30 10:00 tty # Terminal
6 crw----- 1 root root 10, 1 Dec 30 10:00 psaux # Mouse PS/2

```

El primer carácter indica el tipo:

- **b** (block): Dispositivos de bloque (discos, particiones)
- **c** (character): Dispositivos de carácter (terminales, puertos serie)

Los dos números importantes son el *major* y *minor number*:

```

1 brw-rw---- 1 root disk 8, 1 Dec 30 10:00 sda1
2                               ^ ^
3                               | |
4                               Major Minor
5
6 Major number: identifica el driver (8 = SCSI/SATA disk)
7 Minor number: identifica el dispositivo específico (1 = primera partición)

```

El kernel usa el major number para determinar qué driver debe manejar las operaciones en ese dispositivo. El driver usa el minor number para saber cuál de sus dispositivos se está accediendo. Esta abstracción permite que escribir a `/dev/sda1` sea idéntico sintácticamente a escribir a cualquier archivo regular. El kernel se encarga de rutear la operación al driver correcto.

13.10.3. Schedulers de I/O en Linux

Linux provee varios schedulers de disco que se pueden cambiar dinámicamente según las necesidades del sistema. Cada uno implementa diferentes estrategias de los algoritmos que vimos.

CFQ (Completely Fair Queuing):

- Default en kernels antiguos
- Similar a round-robin entre procesos
- Busca fairness

Deadline:

- Garantiza tiempo máximo de espera
- Usa read deadline y write deadline
- Bueno para sistemas de tiempo real

Noop (No Operation):

- Solo merge de requests consecutivos
- Mínimo overhead
- Óptimo para SSDs (que no tienen seek time)

mq-deadline / BFQ / Kyber:

- Schedulers modernos para dispositivos multi-cola (NVMe)
- Aprovechan paralelismo de SSDs modernos

Ver y cambiar el scheduler (dinámicamente):

```
1 # Ver scheduler actual
2 $ cat /sys/block/sda/queue/scheduler
3 [mq-deadline] kyber bfq none
4
5 # Cambiar scheduler
6 $ echo bfq > /sys/block/sda/queue/scheduler
```

La elección del scheduler puede tener impacto significativo en workloads específicos. Para SSDs, `noop` o `none` típicamente dan mejor rendimiento porque el overhead del scheduling es desperdicio. Para HDDs con carga mixta, `mq-deadline` o `bfq` típicamente son mejores.

13.10.4. ioctl: Operaciones Específicas de Dispositivo

ioctl (Input/Output Control): Syscall que permite ejecutar operaciones específicas de un dispositivo que no son read/write estándar. Es la "puerta trasera" para control de dispositivo.

No todas las operaciones de dispositivo son simplemente leer o escribir datos. A veces necesitas configurar parámetros, obtener información del dispositivo, o ejecutar comandos especiales. `ioctl` es el syscall catch-all para estas operaciones.

Ejemplo conceptual:

```
1 #include <sys/ioctl.h>
2
3 int fd = open("/dev/sda", O_RDONLY);
4
5 // Obtener tamaño del disco
6 unsigned long long size;
7 ioctl(fd, BLKGETSIZE64, &size);
8 printf("Disk size: %llu bytes\n", size);
9
10 // Obtener tamaño de bloque
11 int block_size;
12 ioctl(fd, BLKBSZGET, &block_size);
13 printf("Block size: %d bytes\n", block_size);
14
15 close(fd);
```

Uso típico de ioctl:

- Configurar baudrate en puerto serie
- Cambiar resolución y modos de color en terminal
- Obtener geometría de disco (cilindros, heads, sectores)
- Controlar volumen en dispositivos de audio
- Configurar parámetros de captura de video
- Enviar comandos SCSI raw a discos
- Prácticamente cualquier operación específica de dispositivo

Cada driver define sus propios códigos de ioctl. Por ejemplo, `BLKGETSIZE64` es específico de block devices, mientras que `TCGETS` es específico de terminales. Esta flexibilidad hace que ioctl sea tremendamente poderoso pero también algo caótico: no hay una interfaz uniforme como con read/write.

13.11. 10. Síntesis y Puntos Clave

13.11.1. Conceptos Fundamentales

El *subsistema de I/O* enfrenta una complejidad única en el sistema operativo: debe coordinar una enorme diversidad de dispositivos con rangos de velocidad que van de 6 órdenes de magnitud (desde teclados a 10 bytes/seg hasta SSDs NVMe a 7 GB/seg). La necesidad de abstracción (device independence) permite que las aplicaciones usen interfaces uniformes sin preocuparse por los detalles de cada dispositivo.

Los dispositivos se clasifican por **transferencia** (bloque, carácter, red) y por **acceso** (compartibles, dedicados). Los controladores de dispositivo actúan como interfaz entre el bus del sistema y el dispositivo físico, y el driver del SO interactúa con el controlador, no directamente con el dispositivo.

Las tres *técnicas de I/O* representan una evolución en eficiencia:

Polling: CPU espera activamente (busy-wait). Simple pero extremadamente ineficiente. Solo útil si el dispositivo responde en < 100 ciclos.

Interrupciones: Dispositivo avisa cuando termina. Mucho mejor uso de CPU, pero overhead de context switch en cada evento. Ideal para eventos esporádicos y transferencias pequeñas.

DMA: Transferencia directa sin CPU. Óptimo para bloques grandes. Es obligatorio en dispositivos modernos de alta velocidad.

13.11.2. Estructura y Rendimiento de Discos:

La geometría física del disco (plato $\rightarrow$ pista $\rightarrow$ sector $\rightarrow$ cilindro) determina fundamentalmente su rendimiento. El tiempo de acceso total se descompone en:

$$T_{total} = T_{seek} + T_{rotacional} + T_{transferencia}$$

Valores típicos en HDD de 7200 RPM:

Seek promedio: ~ 9 ms

Latencia rotacional promedio: ~ 4.17 ms

Transferencia de 4 KiB: ~ 0.027 ms (despreciable)

Leer 10 sectores consecutivos toma apenas más tiempo que leer 1 sector (13.44 ms vs 13.2 ms), pero leer 10 sectores dispersos toma $10\times$ más (132 ms). Esta es la razón fundamental por la que necesitamos algoritmos de scheduling de disco.

13.11.3. Scheduling de Disco

El objetivo del disk scheduling es minimizar el movimiento del brazo. Los algoritmos principales son:

FCFS: Simple, justo, pero muy ineficiente SSTF: Greedy, mejor throughput, pero puede causar starvation SCAN/C-SCAN: Tipo “ascensor”, sin starvation LOOK/C-LOOK: Optimización de SCAN (no va hasta extremos innecesariamente) FSCAN/N-STEP: Variantes que congelan la cola para prevenir starvation

13.11.4. RAID

RAID combina múltiples discos para mejorar rendimiento y/o confiabilidad:

RAID 0 (Striping): Capacidad = $N \times C$, Tolerancia = 0 discos, Uso = máximo rendimiento

RAID 1 (Mirroring): Capacidad = C (50, Tolerancia = $N-1$ discos, Uso = datos críticos

RAID 5 (Paridad): Capacidad = $(N - 1) \times C$, Tolerancia = 1 disco, Uso = balance óptimo

Propiedad XOR para recuperación:

```
1 Si A XOR B XOR C = Paridad
2 Entonces: A = B XOR C XOR Paridad
```

13.11.5. Software de I/O

Buffering vs Caching:

- Buffer: Datos en tránsito, temporal, una copia
- Cache: Datos duplicados, persistente (en cache), dos copias

Double buffering permite overlap: mientras el consumidor procesa un buffer, el productor llena el otro. Mejora típica: 50-100 % en throughput.

Ring buffers extienden esto a N buffers circularmente, suavizando bursts y permitiendo máxima concurrencia.

Buffer cache del SO es crítico: un hit evita acceso a disco (~10ms → ~100ns), aceleración de 100.000×. Hit rate típico: 85-95 %.

Device Independence: Aplicaciones usan interfaz uniforme (read/write), drivers específicos manejan hardware particular, VFS abstraee diferentes file systems.

Capas del subsistema I/O:

```
1 Aplicación
2   | syscall
3 File System
4   |
5 Block Layer / Cache
6   |
7 Device Driver
8   |
9 Hardware Controller
10  |
11 Dispositivo
```

13.11.6. Errores Comunes a Evitar

- Confundir seek time con latencia rotacional

```
1 Seek = movimiento del brazo (cambia cilindros)
2 Rotacional = espera a que pase el sector (fijo por RPM)
```

- En SCAN/C-SCAN, olvidar ir hasta el extremo

```
1 SCAN va hasta 199 (o 0), no hasta última solicitud
2 LOOK va solo hasta última solicitud
```

- En C-SCAN/C-LOOK, no contar el salto al inicio

```
1 C-SCAN: movimiento incluye |MAX-pos| + |MAX-0| + |primera_solicitud-0|
2 El salto de MAX a 0 SÍ cuenta como movimiento
```

- Calcular mal capacidad en RAID 5

```
1 Correcto: (N-1) × C
2 Incorrecto: N × C (olvidan el disco de paridad)
```

- Confundir buffer con cache

```
1 Buffer: temporal durante transferencia (tránsito)
2 Cache: copia para acceso rápido (duplicado)
```

- No considerar que DMA usa cycle stealing

```
1 DMA "roba" ciclos del bus periódicamente
2 CPU puede ralentizarse ligeramente (pero sigue siendo mejor que interrupciones)
```

- Pensar que SSTF es siempre óptimo

```
1 SSTF puede causar starvation
2 LOOK/C-LOOK son mejor balance en la práctica
```

13.11.7. Tips para Ejercicios Numéricos

Cálculo de tiempos: 1. Separar claramente: seek, rotacional, transferencia 2. Usar unidades consistentes (ms típicamente) 3. Recordar: transferencia suele ser despreciable en HDD 4. Para múltiples sectores consecutivos: solo 1 seek + 1 rotacional

Scheduling: 1. Dibujar timeline visual con las posiciones 2. Para SCAN/C-SCAN: marcar claramente dirección inicial 3. Verificar que todas las solicitudes se atiendan 4. Revisar si hay starvation posible (SSTF)

RAID: 1. Identificar qué se pregunta: capacidad, tolerancia, o ambos 2. Fórmulas clave: RAID 0: $N \times C$ RAID 1: C (con 2 discos) RAID 5: $(N-1) \times C$ 3. Para paridad XOR: aplicar bit a bit, recordar propiedad asociativa

13.12. 12. Conexiones con Otros Capítulos

El I/O no existe en aislamiento. Se conecta profundamente con todos los componentes del sistema operativo que estudiamos anteriormente.

Capítulo 1: Introducción

Las interrupciones que mencionamos al principio del curso se usan extensivamente en I/O. Cada vez que un disco completa una operación, genera una interrupción. DMA es un caso especial de hardware especializado que trabaja junto con el mecanismo de interrupciones. Los modos de operación (kernel/user) son críticos para I/O protegido: solo el kernel puede acceder directamente a dispositivos.

Capítulo 2: Procesos

Los procesos se bloquean esperando I/O (transición a estado BLOCKED). Cuando solicitas leer un archivo, tu proceso se bloquea hasta que el disco complete la operación. El context switch ocurre precisamente aquí: mientras esperas I/O, otro proceso usa la CPU. Las syscalls read/write que vimos interactúan directamente con el scheduler de procesos.

Capítulo 3: Planificación de CPU

La distinción entre procesos I/O-bound y CPU-bound es fundamental. Los procesos I/O-bound pasan la mayoría del tiempo esperando I/O y tienen bursts de CPU cortos. Los algoritmos de scheduling de CPU consideran estos I/O bursts para dar mejor servicio interactivo. Típicamente, procesos I/O-bound reciben mayor prioridad para mejorar responsiveness.

Capítulo 5: Sincronización

Los drivers de dispositivos usan locks para proteger estructuras compartidas como las colas de I/O. Son posibles race conditions si múltiples CPUs acceden a la cola simultáneamente. Los interrupt handlers deben ser thread-safe porque pueden ejecutarse en cualquier momento, interrumpiendo código del kernel que podría estar accediendo a las mismas estructuras.

Capítulo 7-8: Memoria El buffer cache reside en memoria RAM. En Linux modernos, el page cache y el buffer cache están unificados. Memory-mapped I/O usa directamente el espacio de direcciones virtual para acceder a dispositivos. El swap (memoria virtual usando disco) es fundamentalmente una operación de I/O: cuando hay page fault y la página está en swap, el SO debe leerla desde disco.

Capítulo 9: File Systems

Los file systems dependen completamente del subsistema de I/O. Los bloques lógicos del file system se mapean a sectores físicos del disco mediante la geometría. Los algoritmos de scheduling que estudiamos afectan directamente el rendimiento del file system: leer un archivo fragmentado es mucho más lento que uno contiguo. RAID típicamente se implementa por debajo del file system, en el block device layer.

Ejemplo integrador:

```
1 Aplicación llama read(fd, buf, 4096):
2 1. [Cap 9] VFS traduce fd a inodo
3 2. [Cap 9] File system (EXT4) determina qué bloque leer
4 3. [Cap 8] Verificar si bloque está en buffer cache (memoria)
5 4. Si no: [Cap 10] Generar solicitud de I/O a disco
6 5. [Cap 10] Scheduler de disco ordena la solicitud (LOOK)
7 6. [Cap 10] DMA transfiere datos disco → RAM
8 7. [Cap 10] Interrupción notifica completación
```

```

9 8. [Cap 2] Proceso despierta (BLOCKED → READY)
10 9. [Cap 3] Scheduler de CPU lo ejecuta eventualmente
11 10. [Cap 8] Copiar datos de buffer cache a espacio usuario

```

13.13. 13. Ejercicios Adicionales (Práctica)

13.13.1. Ejercicio 1: Técnicas de I/O

Un dispositivo de red recibe paquetes de 1500 bytes a una tasa de 100 Mbps. ¿Qué técnica de I/O recomendarías: polling, interrupciones o DMA? Justificar.

Respuesta: - Calcular frecuencia de paquetes: ~8333 paquetes/seg - Polling: CPU 100 % ocupada verificando → inviable - Interrupciones: ~8333 interrupciones/seg → overhead alto - **DMA es óptimo:** Transfiere cada paquete sin CPU, solo 1 interrupción por paquete - Plus: Ring buffers para batch de paquetes → aún mejor

13.13.2. Ejercicio 2: Tiempos de Acceso

Se tiene un disco de 10000 RPM, seek promedio 6 ms, transferencia 200 MiB/s. Calcular tiempo para leer 100 sectores de 512 bytes:

- a) Consecutivos en la misma pista
- b) Dispersos aleatoriamente

Respuesta:

```

1 Latencia rotacional = (60/10000)/2 = 3 ms
2
3 a) Consecutivos:
4   T = 6 ms (seek) + 3 ms (rot) + (100×512)/(200×10^6) = 9.25 ms
5
6 b) Dispersos:
7   T = 100 × (6 + 3 + 0.0025) = 900.25 ms
8
9   ~97× más lento!

```

13.13.3. Ejercicio 3: RAID Capacity

Se tienen 6 discos de 2 TiB cada uno. Calcular capacidad útil para:

- a) RAID 0

- b) RAID 1
- c) RAID 5

Respuesta:

```
1 a) RAID 0:  $6 \times 2 \text{ TiB} = 12 \text{ TiB}$  (sin redundancia)
2
3 b) RAID 1:  $3 \times 2 \text{ TiB} = 6 \text{ TiB}$  (3 pares espejo)
4
5 c) RAID 5:  $(6-1) \times 2 \text{ TiB} = 10 \text{ TiB}$  (1 disco de paridad distribuida)
```

13.13.4. Ejercicio 4: Scheduling Avanzado

Teniendo los siguientes datos, Cola: [25, 179, 58, 120, 77, 190, 13, 85], posición: 100, dirección: DOWN, disco [0-199]. Calcular movimiento para LOOK y C-LOOK.

Respuesta:

```
1 LOOK (DOWN luego UP):
2 100 → 85 → 77 → 58 → 25 → 13 (fin de DOWN) → 120 → 179 → 190
3 Movimiento: 15+8+19+33+12+107+59+11 = 264
4
5 C-LOOK (DOWN hasta fin, saltar a máximo, continuar DOWN):
6 100 → 85 → 77 → 58 → 25 → 13 → 190 (salto) → 179 → 120
7 Movimiento: 15+8+19+33+12+177+11+59 = 334
```

Este capítulo completó la visión integral del sistema operativo: gestión de procesos, memoria, file systems, y ahora I/O. Entender cómo el SO coordina la CPU, RAM y dispositivos periféricos es esencial para comprender el funcionamiento completo de un sistema de computación moderno.

14 Recursos Adicionales

14.1. Bibliografía Recomendada

Para profundizar en teoría:

- Silberschatz, Galvin, Gagne. *Operating System Concepts* (10ª ed.). El clásico académico, excelente para fundamentos teóricos.
- Tanenbaum, Bos. *Modern Operating Systems* (4ª ed.). Más orientado a implementación, con buenos ejemplos.

Para programación de sistemas:

- Stevens, Rago. *Advanced Programming in the UNIX Environment* (3ª ed.). La biblia para syscalls y programación en Unix/Linux.
- Kerrisk. *The Linux Programming Interface*. Exhaustivo y actualizado específicamente para Linux.

Para internals del kernel:

- Love. *Linux Kernel Development* (3ª ed.). Accesible introducción al desarrollo del kernel.
- Bovet, Cesati. *Understanding the Linux Kernel* (3ª ed.). Más detallado sobre implementación interna.

14.2. Sobre este Libro

Versión actual: 1.0 (Mayo de 2026)

Repositorio: [<https://github.com/dperalta86/Libro-Sistemas-Operativos>]

Licencia: Creative Commons BY-SA 4.0

Contribuir: - Reportá errores abriendo un issue en GitHub - Sugerí mejoras mediante pull requests

- Compartí tu feedback directamente

Este es un proyecto colaborativo y en evolución. Tu participación lo hace mejor.

15 Glosario de Términos

Este glosario recopila los términos técnicos más importantes utilizados a lo largo del libro. Se mantiene la nomenclatura en inglés seguida de su traducción al español cuando corresponde.

15.1. A

Address Space (Espacio de direcciones): Conjunto de direcciones de memoria que un proceso puede utilizar. En sistemas con memoria virtual, cada proceso tiene su propio espacio de direcciones virtual independiente.

Aging (Envejecimiento): Técnica utilizada en algoritmos de planificación para prevenir *starvation*. Incrementa gradualmente la prioridad de procesos que llevan mucho tiempo esperando.

Atomic Operation (Operación atómica): Operación que se ejecuta completamente o no se ejecuta en absoluto, sin estados intermedios visibles. No puede ser interrumpida y es indivisible desde el punto de vista de otros procesos o hilos.

15.2. B

Backing Store: Área de almacenamiento secundario (generalmente disco) utilizada para mantener páginas que fueron removidas de memoria principal en sistemas de memoria virtual.

Blocking (Bloqueo): Estado en el que un proceso o hilo no puede continuar su ejecución hasta que ocurra algún evento específico (I/O, señal, liberación de recurso).

Busy Waiting (Espera activa): Técnica de sincronización donde un proceso o hilo consume ciclos de CPU activamente verificando una condición en un loop hasta que se cumpla. Ineficiente pero útil en contextos específicos.

Burst: Período continuo de actividad. *CPU burst* es el tiempo que un proceso usa la CPU antes de bloquearse, *I/O burst* es el tiempo dedicado a operaciones de entrada/salida.

15.3. C

Cache: Memoria pequeña y rápida que almacena copias de datos frecuentemente accedidos para reducir el tiempo de acceso promedio.

Context Switch (Cambio de contexto): Proceso de guardar el estado de un proceso o hilo y restaurar el estado de otro. Incluye registros de CPU, contador de programa, puntero de pila y otra información de estado.

Critical Section (Sección crítica): Segmento de código que accede a recursos compartidos y que no debe ser ejecutado concurrentemente por más de un hilo o proceso.

CPU-bound: Proceso que pasa la mayor parte de su tiempo usando la CPU, realizando cálculos intensivos. Contrasta con *I/O-bound*.

15.4. D

Daemon (Demonio): Proceso que corre en background, típicamente iniciado al arranque del sistema, que proporciona servicios sin interacción directa del usuario.

Deadlock (Interbloqueo, Abrazo mortal): Situación donde dos o más procesos están esperando indefinidamente por recursos que están siendo retenidos por otros procesos del mismo conjunto.

Device Driver (Controlador de dispositivo): Software que proporciona una interfaz entre el sistema operativo y un dispositivo de hardware específico.

DMA (Direct Memory Access): Característica que permite a ciertos subsistemas de hardware acceder a memoria del sistema independientemente de la CPU, reduciendo la carga de procesamiento para operaciones de I/O.

15.5. F

File Descriptor (Descriptor de archivo): Entero que identifica un archivo abierto en un proceso. En Unix/Linux, los descriptors 0, 1 y 2 son stdin, stdout y stderr respectivamente.

Fork: Syscall en Unix/Linux que crea un nuevo proceso (hijo) duplicando el proceso actual (padre). El proceso hijo es una copia casi exacta del padre.

Fragmentation (Fragmentación): Desperdicio de espacio de memoria o almacenamiento. *Externa:* espacios libres no contiguos que no pueden usarse. *Interna:* espacio desperdiciado dentro de bloques asignados.

Frame: Unidad física de memoria. En memoria virtual, la memoria física se divide en frames de tamaño fijo (típicamente 4KB) donde se cargan las páginas.

15.6. I

I/O-bound: Proceso que pasa la mayor parte de su tiempo esperando operaciones de entrada/salida. Genera muchos *context switches*.

Interrupt (Interrupción): Señal al procesador indicando que un evento requiere atención inmediata. Causa que el procesador suspenda su ejecución actual y ejecute un *interrupt handler*.

IPC (Inter-Process Communication): Mecanismos que permiten a procesos intercambiar datos y sincronizar sus acciones. Incluyen pipes, sockets, memoria compartida, colas de mensajes, etc.

IRQ (Interrupt Request): Línea de hardware utilizada por dispositivos para señalar al procesador que requieren atención.

15.7. K

Kernel (Núcleo): Componente central del sistema operativo que tiene control completo sobre el hardware y opera en modo privilegiado (*kernel mode*).

Kernel Mode (Modo núcleo, Modo supervisor): Modo de operación de la CPU que permite ejecución de todas las instrucciones y acceso a todo el hardware. Opuesto a *user mode*.

Kernel Space: Área de memoria reservada para el kernel del sistema operativo, inaccesible directamente desde *user space*.

15.8. L

Latency (Latencia): Tiempo de retardo entre el inicio de una solicitud y el comienzo de su respuesta. Diferente de *throughput*.

Livelock: Situación similar a *deadlock* donde procesos cambian constantemente su estado en respuesta a otros procesos sin hacer progreso real.

Lock (Cerrojo, Candado): Mecanismo de sincronización que previene el acceso simultáneo a un recurso compartido. Puede ser *mutex* (mutual exclusion) o *read-write lock*.

15.9. M

Memory Leak (Fuga de memoria): Error de programación donde memoria asignada dinámicamente no es liberada cuando ya no es necesaria, causando pérdida gradual de memoria disponible.

MMU (Memory Management Unit): Componente de hardware responsable de traducir direcciones virtuales a direcciones físicas y hacer cumplir las protecciones de memoria.

Mutex (Mutual Exclusion): Mecanismo de sincronización que permite a un solo hilo acceder a un recurso compartido a la vez. Ver *lock*.

15.10. O

Overhead (Sobrecarga): Trabajo adicional o recursos consumidos por el sistema operativo para gestionar recursos, que no contribuye directamente al trabajo útil de las aplicaciones. Por ejemplo, el tiempo de *context switch*.

15.11. P

Page (Página): Unidad lógica de memoria en sistemas de memoria virtual. Los espacios de direcciones de procesos se dividen en páginas de tamaño fijo.

Page Fault (Fallo de página): Excepción que ocurre cuando un proceso intenta acceder a una página que no está actualmente en memoria física. El sistema operativo debe cargarla desde *backing store*.

Page Table (Tabla de páginas): Estructura de datos que mantiene el mapeo entre direcciones virtuales (páginas) y direcciones físicas (frames).

PCB (Process Control Block): Estructura de datos en el kernel que contiene toda la información necesaria para gestionar un proceso (PID, estado, registros, prioridad, etc.).

PID (Process ID): Identificador numérico único asignado a cada proceso por el sistema operativo.

Pipe (Tubería): Mecanismo de IPC que permite comunicación unidireccional entre procesos, típicamente entre proceso padre e hijo.

Polling: Técnica donde el procesador verifica repetidamente el estado de un dispositivo para determinar si está listo. Alternativa a *interrupts*.

Preemption (Apropiación, Desalojo): Acción del planificador de interrumpir un proceso en ejecución para dar la CPU a otro proceso de mayor prioridad o porque expiró su quantum.

Priority Inversion (Inversión de prioridad): Situación donde un proceso de baja prioridad retiene un recurso necesitado por uno de alta prioridad, efectivamente invirtiendo sus prioridades.

Process (Proceso): Programa en ejecución. Incluye el código del programa, datos, pila, heap, y estado de ejecución.

15.12. Q

Quantum (Cuanto de tiempo, Time slice): Período de tiempo asignado a un proceso en algoritmos de planificación por turnos (*round-robin*). Al expirar, ocurre un *context switch*.

Queue (Cola): Estructura de datos FIFO utilizada por el planificador para organizar procesos. Existen ready queue, wait queue, etc.

15.13. R

Race Condition (Condición de carrera): Situación donde el resultado de una operación depende del timing o secuencia de eventos incontrolables. Ocurre cuando múltiples hilos/procesos acceden concurrentemente a recursos compartidos.

Real-time System (Sistema de tiempo real): Sistema donde la corrección no solo depende del resultado lógico sino también del tiempo en que se produce. *Hard real-time*: deadlines estrictos. *Soft real-time*: deadlines flexibles.

Reentrant Code (Código reentrante): Código que puede ser ejecutado concurrentemente por múltiples hilos sin causar resultados incorrectos. No usa variables globales ni estáticas.

Response Time (Tiempo de respuesta): Tiempo desde que se hace una solicitud hasta que se recibe la primera respuesta (no la respuesta completa).

15.14. S

Scheduler (Planificador): Componente del sistema operativo que decide qué proceso ejecutar siguiente y durante cuánto tiempo.

Semaphore (Semáforo): Mecanismo de sincronización que controla el acceso a recursos compartidos mediante un contador. *Binary semaphore* (similar a mutex), *Counting semaphore* (permite N accesos simultáneos).

Signal (Señal): Mecanismo de notificación asíncrona en Unix/Linux. Permite comunicar eventos a procesos (ej: SIGKILL, SIGTERM, SIGUSR1).

Socket: Endpoint de comunicación de red. Abstracción que permite comunicación entre procesos en diferentes máquinas.

Spinlock: Tipo de *lock* que usa *busy waiting*. El hilo que intenta adquirir el lock “gira” en un loop verificando constantemente. Eficiente para períodos cortos de espera.

Starvation (Inanición): Situación donde un proceso nunca obtiene los recursos que necesita porque otros procesos son constantemente favorecidos.

Swapping: Técnica de gestión de memoria donde procesos completos se mueven entre memoria principal y *backing store*.

Syscall (System Call) (Llamada al sistema): Interfaz programática que permite a programas en *user mode* solicitar servicios del kernel. Ejemplos: `open()`, `read()`, `fork()`, `exec()`.

15.15. T

Thrashing: Situación donde el sistema pasa más tiempo manejando *page faults* (paginando) que ejecutando procesos útiles. Indica memoria insuficiente.

Thread (Hilo): Unidad básica de ejecución dentro de un proceso. Múltiples hilos en un proceso comparten el mismo espacio de direcciones pero tienen sus propias pilas y registros.

Throughput (Rendimiento, Capacidad de procesamiento): Cantidad de trabajo completado por unidad de tiempo. Por ejemplo, procesos completados por segundo.

TLB (Translation Lookaside Buffer): Caché de hardware que almacena traducciones recientes de direcciones virtuales a físicas para acelerar el acceso a memoria.

Turnaround Time (Tiempo de retorno): Tiempo total desde que un proceso es enviado hasta que completa su ejecución. Incluye tiempo de espera, tiempo de CPU y tiempo de I/O.

15.16. U

User Mode (Modo usuario): Modo de operación de la CPU con privilegios restringidos. Los programas de usuario ejecutan en este modo y deben usar *syscalls* para acceder al hardware.

User Space: Área de memoria donde ejecutan los programas de usuario, separada y protegida del *kernel space*.

15.17. V

Virtual Memory (Memoria virtual): Técnica que permite ejecutar programas que son más grandes que la memoria física disponible. Crea la abstracción de un espacio de direcciones grande y contiguo para cada proceso.

Virtual Address: Dirección de memoria generada por la CPU, que debe ser traducida a una dirección física por la MMU antes de acceder a memoria.

15.18. W

Wait Queue: Cola donde procesos/hilos esperan por un evento específico (disponibilidad de recurso, completitud de I/O, etc.).

Working Set: Conjunto de páginas que un proceso está usando activamente en un período de tiempo dado. Importante para políticas de reemplazo de páginas.

15.19. Z

Zombie Process (Proceso zombi): Proceso que ha terminado su ejecución pero su entrada en la tabla de procesos no ha sido removida porque su proceso padre no ha leído su estado de salida con `wait()`.

15.20. Referencias

Los términos en este glosario son utilizados conforme a las definiciones estándar en la literatura de sistemas operativos, particularmente: - Silberschatz, Galvin, Gagne. *Operating System Concepts* - Tanenbaum, Bos. *Modern Operating Systems* - Stevens, Rago. *Advanced Programming in the UNIX Environment*

